

Métaheuristiques pour l'optimisation multi-objectifs

Thibaut Lust

M2 AI2D - MADMC - 2025



SORBONNE
UNIVERSITÉ



Rappel Métaheuristiques

- Solution unique

- Population de solutions

Rappel optimisation multi-objectifs

Métaheuristiques pour l'optimisation multi-objectifs

Mise à jour d'une archive de Pareto

- Simple liste

- Quad-tree

- ND-Tree

Rappel Métaheuristiques

- Solution unique

- Population de solutions

Rappel optimisation multi-objectifs

Métaheuristiques pour l'optimisation multi-objectifs

Mise à jour d'une archive de Pareto

- Simple liste

- Quad-tree

- ND-Tree

Métaheuristiques

Le mot **Métaheuristique** est dérivé de la composition de deux mots grecs :

- ▶ Meta : qui est un suffixe signifiant “au-delà”, “dans un niveau supérieur”
- ▶ Heuristique : qui vient du verbe “heuriskein” et qui signifie “trouver”

Plusieurs définitions ont été proposées pour expliquer ce qu’est une métaheuristique.

Aucune de ces définitions n’est universellement reconnue.

- ▶ I.H. Osman and G. Laporte, Metaheuristics : a bibliography. Annals of Operations Research 63, 513-623, 1996 :
“A metaheuristic is formally defined as an iterative generation process which guides a subordinate heuristic by combining intelligently different concepts for exploring and exploiting the search space, learning strategies are used to structure information in order to find efficiently near-optimal solutions.”
- ▶ S. Voß, S. Martello, I.H. Osman and C. Roucairol (Eds), Meta-Heuristics - Advances and Trends in Local Search Paradigms for Optimization. Kluwer Academic Publishers, Dordrecht, The Netherlands, (1999)
“A metaheuristic is an iterative master process that guides and modifies the operations of subordinate heuristics to efficiently produce high-quality solutions. It may manipulate a complete (or incomplete) single solution or a collection of solutions at each iteration. The subordinate heuristics may be high (or low) level procedures, or a simple local search, or just a constructive method’.”

- ▶ T. Stützle, Local Search Algorithms for Combinatorial Problems - Analysis, Algorithms and New Applications. DISKI - Dissertationen zur Künstlichen Intelligenz. Infix, Sankt Augustin, Germany (1999)

"Metaheuristics are typically high level strategies which guide an underlying more problem specific heuristic, to increase their performance. The main goal is to avoid the disadvantages of iterative improvement and, in particular, multiple descent by allowing the local search to escape from local optima. This is achieved by either allowing worsening moves or generating new starting solutions for the local search in a more "intelligent" way than just providing random initial solutions."

- ▶ Metaheuristics Network Website <http://www.metaheuristics.org>

"A metaheuristic is a set of concepts that can be used to define heuristic methods that can be applied to a wide set of different problems. In other words, a metaheuristic can be seen as a general algorithmic framework which can be applied to different optimization problems with relatively few modifications to make them adapted to a specific problem."

Propriétés fondamentales des métaheuristiques

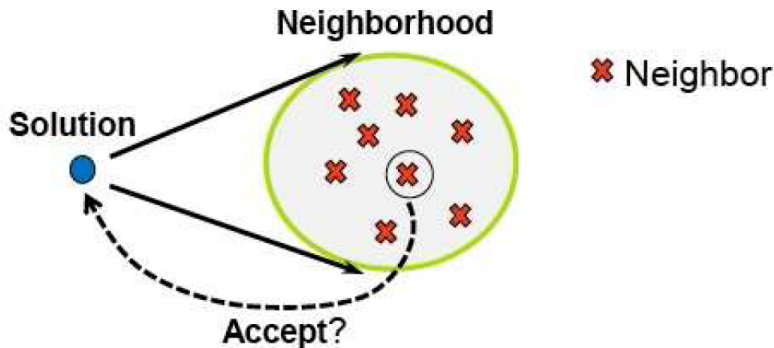
- ▶ Stratégies qui permettent de guider la recherche d'une solution optimale
- ▶ Explorer l'espace de recherche efficacement
- ▶ Les techniques qui vont de la simple procédure de recherche locale à des processus d'apprentissage complexes
- ▶ En général non-déterministes et ne donnent aucune garantie d'optimalité
- ▶ Peuvent contenir des mécanismes qui permettent d'éviter d'être bloqué dans des régions de l'espace de recherche
- ▶ Peuvent être décrites de manière abstraite, sans faire appel à un problème spécifique
- ▶ Peuvent faire appel à des heuristiques qui tiennent compte de la spécificité du problème traité, mais contrôlées par une stratégie de niveau supérieur
- ▶ Peuvent faire usage de l'expérience accumulée durant la recherche de l'optimum, pour mieux guider la suite du processus de recherche

Métaheuristiques à solution unique

- ▶ Recherche locale, algorithmes de descente
- ▶ Recuit Simulé (Simulated Annealing) (Kirkpatrick *et al*, 1983)
- ▶ Recherche Tabou (Tabu search) (Glover, 1986)

Voisinage d'une solution

- ▶ S : ensemble de solutions (espace de recherche)
- ▶ $f : S \rightarrow \mathbb{R}$: fonction objectif à optimiser
- ▶ $\mathcal{N}(s)$: ensemble des voisins d'une solution s



Recherche locale (LS)

3 ingrédients indispensables :

- ▶ S : ensemble de solutions (espace de recherche)
- ▶ $f : S \rightarrow \mathbb{R}$: fonction objectif à optimiser
- ▶ $\mathcal{N}(s)$: ensemble des voisins d'une solution s

Algorithme d'une recherche locale

Choisir solution initiale $s \in S$

répéter

 choisir $s' \in \mathcal{N}(s)$

$s \leftarrow s'$

jusqu'à critère d'arrêt vérifié

Méthode de descente - best improvement

Hill-climbing - best improvement (minimisation)

Choisir solution initiale $s \in S$

répéter

choisir $s' \in \mathcal{N}(s)$ telle que $f(s')$ est minimale

si $f(s') \leq f(s)$ **alors**

$s \leftarrow s'$

jusqu'à s minimum local (i.e. $f(s') > f(s)$)

Méthode de descente - first improvement

Hill-climbing - first improvement (minimisation)

Choisir solution initiale $s \in S$

répéter

 choisir $s' \in \mathcal{N}(s)$ aléatoirement

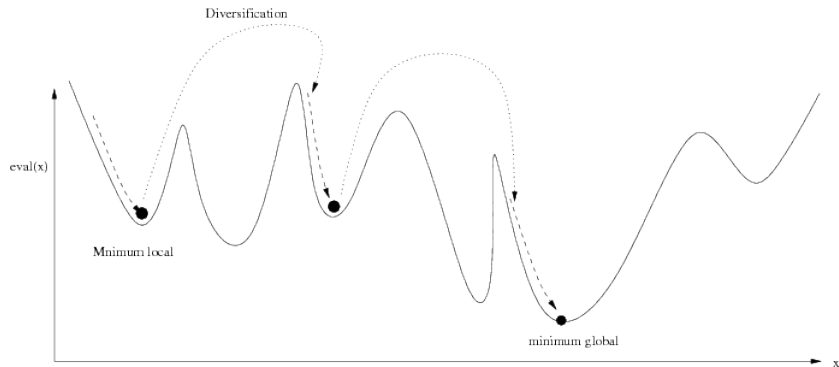
si $f(s') \leq f(s)$ **alors**

$s \leftarrow s'$

fin si

jusqu'à s minimum local (nbr d'essais $>$ taille du voisinage)

Méthode de descente - Inconvénients



Autre méthode de recherche locale

Recherche locale aléatoire - Random Walk

Choisir solution initiale $s \in S$

répéter

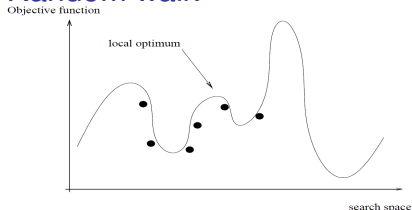
 choisir $s' \in \mathcal{N}(s)$ aléatoirement

$s \leftarrow s'$

jusqu'à nbr d'iter. $> \text{maxNbrIter}$

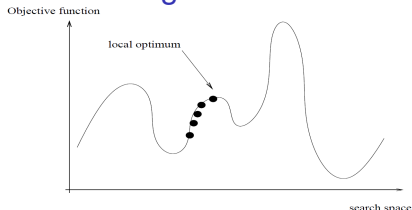
Recherche locale : random walk / hill-climbing

Random walk



Maximal exploration,
diversification

Hill-climbing



Maximal exploitation,
intensification

Enjeu principal : compromis exploration/exploitation

S'échapper d'optimum local → recuit simulé, recherche tabou.

Recuit Simulé (Simulated Annealing)

Utilisé depuis les années 80 :

- ▶ Metropolis (1953) simulation du refroidissement de matériaux (thermodynamique)
- ▶ Kirkpatrick (1983) utilisation pour la résolution de problèmes d'optimisation

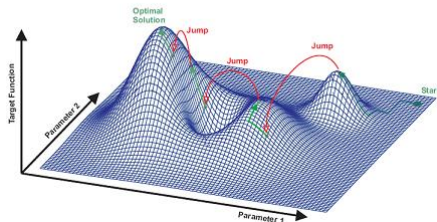
But : échapper aux optima locaux

Principe : probabilité non nulle de sélection d'une solution voisine dégradée

Recuit Simulé : analogie

Système physique	Problème d'optimisation
Energie	Fonction objectif
Etats du système	Solution
Etats de basse énergie	Bonne solution
Température	Paramètre de contrôle

Simulated Annealing



Recuit simulé

Choisir solution initiale $s \in S$ et température initiale T

répéter

Choisir aléatoirement $s' \in \mathcal{N}(s)$, $\Delta = f(s') - f(s)$

Si $\Delta < 0$ **alors**

$s \leftarrow s'$

sinon

u nombre aléatoire de $[0, 1]$

si $u < e^{-\frac{\Delta}{T}}$ **alors**

$s \leftarrow s'$

fin si

fin si

Mise à jour de la température

jusqu'à Critère d'arrêt vérifié

Recuit simulé : remarques

Si $\Delta > 0$ alors la probabilité $e^{\frac{-\Delta}{T}}$ est proche de 0 lorsque :

- ▶ la différence $|\Delta = f(s') - f(s)|$ est grande
- ▶ la température est petite

Conséquences :

- ▶ Lorsque la température est grande (début de recherche) :
→ recherche aléatoire
- ▶ Lorsque la température est petite (fin de la recherche) : →
descente

Recuit simulé : température initiale

Evaluer $\Delta_0 = f(s'_0) - f(s_0)$:

- ▶ Choisir n (grand si possible) solutions aléatoires initiales s_0 et une solution voisine s'_0
- ▶ Calculer la moyenne de Δ_0 sur l'échantillon

Température initiale T_0 telle que $p_0 = e^{\frac{-\Delta_0}{T_0}}$ désiré :

- ▶ Qualité “médiocre” ($p_0 = 0.5$) : démarrage à haute température
- ▶ Qualité “bonne” ($p_0 = 0.2$) : démarrage à basse température

Recuit simulé : décroissance de température

Décroissance suivant une loi géométrique $T_{k+1} = \alpha T_k$ suivant $0.8 \leq \alpha < 1.0$

Changement par pallier de température suivant l'une des deux conditions (par exemple) :

- ▶ $10 \cdot N$ perturbations acceptées (mouvement de solution)
- ▶ $100 \cdot N$ perturbations tentées (mouvement ou non mouvement)

où N est un paramètre qui décrit la taille du problème (nombre de villes, de variables, etc.)

Critère d'arrêt : Par exemple après 3 palliers successifs sans aucune acceptation.

Recherche Tabou (Tabu search)

Introduite par Glover en 1986 :



F. Glover

Future paths for Integer Programming and Links to Artificial Intelligence.

Computers and Operations Research, (5) :533-549, 1986.

But : Echapper aux optima locaux

Principe : Introduction d'une notion de mémoire dans la stratégie d'exploration

Interdiction de reprendre des solutions déjà (ou récemment)
rencontrées

Recherche tabou

Choisir solution initiale $s \in S$

Initialiser la liste Tabou M

répéter

Choisir $s' \in \mathcal{N}(s)$ telle que :

$f(s')$ meilleure solution de $\mathcal{N}(s)$ ET critère d'aspiration

vérifié

OU $f(s')$ meilleure solution de $\mathcal{N}(s)$ non taboue

$s \leftarrow s'$

Mise à la jour de la liste tabou M

jusqu'à Critère d'arrêt vérifié

Recherche tabou : mémoire des tabous

Les tabous sont des mouvements tabous pendant une durée t (longueur de la liste tabou) :

Exemple

Problème du sac à dos avec $n = 10$

$M = (0, 3, 0, 0, 0, 0, 0, 0, 0, 0)$

→ Le deuxième objet ne peut être modifié pendant 3 itérations.

$M = (0, 2, 0, 0, 3, 0, 0, 0, 0, 0)$

→ Le cinquième objet ne peut être modifié pendant 3 itérations.

Etc.

Lorsqu'un mouvement est effectué : interdiction pendant t itérations.

⚠ Si t est trop faible, tabou peu efficace, si t trop grand, trop peu de voisins admissibles.

Recherche tabou : critère d'aspiration

Permet de lever l'interdiction lorsqu'un mouvement interdit permet d'obtenir la meilleure solution enregistrée jusqu'à présent.

Exemple

Problème du sac à dos avec $n = 10$

$M = (0, 2, 0, 0, 3, 0, 0, 0, 0, 0)$

→ Si un mouvement sur le deuxième objet permet d'améliorer la meilleure solution, le mouvement peut être réalisé.

Autres méthodes de recherche locale

- ▶ Greedy randomized adaptive search procedure (GRASP) (construction qui combine les méthodes gloutonnes et aléatoires)
- ▶ Variable neighborhood search (VNS) (changement de voisinage lorsque l'on tombe dans un minimum local)
- ▶ Very large-scale neighborhood search (utilisation d'heuristique/méthode exacte pour l'exploration du voisinage)
- ▶ Etc.

Méthode à population de solutions

- ▶ Parfois appelées **méthodes évolutives** : font évoluer une population de solutions selon des règles bien précises
- ▶ Alternance de périodes d'adaptation individuelle et de périodes de coopération pendant lesquelles les individus peuvent échanger de l'information

Méthode évolutive

Générer une population initiale d'individus

Tant que aucun critère d'arrêt n'est satisfait faire

 Exécuter une procédure de coopération

 Exécuter une procédure d'adaptation individuelle

Fin du tant que

Principales caractéristiques

- ▶ Types d'individus (individus $\neq$ solutions)
- ▶ Type d'évolution (survie de la population) : remplacement générationnel ou remplacement stationnaire
- ▶ Sources d'information : à partir de quelle information les nouveaux individus sont générés
- ▶ Intensification : adaptation individuelle
- ▶ Diversification : bruitage appliqué sur chacun des individus

Algorithmes génétiques



J. Holland

Adaptation In Natural And Artificial Systems.

Adaptation In Natural And Artificial Systems, University of Michigan Press (1975).

Générer une population initiale P_0 de p individus

$i = 0$

Tant que aucun critère d'arrêt n'est satisfait faire

$i = i + 1$

$P_i = \emptyset$

 Répéter p fois

 Créer un enfant E en croisant I_1 et I_2 de P_{i-1}

 Appliquer l'opérateur de mutation à E

 Rajouter l'enfant modifiée à P_i

Fin du tant que

- ▶ Types d'individus : en général les solutions
- ▶ Type d'évolution : remplacement générationnel avec une population de taille constante
- ▶ Sources d'information : deux parents
- ▶ Intensification : aucune
- ▶ Diversification : mutation

Autres méthodes à population de solutions

- ▶ Colonies de fourmis
- ▶ Recherche dispersée (Scatter search)
- ▶ Méthodes à essaims particulières (Particle swarm optimization)
- ▶ Etc.

Théorème “no-free-lunch”

- ▶ Méthode d'optimisation universelle n'existe pas. La seule façon qu'une stratégie soit meilleure qu'une autre c'est de la spécialiser à la structure du problème considéré.
- ▶ Aucune métaheuristique ne bat tous les autres métaheuristicques sur tous les problèmes



D.H. Wolpert and W.G. Macready

No Free Lunch Theorems for Optimization.

IEEE Transactions on Evolutionary Computation, (1) :67, 1997.



Y.C. Ho and D.L. Pepyne

Simple Explanation of the No-Free-Lunch Theorem and Its Implications.

Journal of Optimization Theory and Applications, 115 :549, 2002.

Rappel Métaheuristiques

Solution unique

Population de solutions

Rappel optimisation multi-objectifs

Métaheuristiques pour l'optimisation multi-objectifs

Mise à jour d'une archive de Pareto

Simple liste

Quad-tree

ND-Tree

Mathematical Formulation

$$\begin{array}{ll}\min & f(x) \\ \text{subject to} & g(x) \leq 0 \\ & x \in \mathbb{R}^n\end{array}$$

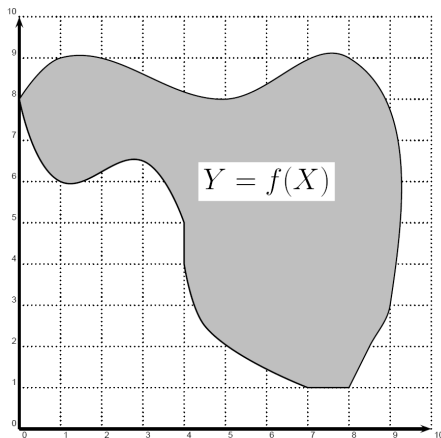
$$\begin{array}{ll}x \in \mathbb{R}^n & \rightarrow n \text{ variables, } i = 1, \dots, n \\ g : \mathbb{R}^n \rightarrow \mathbb{R}^m & \rightarrow m \text{ constraints, } j = 1 \dots, m \\ f : \mathbb{R}^n \rightarrow \mathbb{R}^p & \rightarrow p \text{ objective functions, } k = 1 \dots p\end{array}$$

Feasible Sets

- ▶ $X = \{x \in \mathbb{R}^n : g(x) \leq 0\}$:
feasible set in decision space

Feasible Sets

- ▶ $X = \{x \in \mathbb{R}^n : g(x) \leq 0\}$:
feasible set in decision space
- ▶ $Y = f(X) = \{f(x) : x \in X\}$:
feasible set in objective
space



Pareto optimality

Let y^1 and $y^2 \in \mathbb{R}^p$:

1. Weak component-wise order : $y^1 \leq y^2 \Leftrightarrow y_k^1 \leq y_k^2$ for $k = 1, \dots, p$
2. Component-wise order : $y^1 \prec y^2 \Leftrightarrow y_k^1 \leq y_k^2$ for $k = 1, \dots, p$ and $y^1 \neq y^2$
3. Strict component-wise order : $y^1 < y^2 \Leftrightarrow y_k^1 < y_k^2$ for $k = 1, \dots, p$

Pareto optimality

Let y^1 and $y^2 \in \mathbb{R}^p$:

1. Weak component-wise order : $y^1 \leq y^2 \Leftrightarrow y_k^1 \leq y_k^2$ for $k = 1, \dots, p$
2. Component-wise order : $y^1 \prec y^2 \Leftrightarrow y_k^1 \leq y_k^2$ for $k = 1, \dots, p$ and $y^1 \neq y^2$
3. Strict component-wise order : $y^1 < y^2 \Leftrightarrow y_k^1 < y_k^2$ for $k = 1, \dots, p$

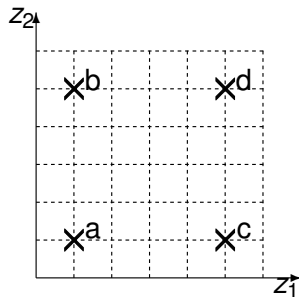
In multiobjective optimization (x : feasible solution, $f(x)$: evaluation) :

1. if $f(x) \leq f(x')$: $f(x)$ weakly Pareto dominates $f(x')$
2. if $f(x) \prec f(x')$: $f(x)$ Pareto dominates $f(x')$
3. if $f(x) < f(x')$: $f(x)$ strictly Pareto dominates $f(x')$

$$f(x) < f(x') \Rightarrow f(x) \prec f(x') \Rightarrow f(x) \leq f(x')$$

$\leq, \prec, <$ are not a total order

Example



- ▶ $a < d \Rightarrow a \prec d \Rightarrow a \leq d$
- ▶ $a \prec b \Rightarrow a \leq b$
- ▶ $a \prec c \Rightarrow a \leq c$
- ▶ $b \prec d \Rightarrow b \leq d$
- ▶ $c \prec d \Rightarrow c \leq d$
- ▶ $b \parallel c \rightarrow$ In multiobjective optimization : solutions cannot always be compared.

Example

$$\min f_1(x_1, x_2, x_3, x_4) = 3x_1 + 2x_2 - 4x_3 + x_4$$

$$\min f_2(x_1, x_2, x_3, x_4) = x_1 - 3x_2 + 4x_3 - 2x_4$$

$$\min f_3(x_1, x_2, x_3, x_4) = -x_1 + x_2 + 3x_3 + x_4$$

$$(n = 4, p = 3)$$

Example

$$\min f_1(x_1, x_2, x_3, x_4) = 3x_1 + 2x_2 - 4x_3 + x_4$$

$$\min f_2(x_1, x_2, x_3, x_4) = x_1 - 3x_2 + 4x_3 - 2x_4$$

$$\min f_3(x_1, x_2, x_3, x_4) = -x_1 + x_2 + 3x_3 + x_4$$

$(n = 4, p = 3)$

- $X = \{x^1 = (2, 1, 3, 1), x^2 = (\frac{-2}{9}, \frac{-44}{9}, \frac{8}{9}, 10), x^3 = (1, 2, 2, 2), x^4 = (\frac{17}{2}, \frac{9}{2}, 4, 1), x^5 = (3, -10, 2, 16)\} :$
arbitrary feasible set in decision space

Example

$$\min f_1(x_1, x_2, x_3, x_4) = 3x_1 + 2x_2 - 4x_3 + x_4$$

$$\min f_2(x_1, x_2, x_3, x_4) = x_1 - 3x_2 + 4x_3 - 2x_4$$

$$\min f_3(x_1, x_2, x_3, x_4) = -x_1 + x_2 + 3x_3 + x_4$$

$(n = 4, p = 3)$

- ▶ $X = \{x^1 = (2, 1, 3, 1), x^2 = (\frac{-2}{9}, \frac{-44}{9}, \frac{8}{9}, 10), x^3 = (1, 2, 2, 2), x^4 = (\frac{17}{2}, \frac{9}{2}, 4, 1), x^5 = (3, -10, 2, 16)\}$:
arbitrary feasible set in decision space
- ▶ $Y = f(X) = \{f(x^1), f(x^2), f(x^3), f(x^4), f(x^5)\} =$
 $\{(-3, 9, 9), (-4, -2, -8), (1, -1, 9), (\frac{39}{2}, 9, 9), (-3, 9, 9)\}$:
feasible set in objective space

Example

$$\min f_1(x_1, x_2, x_3, x_4) = 3x_1 + 2x_2 - 4x_3 + x_4$$

$$\min f_2(x_1, x_2, x_3, x_4) = x_1 - 3x_2 + 4x_3 - 2x_4$$

$$\min f_3(x_1, x_2, x_3, x_4) = -x_1 + x_2 + 3x_3 + x_4$$

$(n = 4, p = 3)$

- ▶ $X = \{x^1 = (2, 1, 3, 1), x^2 = (\frac{-2}{9}, \frac{-44}{9}, \frac{8}{9}, 10), x^3 = (1, 2, 2, 2), x^4 = (\frac{17}{2}, \frac{9}{2}, 4, 1), x^5 = (3, -10, 2, 16)\}$:
arbitrary feasible set in decision space
- ▶ $Y = f(X) = \{f(x^1), f(x^2), f(x^3), f(x^4), f(x^5)\} =$
 $\{(-3, 9, 9), (-4, -2, -8), (1, -1, 9), (\frac{39}{2}, 9, 9), (-3, 9, 9)\}$:
feasible set in objective space

What can you say about

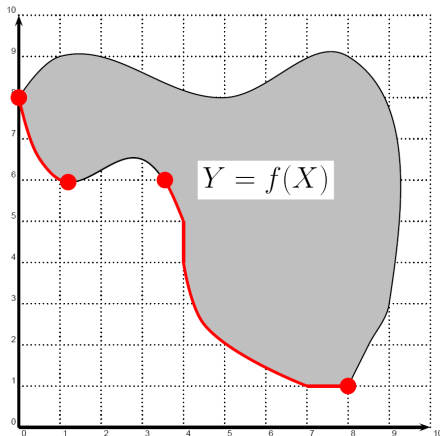
$f(x^1), f(x^2), f(x^3), f(x^4), f(x^5)$ ($\leq, \prec, <, ||$) ?

Weakly Efficient Solutions

► Weakly Efficient Solutions

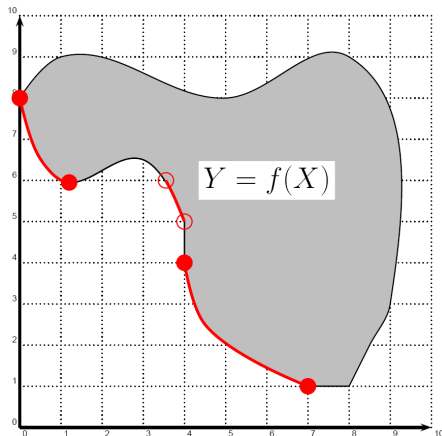
X_{WE}

- There is no x with $f(x) < f(\hat{x})$
- $f(\hat{x})$ is weakly dominated
- $Y_{wN} := f(X_{WE})$



Efficient Solutions

- ▶ Efficient Solutions X_E
 - ▶ There is no x with $f(x) \prec f(\hat{x})$
 - ▶ $f(\hat{x})$ is non-dominated
 - ▶ $Y_N := f(X_E) \rightarrow$ Pareto front.



Lexicographic Optimality

$$\operatorname{lexmin}_{x \in X}(f_1(x), \dots, f_p(x))$$

- ▶ Let y^1 and $y^2 \in \mathbb{R}^p$:
Lexicographic order : $y^1 <_{lex} y^2$ if $y_q^1 < y_q^2$ where $q = \min\{k : y_k^1 \neq y_k^2\}$
- ▶ Lexicographic optimal solution :
A feasible solution $\hat{x} \in X$ is lexicographically optimal or a lexicographic solution if there is no $x \in X$ such that $f(x) <_{lex} f(\hat{x})$.
- ▶ The relation $<_{lex}$ is a total preorder - all solutions can be compared, ex-aequo are possible.

Examples

► Example 1 :

- $f(x) = (f_1(x), f_2(x), f_3(x))$
- $Y = f(X) = \{f(x^1), f(x^2), f(x^3), f(x^4)\} =$
 $\{(-3, 10, 10), (-3, 11, 11), (-3, 10, 12), (-2, 8, 7)\}$

Examples

► Example 1 :

- $f(x) = (f_1(x), f_2(x), f_3(x))$
- $Y = f(X) = \{f(x^1), f(x^2), f(x^3), f(x^4)\} =$
 $\{(-3, 10, 10), (-3, 11, 11), (-3, 10, 12), (-2, 8, 7)\}$
- x^1 is lexicographically optimal

► Example 2 :

- $f(x) = (f_3(x), f_1(x), f_2(x))$
- $Y = f(X) = \{f(x^1), f(x^2), f(x^3), f(x^4)\} =$
 $\{(10, 10, -3), (11, 11, -3), (12, 10, -3), (7, 8, -2)\}$

Examples

► Example 1 :

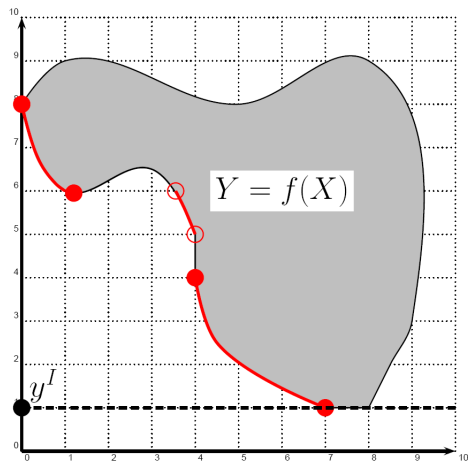
- $f(x) = (f_1(x), f_2(x), f_3(x))$
- $Y = f(X) = \{f(x^1), f(x^2), f(x^3), f(x^4)\} =$
 $\{(-3, 10, 10), (-3, 11, 11), (-3, 10, 12), (-2, 8, 7)\}$
- x^1 is lexicographically optimal

► Example 2 :

- $f(x) = (f_3(x), f_1(x), f_2(x))$
- $Y = f(X) = \{f(x^1), f(x^2), f(x^3), f(x^4)\} =$
 $\{(10, 10, -3), (11, 11, -3), (12, 10, -3), (7, 8, -2)\}$
- x^4 is lexicographically optimal

Ideal and Nadir Points

- Ideal point y^I
 $y_k^I = \min\{y_k : y \in Y\}$



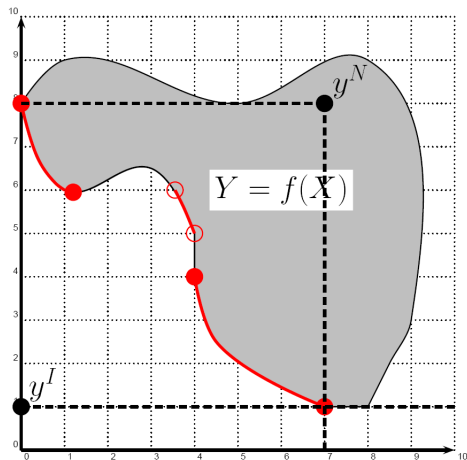
Ideal and Nadir Points

- Ideal point y^I

$$y_k^I = \min\{y_k : y \in Y\}$$

- Nadir point y^N

$$y_k^N = \max\{y_k : Y \in Y_N\}$$



Ideal and Nadir Points

- ▶ Ideal point y^I

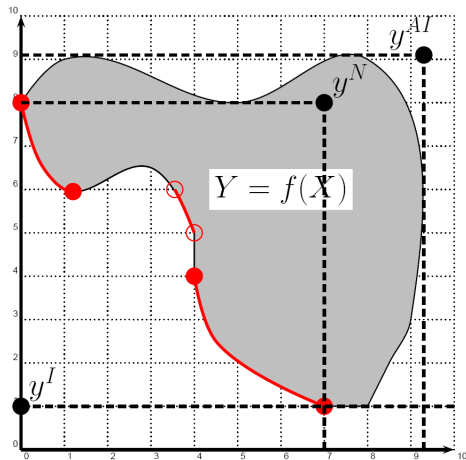
$$y_k^I = \min\{y_k : y \in Y\}$$

- ▶ Nadir point y^N

$$y_k^N = \max\{y_k : Y \in Y_N\}$$

- ▶ Anti-Ideal point y^{AI}

$$y_k^{AI} = \max\{y_k : y \in Y\}$$



Ideal and Nadir Points

- ▶ Ideal point y^I

$$y_k^I = \min\{y_k : y \in Y\}$$

- ▶ Nadir point y^N

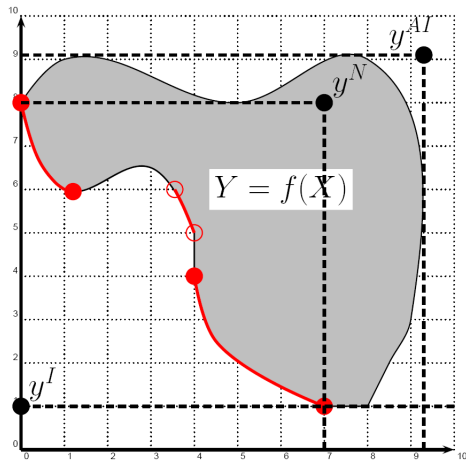
$$y_k^N = \max\{y_k : Y \in Y_N\}$$

- ▶ Anti-Ideal point y^{AI}

$$y_k^{AI} = \max\{y_k : y \in Y\}$$

- ▶ Utopian point y^U

$$y_k^U = y_k^I - \epsilon_k$$



Bounds

For any $y \in Y_N$, we have :

- ▶ $y_k^l \leq y_k$

- ▶ $y_k \leq y_k^N$

y^l and y^N are tight lower and upper bounds on the efficient set.

Multiobjective Combinatorial Optimization Problems (MOCO problems)

$$\begin{array}{ll} \underset{x}{\text{“min”}} & f(x) = Cx \\ \text{subject to} & \{x : Ax \leq b\} \\ & x \in \{0, 1\}^n \end{array}$$

$$\begin{array}{lll} x \in \{0, 1\}^n & \longrightarrow & n \text{ variables, } i = 1, \dots, n \\ C \in \mathbb{Z}^{p \times n} & \longrightarrow & p \text{ objective functions, } k = 1, \dots, p \\ A \in \mathbb{Z}^{m \times n} & \longrightarrow & m \text{ constraints, } j = 1, \dots, m \end{array}$$

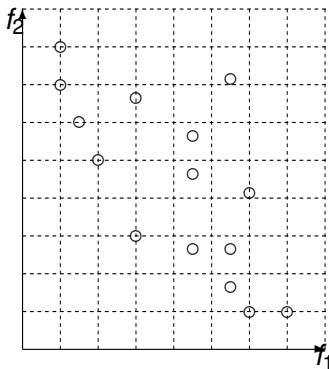
Combinatorial structure : path, trees, flow, tour, etc.

Feasible Sets

- ▶ $X = \{x \in \{0, 1\}^n : Ax \leq b\}$:
feasible set in the decision
space

Feasible Sets

- ▶ $X = \{x \in \{0, 1\}^n : Ax \leq b\}$:
feasible set in the decision
space
- ▶ $Y = f(X) = \{Cx : x \in X\}$:
feasible set in the objective
space



Example : Assignment Problem

$$\min f_k(x) = \sum_{i=1}^n \sum_{j=1}^n c_{ij}^k x_{ij} \quad k = 1, \dots, p$$

$$\text{subject to} \quad \sum_{i=1}^n x_{ij} = 1 \quad j = 1, \dots, n$$

$$\sum_{j=1}^n x_{ij} = 1 \quad i = 1, \dots, n$$

$$x_{ij} \in \{0, 1\} \quad i, j = 1, \dots, n$$

Example :

$$C^1 := (c_{ij}^1) = \begin{pmatrix} 5 & 1 & 4 & 7 \\ 6 & 2 & 2 & 6 \\ 2 & 8 & 4 & 4 \\ 3 & 5 & 7 & 1 \end{pmatrix}$$

$$C^2 := (c_{ij}^2) = \begin{pmatrix} 3 & 6 & 4 & 2 \\ 1 & 3 & 8 & 3 \\ 5 & 2 & 2 & 3 \\ 4 & 2 & 3 & 5 \end{pmatrix}$$

Example : Assignment Problem

$$\min f_k(x) = \sum_{i=1}^n \sum_{j=1}^n c_{ij}^k x_{ij} \quad k = 1, \dots, p$$

$$\text{subject to} \quad \sum_{i=1}^n x_{ij} = 1 \quad j = 1, \dots, n$$

$$\sum_{j=1}^n x_{ij} = 1 \quad i = 1, \dots, n$$

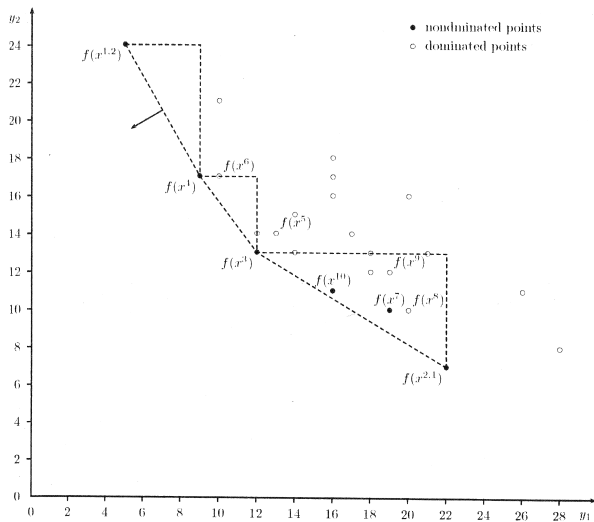
$$x_{ij} \in \{0, 1\} \quad i, j = 1, \dots, n$$

Example :

$$C^1 := (c_{ij}^1) = \begin{pmatrix} 5 & 1 & 4 & 7 \\ 6 & 2 & 2 & 6 \\ 2 & 8 & 4 & 4 \\ 3 & 5 & 7 & 1 \end{pmatrix}$$

$$C^2 := (c_{ij}^2) = \begin{pmatrix} 3 & 6 & 4 & 2 \\ 1 & 3 & 8 & 3 \\ 5 & 2 & 2 & 3 \\ 4 & 2 & 3 & 5 \end{pmatrix}$$

Assignment Problem

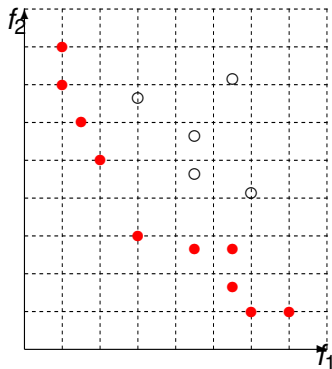


Weakly Efficient Set

- ▶ Weakly efficient solution $x \in X$: a feasible solution $x^* \in X$ is called *weakly efficient* if there does not exist any other feasible solution $x \in X$ such as $f(x) < f(x^*)$.

Weakly Efficient Set

- ▶ Weakly efficient solution $x \in X$: a feasible solution $x^* \in X$ is called *weakly efficient* if there does not exist any other feasible solution $x \in X$ such as $f(x) < f(x^*)$.
 - ▶ $X_{wE} \in X$: set of weakly efficient solutions (weakly efficient set).
 - ▶ $Y_{wN} := f(X_{wE}) (\in Y)$: set of weakly non-dominated points.

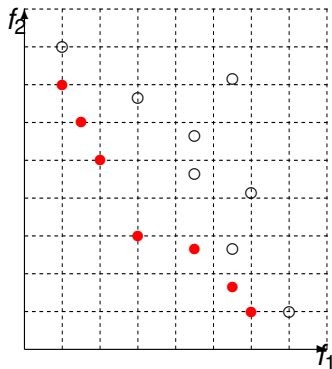


Efficient Set

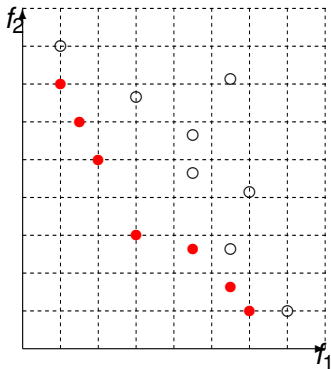
- ▶ Efficient solution : a feasible solution $x^* \in X$ is called *efficient* if there does not exist any other feasible solution $x \in X$ such as $f(x) \prec f(x^*)$.

Efficient Set

- ▶ Efficient solution : a feasible solution $x^* \in X$ is called *efficient* if there does not exist any other feasible solution $x \in X$ such as $f(x) \prec f(x^*)$.
 - ▶ $X_E \in X$: set of efficient solutions (efficient set).
 - ▶ $Y_N := f(X_E) (\in Y)$ (Pareto front, set of non-dominated points).



Efficient Set



In general, in combinatorial multiobjective optimization, we try to generate the efficient set.

Classification of efficient sets

Hansen 1979 :

- ▶ $x_1, x_2 \in X_E$ are equivalent if $f(x_1) = f(x_2)$
- ▶ Complete set : $\hat{X} \subset X_E$ such that for all $y \in Y_N$ there is $x \in \hat{X}$ with $f(x) = y$
- ▶ Minimal complete set (or reduced efficient set) contains no equivalent solutions
- ▶ Maximal complete set contains all equivalent solutions (= efficient set)

Classification of efficient sets

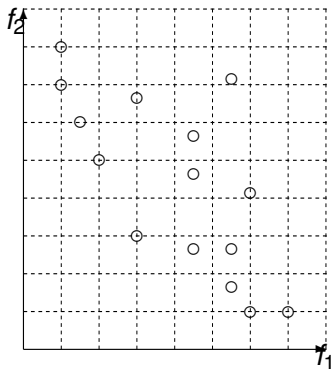
Hansen 1979 :

- ▶ $x_1, x_2 \in X_E$ are equivalent if $f(x_1) = f(x_2)$
- ▶ Complete set : $\hat{X} \subset X_E$ such that for all $y \in Y_N$ there is $x \in \hat{X}$ with $f(x) = y$
- ▶ Minimal complete set (or reduced efficient set) contains no equivalent solutions
- ▶ Maximal complete set contains all equivalent solutions (= efficient set)

→ Efficient set is unique, but the reduced efficient set is not.

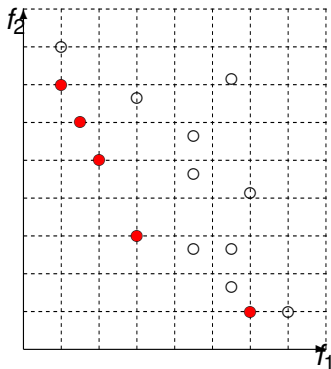
Supported - Non-supported efficient solutions

- ▶ Supported efficient solutions
 X_{SE} : There is $\lambda > 0$ with
 $\lambda^T C\hat{x} \leq \lambda^T Cx$ for all $x \in X$.
 - ▶ $C\hat{x}$ is extreme point of $\text{conv}(Y) + \mathbb{R}_{\geq}^p \rightarrow X_{SE1}$
 - ▶ $C\hat{x}$ is in the relative interior of the faces of $\text{conv}(Y) + \mathbb{R}_{\geq}^p \rightarrow X_{SE2}$
- ▶ Non-supported efficient solutions
 X_{NE} : $C\hat{x}$ is in the interior of $\text{conv}(Y) + \mathbb{R}_{\geq}^p$



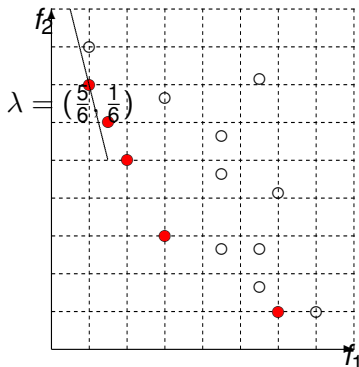
Supported - Non-supported efficient solutions

- ▶ **Supported efficient solutions**
 X_{SE} : There is $\lambda > 0$ with
 $\lambda^T C\hat{x} \leq \lambda^T Cx$ for all $x \in X$.
 - ▶ $C\hat{x}$ is extreme point of $\text{conv}(Y) + \mathbb{R}_{\geq}^p \rightarrow X_{SE1}$
 - ▶ $C\hat{x}$ is in the relative interior of the faces of $\text{conv}(Y) + \mathbb{R}_{\geq}^p \rightarrow X_{SE2}$
- ▶ **Non-supported efficient solutions**
 X_{NE} : $C\hat{x}$ is in the interior of $\text{conv}(Y) + \mathbb{R}_{\geq}^p$



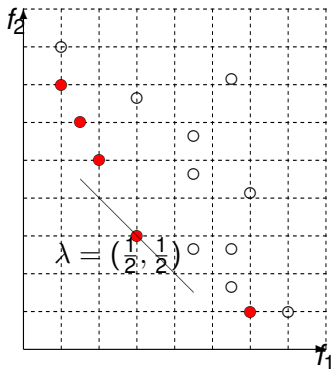
Supported - Non-supported efficient solutions

- ▶ **Supported efficient solutions**
 X_{SE} : There is $\lambda > 0$ with
 $\lambda^T C\hat{x} \leq \lambda^T Cx$ for all $x \in X$.
 - ▶ $C\hat{x}$ is extreme point of $\text{conv}(Y) + \mathbb{R}_{\geq}^p \rightarrow X_{SE1}$
 - ▶ $C\hat{x}$ is in the relative interior of the faces of $\text{conv}(Y) + \mathbb{R}_{\geq}^p \rightarrow X_{SE2}$
- ▶ **Non-supported efficient solutions**
 X_{NE} : $C\hat{x}$ is in the interior of $\text{conv}(Y) + \mathbb{R}_{\geq}^p$



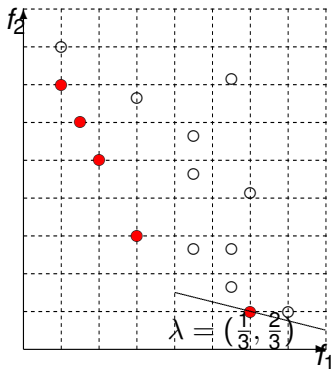
Supported - Non-supported efficient solutions

- ▶ **Supported efficient solutions**
 X_{SE} : There is $\lambda > 0$ with
 $\lambda^T C\hat{x} \leq \lambda^T Cx$ for all $x \in X$.
 - ▶ $C\hat{x}$ is extreme point of $\text{conv}(Y)$
 $+ \mathbb{R}_{\geq}^p \rightarrow X_{SE1}$
 - ▶ $C\hat{x}$ is in the relative interior of
the faces of $\text{conv}(Y) +$
 $\mathbb{R}_{\geq}^p \rightarrow X_{SE2}$
- ▶ **Non-supported efficient solutions**
 X_{NE} : $C\hat{x}$ is in the interior of
 $\text{conv}(Y) + \mathbb{R}_{\geq}^p$



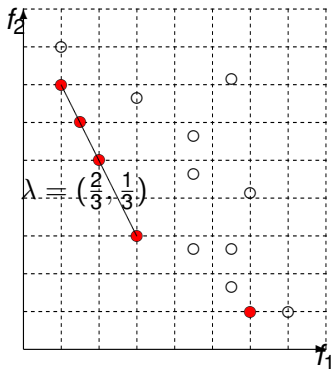
Supported - Non-supported efficient solutions

- ▶ **Supported efficient solutions**
 X_{SE} : There is $\lambda > 0$ with
 $\lambda^T C\hat{x} \leq \lambda^T Cx$ for all $x \in X$.
 - ▶ $C\hat{x}$ is extreme point of $\text{conv}(Y)$
 $+ \mathbb{R}_{\geq}^p \rightarrow X_{SE1}$
 - ▶ $C\hat{x}$ is in the relative interior of
the faces of $\text{conv}(Y) +$
 $\mathbb{R}_{\geq}^p \rightarrow X_{SE2}$
- ▶ **Non-supported efficient solutions**
 X_{NE} : $C\hat{x}$ is in the interior of
 $\text{conv}(Y) + \mathbb{R}_{\geq}^p$



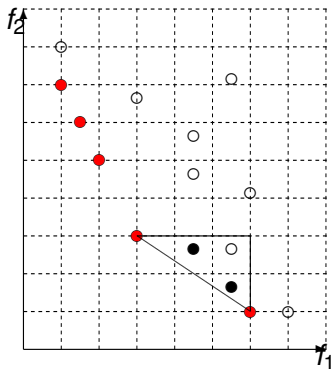
Supported - Non-supported efficient solutions

- ▶ **Supported efficient solutions**
 X_{SE} : There is $\lambda > 0$ with
 $\lambda^T C\hat{x} \leq \lambda^T Cx$ for all $x \in X$.
 - ▶ $C\hat{x}$ is extreme point of $\text{conv}(Y) + \mathbb{R}_{\geq}^p \rightarrow X_{SE1}$
 - ▶ $C\hat{x}$ is in the relative interior of the faces of $\text{conv}(Y) + \mathbb{R}_{\geq}^p \rightarrow X_{SE2}$
- ▶ **Non-supported efficient solutions**
 X_{NE} : $C\hat{x}$ is in the interior of $\text{conv}(Y) + \mathbb{R}_{\geq}^p$

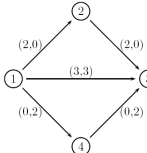


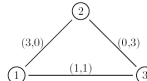
Supported - Non-supported efficient solutions

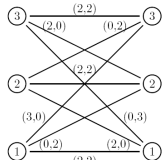
- ▶ Supported efficient solutions
 X_{SE} : There is $\lambda > 0$ with
 $\lambda^T C\hat{x} \leq \lambda^T Cx$ for all $x \in X$.
 - ▶ $C\hat{x}$ is extreme point of $\text{conv}(Y) + \mathbb{R}_{\geq}^p \rightarrow X_{SE1}$
 - ▶ $C\hat{x}$ is in the relative interior of the faces of $\text{conv}(Y) + \mathbb{R}_{\geq}^p \rightarrow X_{SE2}$
- ▶ Non-supported efficient solutions
 X_{NE} : $C\hat{x}$ is in the interior of $\text{conv}(Y) + \mathbb{R}_{\geq}^p$



Non-supported Efficient Solutions

- Shortest path:  $Y_N = \left\{ \begin{pmatrix} 4 \\ 0 \end{pmatrix}, \begin{pmatrix} 3 \\ 3 \end{pmatrix}, \begin{pmatrix} 0 \\ 4 \end{pmatrix} \right\}$

- Spanning tree:  $Y_N = \left\{ \begin{pmatrix} 4 \\ 1 \end{pmatrix}, \begin{pmatrix} 3 \\ 3 \end{pmatrix}, \begin{pmatrix} 1 \\ 4 \end{pmatrix} \right\}$

- Assignment:  $Y_N = \left\{ \begin{pmatrix} 7 \\ 0 \end{pmatrix}, \begin{pmatrix} 4 \\ 4 \end{pmatrix}, \begin{pmatrix} 0 \\ 7 \end{pmatrix} \right\}$

Number of Efficient Solutions

Empirically often :

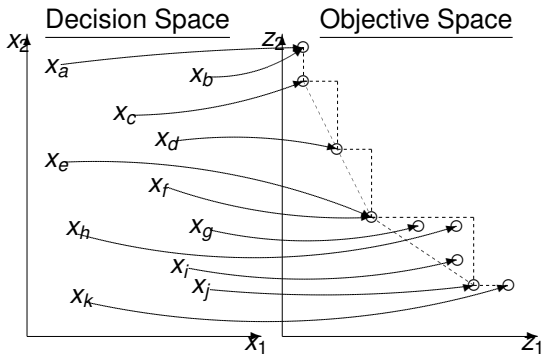
- ▶ $|X_{SE}|$ (supported) grows polynomially with instance size

Number of Efficient Solutions

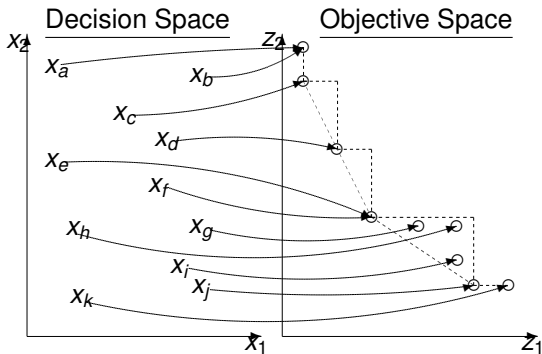
Empirically often :

- ▶ $|X_{SE}|$ (supported) grows polynomially with instance size
- ▶ $|X_{NE}|$ (non-supported) grows exponentially with instance size

Summary

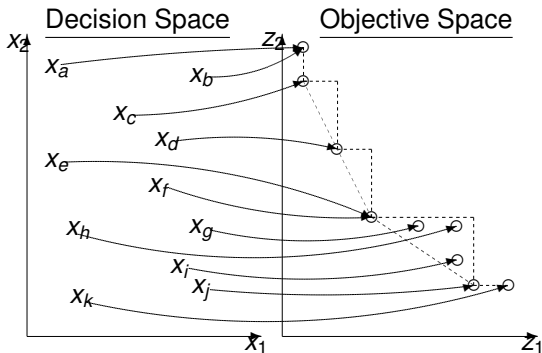


Summary



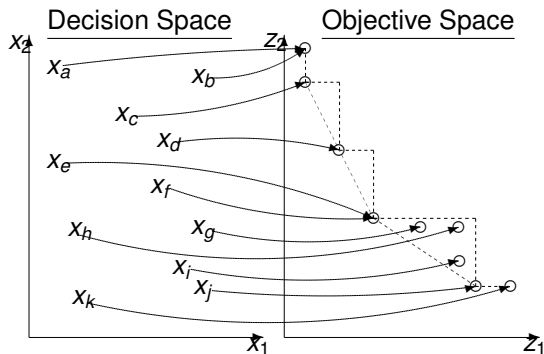
► $X_{WE} =$

Summary



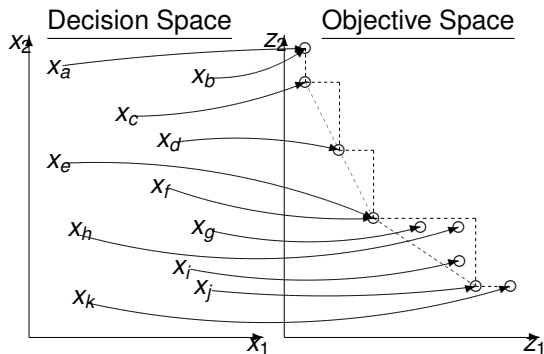
- $X_{WE} = \{x_a, x_b, x_c, x_d, x_e, x_f, x_g, x_h, x_i, x_j, x_k\}$ (Weakly Efficient Set)

Summary



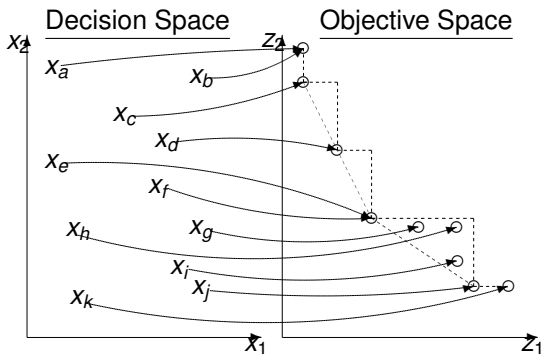
- $X_{WE} = \{x_a, x_b, x_c, x_d, x_e, x_f, x_g, x_h, x_i, x_j, x_k\}$ (Weakly Efficient Set)
- $X_E =$

Summary



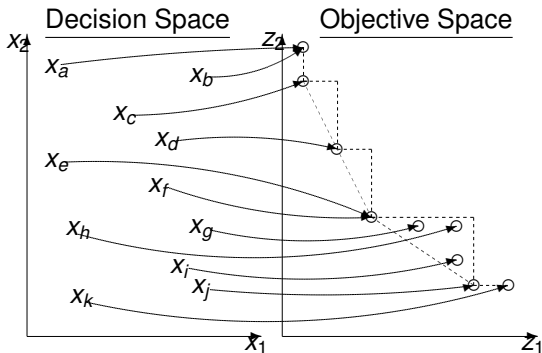
- $X_{WE} = \{x_a, x_b, x_c, x_d, x_e, x_f, x_g, x_h, x_i, x_j, x_k\}$ (Weakly Efficient Set)
- $X_E = \{x_c, x_d, x_e, x_f, x_g, x_i, x_j\}$ (Efficient Set)

Summary



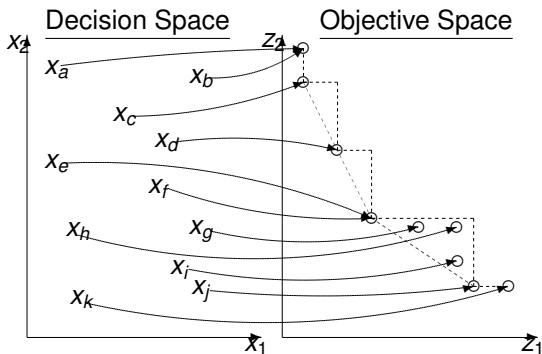
- $X_{WE} = \{x_a, x_b, x_c, x_d, x_e, x_f, x_g, x_h, x_i, x_j, x_k\}$ (Weakly Efficient Set)
- $X_E = \{x_c, x_d, x_e, x_f, x_g, x_i, x_j\}$ (Efficient Set)
- $X_{SE} =$

Summary



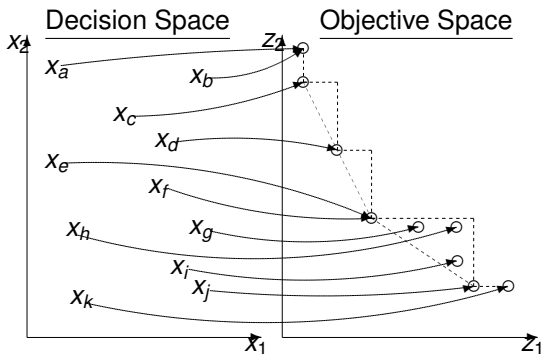
- ▶ $X_{WE} = \{x_a, x_b, x_c, x_d, x_e, x_f, x_g, x_h, x_i, x_j, x_k\}$ (Weakly Efficient Set)
- ▶ $X_E = \{x_c, x_d, x_e, x_f, x_g, x_i, x_j\}$ (Efficient Set)
- ▶ $X_{SE} = \{x_c, x_d, x_e, x_f, x_j\}$ (Supported Efficient Solutions Set)

Summary



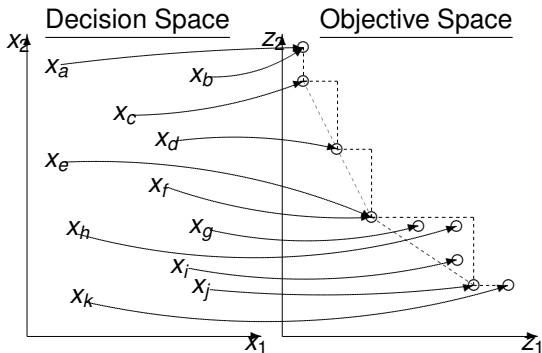
- ▶ $X_{WE} = \{x_a, x_b, x_c, x_d, x_e, x_f, x_g, x_h, x_i, x_j, x_k\}$ (Weakly Efficient Set)
- ▶ $X_E = \{x_c, x_d, x_e, x_f, x_g, x_i, x_j\}$ (Efficient Set)
- ▶ $X_{SE} = \{x_c, x_d, x_e, x_f, x_j\}$ (Supported Efficient Solutions Set)
- ▶ $X_{SE1} =$

Summary



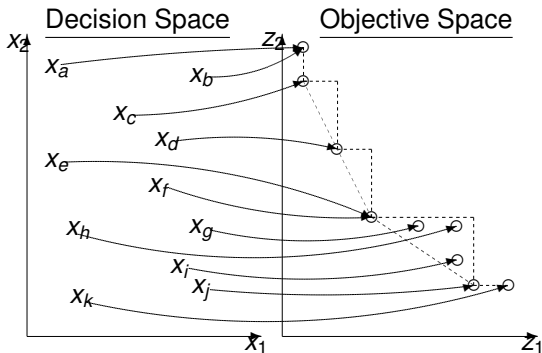
- ▶ $X_{WE} = \{x_a, x_b, x_c, x_d, x_e, x_f, x_g, x_h, x_i, x_j, x_k\}$ (Weakly Efficient Set)
- ▶ $X_E = \{x_c, x_d, x_e, x_f, x_j\}$ (Efficient Set)
- ▶ $X_{SE} = \{x_c, x_d, x_e, x_f, x_j\}$ (Supported Efficient Solutions Set)
- ▶ $X_{SE1} = \{x_c, x_e, x_f, x_j\}$ (Extreme Supported Efficient Solutions Set)

Summary



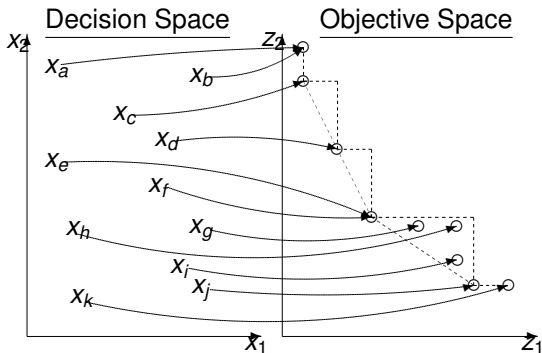
- ▶ $X_{WE} = \{x_a, x_b, x_c, x_d, x_e, x_f, x_g, x_h, x_i, x_j, x_k\}$ (Weakly Efficient Set)
- ▶ $X_E = \{x_c, x_d, x_e, x_f, x_g, x_i, x_j\}$ (Efficient Set)
- ▶ $X_{SE} = \{x_c, x_d, x_e, x_f, x_j\}$ (Supported Efficient Solutions Set)
- ▶ $X_{SE1} = \{x_c, x_e, x_f, x_j\}$ (Extreme Supported Efficient Solutions Set)
- ▶ $X_{SE2} =$

Summary



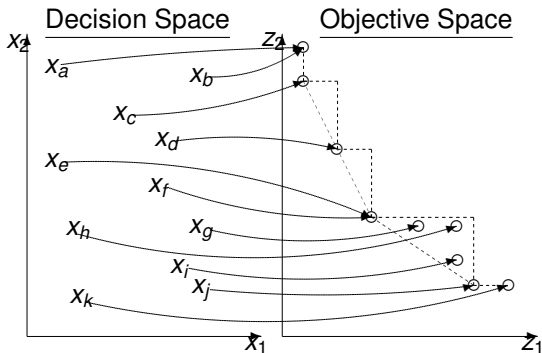
- ▶ $X_{WE} = \{x_a, x_b, x_c, x_d, x_e, x_f, x_g, x_h, x_i, x_j, x_k\}$ (Weakly Efficient Set)
- ▶ $X_E = \{x_c, x_d, x_e, x_f, x_g, x_i, x_j\}$ (Efficient Set)
- ▶ $X_{SE} = \{x_c, x_d, x_e, x_f, x_j\}$ (Supported Efficient Solutions Set)
- ▶ $X_{SE1} = \{x_c, x_e, x_f, x_j\}$ (Extreme Supported Efficient Solutions Set)
- ▶ $X_{SE2} = \{x_d\}$ (Non-Extreme Supported Efficient Solutions Set)

Summary



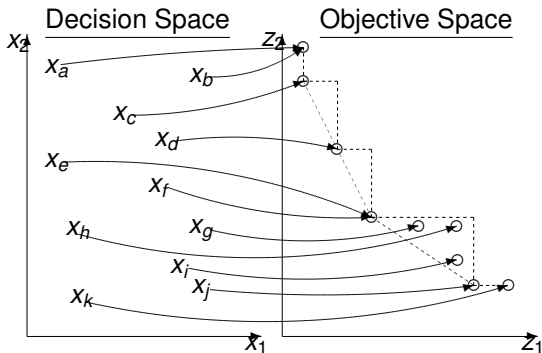
- ▶ $X_{WE} = \{x_a, x_b, x_c, x_d, x_e, x_f, x_g, x_h, x_i, x_j, x_k\}$ (Weakly Efficient Set)
- ▶ $X_E = \{x_c, x_d, x_e, x_f, x_g, x_i, x_j\}$ (Efficient Set)
- ▶ $X_{SE} = \{x_c, x_d, x_e, x_f, x_j\}$ (Supported Efficient Solutions Set)
- ▶ $X_{SE1} = \{x_c, x_e, x_f, x_j\}$ (Extreme Supported Efficient Solutions Set)
- ▶ $X_{SE2} = \{x_d\}$ (Non-Extreme Supported Efficient Solutions Set)
- ▶ $X_{NE} =$

Summary



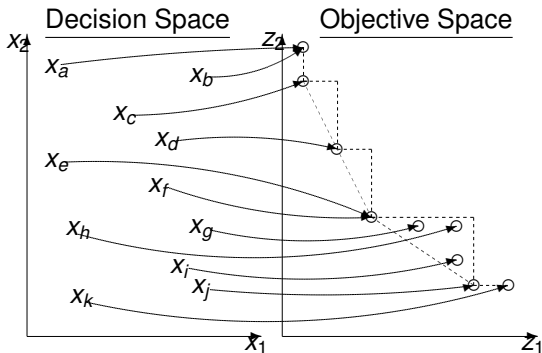
- ▶ $X_{WE} = \{x_a, x_b, x_c, x_d, x_e, x_f, x_g, x_h, x_i, x_j, x_k\}$ (Weakly Efficient Set)
- ▶ $X_E = \{x_c, x_d, x_e, x_f, x_g, x_i, x_j\}$ (Efficient Set)
- ▶ $X_{SE} = \{x_c, x_d, x_e, x_f, x_j\}$ (Supported Efficient Solutions Set)
- ▶ $X_{SE1} = \{x_c, x_e, x_f, x_j\}$ (Extreme Supported Efficient Solutions Set)
- ▶ $X_{SE2} = \{x_d\}$ (Non-Extreme Supported Efficient Solutions Set)
- ▶ $X_{NE} = \{x_g, x_i\}$ (Non-Supported Efficient Set)

Summary



- ▶ $X_{WE} = \{x_a, x_b, x_c, x_d, x_e, x_f, x_g, x_h, x_i, x_j, x_k\}$ (Weakly Efficient Set)
- ▶ $X_E = \{x_c, x_d, x_e, x_f, x_g, x_i, x_j\}$ (Efficient Set)
- ▶ $X_{SE} = \{x_c, x_d, x_e, x_f, x_j\}$ (Supported Efficient Solutions Set)
- ▶ $X_{SE1} = \{x_c, x_e, x_f, x_j\}$ (Extreme Supported Efficient Solutions Set)
- ▶ $X_{SE2} = \{x_d\}$ (Non-Extreme Supported Efficient Solutions Set)
- ▶ $X_{NE} = \{x_g, x_i\}$ (Non-Supported Efficient Set)
- ▶ A minimal complete set=

Summary



- ▶ $X_{WE} = \{x_a, x_b, x_c, x_d, x_e, x_f, x_g, x_h, x_i, x_j, x_k\}$ (Weakly Efficient Set)
- ▶ $X_E = \{x_c, x_d, x_e, x_f, x_g, x_i, x_j\}$ (Efficient Set)
- ▶ $X_{SE} = \{x_c, x_d, x_e, x_f, x_j\}$ (Supported Efficient Solutions Set)
- ▶ $X_{SE1} = \{x_c, x_e, x_f, x_j\}$ (Extreme Supported Efficient Solutions Set)
- ▶ $X_{SE2} = \{x_d\}$ (Non-Extreme Supported Efficient Solutions Set)
- ▶ $X_{NE} = \{x_g, x_i\}$ (Non-Supported Efficient Set)
- ▶ A minimal complete set = $\{x_c, x_d, x_e, x_g, x_i, x_j\}$.

Rappel Métaheuristiques

- Solution unique

- Population de solutions

Rappel optimisation multi-objectifs

Métaheuristiques pour l'optimisation multi-objectifs

Mise à jour d'une archive de Pareto

- Simple liste

- Quad-tree

- ND-Tree

Solving a multiobjective combinatorial optimization problem

Three approaches :

- ▶ **A posteriori** approach : generation of the Pareto optimal set
- ▶ **A priori** approach : elicitation and optimization of an aggregation function (weighted sum, weighted Tchebycheff norm, ordered weighted average, Choquet integral, etc.) → single-objective optimization
- ▶ **Interactive** approach : dynamic elicitation of an aggregation function according to the preferences expressed by the decision maker

Why is it difficult to generate the Pareto optimal set

Why is it difficult to generate the Pareto optimal set

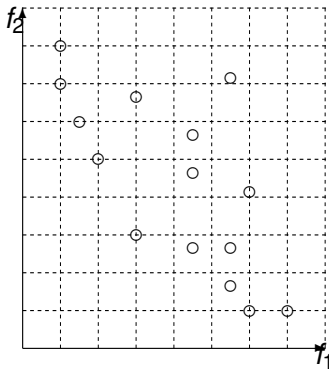
1. Supported and non-supported efficient solutions

$$\min_{x \in D} \sum_{k=1}^p \lambda_k f_k(x)$$

Why is it difficult to generate the Pareto optimal set

1. Supported and non-supported efficient solutions

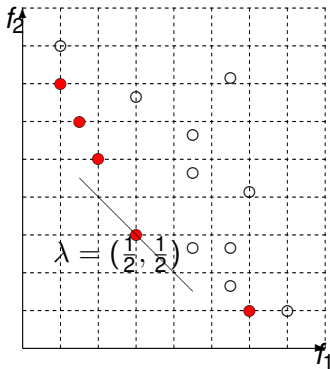
$$\min_{x \in D} \sum_{k=1}^p \lambda_k f_k(x)$$



Why is it difficult to generate the Pareto optimal set

1. Supported and non-supported efficient solutions

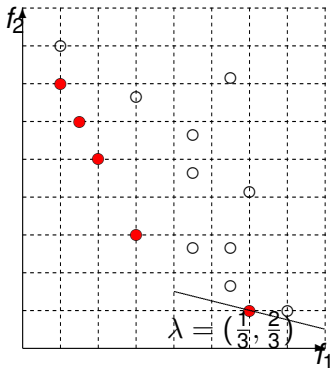
$$\min_{x \in D} \sum_{k=1}^p \lambda_k f_k(x)$$



Why is it difficult to generate the Pareto optimal set

1. Supported and non-supported efficient solutions

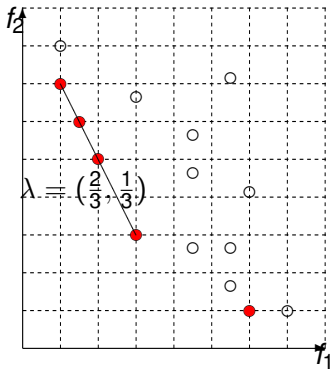
$$\min_{x \in D} \sum_{k=1}^p \lambda_k f_k(x)$$



Why is it difficult to generate the Pareto optimal set

1. Supported and non-supported efficient solutions

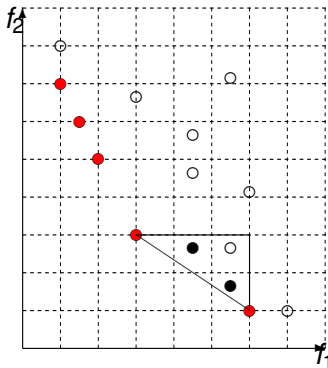
$$\min_{x \in D} \sum_{k=1}^p \lambda_k f_k(x)$$



Why is it difficult to generate the Pareto optimal set

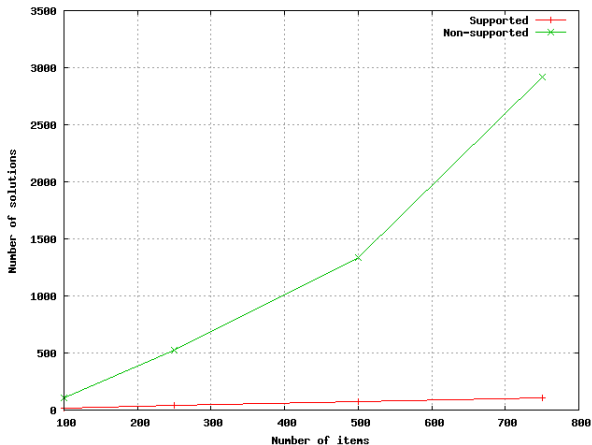
1. Supported and non-supported efficient solutions

$$\min_{x \in D} \sum_{k=1}^p \lambda_k f_k(x)$$



Why is it difficult to solve a MOCO problem ?

2. For many MOCO problems, the number of supported efficient solutions grows *polynomially* with instance size while the number of non-supported efficient solutions grows *exponentially* with instance size



Why is it difficult to solve a MOCO problem ?

3. $\mathcal{NP}$ -Hardness

Combinatorial Problem	$p = 1$	$p \geq 2$
Shortest path	Polynomial	$\mathcal{NP}$ -Hard
Minimum spanning tree	Polynomial	$\mathcal{NP}$ -Hard
Assignment	Polynomial	$\mathcal{NP}$ -Hard
Minimal cut s-t	Polynomial	$\mathcal{NP}$ -Hard
Knapsack	$\mathcal{NP}$ -Hard	$\mathcal{NP}$ -Hard
Traveling salesman	$\mathcal{NP}$ -Hard	$\mathcal{NP}$ -Hard

Why is it difficult to solve a MOCO problem ?

3. $\mathcal{NP}$ -Hardness

Combinatorial Problem	$p = 1$	$p \geq 2$
Shortest path	Polynomial	$\mathcal{NP}$ -Hard
Minimum spanning tree	Polynomial	$\mathcal{NP}$ -Hard
Assignment	Polynomial	$\mathcal{NP}$ -Hard
Minimal cut s-t	Polynomial	$\mathcal{NP}$ -Hard
Knapsack	$\mathcal{NP}$ -Hard	$\mathcal{NP}$ -Hard
Traveling salesman	$\mathcal{NP}$ -Hard	$\mathcal{NP}$ -Hard

4. Intractability

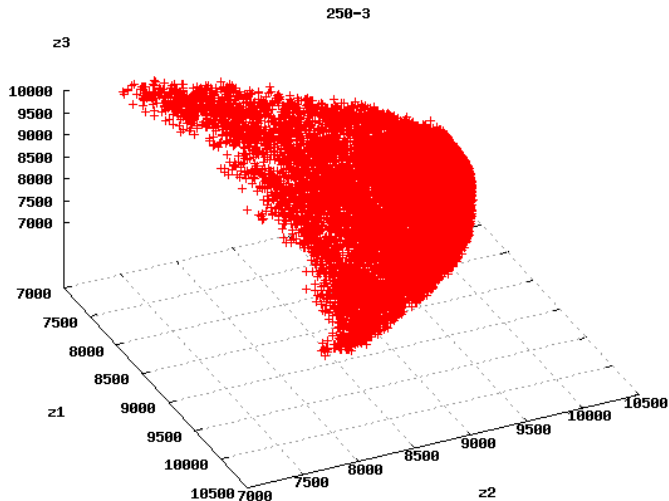
Most of the standard MOCO problems have been proved to be intractable, that is the number of non-dominated points can be exponential in the size of the instance

Why is it difficult to solve a MOCO problem ?

5. Many non-dominated points (knapsack instance, 3 crit, 250 items)

Why is it difficult to solve a MOCO problem ?

5. Many non-dominated points (knapsack instance, 3 crit, 250 items)



Methods

- ▶ Exact methods (two-phase method, ϵ -constraint, branch and bound, dynamic programming, ...) : work only for small size multiobjective problems (1000 items for a biobjective knapsack, 100 items for a three-objective knapsack)
- ▶ Approximation algorithms : few
- ▶ Metaheuristics : quite a lot, adapted since 1984

Metaheuristics for solving MOCO problems

- ▶ Many papers present the adaptation of metaheuristics to multiobjective problems
- ▶ The majority of the developed methods follows the idea of one (hybrid) metaheuristic
- ▶ But all metaheuristics for MOCO problems use a population P to store the non-dominated solutions (with the non-dominated property : $\forall p_1, p_2 \in P, p_1 \not\prec p_2 \wedge p_2 \not\prec p_1$)
- ▶ Search space : same as in the single-objective problem

Updating the potentially efficient set (Update)

IN $\downarrow$: x (solution)

IN-OUT $\updownarrow$: $\hat{\mathcal{X}}_E$ (set of potentially efficient solutions)

OUT $\uparrow$: Boolean

if $(\nexists x' \in \hat{\mathcal{X}}_E \mid x' \preceq x)$ **then**

$\hat{\mathcal{X}}_E \leftarrow \hat{\mathcal{X}}_E \cup \{x\}$

for all $(x' \in \hat{\mathcal{X}}_E \mid x \prec x')$ **do**

$\hat{\mathcal{X}}_E \leftarrow \hat{\mathcal{X}}_E \setminus \{x'\}$

end for

Return **True**

else

Return **False**

end if

Aim of a metaheuristic for solving MOCO problems

Aim of a metaheuristic for solving MOCO problems

To find a good approximation of a Pareto set = set of feasible solutions

Aim of a metaheuristic for solving MOCO problems

To find a good approximation of a Pareto set = set of feasible solutions

But what is a good approximation ?

Aim of a metaheuristic for solving MOCO problems

To find a good approximation of a Pareto set = set of feasible solutions

But what is a good approximation ?

- ▶ ψ : consists of all potential Pareto sets approximations (set of sets)

Aim of a metaheuristic for solving MOCO problems

To find a good approximation of a Pareto set = set of feasible solutions

But what is a good approximation ?

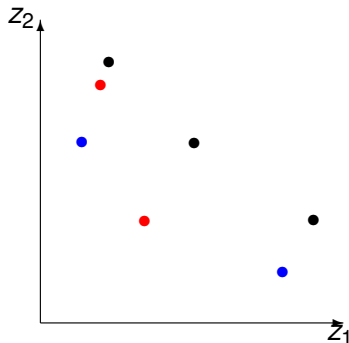
- ▶ ψ : consists of all potential Pareto sets approximations (set of sets)
- ▶ An appropriate order on ψ is required to specify when an approximation is better than another.

Set preference relations : what we can say

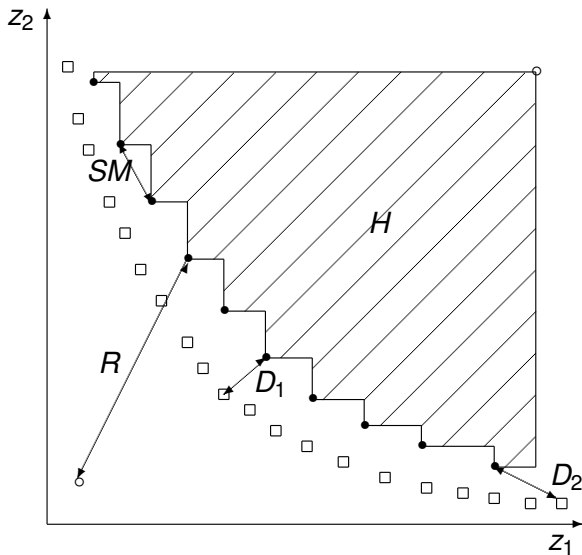
strictly dominates	$A \prec\prec B$	every $b \in B$ is strictly dominated by at least $a \in A$
dominates	$A \prec B$	every $b \in B$ is dominated by at least $a \in A$
better	$A \triangleleft B$	every $b \in B$ is weakly dominated by at least $a \in A$ and $A \neq B$
weakly dominates	$A \preceq B$	every $b \in B$ is weakly dominated by at least $a \in A$
incomparable	$A B$	neither A weakly dominates B nor B weakly dominates A

Set preference relations : what we can say

strictly dominates	$A \prec\prec B$	every $b \in B$ is strictly dominated by at least $a \in A$
dominates	$A \prec B$	every $b \in B$ is dominated by at least $a \in A$
better	$A \triangleleft B$	every $b \in B$ is weakly dominated by at least $a \in A$ and $A \neq B$
weakly dominates	$A \preceq B$	every $b \in B$ is weakly dominated by at least $a \in A$
incomparable	$A B$	neither A weakly dominates B nor B weakly dominates A



Some quality indicators generally used (unary)



Problem with unary quality indicators (Zitzler et al., 2003)

Considering :

- ▶ Combinatorial multiobjective problem with $p \geq 2$
- ▶ A and B two approximation sets
- ▶ A finite combination $\mathbf{I}$ of unary indicators
- ▶ A comparison method C based on $\mathbf{I}$ and an interpretation function E , $C_{\mathbf{I},E}(A, B) = E(\mathbf{I}(A), \mathbf{I}(B))$.

Then, there is no comparison method based on a finite combination of unary quality indicators such that :

$$C_{\mathbf{I},E} \Leftrightarrow A \triangleleft B$$

Problem with unary quality indicators (Zitzler et al., 2003)

Considering :

- ▶ Combinatorial multiobjective problem with $p \geq 2$
- ▶ A and B two approximation sets
- ▶ A finite combination $\mathbf{I}$ of unary indicators
- ▶ A comparison method C based on $\mathbf{I}$ and an interpretation function E , $C_{\mathbf{I},E}(A, B) = E(\mathbf{I}(A), \mathbf{I}(B))$.

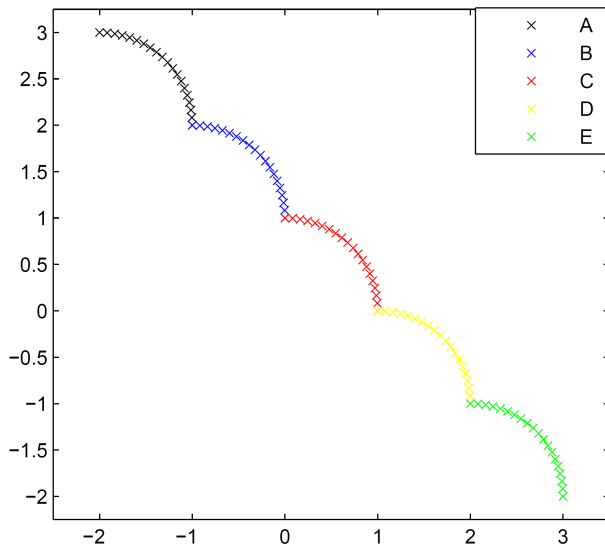
Then, there is no comparison method based on a finite combination of unary quality indicators such that :

$$C_{\mathbf{I},E} \Leftrightarrow A \triangleleft B$$

Example (H indicator)

$A \triangleleft B \Rightarrow H(A) < H(B)$ but we cannot say :
 $H(A) < H(B) \Rightarrow A \triangleleft B$.

Example (Lizarraga et al., 2008)



Multiobjective Neighborhood Search Algorithms : simple adaptation

Multiobjective Simulated Annealing

- ▶ Initial solution

Multiobjective Tabu Search

- ▶ Initial solution

Multiobjective Neighborhood Search Algorithms : simple adaptation

Multiobjective Simulated Annealing

- ▶ Initial solution
- ▶ Neighborhood structure

Multiobjective Tabu Search

- ▶ Initial solution
- ▶ Neighborhood structure

Multiobjective Neighborhood Search Algorithms : simple adaptation

Multiobjective Simulated Annealing

- ▶ Initial solution
- ▶ Neighborhood structure
- ▶ Scalarizing function $s(z(x), \lambda)$

Multiobjective Tabu Search

- ▶ Initial solution
- ▶ Neighborhood structure
- ▶ Scalarizing function $s(z(x), \lambda)$

Multiobjective Neighborhood Search Algorithms : simple adaptation

Multiobjective Simulated Annealing

- ▶ Initial solution
- ▶ Neighborhood structure
- ▶ Scalarizing function $s(z(x), \lambda)$
- ▶ Set of weights (directions) λ

Multiobjective Tabu Search

- ▶ Initial solution
- ▶ Neighborhood structure
- ▶ Scalarizing function $s(z(x), \lambda)$
- ▶ Set of weights (directions) λ

Multiobjective Neighborhood Search Algorithms : simple adaptation

Multiobjective Simulated Annealing

- ▶ Initial solution
- ▶ Neighborhood structure
- ▶ Scalarizing function $s(z(x), \lambda)$
- ▶ Set of weights (directions) λ
- ▶ Simulated Annealing based on s

Multiobjective Tabu Search

- ▶ Initial solution
- ▶ Neighborhood structure
- ▶ Scalarizing function $s(z(x), \lambda)$
- ▶ Set of weights (directions) λ
- ▶ Tabu Search based on s

Multiobjective Neighborhood Search Algorithms : simple adaptation

Multiobjective Simulated Annealing

- ▶ Initial solution
- ▶ Neighborhood structure
- ▶ Scalarizing function $s(z(x), \lambda)$
- ▶ Set of weights (directions) λ
- ▶ Simulated Annealing based on s
- ▶ Merge sets of solutions

Multiobjective Tabu Search

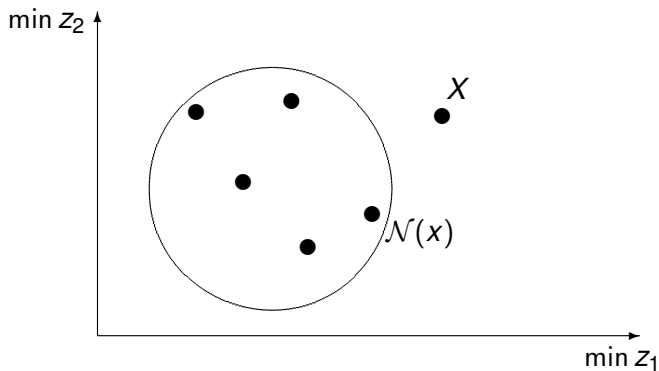
- ▶ Initial solution
- ▶ Neighborhood structure
- ▶ Scalarizing function $s(z(x), \lambda)$
- ▶ Set of weights (directions) λ
- ▶ Tabu Search based on s
- ▶ Merge sets of solutions

Multiobjective case

How to choose a solution of minimal cost ?

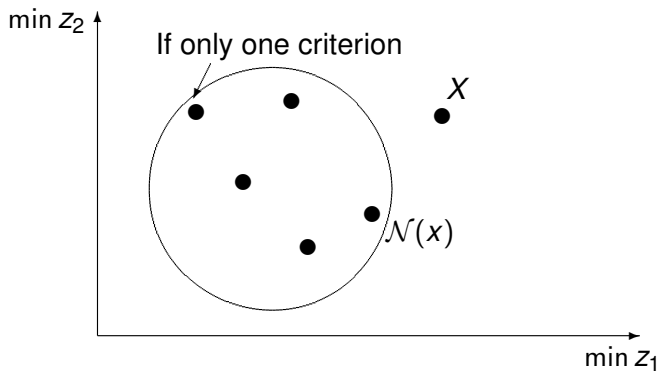
Multiobjective case

How to choose a solution of minimal cost ?



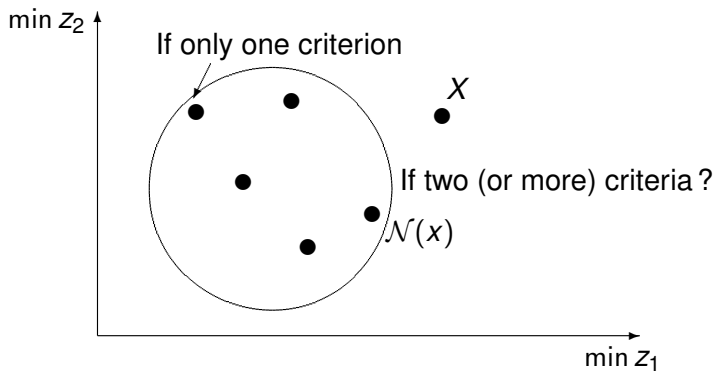
Multiobjective case

How to choose a solution of minimal cost ?



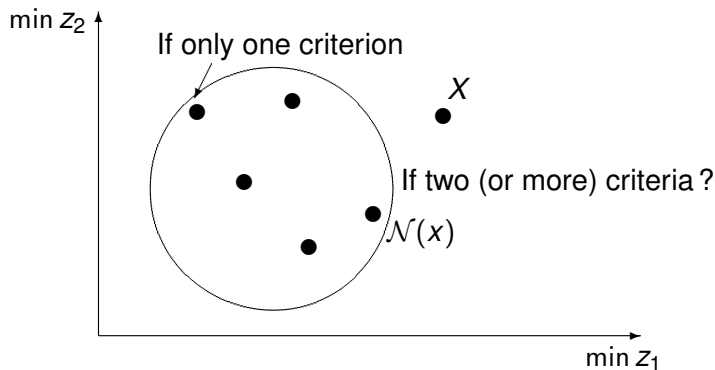
Multiobjective case

How to choose a solution of minimal cost ?



Multiobjective case

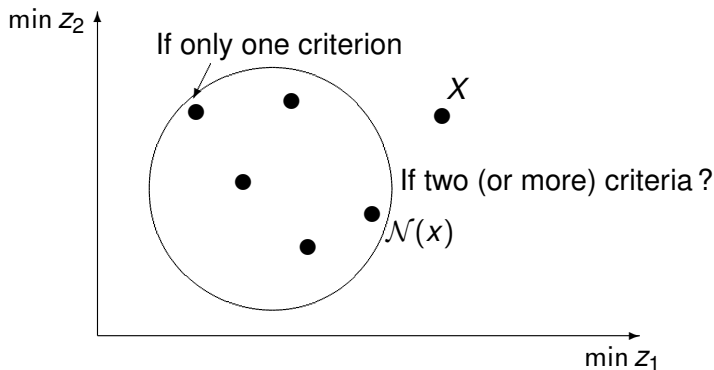
How to choose a solution of minimal cost ?



- Selection of the solution by using different rules (dominance relation, density, ...)

Multiobjective case

How to choose a solution of minimal cost ?



- ▶ Selection of the solution by using different rules (dominance relation, density, ...)
- ▶ Selection of all non-dominated solutions

Example : Pareto Ranking Tabu Search + Density (PRTS+D)

- ▶ Based on Tabu Search
- ▶ Selection of the neighbor based on dominance relation and density

Neighbor Selection of $\text{PRTS}+\text{D}$

Best neighbor selection inspired by the individuals selection of a genetic algorithm (PFGA) [Elaoud et al., 2007], which uses :

- ▶ A rank for each individual, calculated thanks to the strong dominance relation
- ▶ A density, calculated starting from a division of the objectives space in hypervolumes

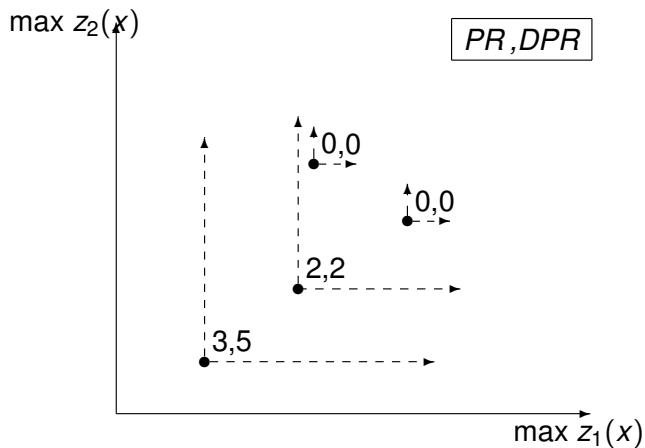


Samya Elaoud, Taïcir Loukil, Jacques Teghem

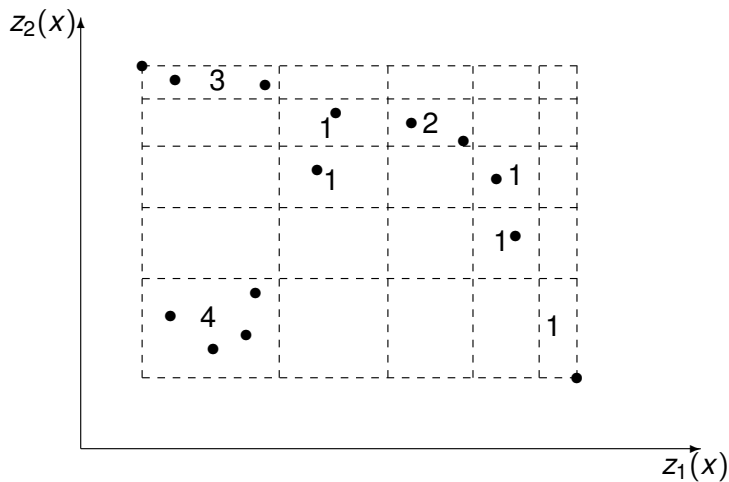
The Pareto fitness genetic algorithm : Test function study.

European Journal of Operational Research, 177(3) :
1703-1719, 2007.

The Double Pareto Ranking *DPR*



The individuals density measure



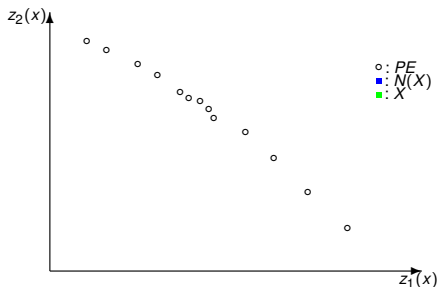
Use of DPR and D in a Tabu method

⇒ Neighbors selection :

- ▶ Population considered for the calculation of DPR and the density D : L neighbors generated at each iteration + potentially efficient solutions
- ▶ The selected neighbor is the one that presents a maximum evaluation for the following function :

$$f(i) = \frac{1}{e^{DPR(i)} \times D(i)}$$

Functioning of PRTS+D



Iteration i

Generate L neighbors Y_l in $N(X)$

Create h^p hypervolumes

For each neighbor Y_l :

 Calculate the rank DPR

 Calculate the density D

 Select the best neighbor $Y^* (\frac{1}{e^{DPR(Y_l)} \times D(Y_l)})$

 Give to the movement $Y^* \rightarrow X$ the Tabu statute

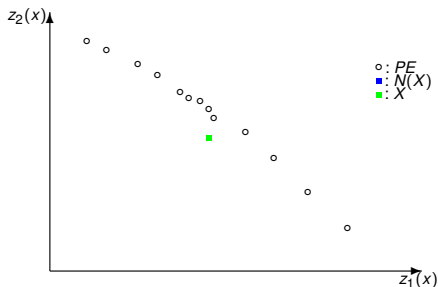
$X \leftarrow Y^*$

For each neighbor Y_l , if $DPR(Y_l) = 0$ then

$PE \leftarrow PE + \{Y_l\}$

Actualize PE

Functioning of PRTS+D



Iteration i

Generate L neighbors Y_l in $N(X)$

Create h^p hypervolumes

For each neighbor Y_l :

 Calculate the rank DPR

 Calculate the density D

 Select the best neighbor $Y^* (\frac{1}{e^{DPR(Y_l)} \times D(Y_l)})$

 Give to the movement $Y^* \rightarrow X$ the Tabu statute

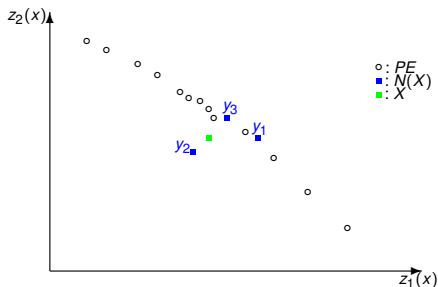
$X \leftarrow Y^*$

For each neighbor Y_l , if $DPR(Y_l) = 0$ then

$PE \leftarrow PE + \{Y_l\}$

Actualize PE

Functioning of PRTS+D



Iteration i

Generate L neighbors Y_l in $N(X)$

Create h^p hypervolumes

For each neighbor Y_l :

 Calculate the rank DPR

 Calculate the density D

 Select the best neighbor $Y^* (\frac{1}{e^{DPR(Y_l)} \times D(Y_l)})$

 Give to the movement $Y^* \rightarrow X$ the Tabu statute

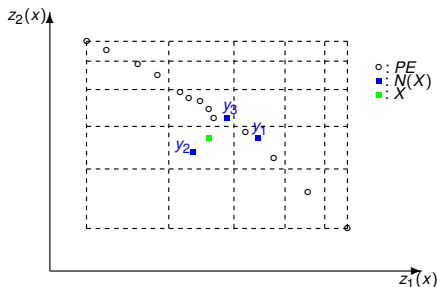
$X \leftarrow Y^*$

For each neighbor Y_l , if $DPR(Y_l) = 0$ then

$PE \leftarrow PE + \{Y_l\}$

Actualize PE

Functioning of PRTS+D



Iteration i

Generate L neighbors Y_l in $N(X)$

Create h^p hypervolumes

For each neighbor Y_l :

 Calculate the rank DPR

 Calculate the density D

 Select the best neighbor $Y^* (\frac{1}{e^{DPR(Y_l)} \times D(Y_l)})$

 Give to the movement $Y^* \rightarrow X$ the Tabu statute

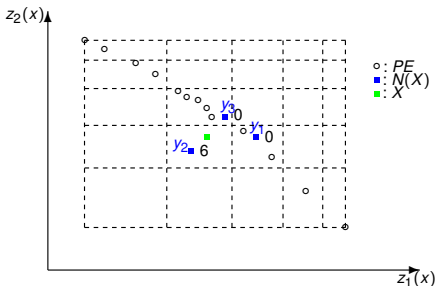
$X \leftarrow Y^*$

For each neighbor Y_l , if $DPR(Y_l) = 0$ then

$PE \leftarrow PE + \{Y_l\}$

Actualize PE

Functioning of PRTS+D



Iteration i

Generate L neighbors Y_l in $N(X)$

Create h^p hypervolumes

For each neighbor Y_l :

Calculate the rank DPR

Calculate the density D

Select the best neighbor $Y^* (\frac{1}{e^{DPR(Y_l)} \times D(Y_l)})$

Give to the movement $Y^* \rightarrow X$ the Tabu statute

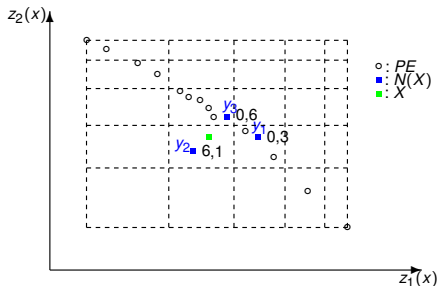
$X \leftarrow Y^*$

For each neighbor Y_l , if $DPR(Y_l) = 0$ then

$PE \leftarrow PE + \{Y_l\}$

Actualize PE

Functioning of PRTS+D



Iteration i

Generate L neighbors Y_l in $N(X)$

Create h^p hypervolumes

For each neighbor Y_l :

 Calculate the rank DPR

Calculate the density D

 Select the best neighbor Y^* ($\frac{1}{e^{DPR(Y_l)} \times D(Y_l)}$)

 Give to the movement $Y^* \rightarrow X$ the Tabu statute

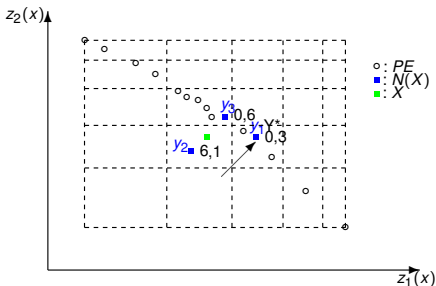
$X \leftarrow Y^*$

For each neighbor Y_l , if $DPR(Y_l) = 0$ then

$PE \leftarrow PE + \{Y_l\}$

Actualize PE

Functioning of PRTS+D



Iteration i

Generate L neighbors Y_l in $N(X)$

Create h^p hypervolumes

For each neighbor Y_l :

Calculate the rank DPR

Calculate the density D

Select the best neighbor Y^* ($\frac{1}{e^{DPR(Y_l)} \times D(Y_l)}$)

Give to the movement $Y^* \rightarrow X$ the Tabu statute

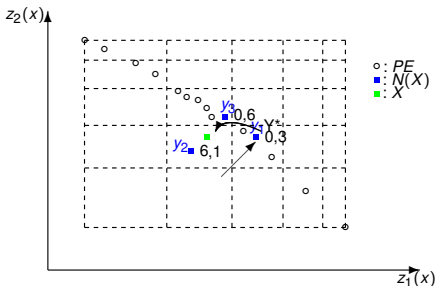
$X \leftarrow Y^*$

For each neighbor Y_l , if $DPR(Y_l) = 0$ then

$PE \leftarrow PE + \{Y_l\}$

Actualize PE

Functioning of PRTS+D



Iteration i

Generate L neighbors Y_l in $N(X)$

Create h^p hypervolumes

For each neighbor Y_l :

 Calculate the rank DPR

 Calculate the density D

Select the best neighbor $Y^* (\frac{1}{e^{DPR(Y_l)} \times D(Y_l)})$

Give to the movement $Y^* \rightarrow X$ the Tabu statute

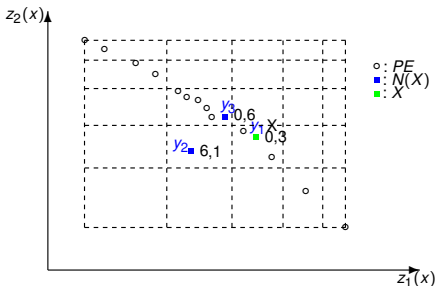
$X \leftarrow Y^*$

For each neighbor Y_l , if $DPR(Y_l) = 0$ then

$PE \leftarrow PE + \{Y_l\}$

Actualize PE

Functioning of PRTS+D



Iteration i

Generate L neighbors Y_l in $N(X)$

Create h^p hypervolumes

For each neighbor Y_l :

 Calculate the rank DPR

 Calculate the density D

Select the best neighbor Y^* ($\frac{1}{e^{DPR(Y_l)} \times D(Y_l)}$)

Give to the movement $Y^* \rightarrow X$ the Tabu statute

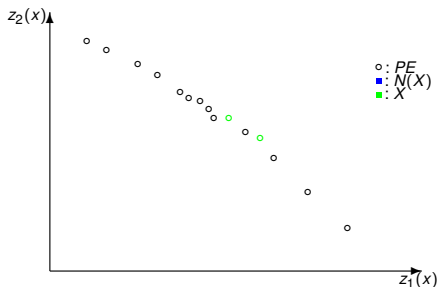
$X \leftarrow Y^*$

For each neighbor Y_l , if $DPR(Y_l) = 0$ then

$PE \leftarrow PE + \{Y_l\}$

Actualize PE

Functioning of PRTS+D



Iteration i

Generate L neighbors Y_l in $N(X)$

Create h^p hypervolumes

For each neighbor Y_l :

 Calculate the rank DPR

 Calculate the density D

 Select the best neighbor $Y^* (\frac{1}{e^{DPR(Y_l)} \times D(Y_l)})$

 Give to the movement $Y^* \rightarrow X$ the Tabu statute

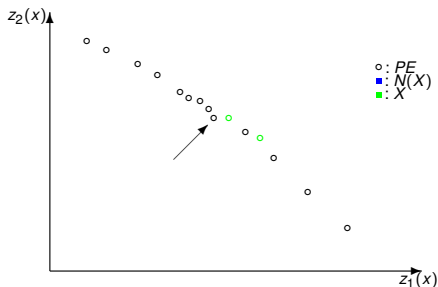
$X \leftarrow Y^*$

For each neighbor Y_l , if $DPR(Y_l) = 0$ then

$PE \leftarrow PE + \{Y_l\}$

Actualize PE

Functioning of PRTS+D



Iteration i

Generate L neighbors Y_l in $N(X)$

Create h^p hypervolumes

For each neighbor Y_l :

 Calculate the rank DPR

 Calculate the density D

 Select the best neighbor $Y^* (\frac{1}{e^{DPR(Y_l)} \times D(Y_l)})$

 Give to the movement $Y^* \rightarrow X$ the Tabu statute

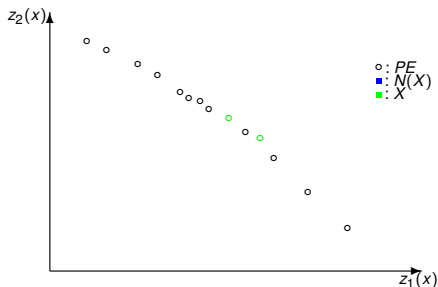
$X \leftarrow Y^*$

For each neighbor Y_l , if $DPR(Y_l) = 0$ then

$PE \leftarrow PE + \{Y_l\}$

Actualize PE

Functioning of PRTS+D



Iteration i

Generate L neighbors Y_l in $N(X)$

Create h^p hypervolumes

For each neighbor Y_l :

 Calculate the rank DPR

 Calculate the density D

 Select the best neighbor Y^* ($\frac{1}{e^{DPR(Y_l)} \times D(Y_l)}$)

 Give to the movement $Y^* \rightarrow X$ the Tabu statute

$X \leftarrow Y^*$

For each neighbor Y_l , if $DPR(Y_l) = 0$ then

$PE \leftarrow PE + \{Y_l\}$

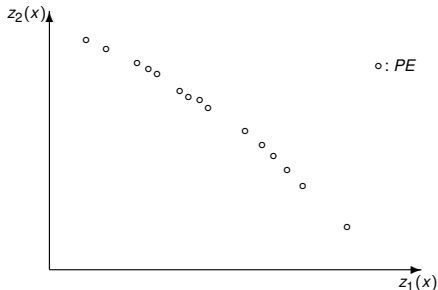
Actualize PE

Hybrid Algorithm

Initialization phase : we apply a local search method or a heuristic to produce at least $r + 1$ non-dominated solutions.

Hybrid Algorithm

Initialization phase : we apply a local search method or a heuristic to produce at least $r + 1$ non-dominated solutions.



Iteration i

Choose a solution X_1 among PE of minimal density

Choose a solution X_2 among the r closest solutions of X_1

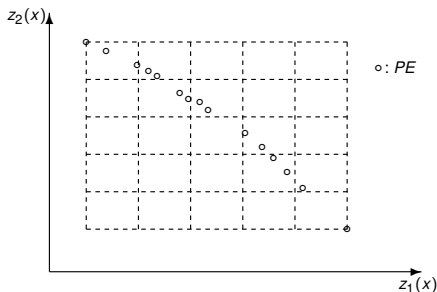
Cross the two solutions X_1 and $X_2 \rightarrow X_3$

If X_3 does not respect the constraints, application of a repair procedure

Apply a local search method from X_3 , stop if no more improvement while it_{stop}

Hybrid Algorithm

Initialization phase : we apply a local search method or a heuristic to produce at least $r + 1$ non-dominated solutions.



Iteration i

Choose a solution X_1 among PE of minimal density

Choose a solution X_2 among the r closest solutions of X_1

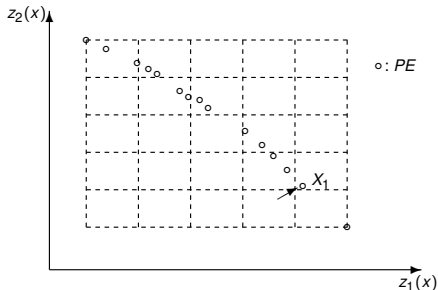
Cross the two solutions X_1 and $X_2 \rightarrow X_3$

If X_3 does not respect the constraints, application of a repair procedure

Apply a local search method from X_3 , stop if no more improvement while it_{stop}

Hybrid Algorithm

Initialization phase : we apply a local search method or a heuristic to produce at least $r + 1$ non-dominated solutions.



Iteration i

Choose a solution X_1 among PE of minimal density

Choose a solution X_2 among the r closest solutions of X_1

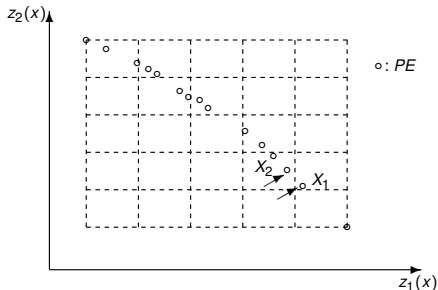
Cross the two solutions X_1 and $X_2 \rightarrow X_3$

If X_3 does not respect the constraints, application of a repair procedure

Apply a local search method from X_3 , stop if no more improvement while it_{stop}

Hybrid Algorithm

Initialization phase : we apply a local search method or a heuristic to produce at least $r + 1$ non-dominated solutions.



Iteration i

Choose a solution X_1 among PE of minimal density

Choose a solution X_2 among the r closest solutions of X_1

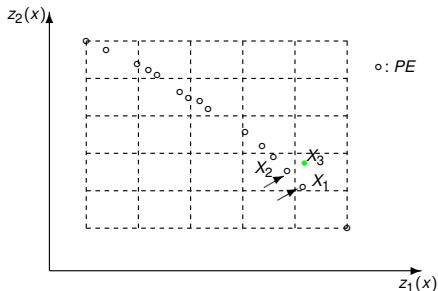
Cross the two solutions X_1 and $X_2 \rightarrow X_3$

If X_3 does not respect the constraints, application of a repair procedure

Apply a local search method from X_3 , stop if no more improvement while it_{stop}

Hybrid Algorithm

Initialization phase : we apply a local search method or a heuristic to produce at least $r + 1$ non-dominated solutions.



Iteration i

Choose a solution X_1 among PE of minimal density

Choose a solution X_2 among the r closest solutions of X_1

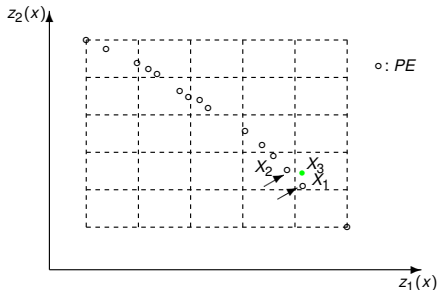
Cross the two solutions X_1 and $X_2 \rightarrow X_3$

If X_3 does not respect the constraints, application of a repair procedure

Apply a local search method from X_3 , stop if no more improvement while it_{stop}

Hybrid Algorithm

Initialization phase : we apply a local search method or a heuristic to produce at least $r + 1$ non-dominated solutions.



Iteration i

Choose a solution X_1 among PE of minimal density

Choose a solution X_2 among the r closest solutions of X_1

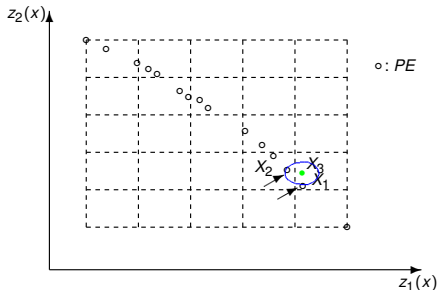
Cross the two solutions X_1 and $X_2 \rightarrow X_3$

If X_3 does not respect the constraints, application of a repair procedure

Apply a local search method from X_3 , stop if no more improvement while it_{stop}

Hybrid Algorithm

Initialization phase : we apply a local search method or a heuristic to produce at least $r + 1$ non-dominated solutions.



Iteration i

Choose a solution X_1 among PE of minimal density

Choose a solution X_2 among the r closest solutions of X_1

Cross the two solutions X_1 and $X_2 \rightarrow X_3$

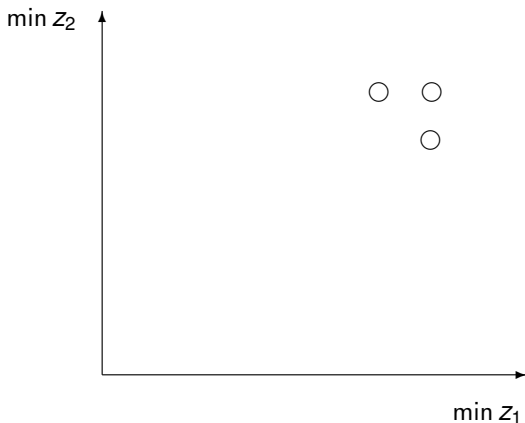
If X_3 does not respect the constraints, application of a repair procedure

Apply a local search method from X_3 , stop if no more improvement while it_{stop}

Pareto Local Search (PLS)

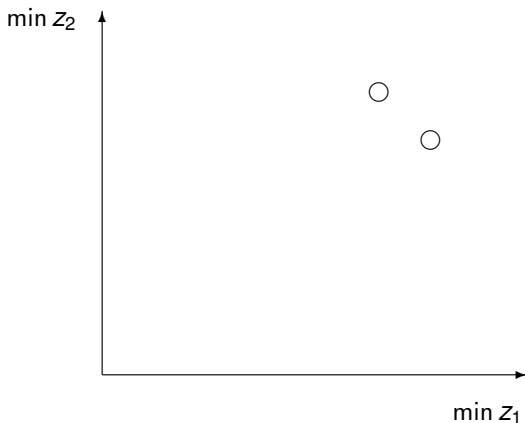
Pareto Local Search (PLS)

Exploration of the neighborhood of each potentially efficient solution generated until all the neighborhoods of all the potentially efficient solutions are explored.



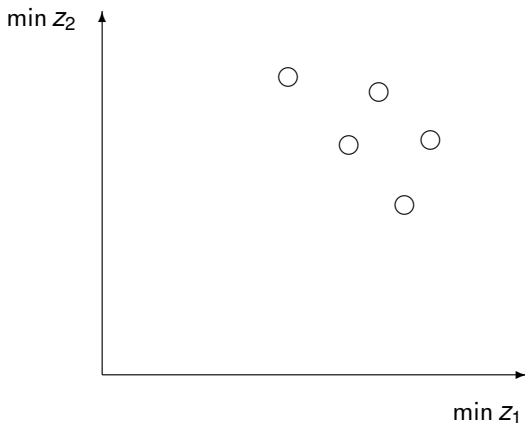
Pareto Local Search (PLS)

Exploration of the neighborhood of each potentially efficient solution generated until all the neighborhoods of all the potentially efficient solutions are explored.



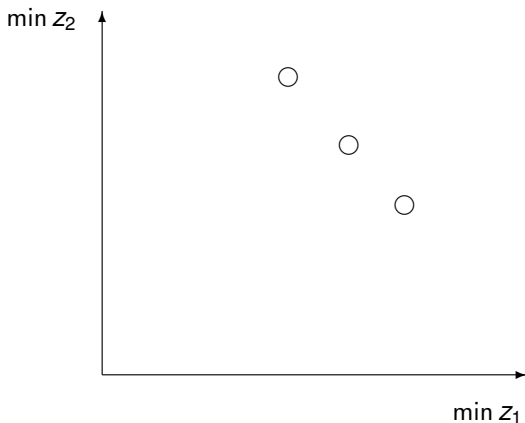
Pareto Local Search (PLS)

Exploration of the neighborhood of each potentially efficient solution generated until all the neighborhoods of all the potentially efficient solutions are explored.



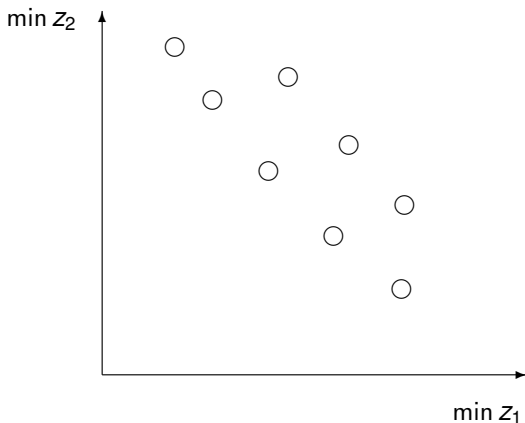
Pareto Local Search (PLS)

Exploration of the neighborhood of each potentially efficient solution generated until all the neighborhoods of all the potentially efficient solutions are explored.



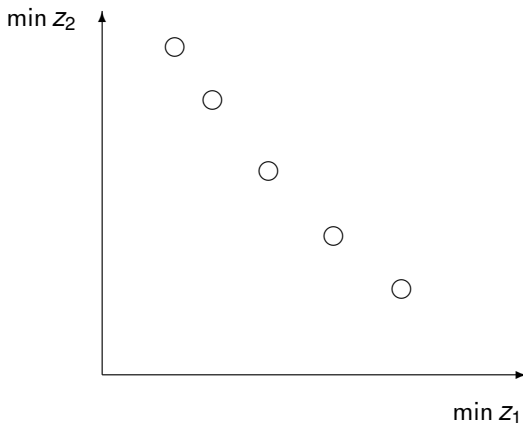
Pareto Local Search (PLS)

Exploration of the neighborhood of each potentially efficient solution generated until all the neighborhoods of all the potentially efficient solutions are explored.



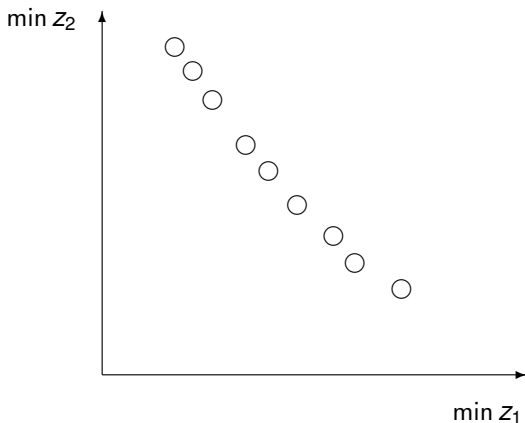
Pareto Local Search (PLS)

Exploration of the neighborhood of each potentially efficient solution generated until all the neighborhoods of all the potentially efficient solutions are explored.



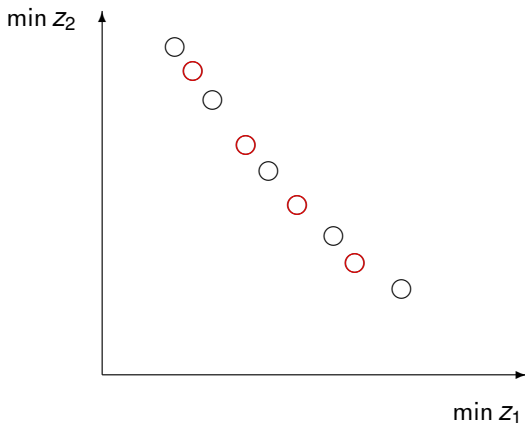
Pareto Local Search (PLS)

Exploration of the neighborhood of each potentially efficient solution generated until all the neighborhoods of all the potentially efficient solutions are explored.



Pareto Local Search (PLS)

Exploration of the neighborhood of each potentially efficient solution generated until all the neighborhoods of all the potentially efficient solutions are explored.



Pareto local search

Parameters $\downarrow$: an initial population P_0 , a neighborhood function $\mathcal{N}(x)$.

Parameters $\uparrow$: an approximation $\tilde{X}_E$ of the efficient set X_E .

--| Initialization of $\tilde{X}_E$ and a population P with the initial population P_0

$\tilde{X}_E \leftarrow P_0$

$P \leftarrow P_0$

--| Initialization of an auxiliary population P_a

$P_a \leftarrow \emptyset$

while $P \neq \emptyset$ **do**

--| Generation of all neighbors p' of each solution $p \in P$

for all $p \in P$ **do**

for all $p' \in \mathcal{N}(p)$ **do**

if $f(p) \not\leq f(p')$ **then**

if $\text{Update}(\tilde{X}_E \downarrow, p' \downarrow)$ **then**

$\text{Update}(P_a \downarrow, p' \downarrow)$

end if

end if

end for

end for

--| P is composed of the new potentially efficient solutions

$P \leftarrow P_a$

--| Reinitialization of P_a

$P_a \leftarrow \emptyset$

end while

Multiobjective genetic algorithm

Parameters $\downarrow$: an initial population P_0 , a procedure $SP1(P)$ of selection of the first parent, a procedure $SP2(P)$ of selection of the second parent, a crossover operator $CO(x_1, x_2)$, a mutation operator $M(x)$, the number nP' of reproduction, a selection procedure $S(P)$.

Parameters $\uparrow$: an approximation $\tilde{X}_E$ of the efficient set X_E .

--| Initialization of the population P

$P \leftarrow P_0$

--| Initialization of $\tilde{X}_E$

for all $x^i \in P$ **do**

 AddSolution($\tilde{X}_E \uparrow, x^i \downarrow$)

end for

repeat

 --| Generation of a new population P' from P and $\tilde{X}_E$

$P' \leftarrow \emptyset$

for $i = 1$ to nP' **do**

$x^1 \leftarrow SP1(\{P \cup \tilde{X}_E\})$

$x^2 \leftarrow SP2(\{P \cup \tilde{X}_E\})$

$x^3 \leftarrow CO(x^1, x^2)$

 AddSolution($\tilde{X}_E \uparrow, x^3 \downarrow$)

$x^4 \leftarrow M(x^3)$

 AddSolution($\tilde{X}_E \uparrow, x^4 \downarrow$)

$P' \leftarrow P' + \{x^4\}$

end for

 --| Selection between $\{P \cup P' \cup \tilde{X}_E\}$ to update the population P

$P \leftarrow S(\{P \cup P' \cup \tilde{X}_E\})$

until stopping criterion met

NSGA-II (most popular MOGA)

- ▶ Developed by Deb *et al.* in 2002



K. Deb, A. Pratap, S. Agarwal and T. Meyarivan

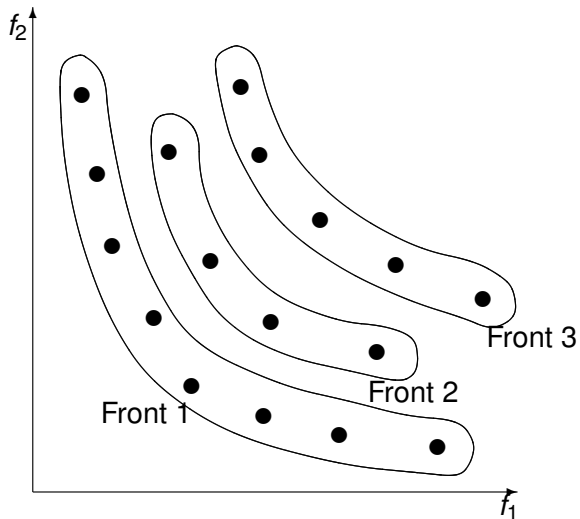
A Fast and Elitist Multiobjective Genetic Algorithm :
NSGA-II.

IEEE Transactions on Evolutionary Computation,
6(2) :182-197, 2002.

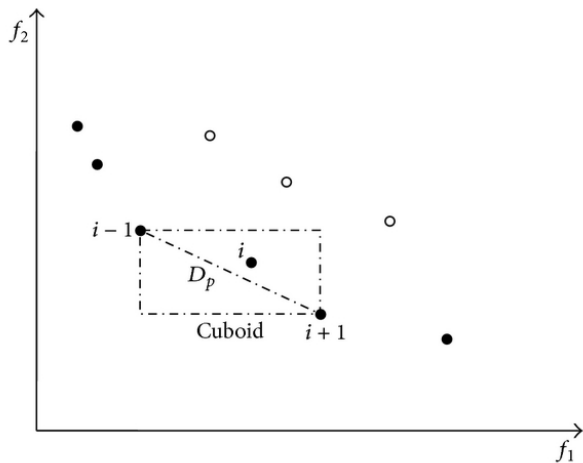
Cited 37590 times ! (October 2020)

- ▶ Fitness based on the Goldberg rank
- ▶ No elitism in the selection
- ▶ The elitist population is used in the selection
- ▶ A diversity measure is introduced with the notion of *crowding* : the value of the perimeter of the hypercube having as vertices the points the closest to the individual for each objective.

Goldberg rank



Crowding distance



Two-Phase Pareto local search

Aim

To speed-up the Pareto local search (very time-consuming if starts from one single solution)

1. Phase 1 : Find a good approximation of the supported efficient solutions

Two-Phase Pareto local search

Aim

To speed-up the Pareto local search (very time-consuming if starts from one single solution)

1. Phase 1 : Find a good approximation of the supported efficient solutions $\rightarrow$ a solver for solving the corresponding single-objective problems

Two-Phase Pareto local search

Aim

To speed-up the Pareto local search (very time-consuming if starts from one single solution)

1. Phase 1 : Find a good approximation of the supported efficient solutions $\rightarrow$ a solver for solving the corresponding single-objective problems
2. Phase 2 : Find a good approximation of the non-supported efficient solutions

Two-Phase Pareto local search

Aim

To speed-up the Pareto local search (very time-consuming if starts from one single solution)

1. Phase 1 : Find a good approximation of the supported efficient solutions → a solver for solving the corresponding single-objective problems
2. Phase 2 : Find a good approximation of the non-supported efficient solutions → a neighborhood (allowing to connect the potentially supported efficient solutions)

Two-Phase Pareto local search

Aim

To speed-up the Pareto local search (very time-consuming if starts from one single solution)

1. Phase 1 : Find a good approximation of the supported efficient solutions → a solver for solving the corresponding single-objective problems
 2. Phase 2 : Find a good approximation of the non-supported efficient solutions → a neighborhood (allowing to connect the potentially supported efficient solutions)
- No numerical parameters have to be tuned

Two-Phase Pareto local search

Aim

To speed-up the Pareto local search (very time-consuming if starts from one single solution)

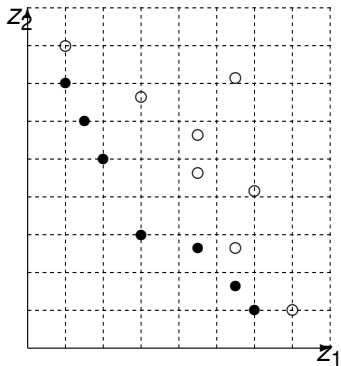
1. Phase 1 : Find a good approximation of the supported efficient solutions → a solver for solving the corresponding single-objective problems
 2. Phase 2 : Find a good approximation of the non-supported efficient solutions → a neighborhood (allowing to connect the potentially supported efficient solutions)
- No numerical parameters have to be tuned
- Natural stopping criterion

Phase 1 : Supported Efficient Solutions

- ▶ Generation of an approximation of the extremal supported efficient solutions

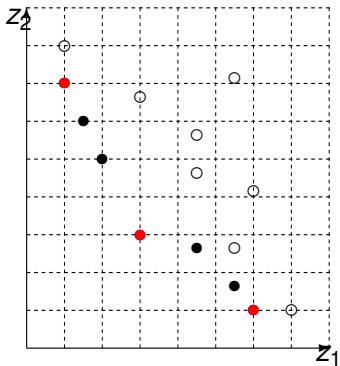
Phase 1 : Supported Efficient Solutions

- Generation of an approximation of the extremal supported efficient solutions



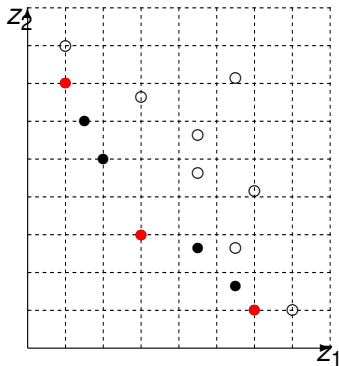
Phase 1 : Supported Efficient Solutions

- Generation of an approximation of the extremal supported efficient solutions



Phase 1 : Supported Efficient Solutions

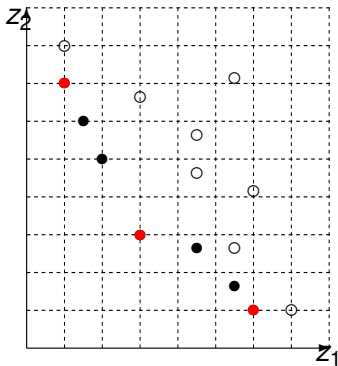
- Generation of an approximation of the extremal supported efficient solutions



- Dichotomic scheme : method that allows to generate the weight sets

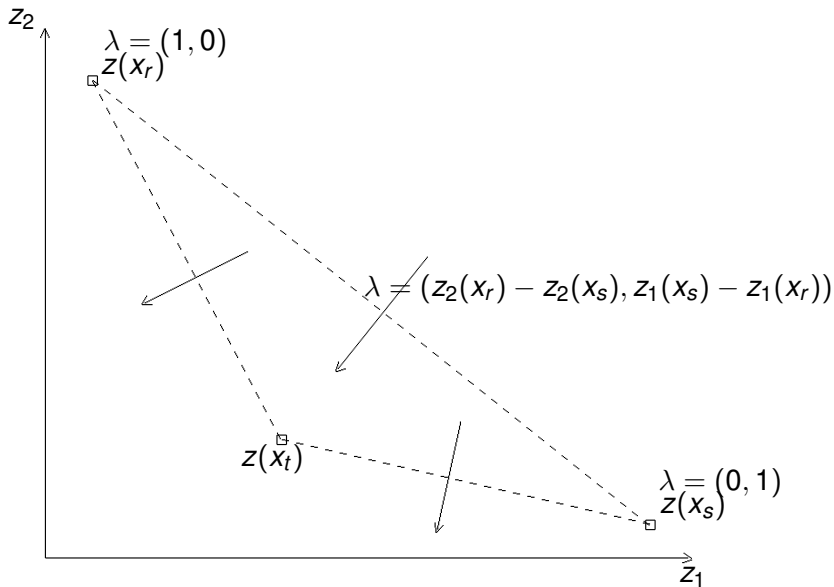
Phase 1 : Supported Efficient Solutions

- Generation of an approximation of the extremal supported efficient solutions



- Dichotomic scheme : method that allows to generate the weight sets
- Single-objective problems obtained are solved with a heuristic

Aneja and Nair Procedure

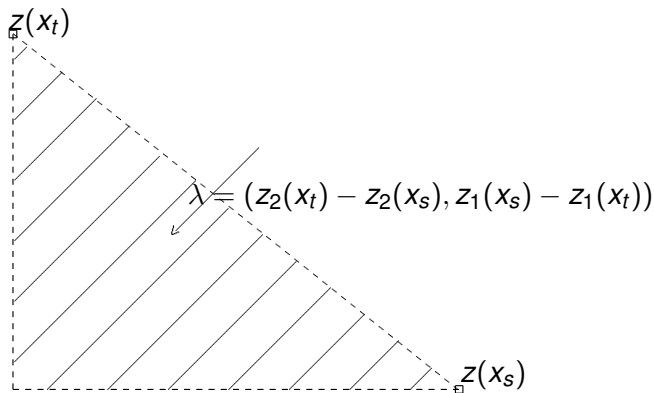


Stopping criterion of the Aneja and Nair procedure

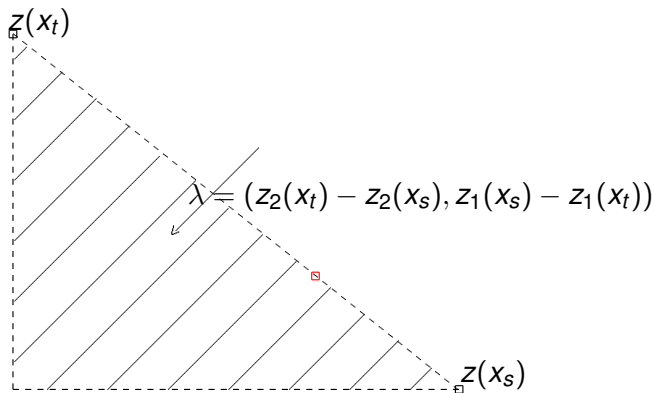
The diagram illustrates the stopping criterion of the Aneja and Nair procedure. It features a dashed line segment connecting two points, $z(x_t)$ at the top-left and $z(x_s)$ at the bottom-right. A solid line segment, representing the vector λ , is drawn perpendicular to the dashed line segment. The vector λ is labeled as $\lambda = (z_2(x_t) - z_2(x_s), z_1(x_s) - z_1(x_t))$. An arrow points from the label λ to the solid line segment.

$$\lambda = (z_2(x_t) - z_2(x_s), z_1(x_s) - z_1(x_t))$$

Stopping criterion of the Aneja and Nair procedure



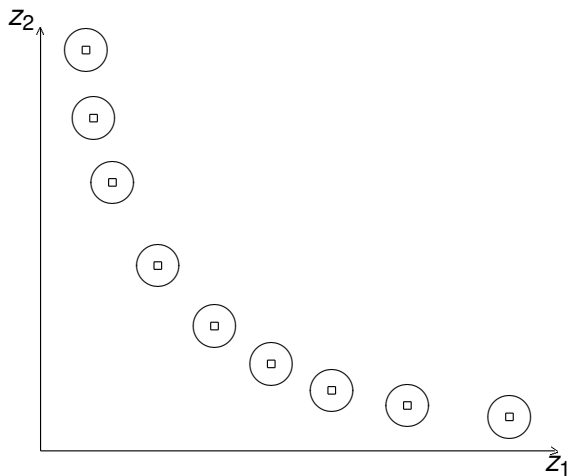
Stopping criterion of the Aneja and Nair procedure



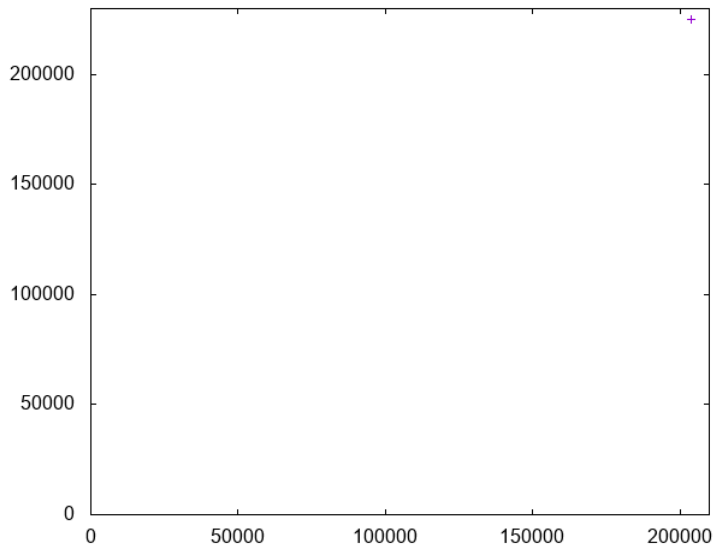
We stop when a point is outside the triangle and does not dominate any other points.

Phase 2 : Finding non-supported efficient solutions

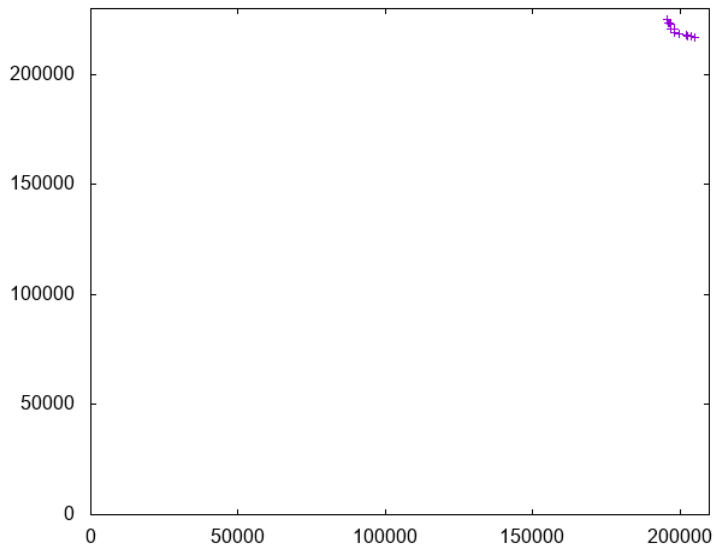
► Pareto Local Search (PLS) :



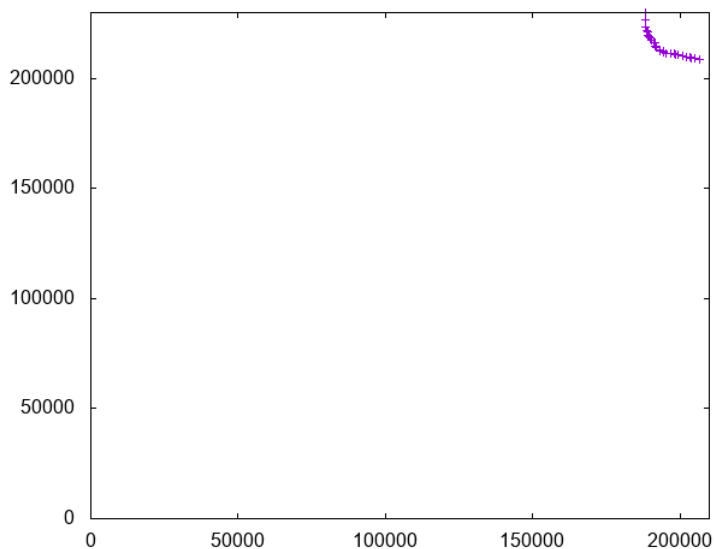
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



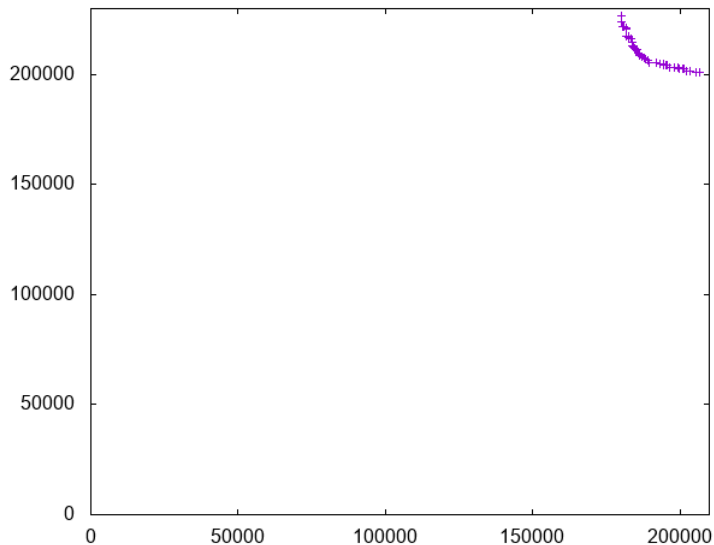
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



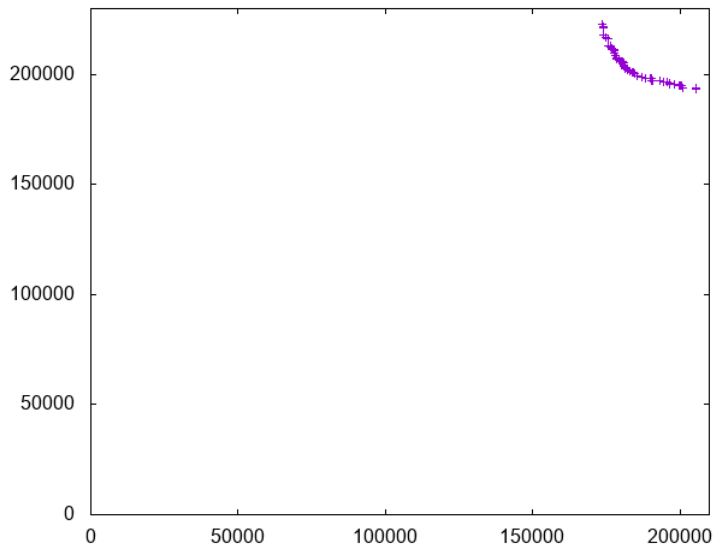
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



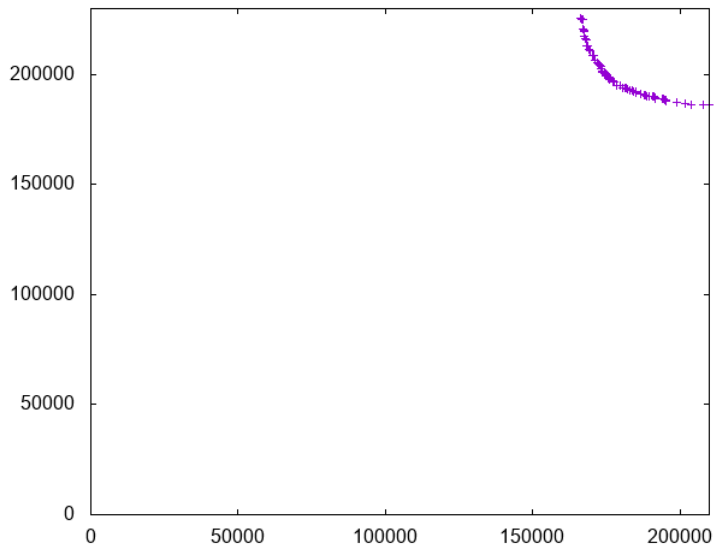
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



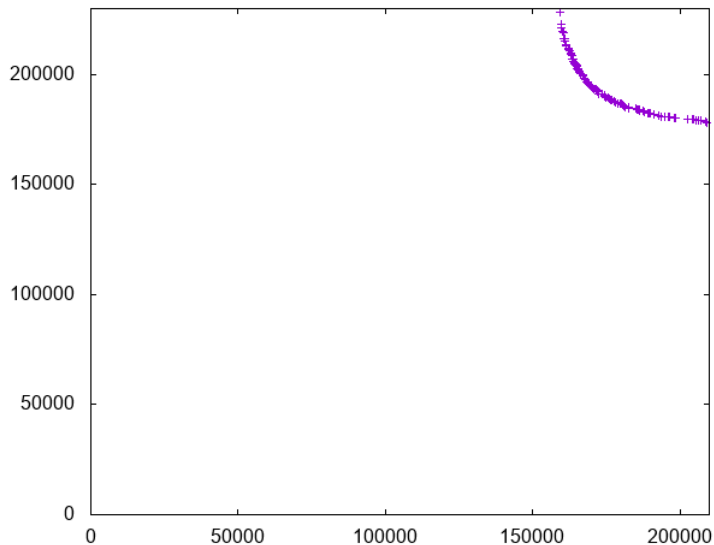
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



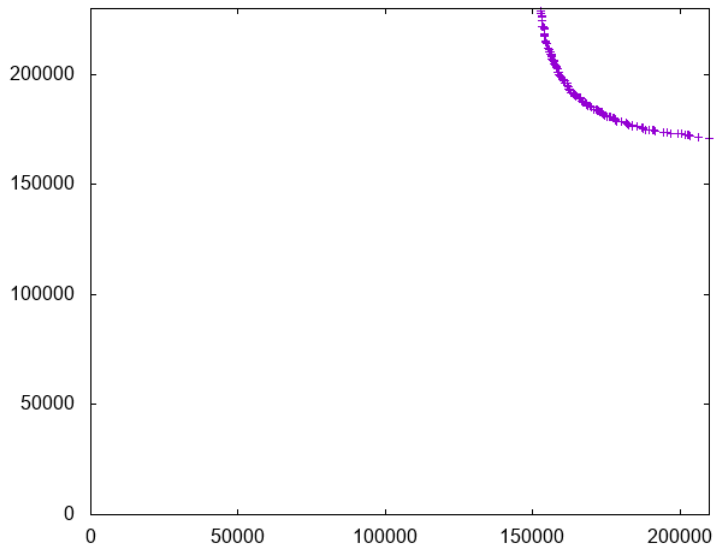
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



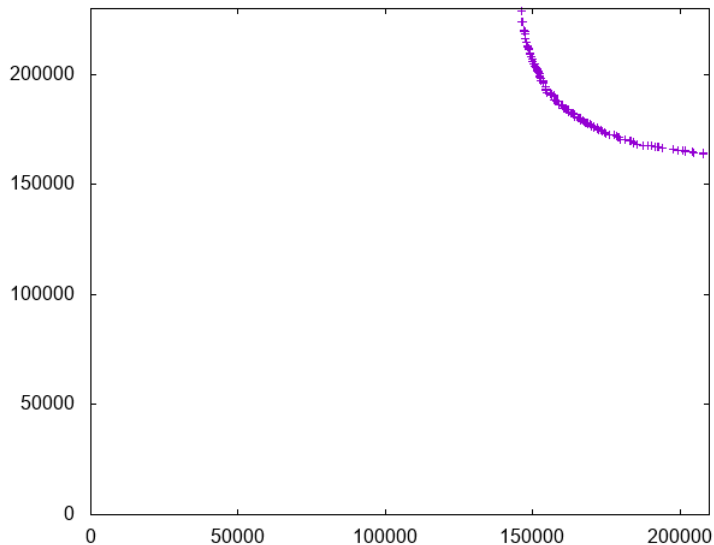
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



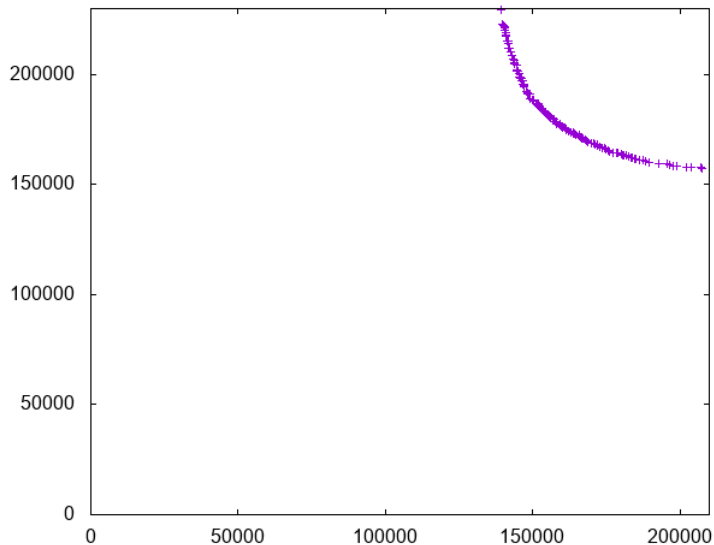
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



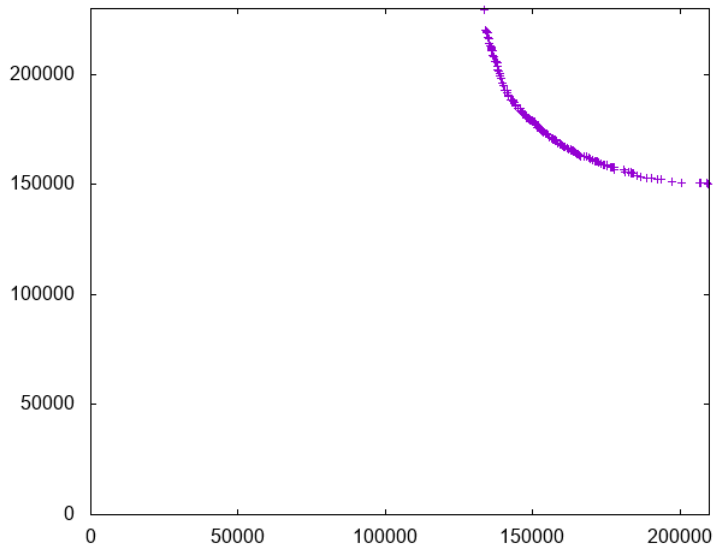
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



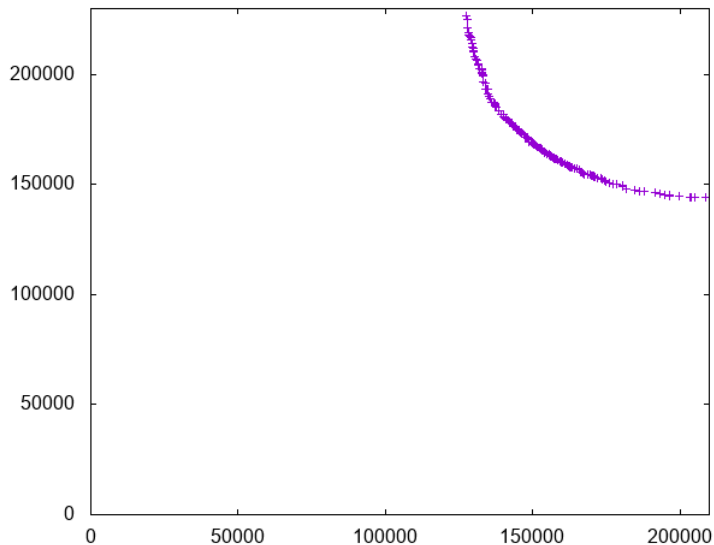
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



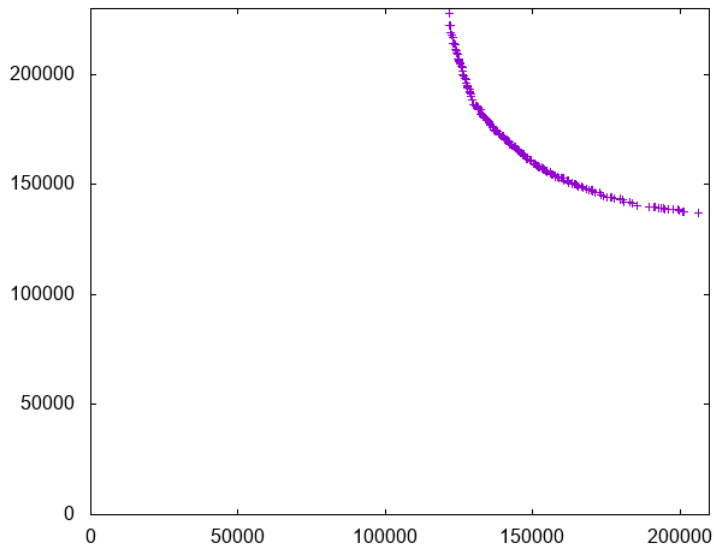
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



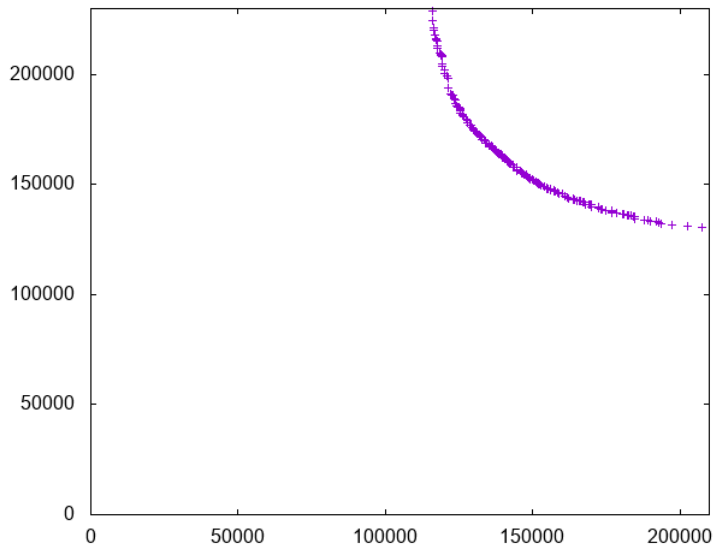
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



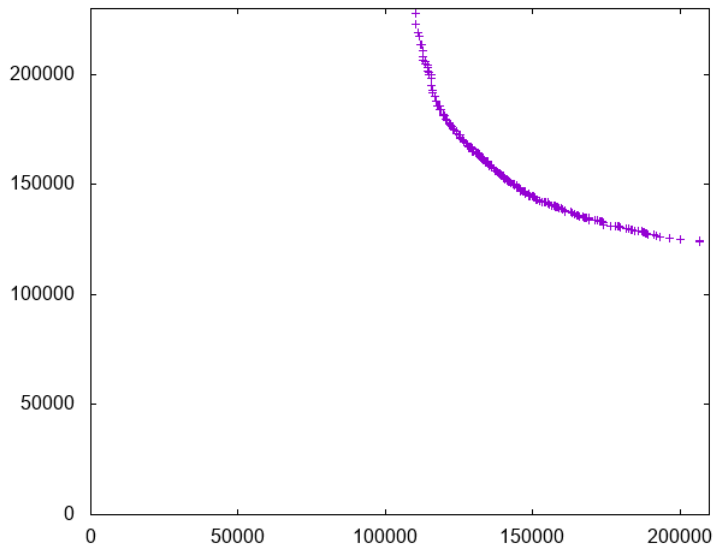
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



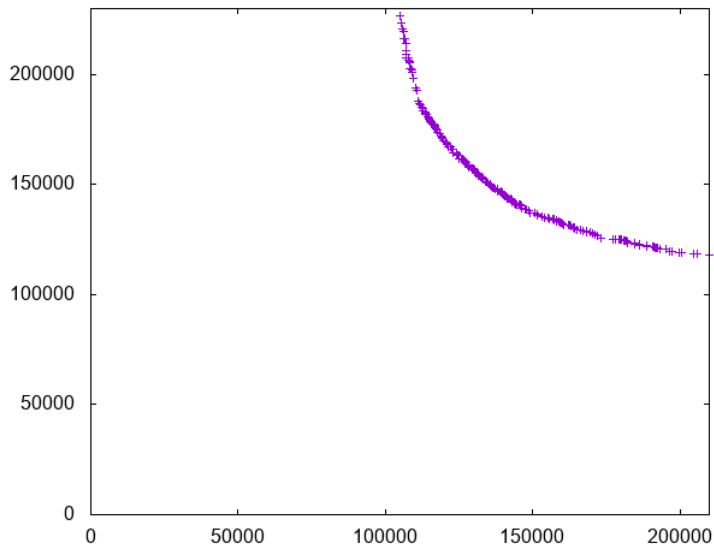
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



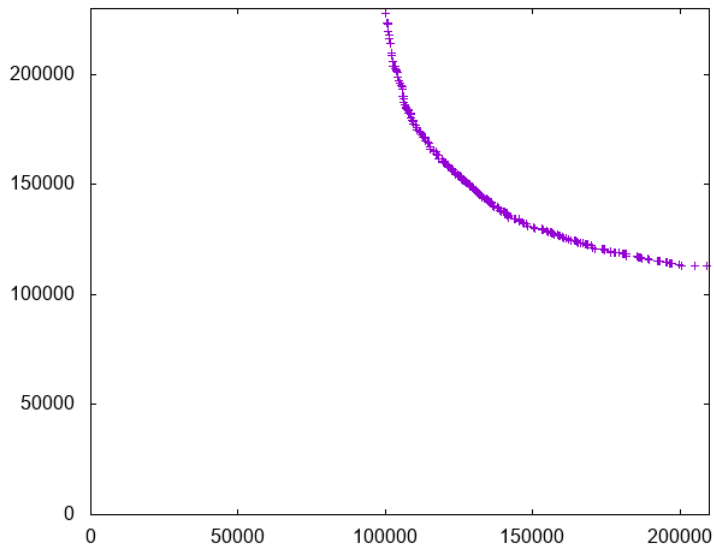
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



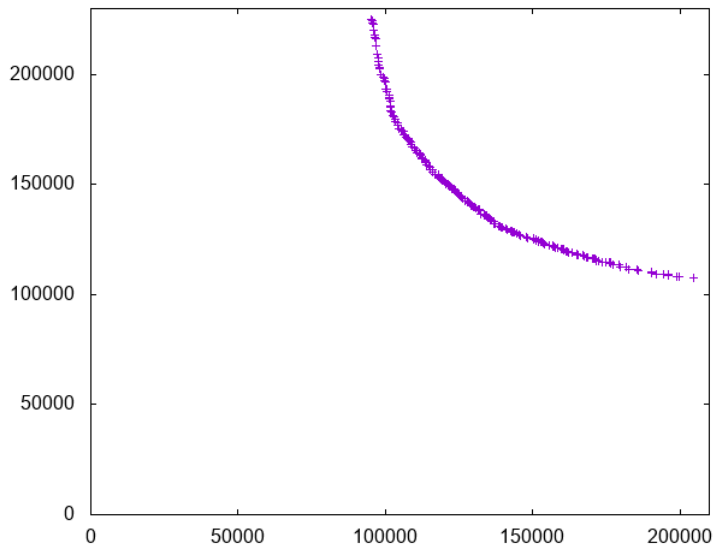
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



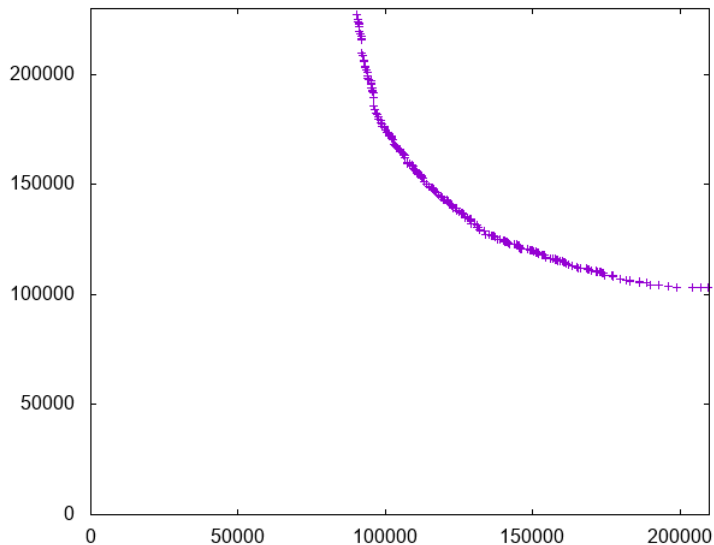
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



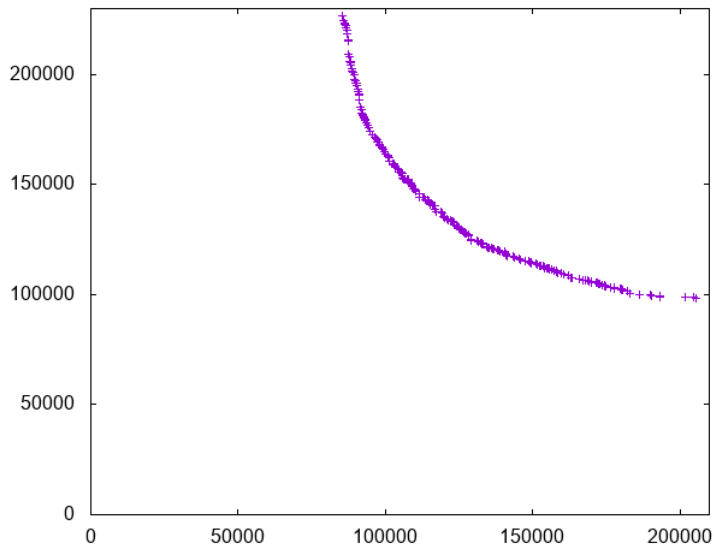
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



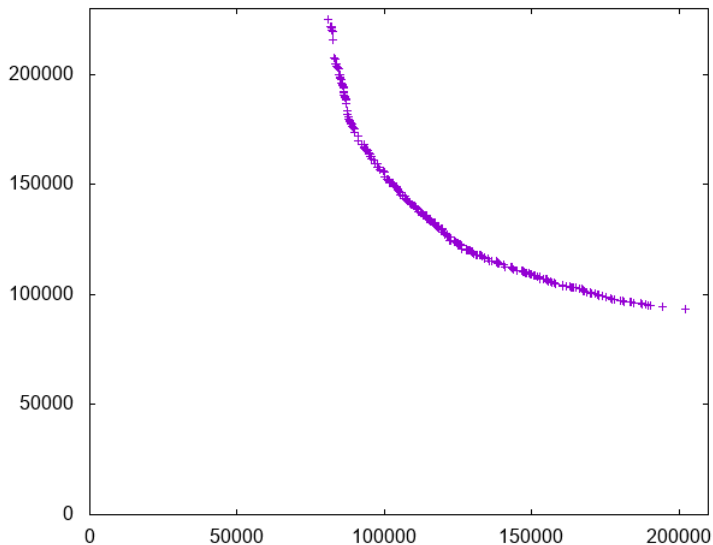
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



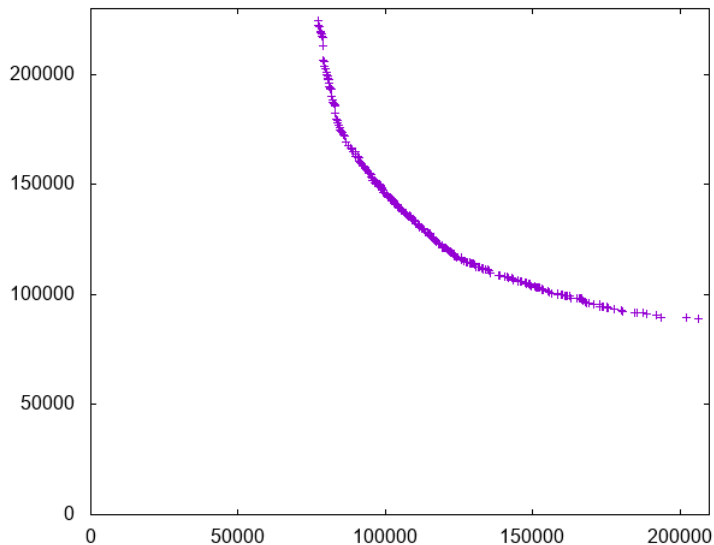
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



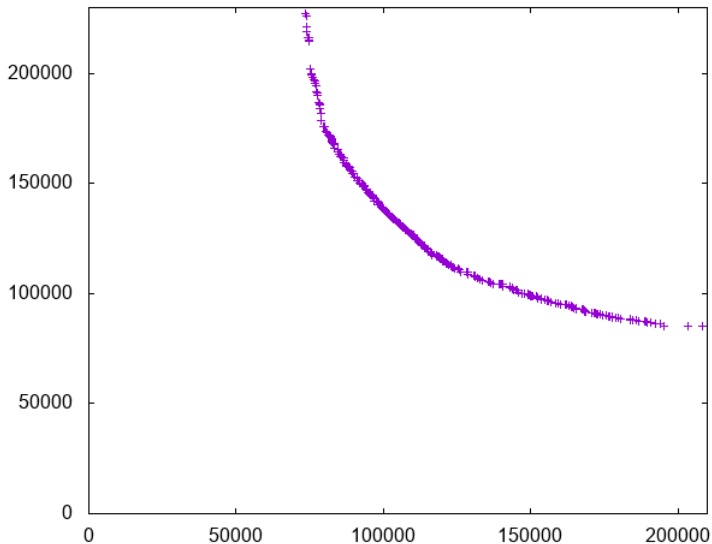
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



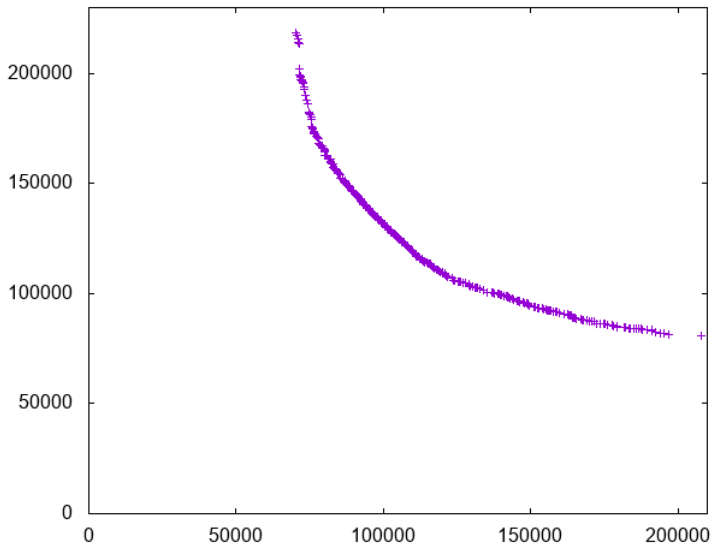
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



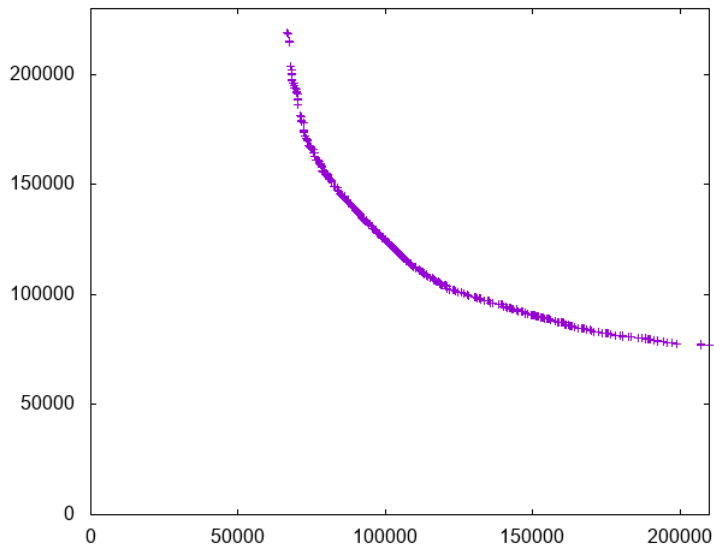
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



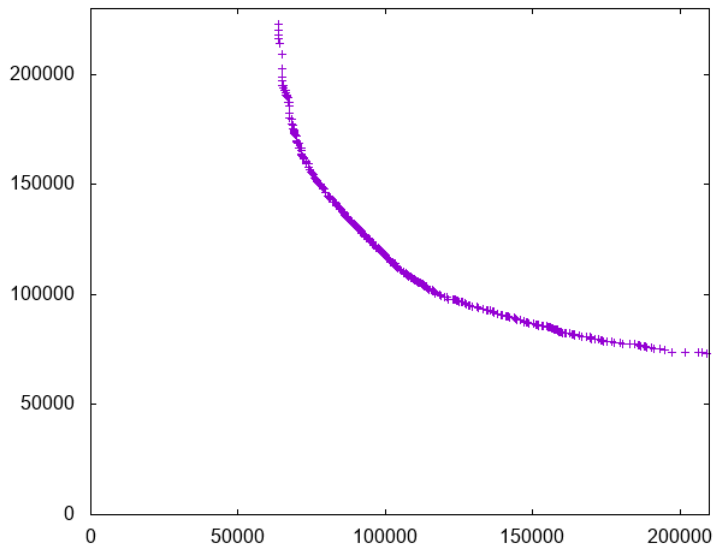
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



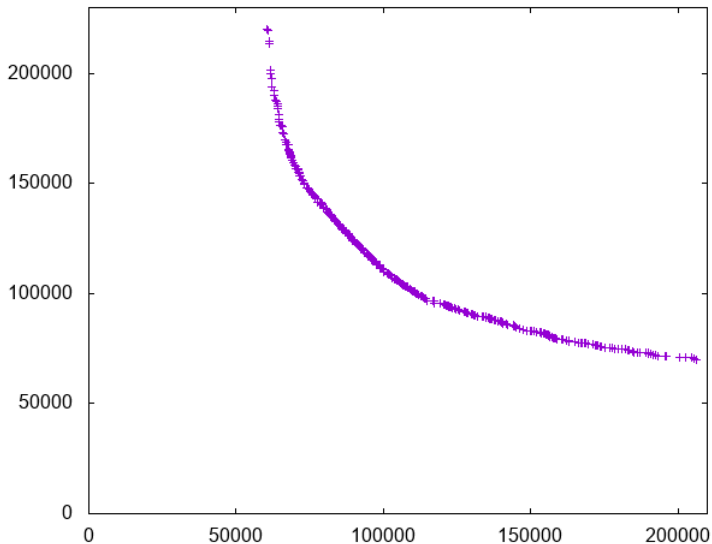
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



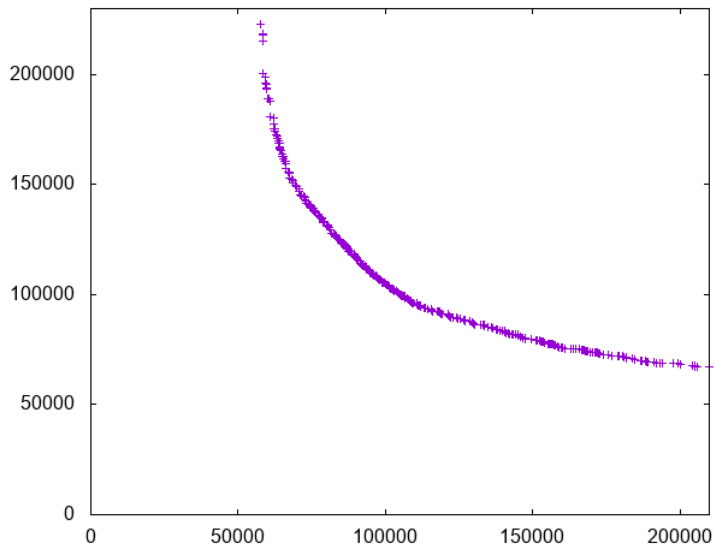
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



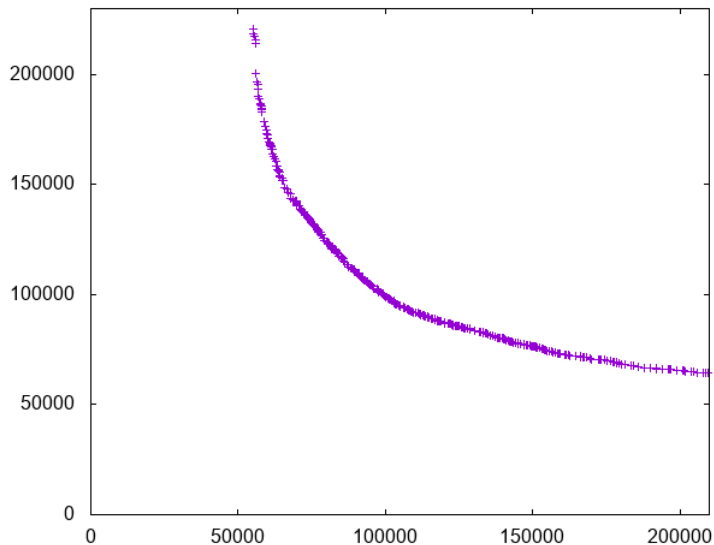
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



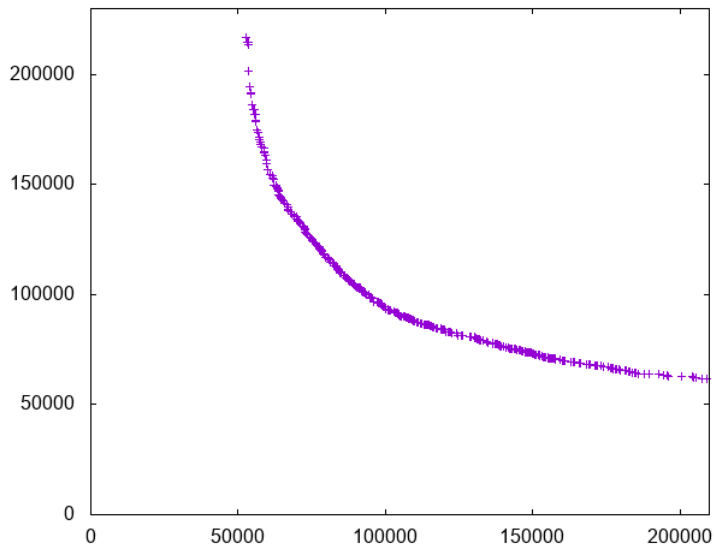
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



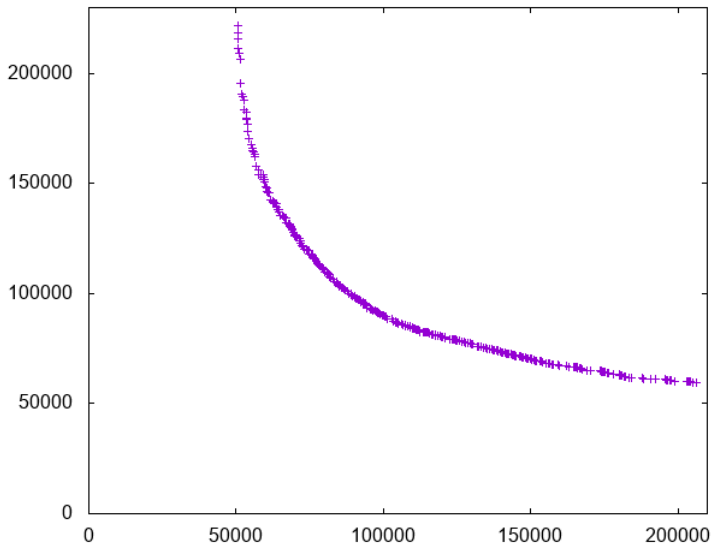
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



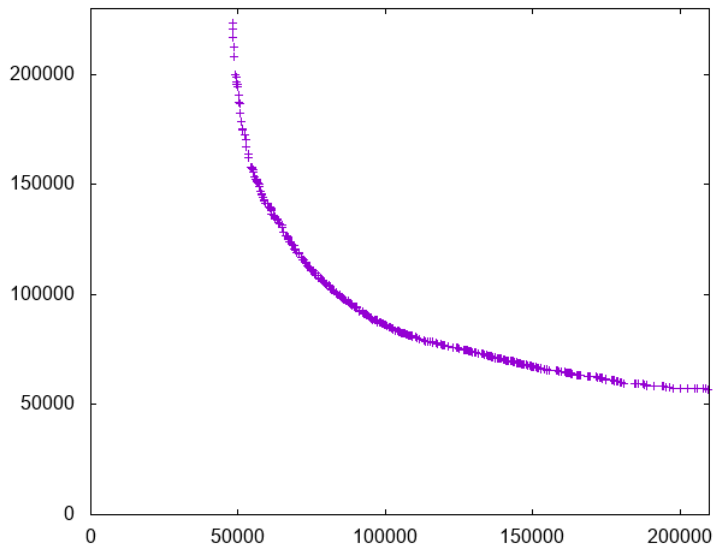
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



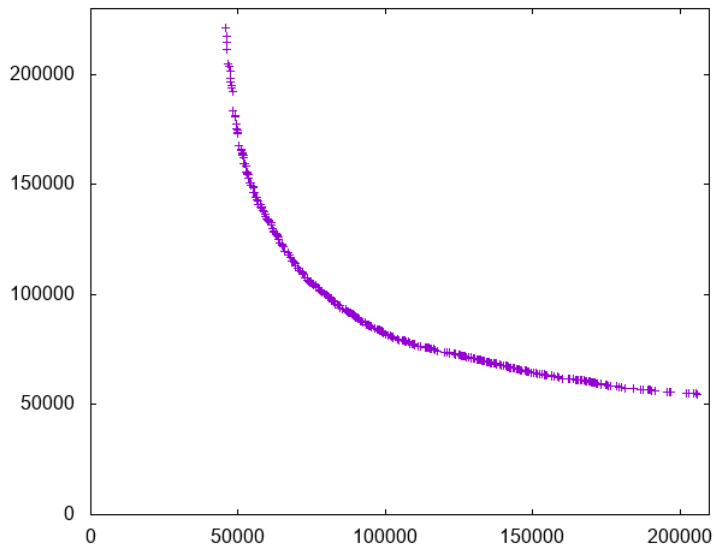
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



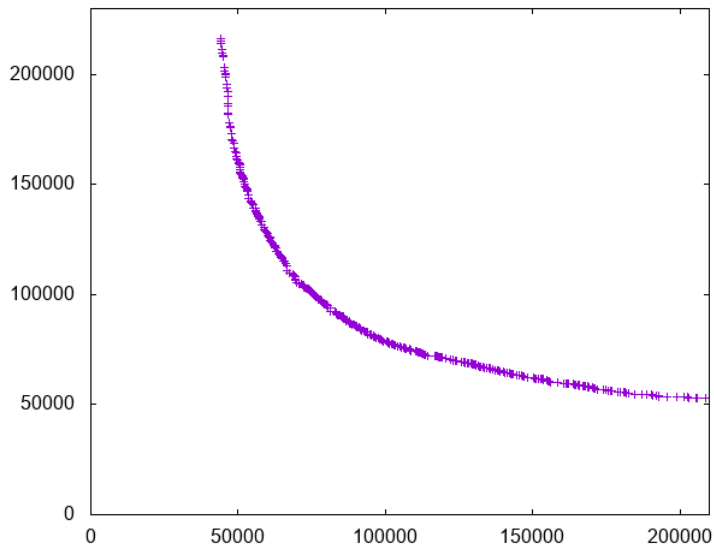
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



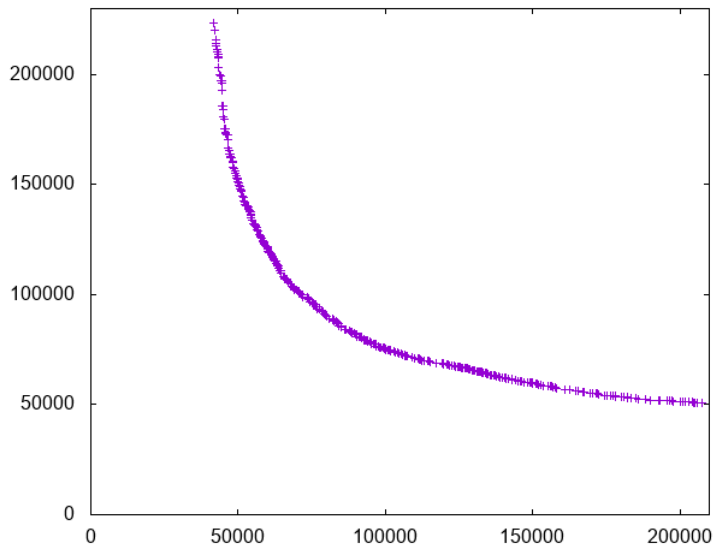
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



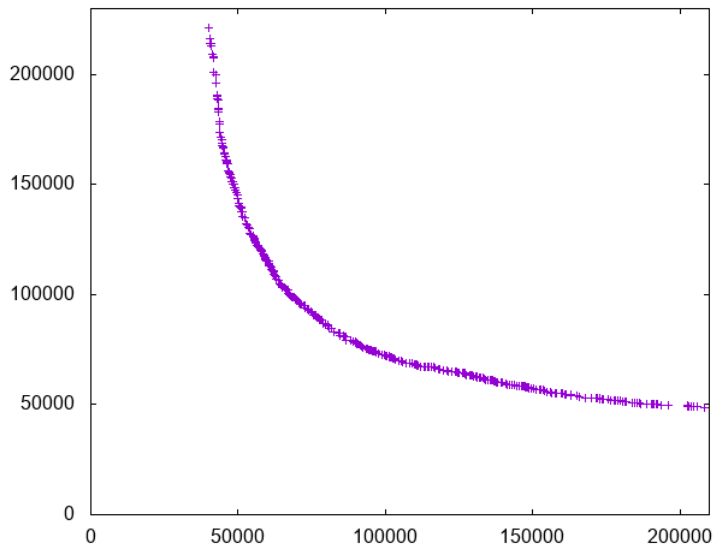
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



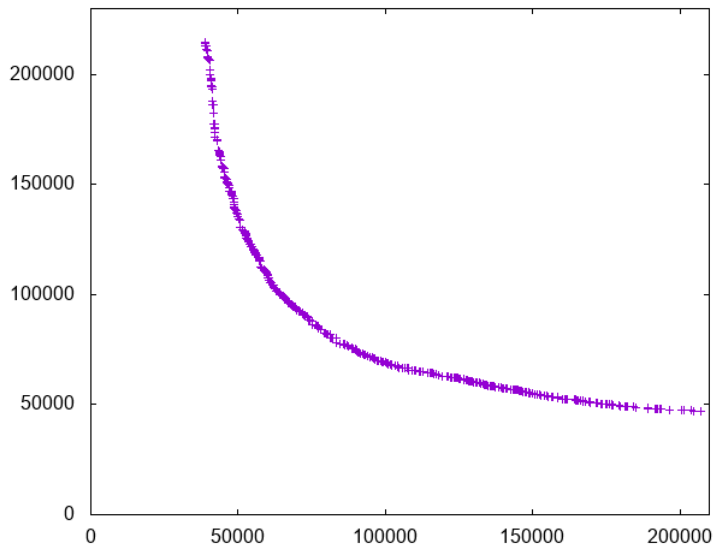
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



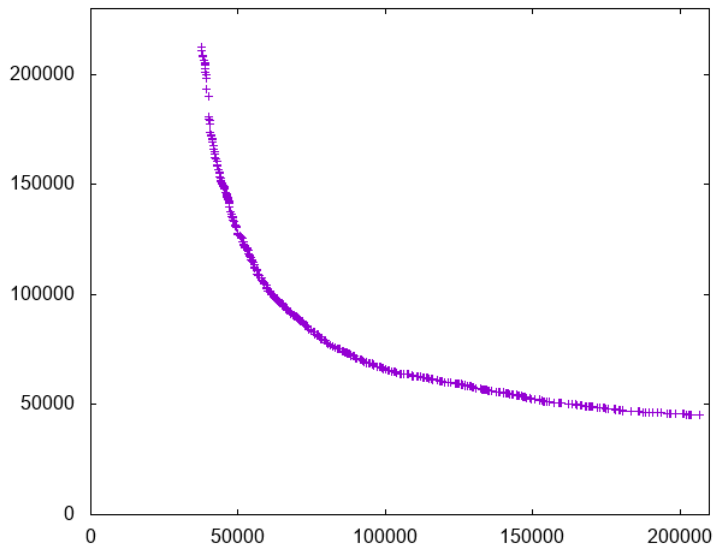
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



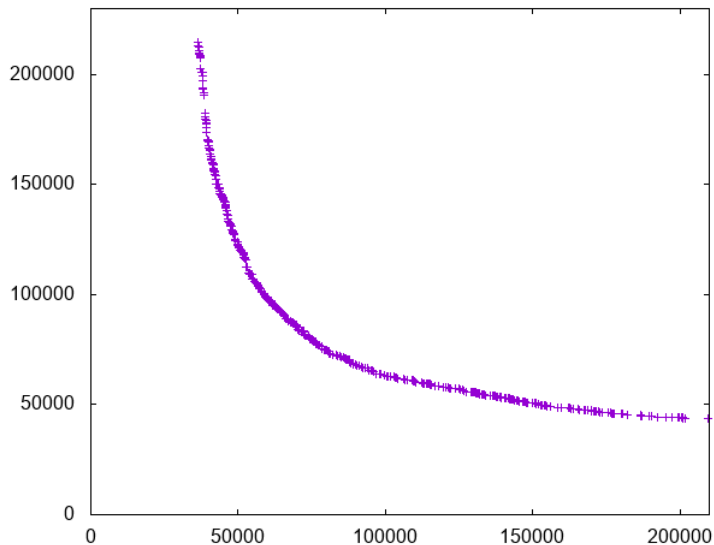
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



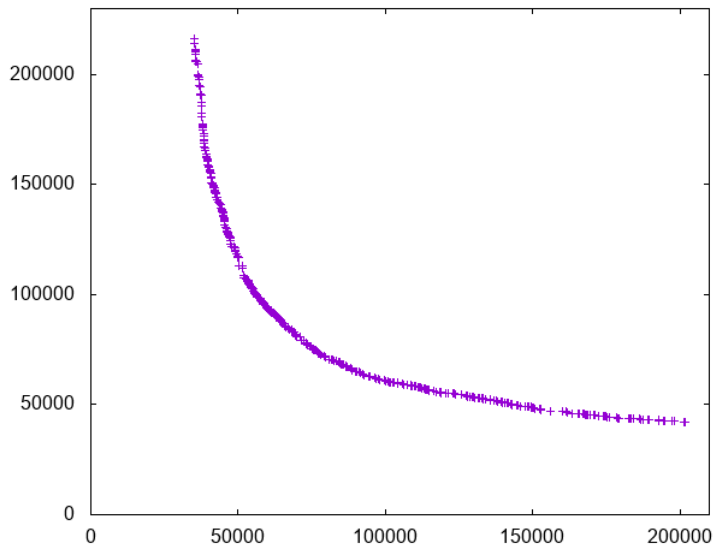
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



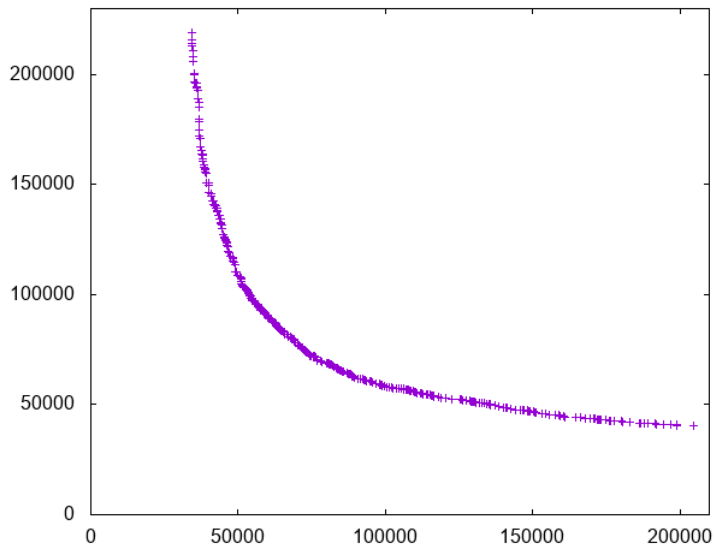
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



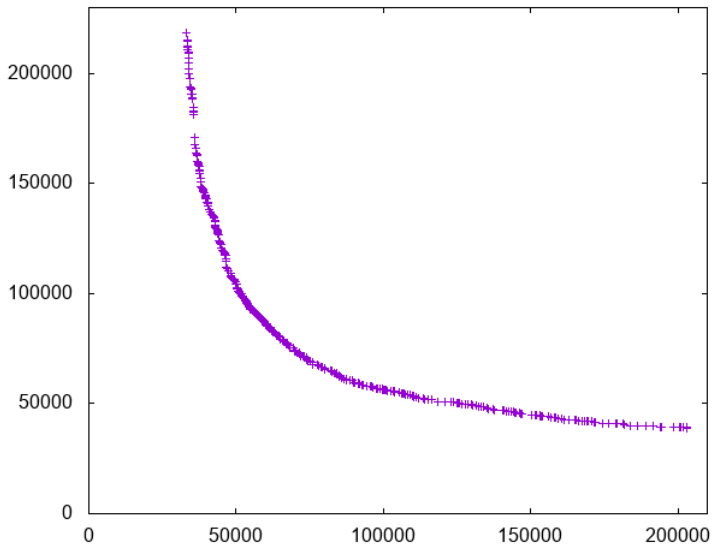
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



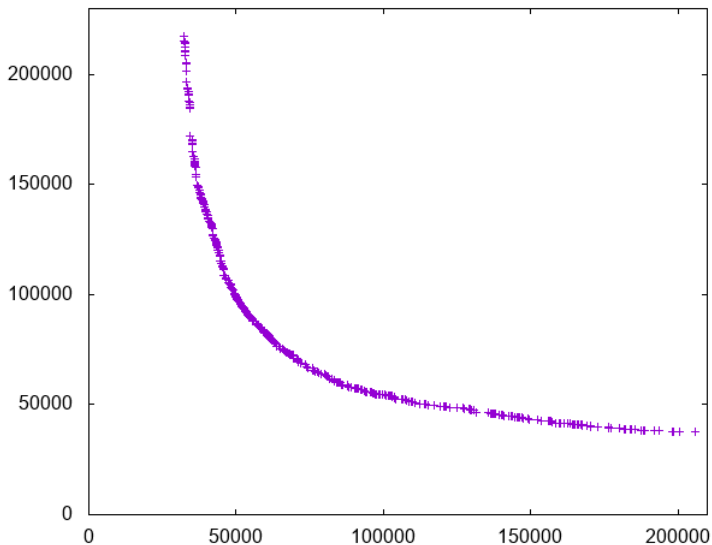
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



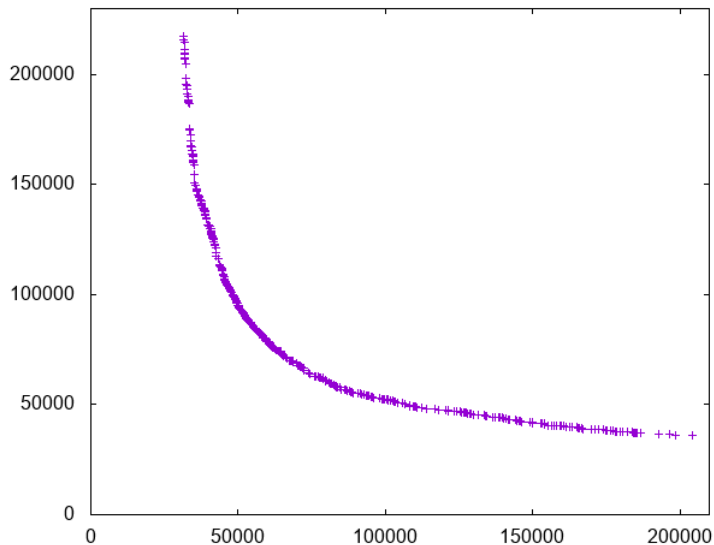
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



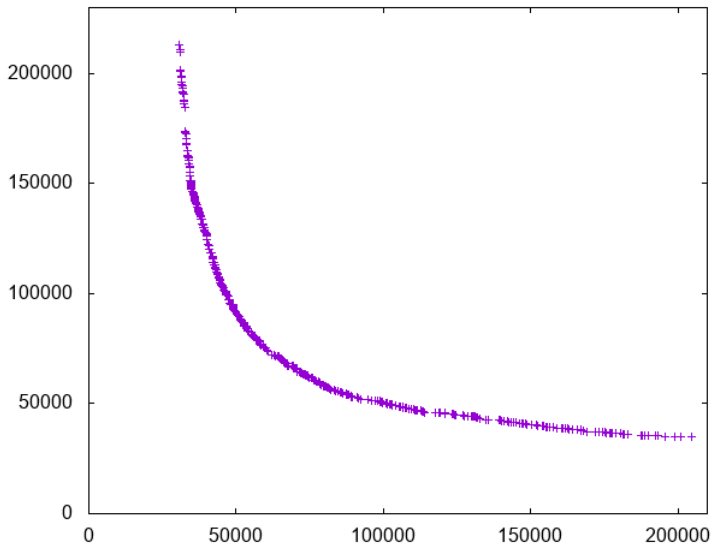
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



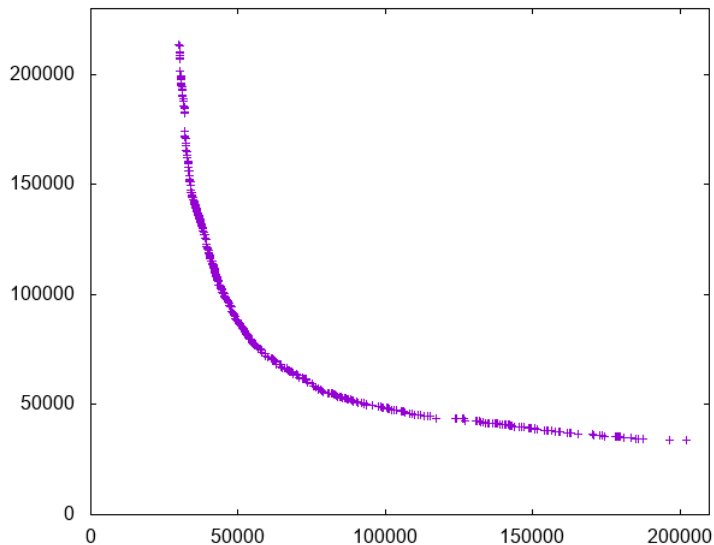
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



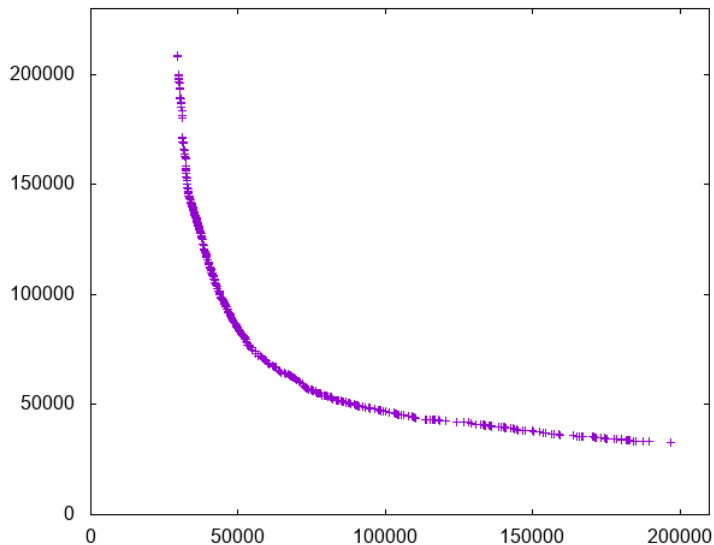
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



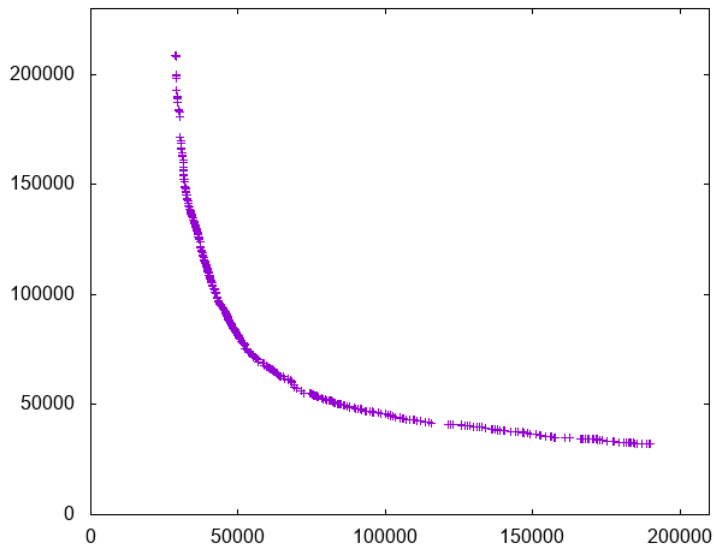
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



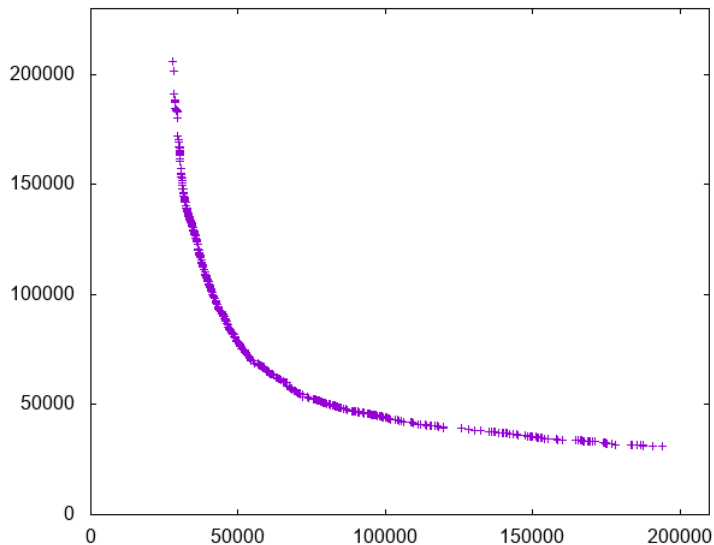
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



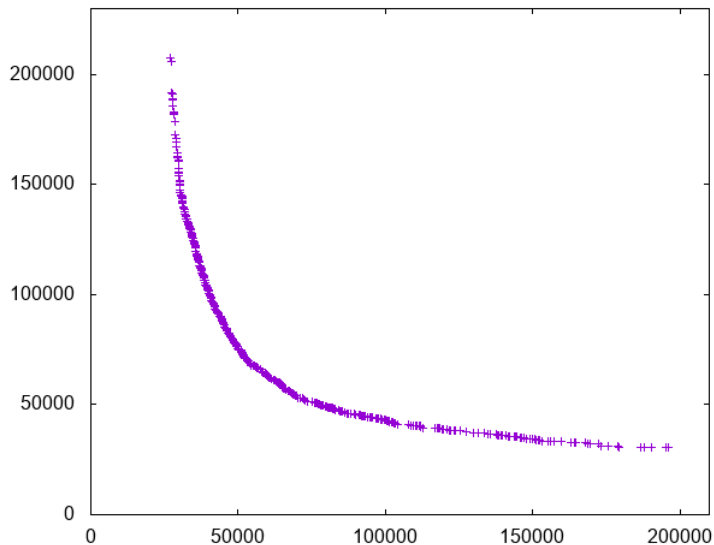
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



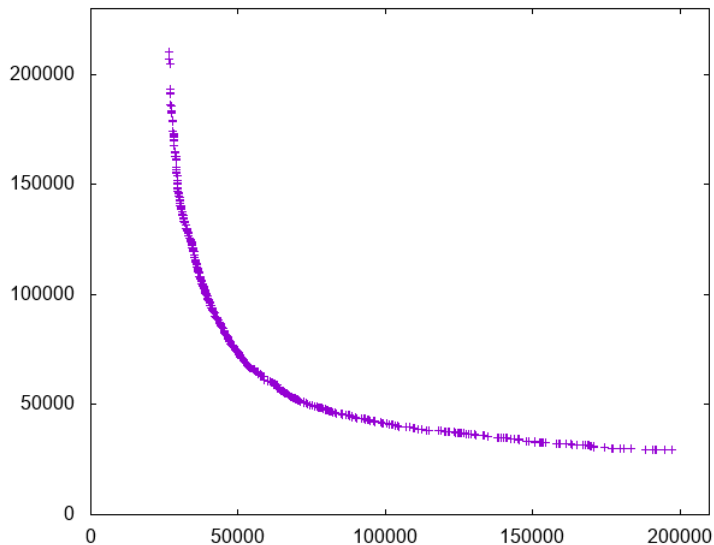
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



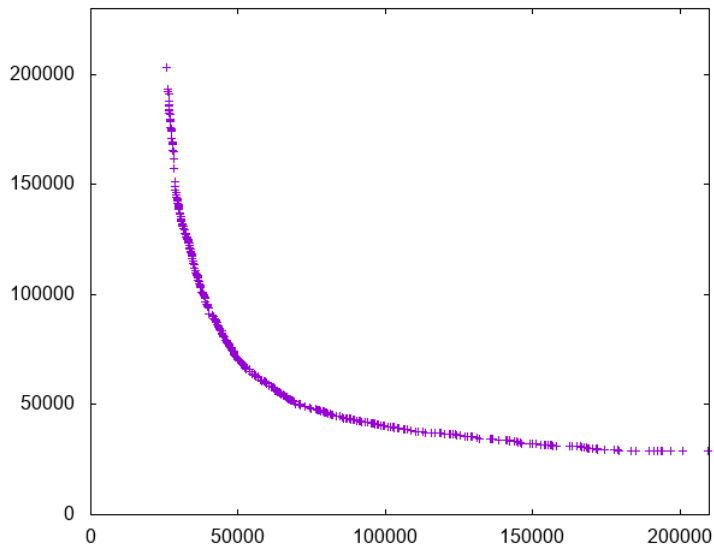
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



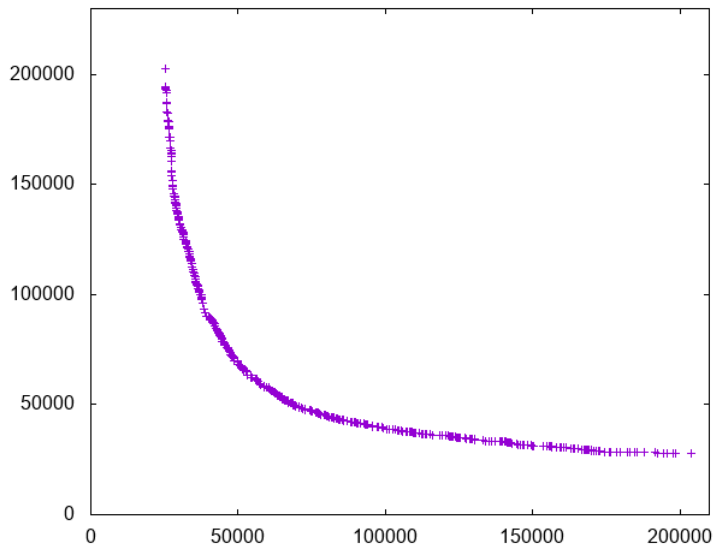
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



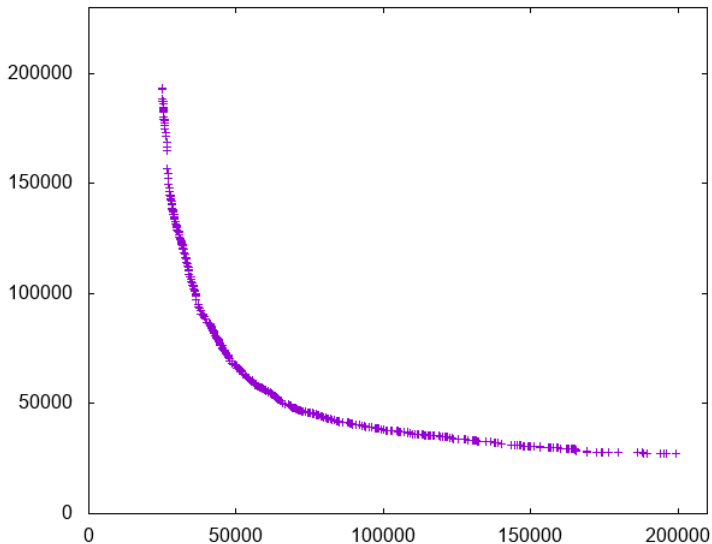
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



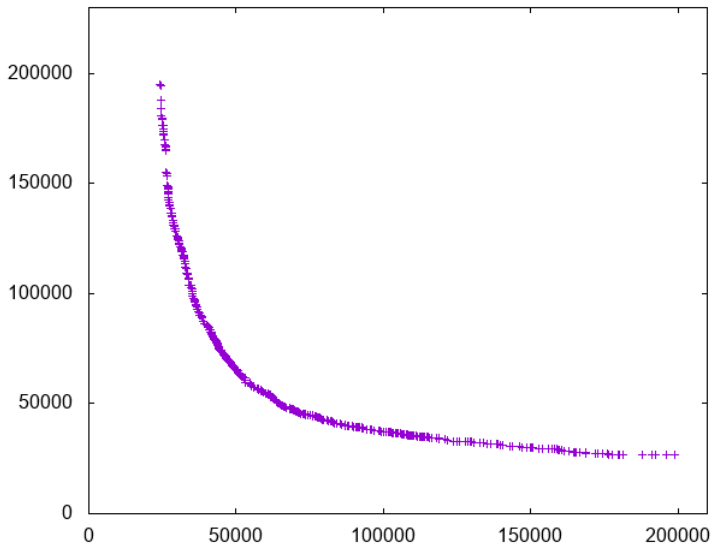
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



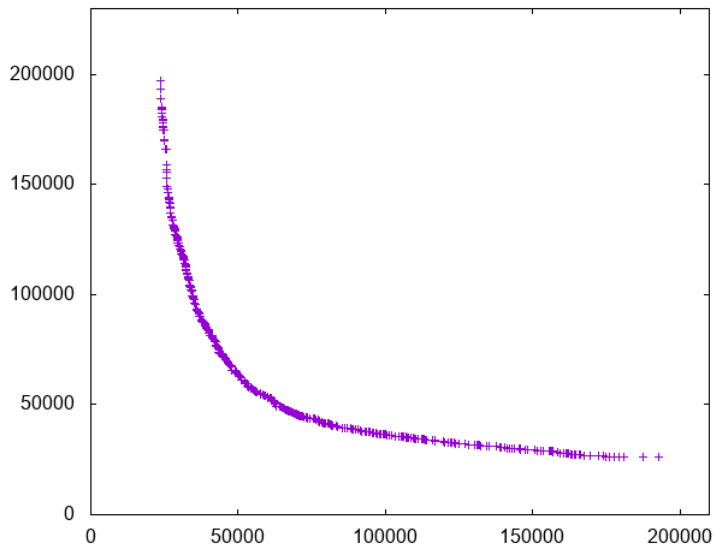
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



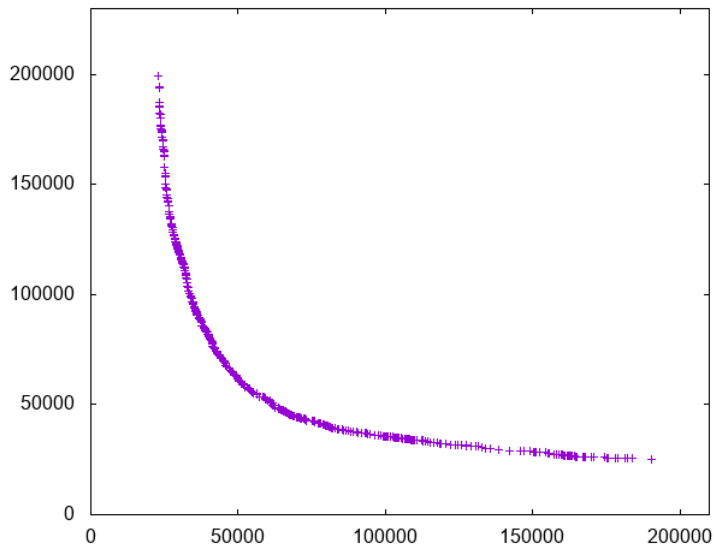
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



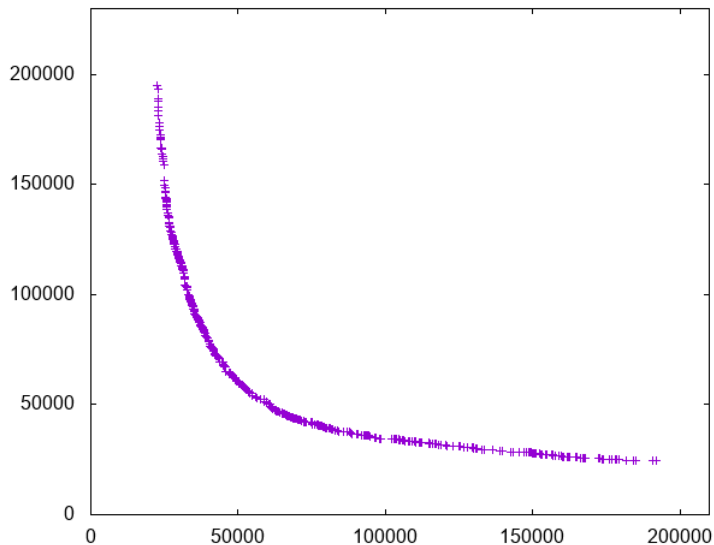
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



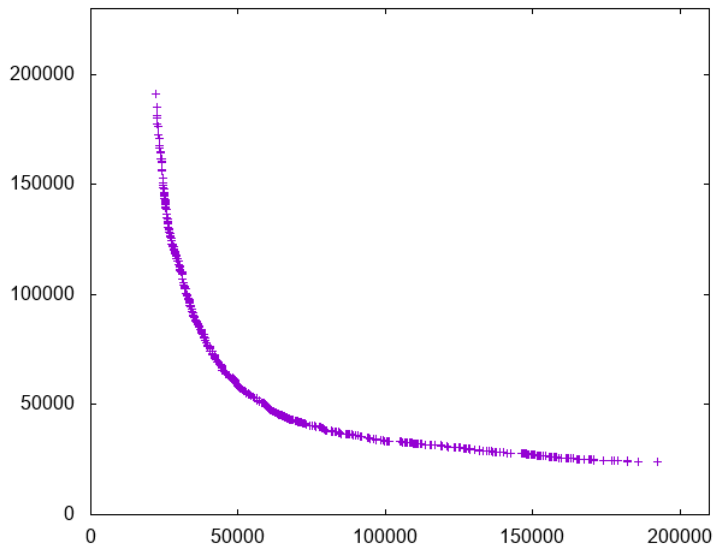
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



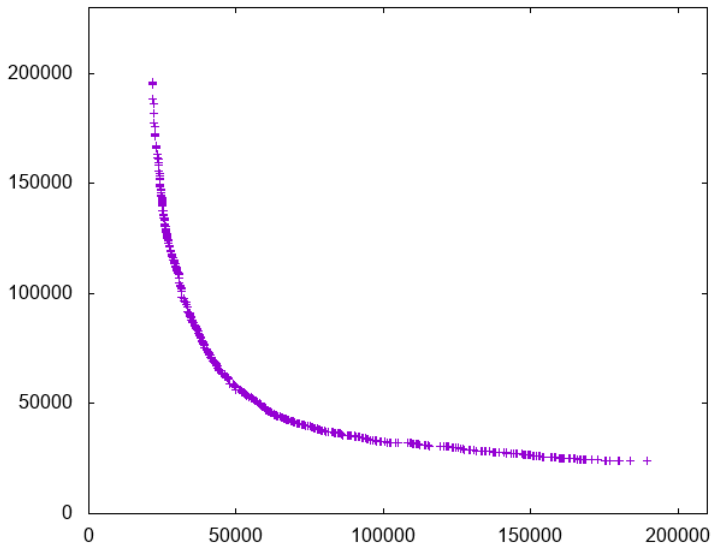
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



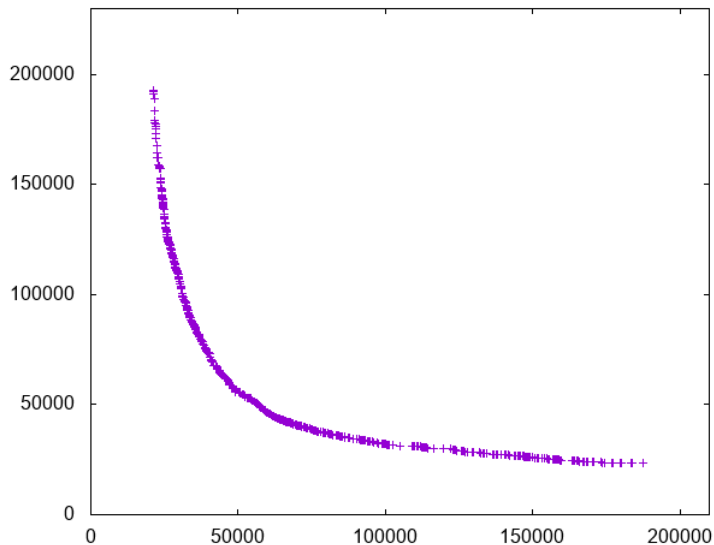
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



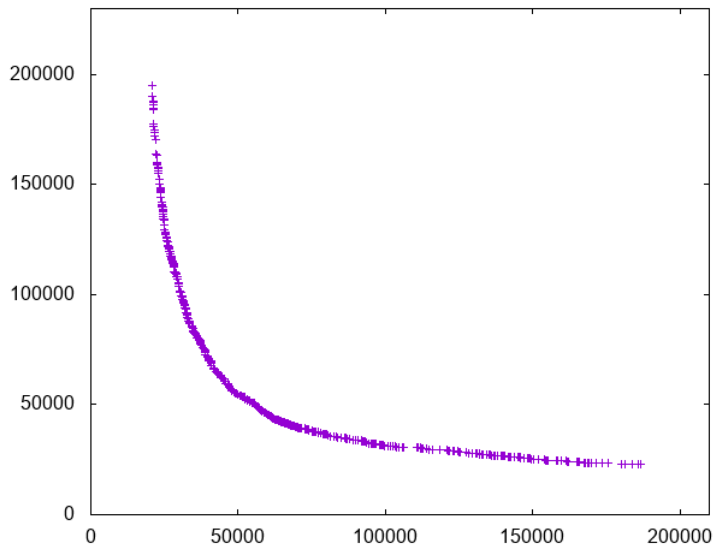
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



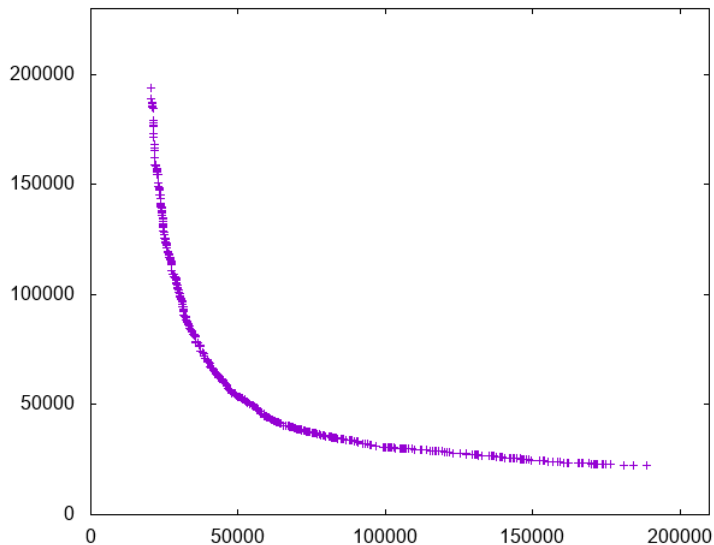
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



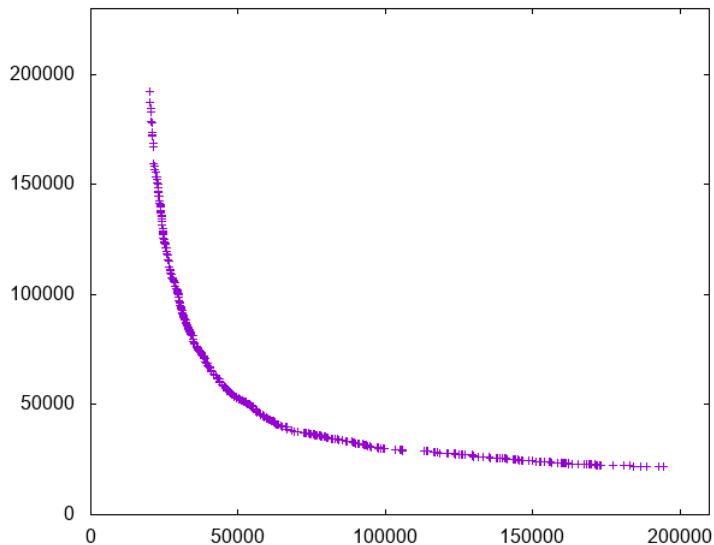
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



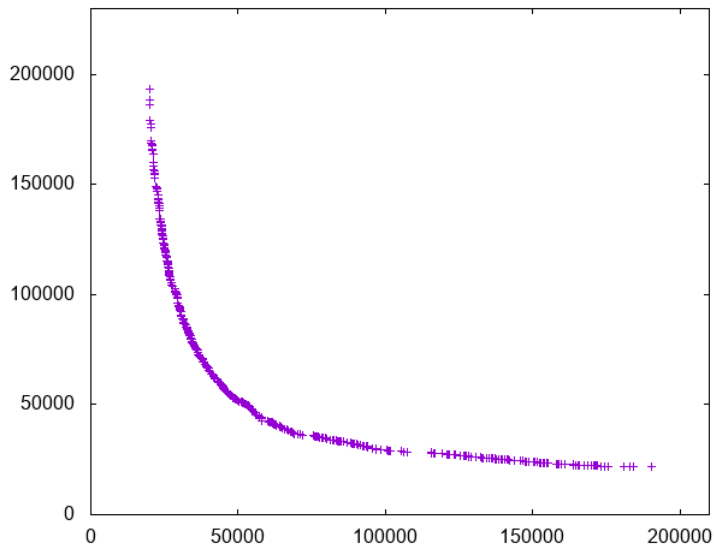
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



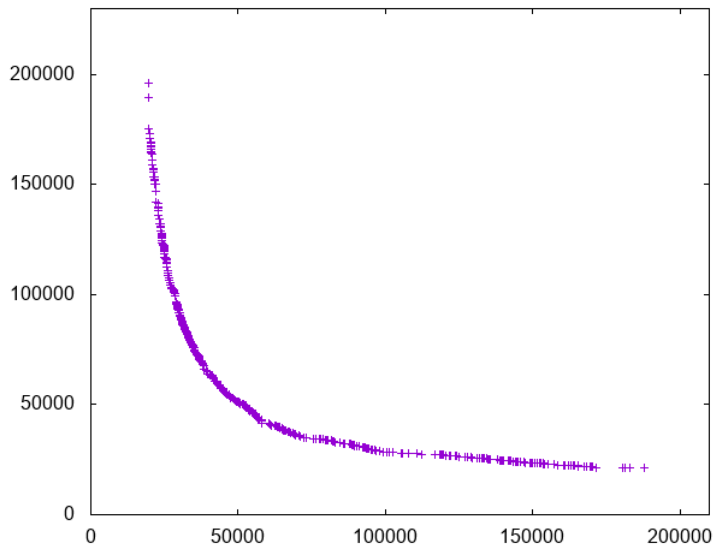
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



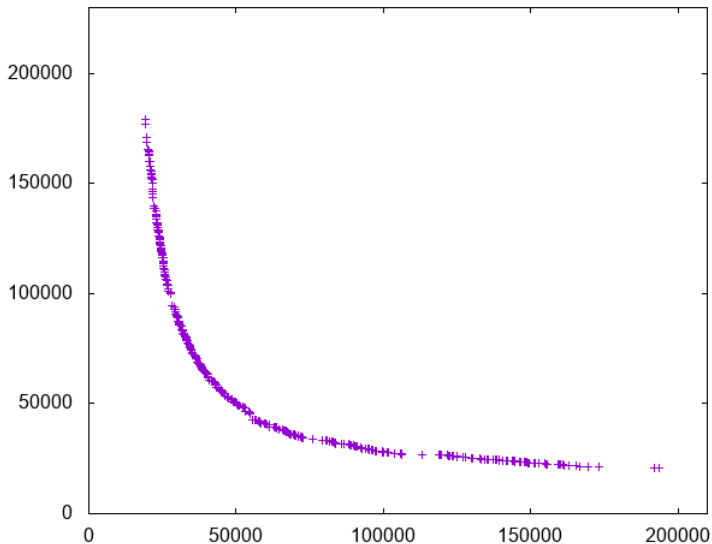
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



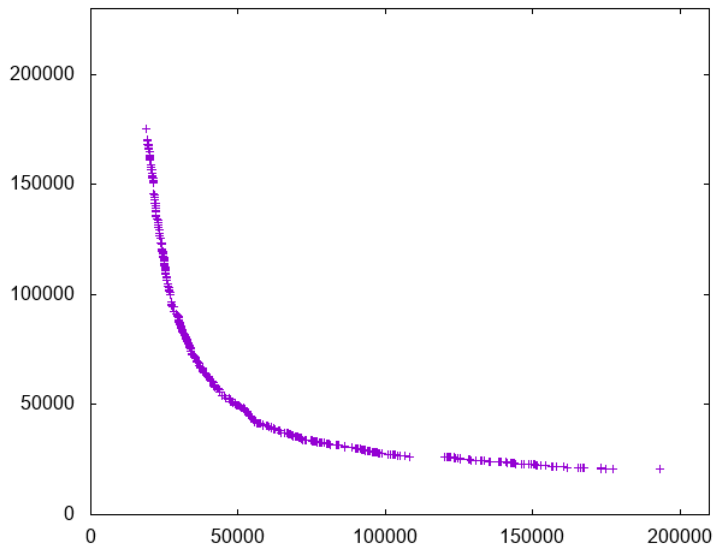
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



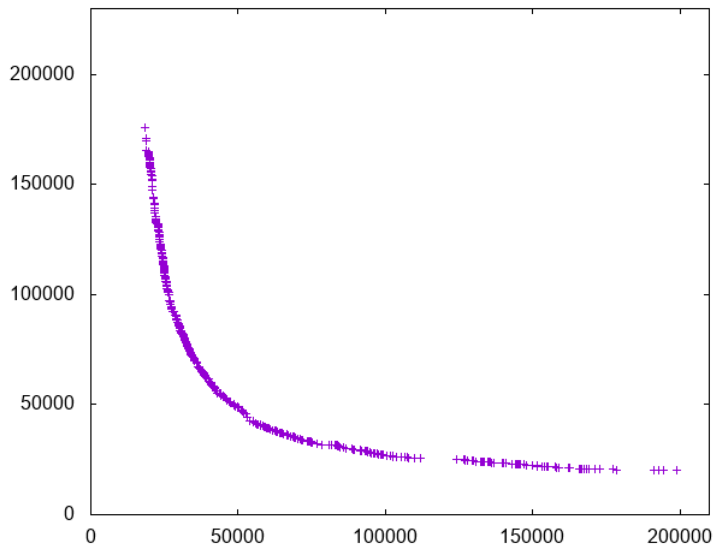
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



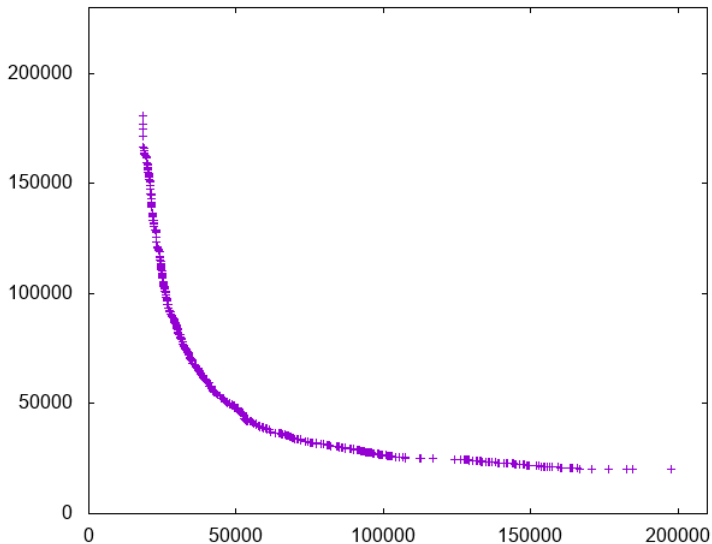
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



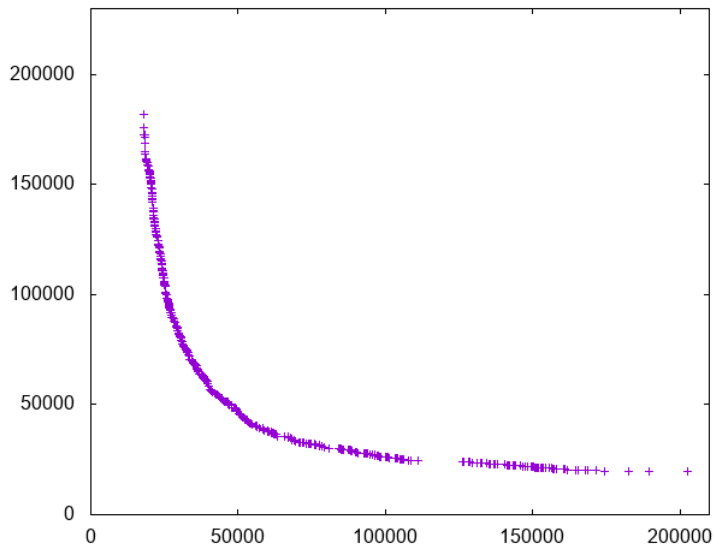
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



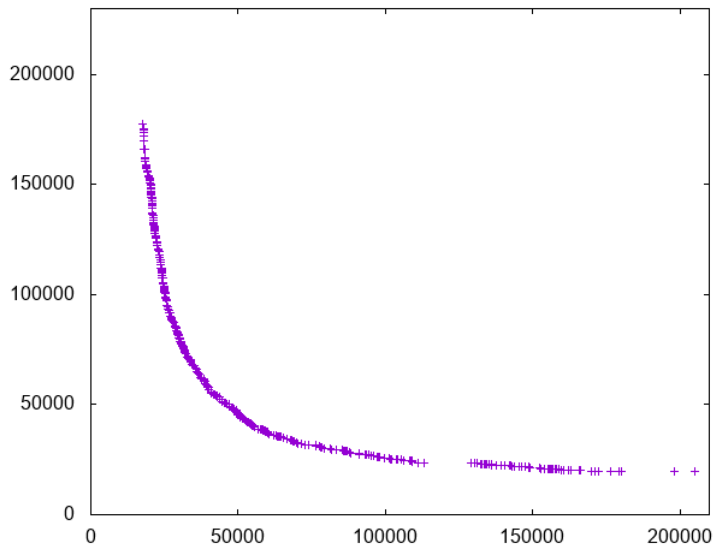
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



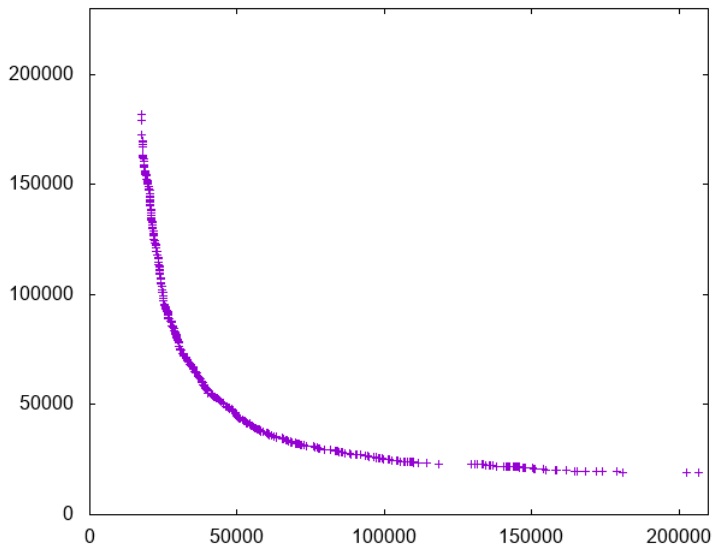
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



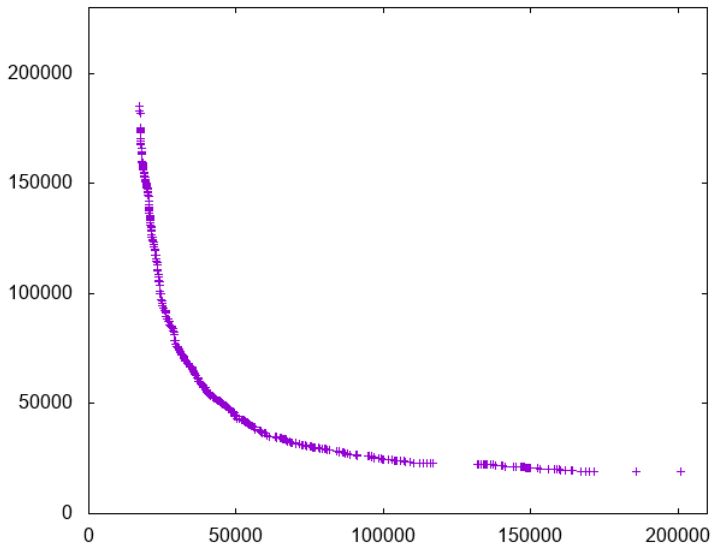
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



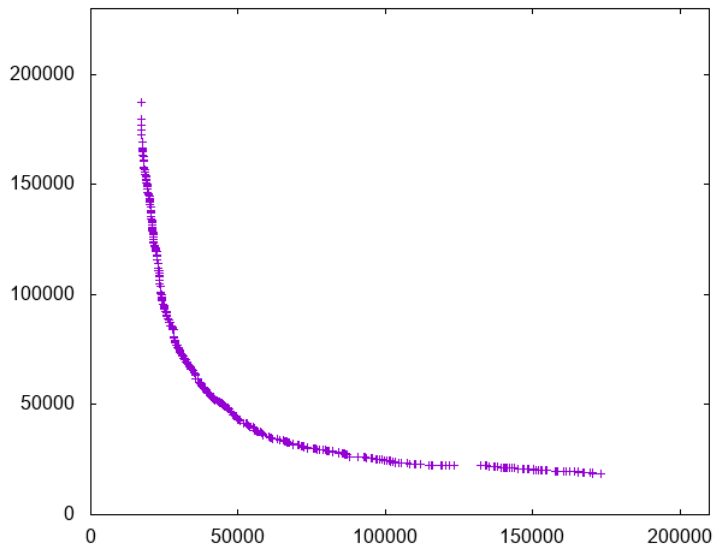
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



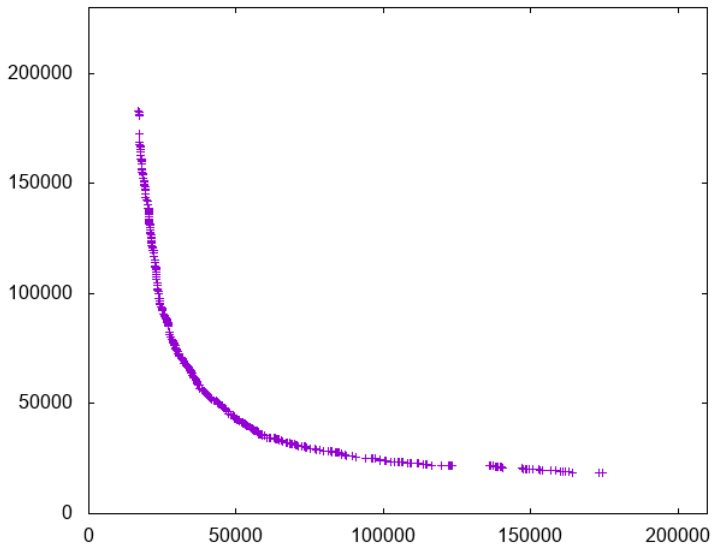
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



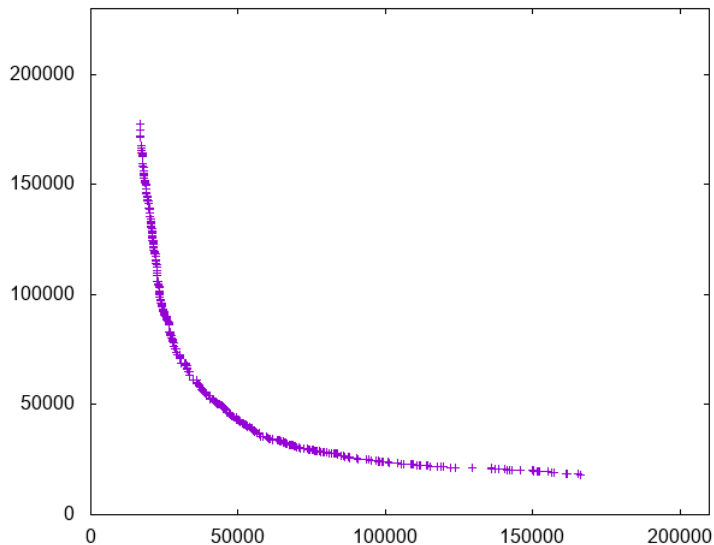
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



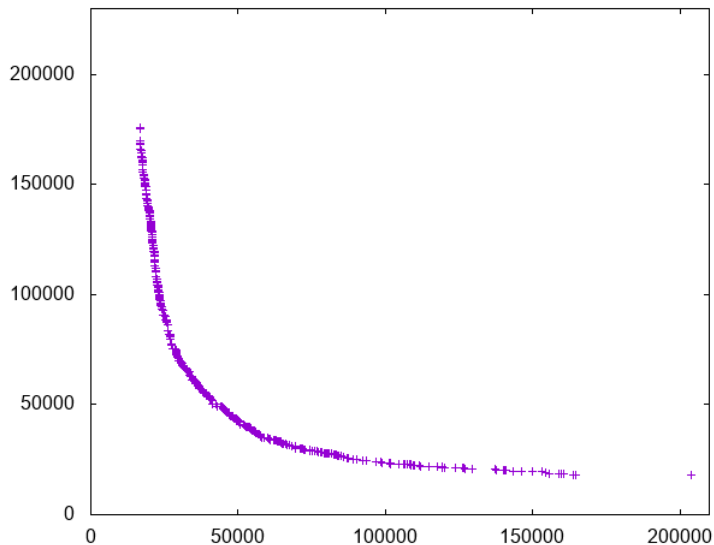
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



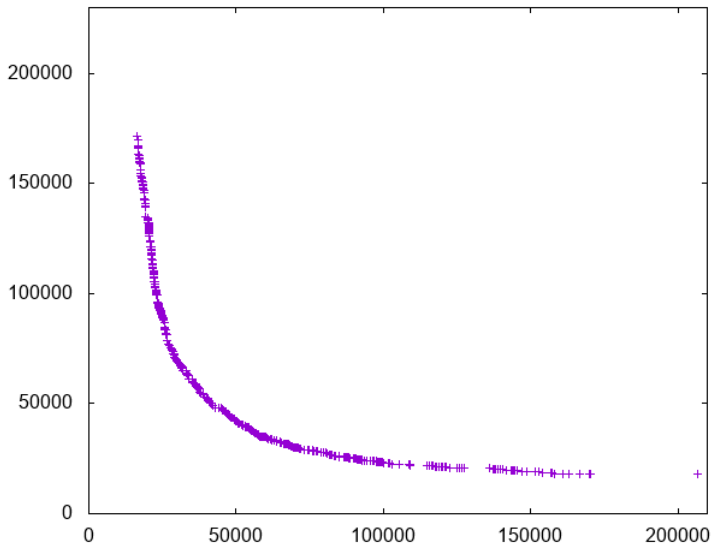
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



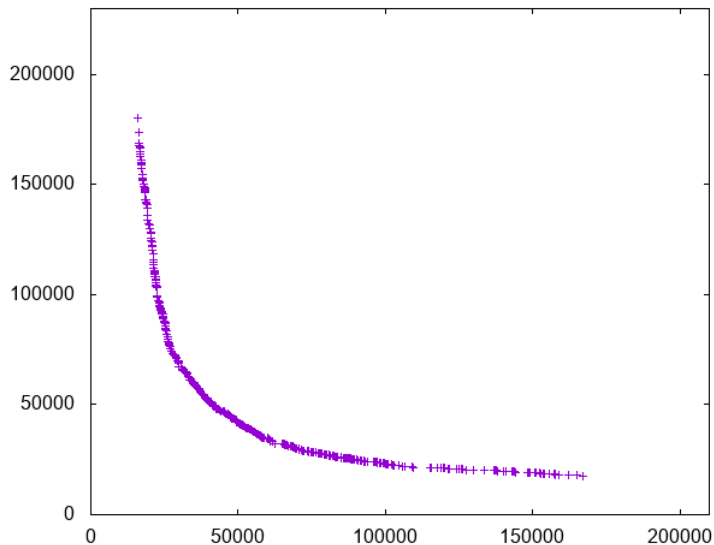
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



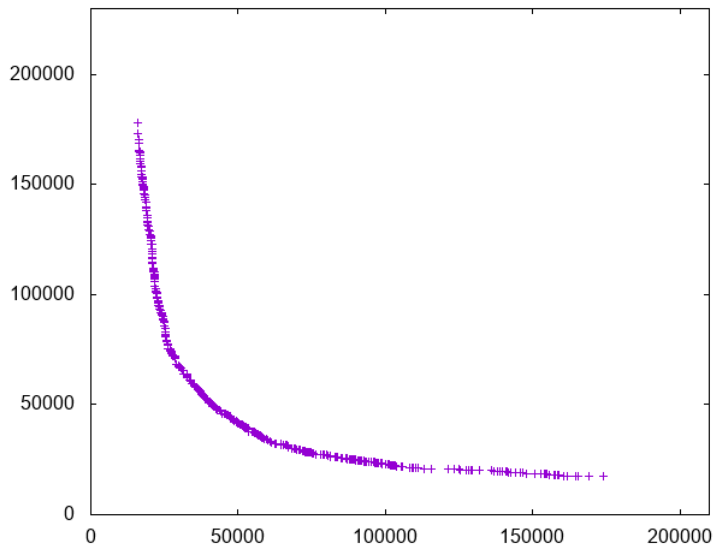
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



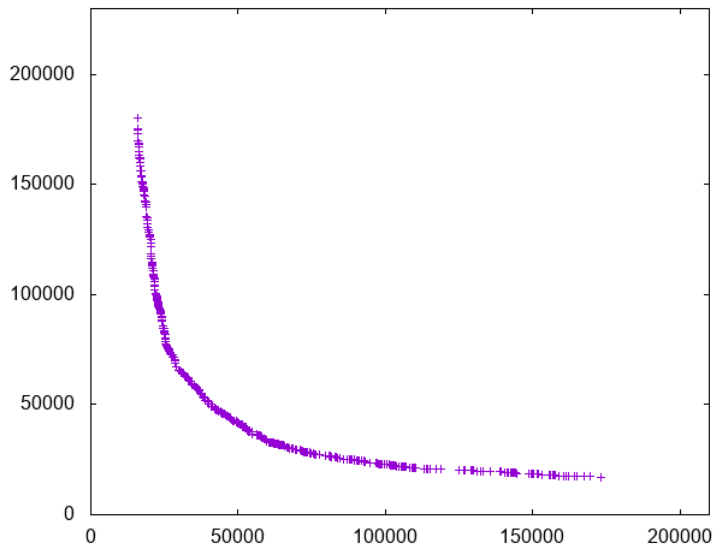
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



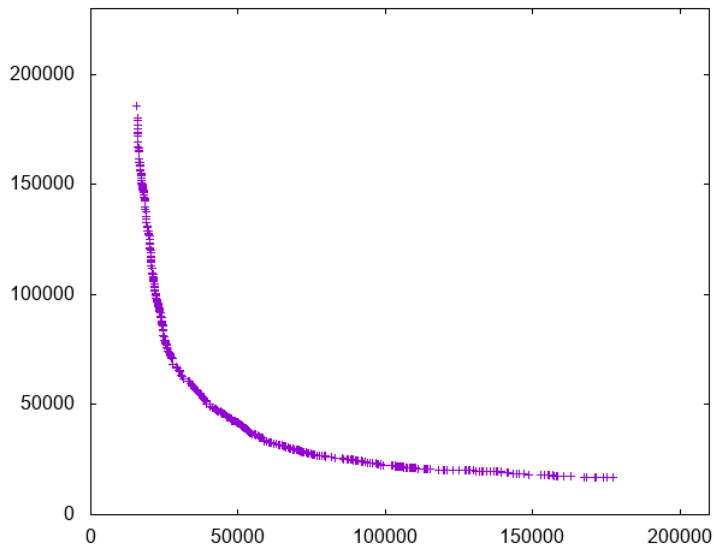
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



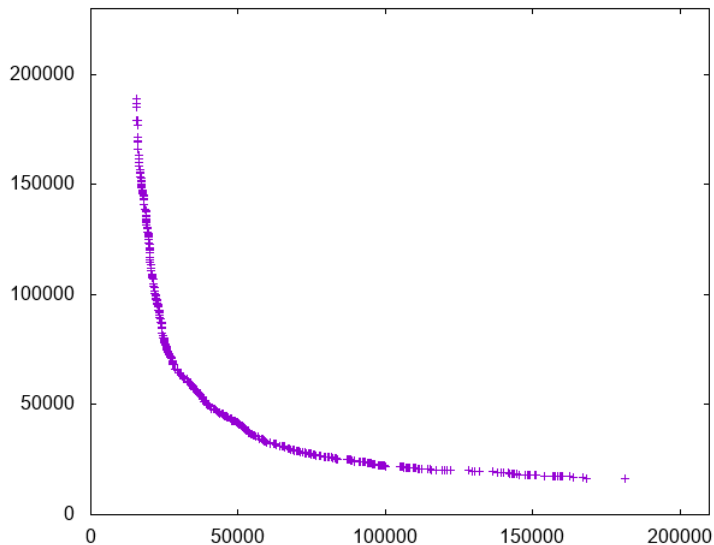
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



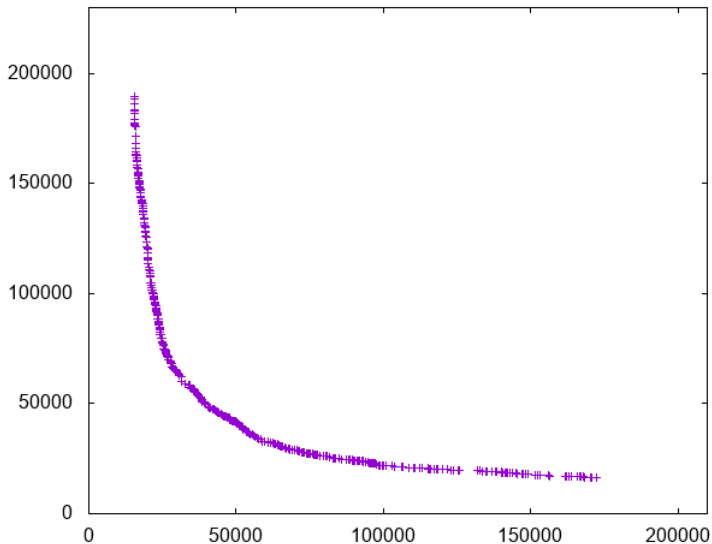
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



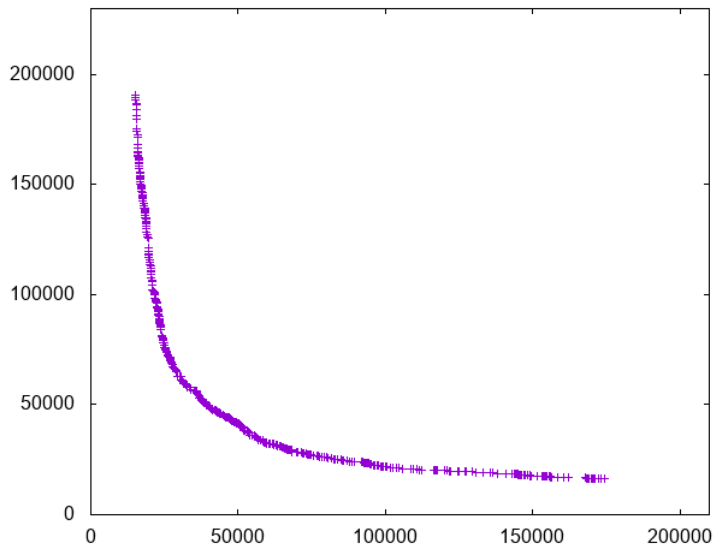
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



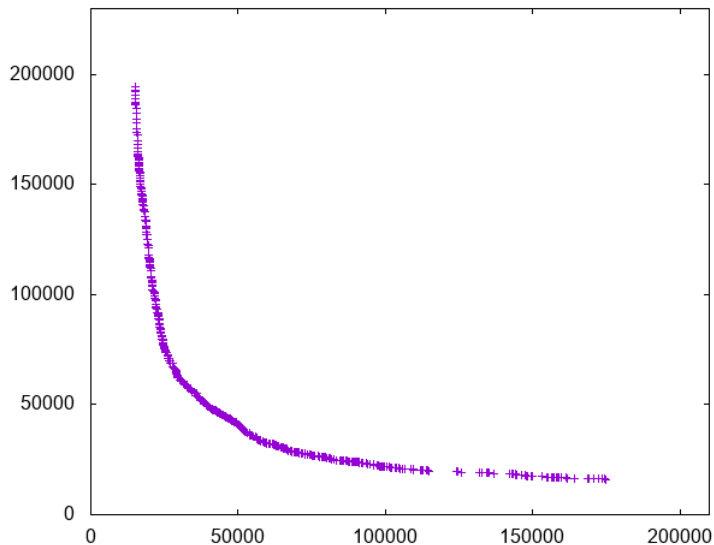
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



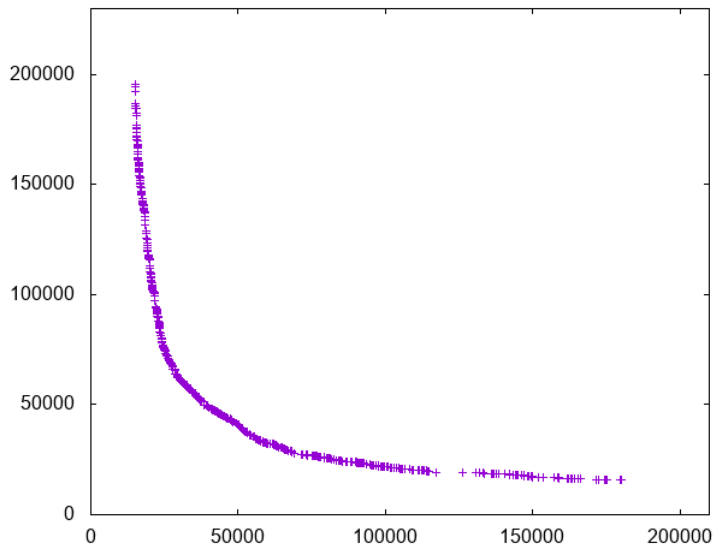
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



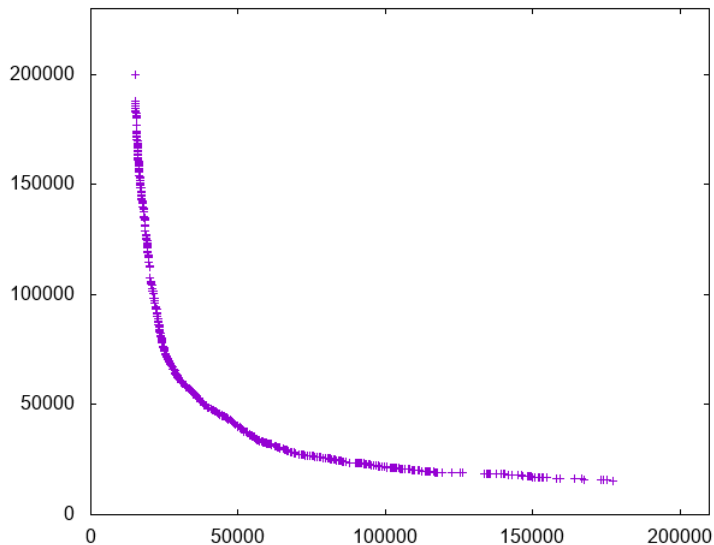
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



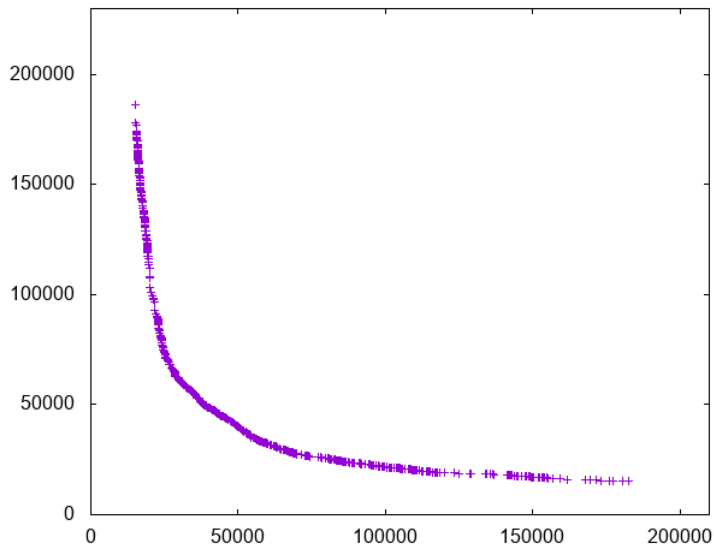
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



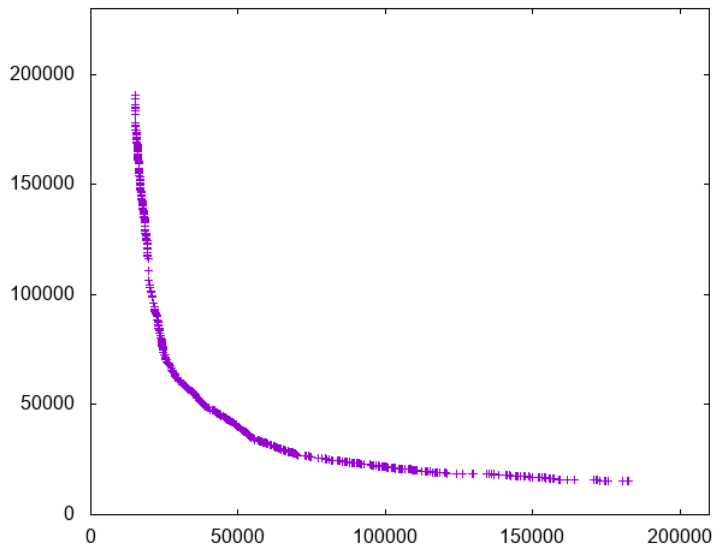
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



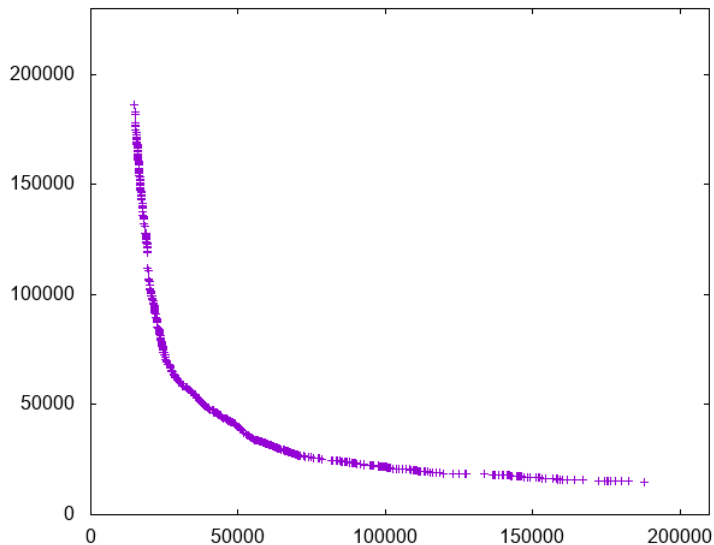
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



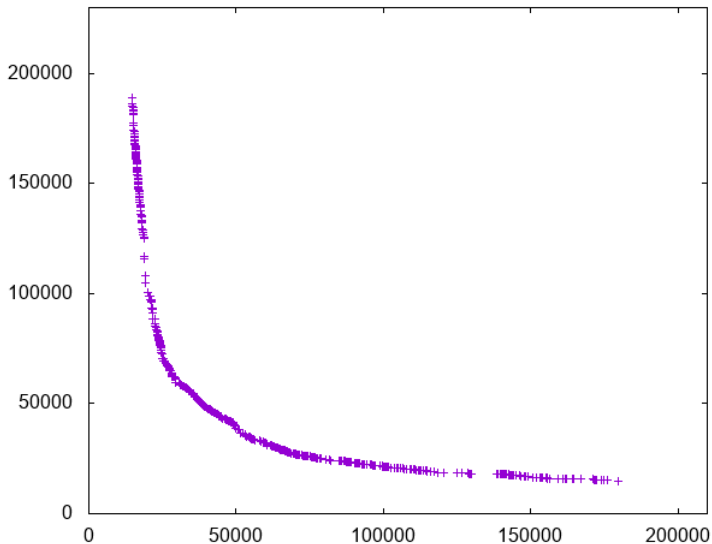
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



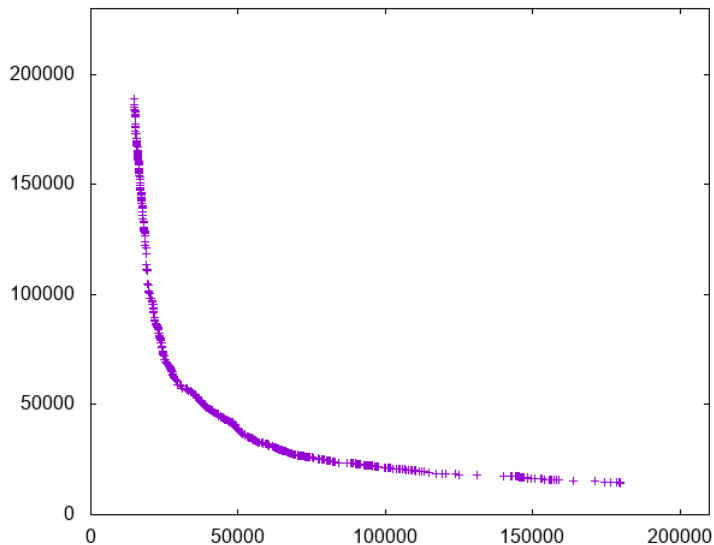
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



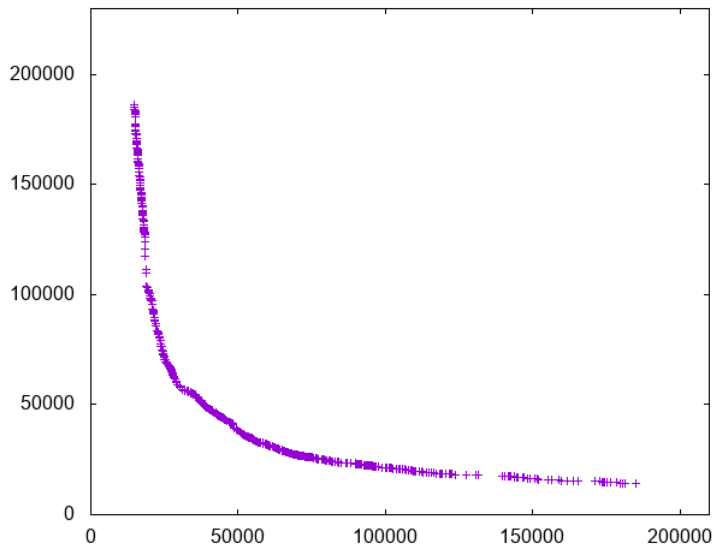
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



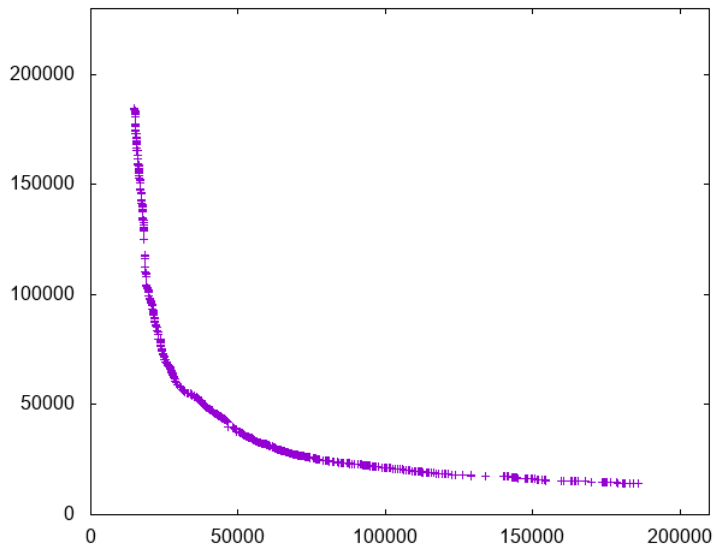
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



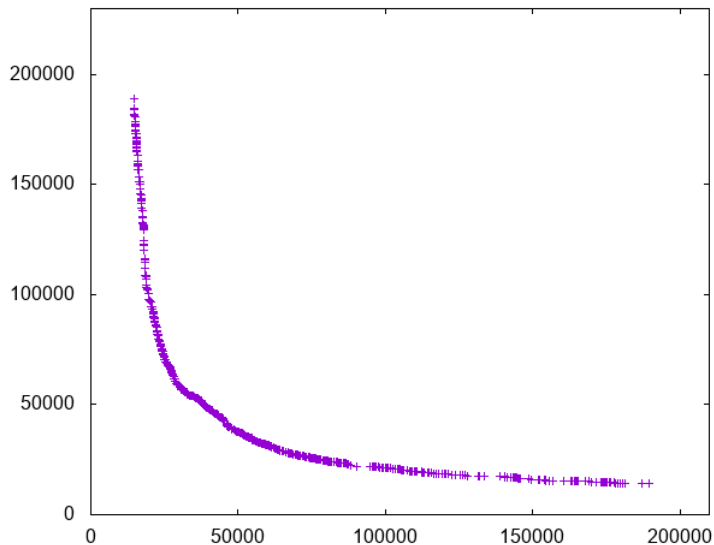
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



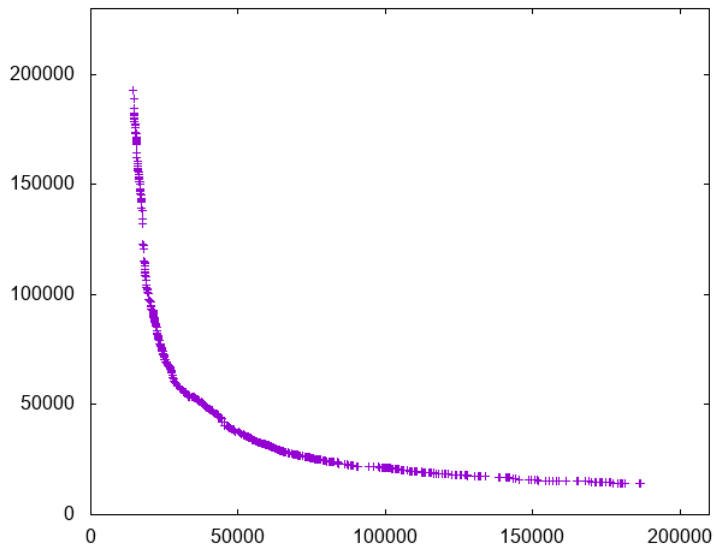
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



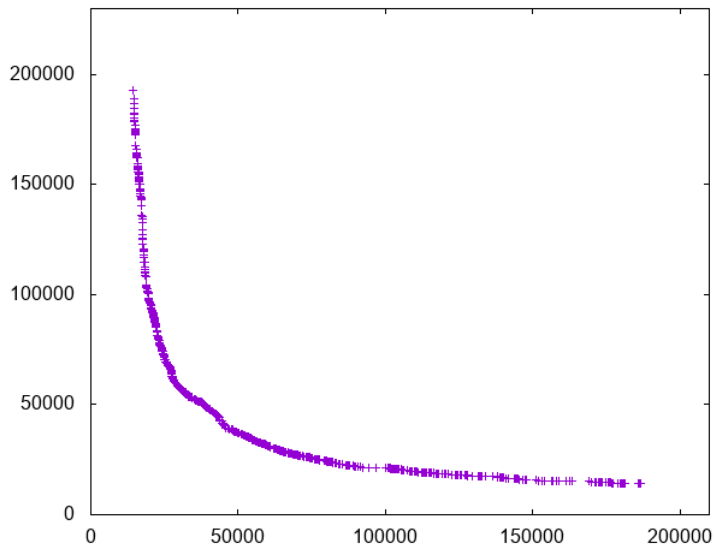
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



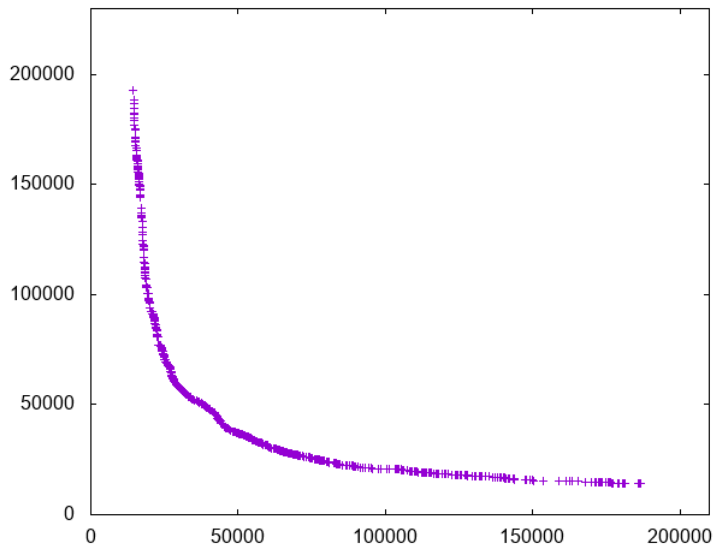
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



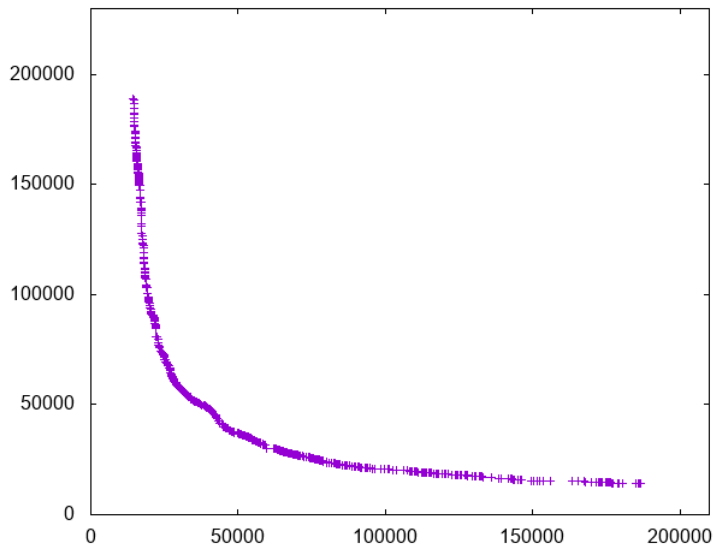
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



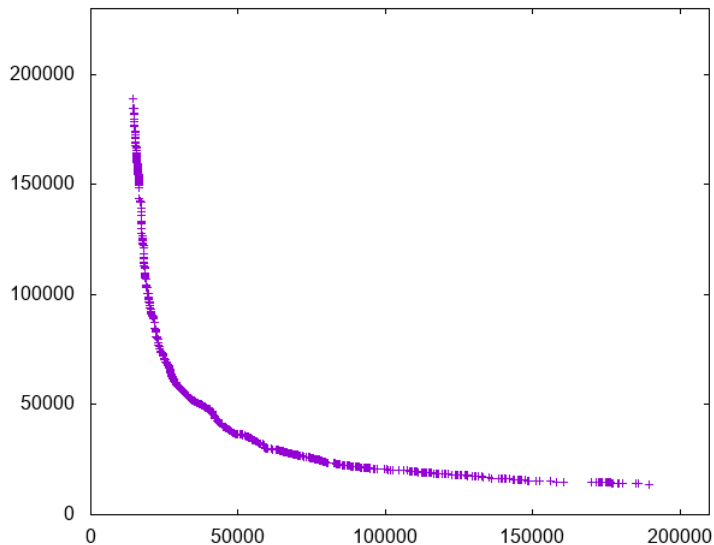
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



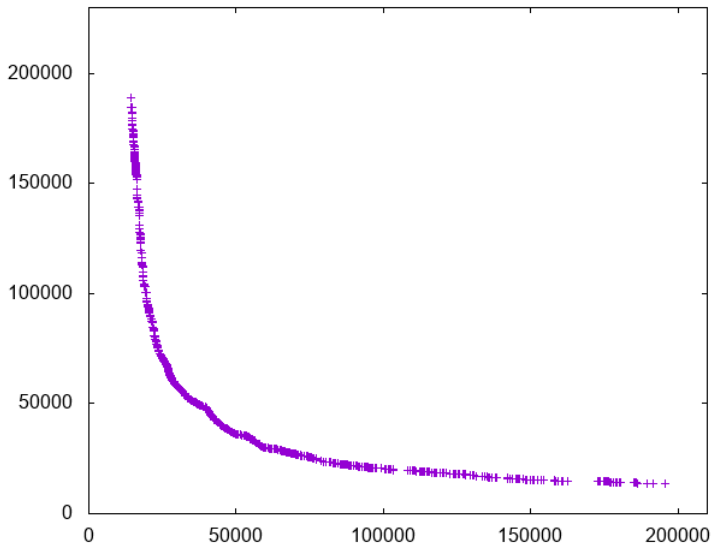
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



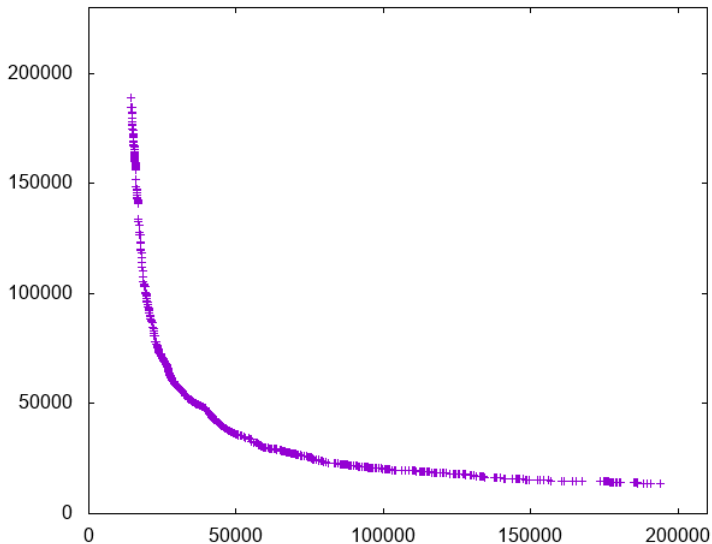
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



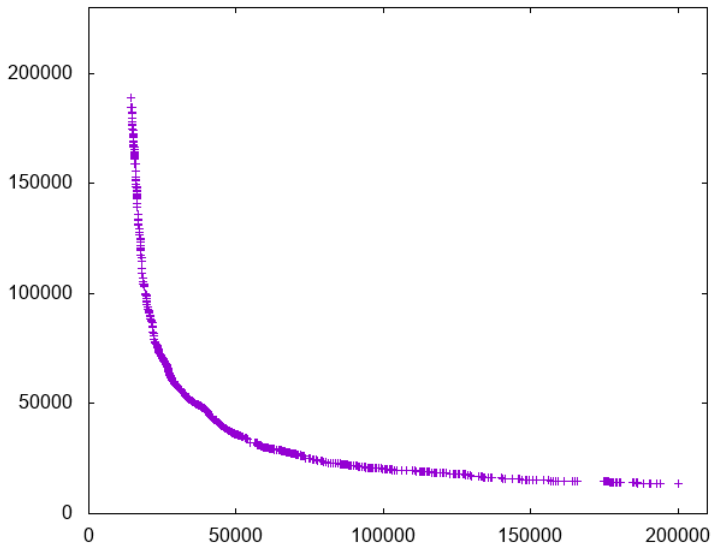
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



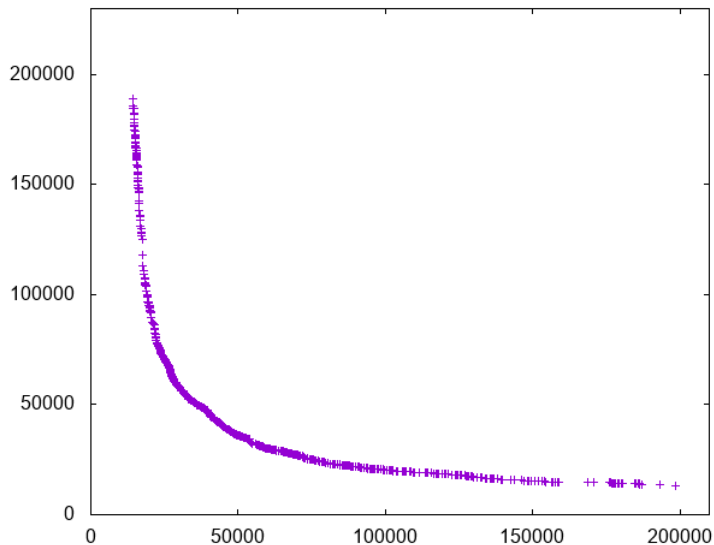
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



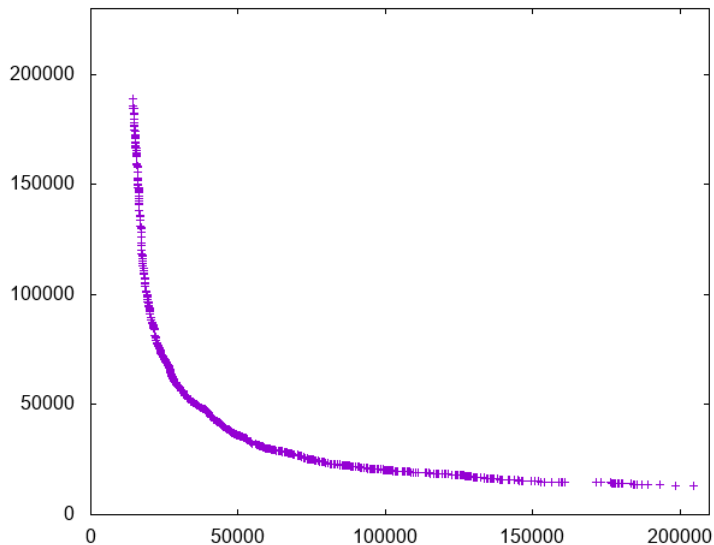
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



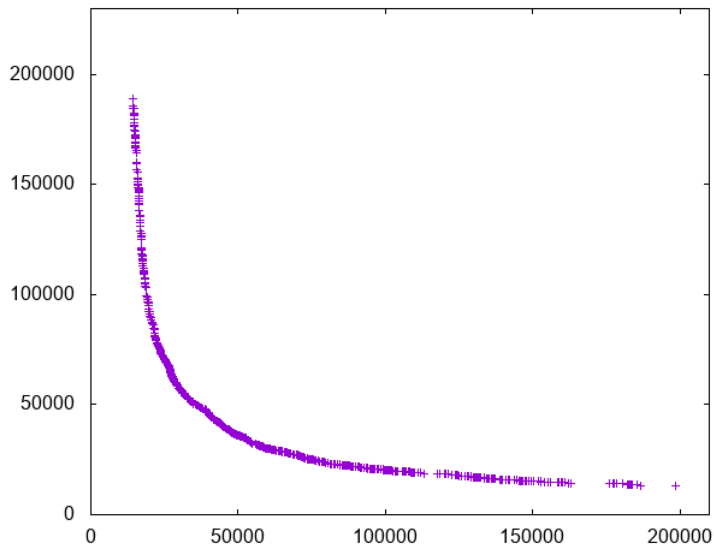
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



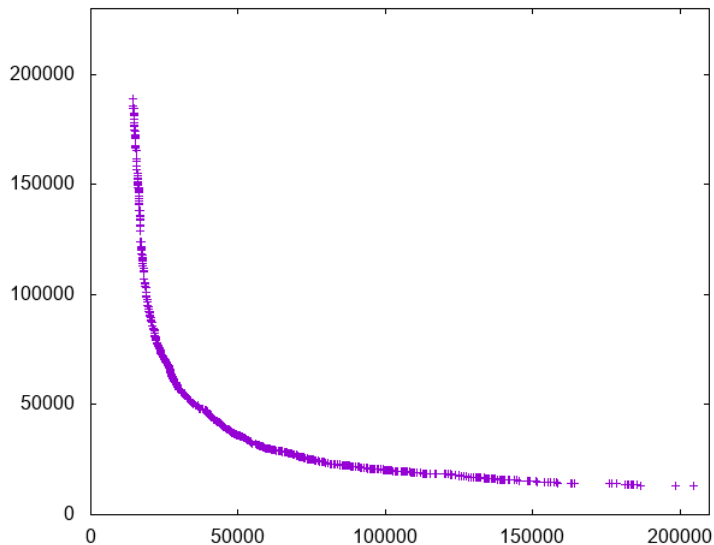
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



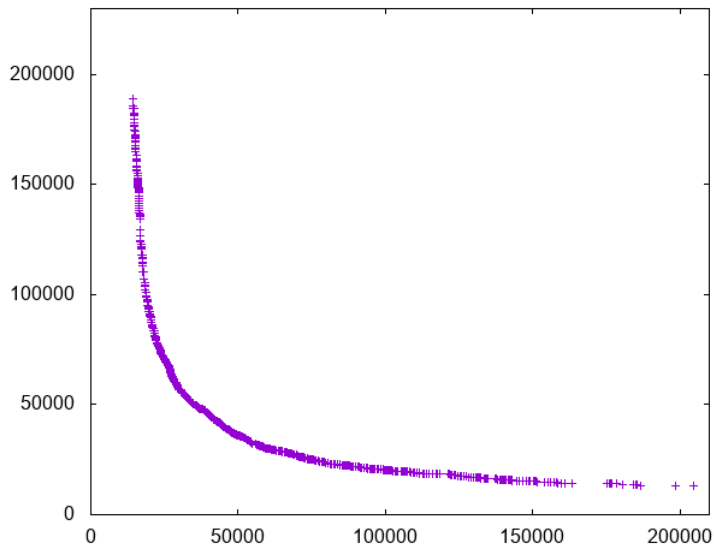
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



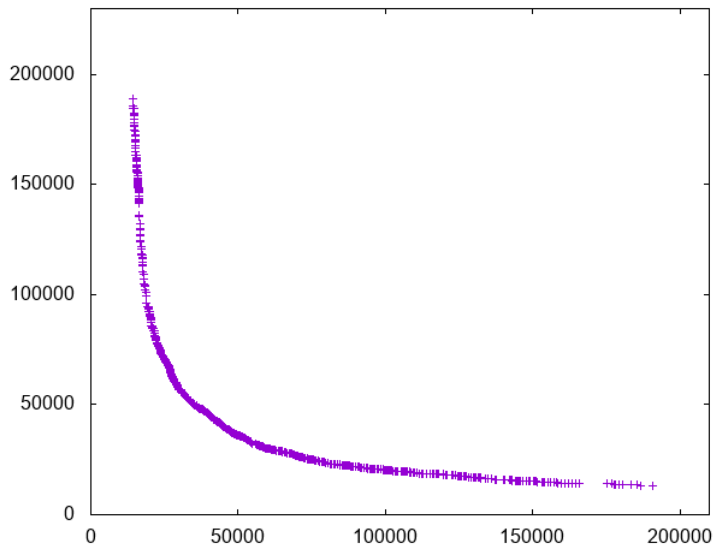
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



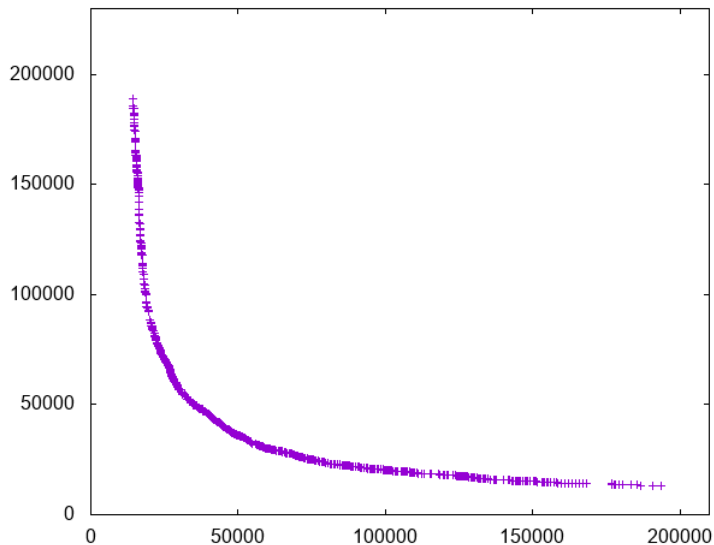
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



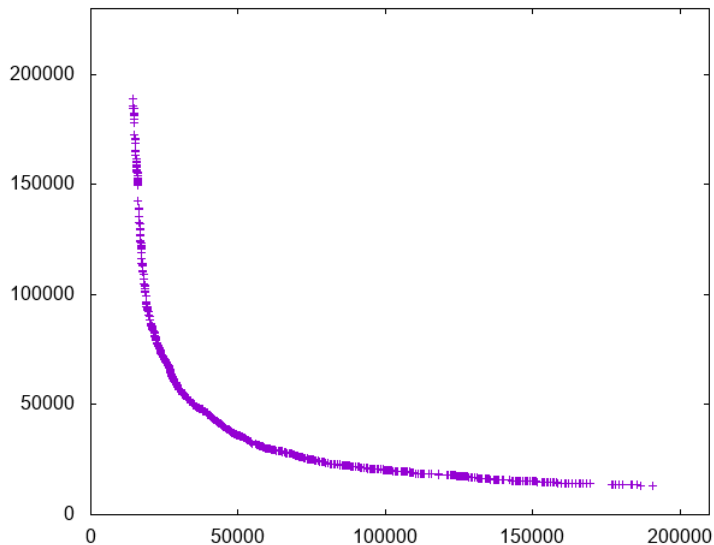
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



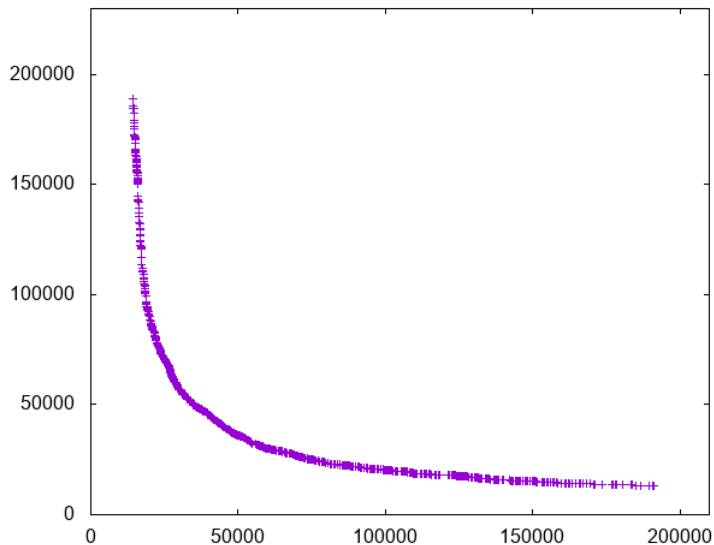
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



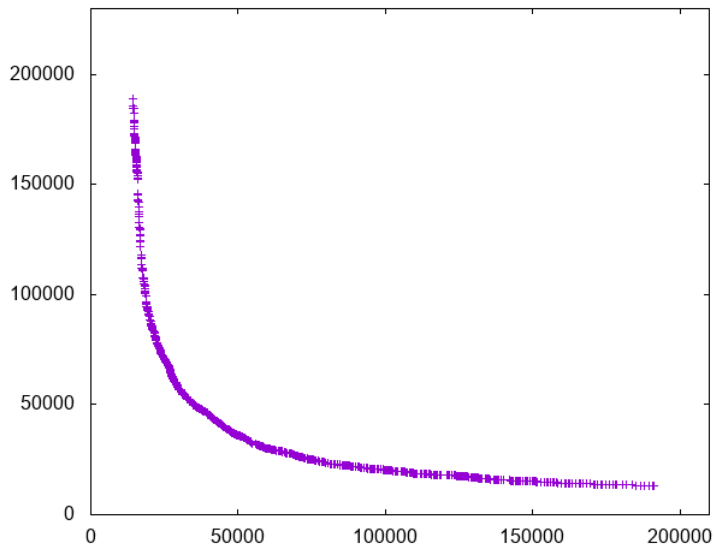
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



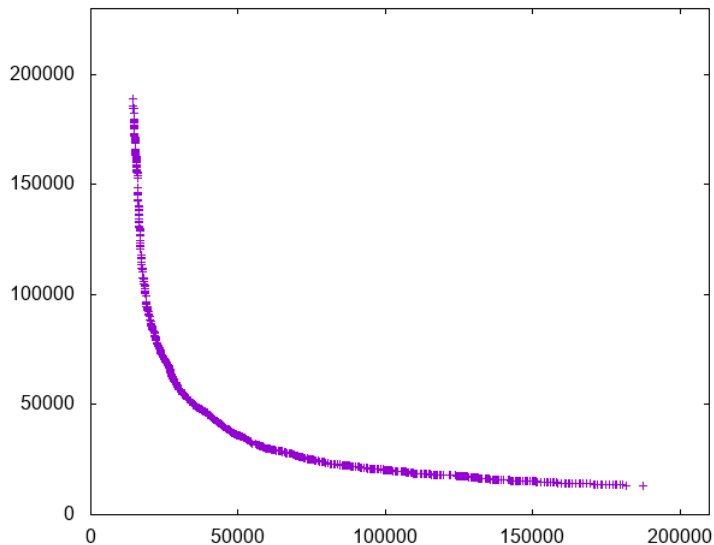
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



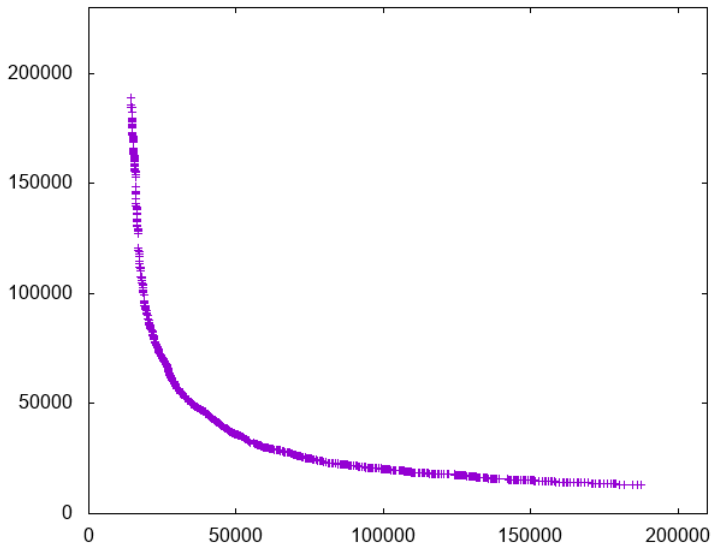
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



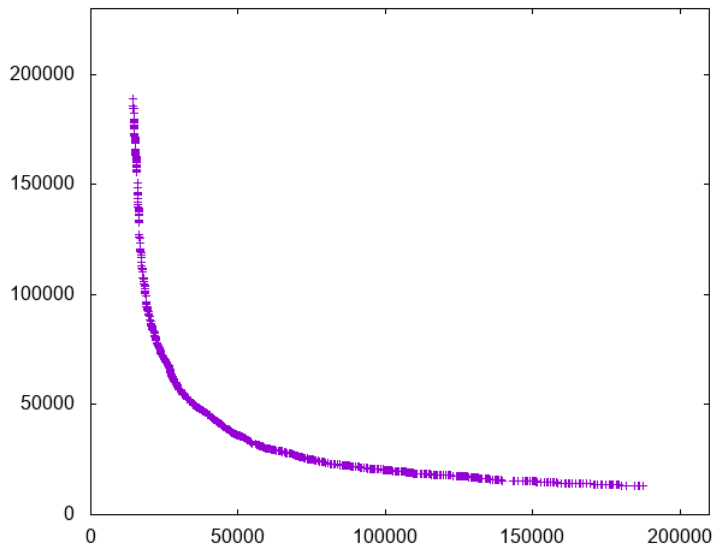
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



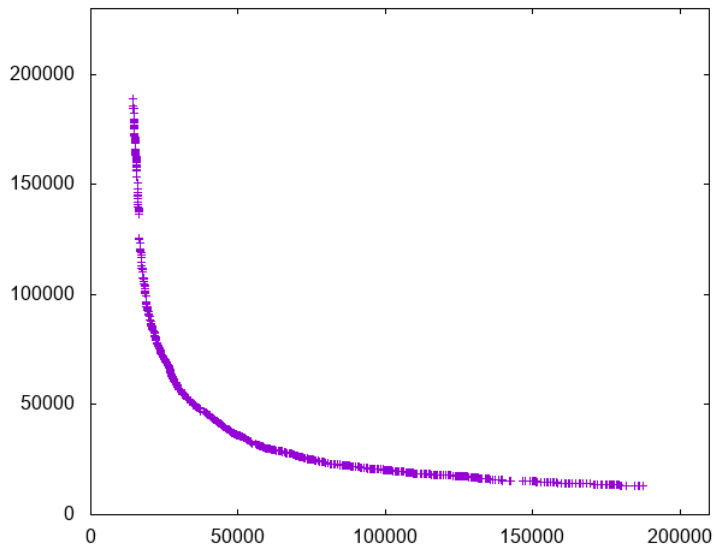
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



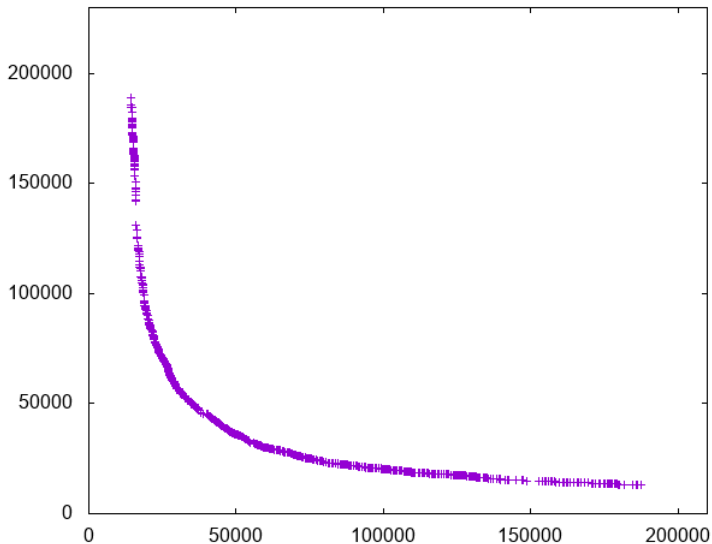
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



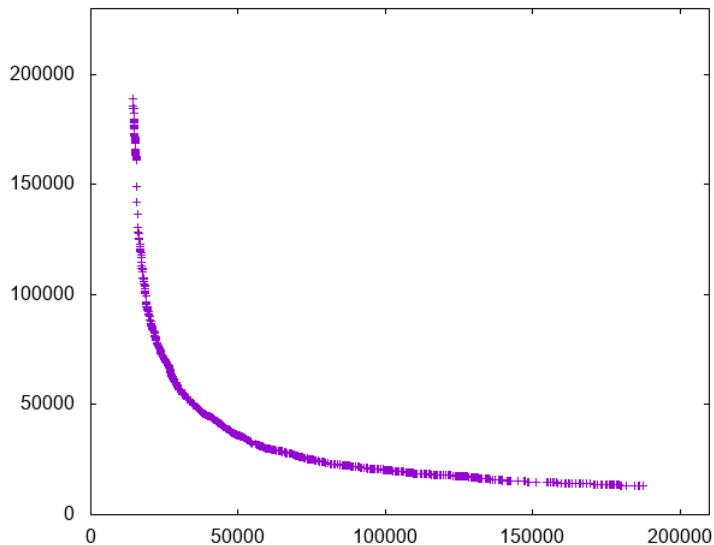
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



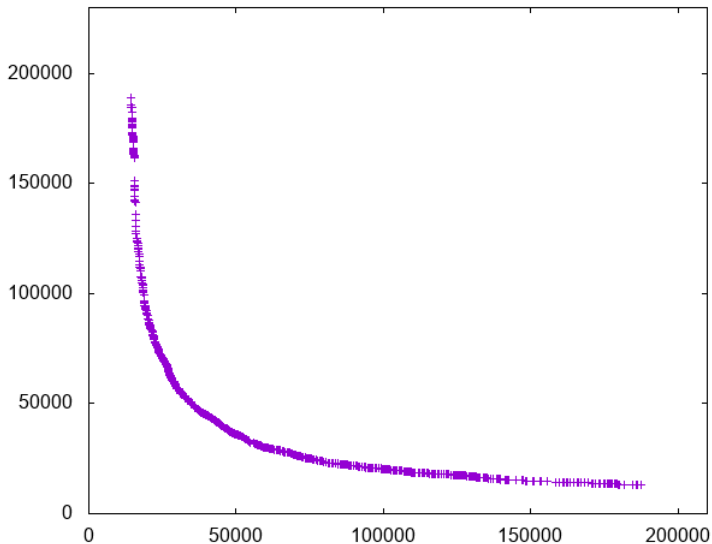
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



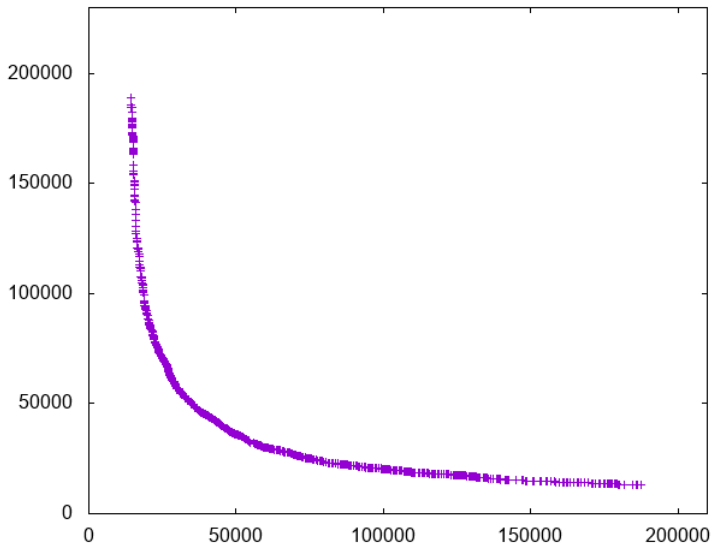
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



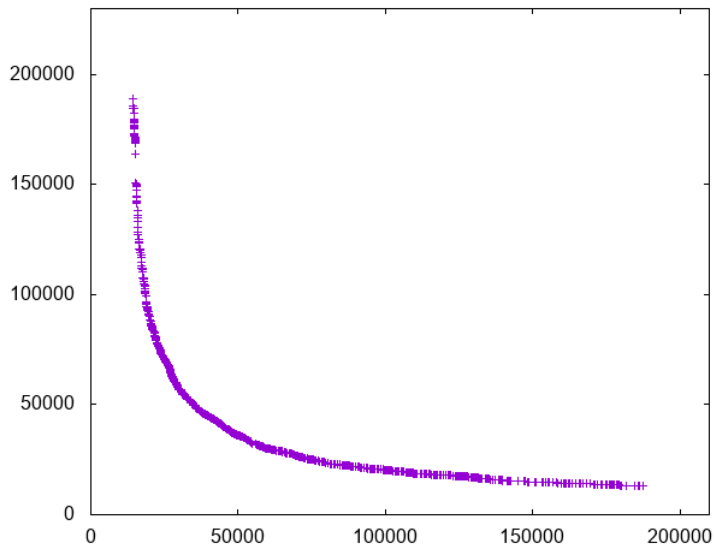
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



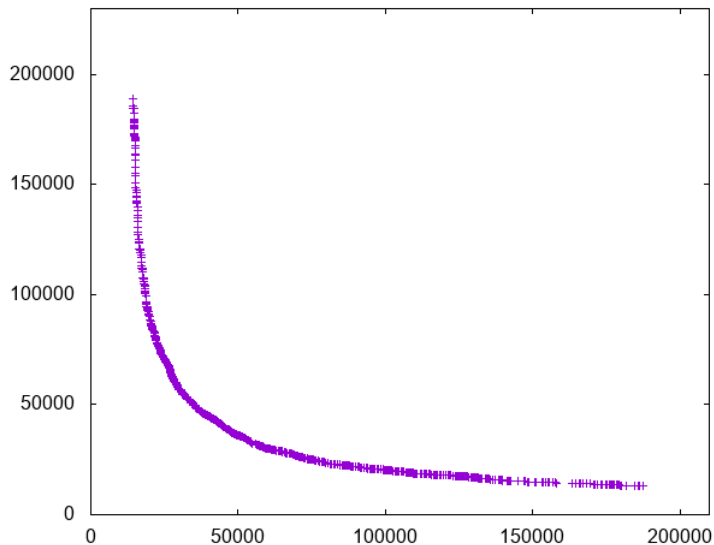
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



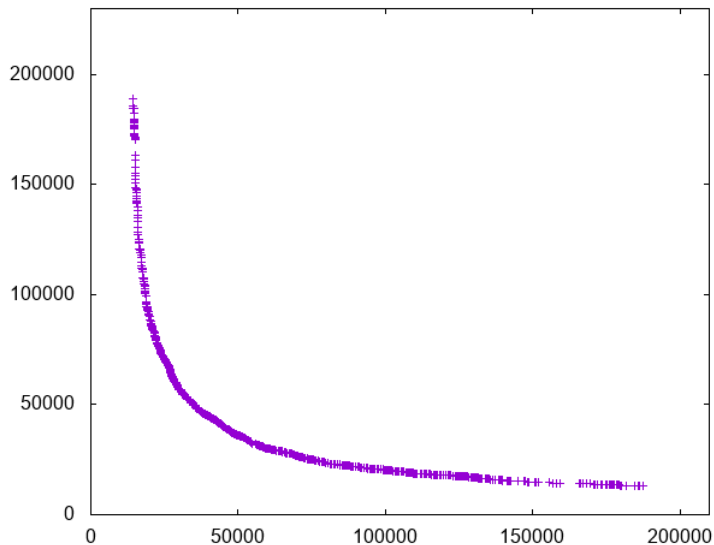
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



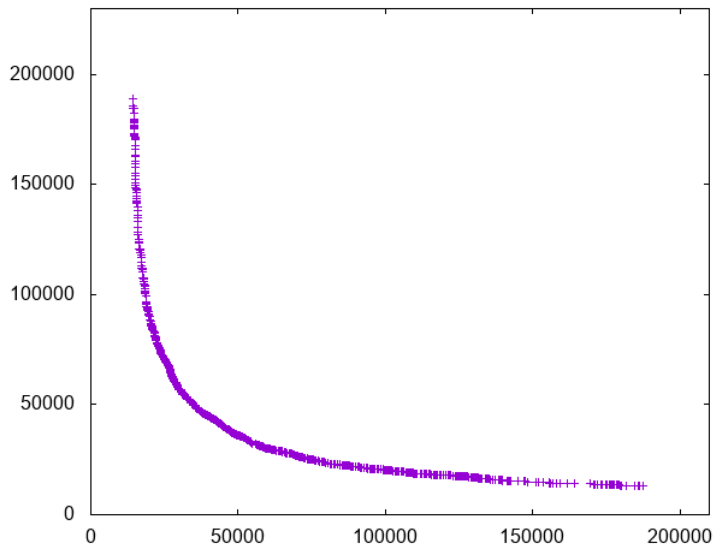
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



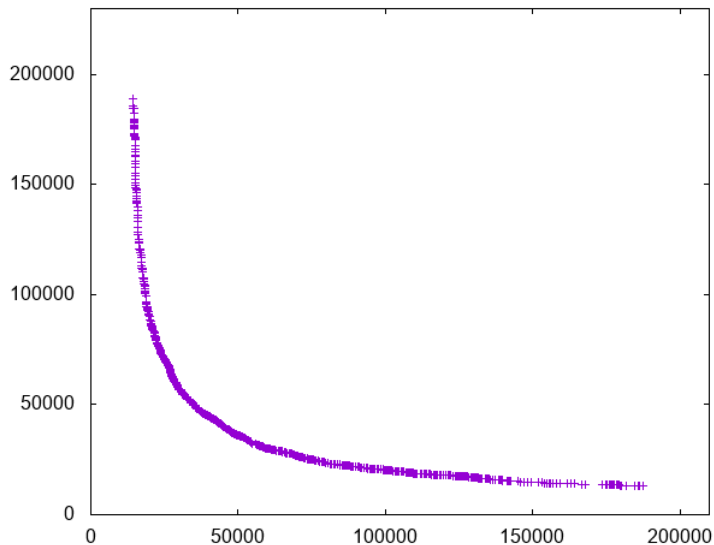
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



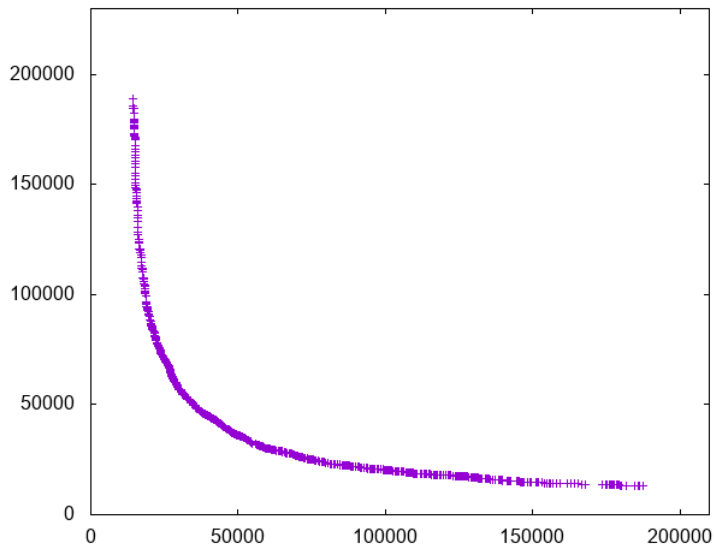
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



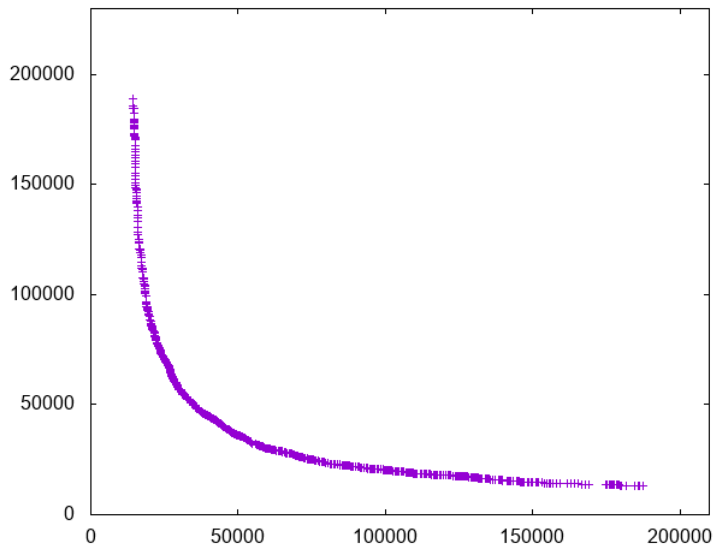
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



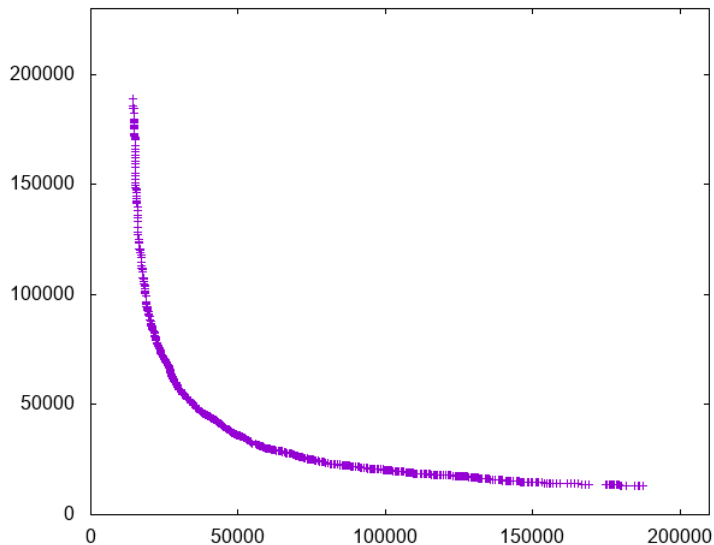
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



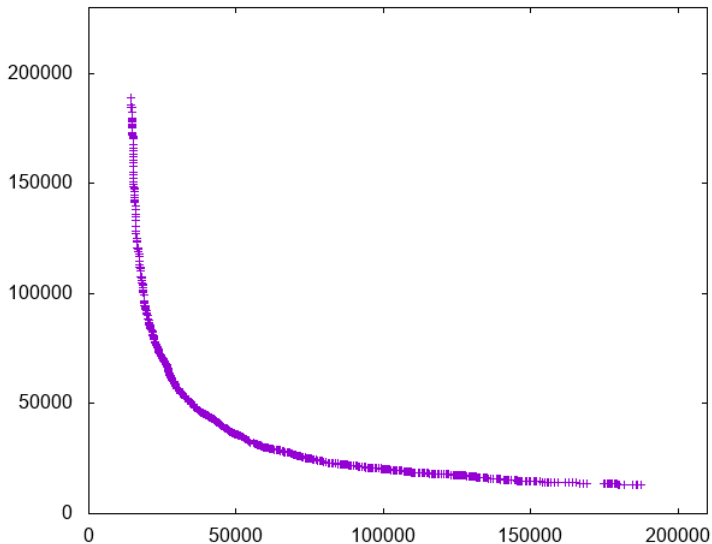
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



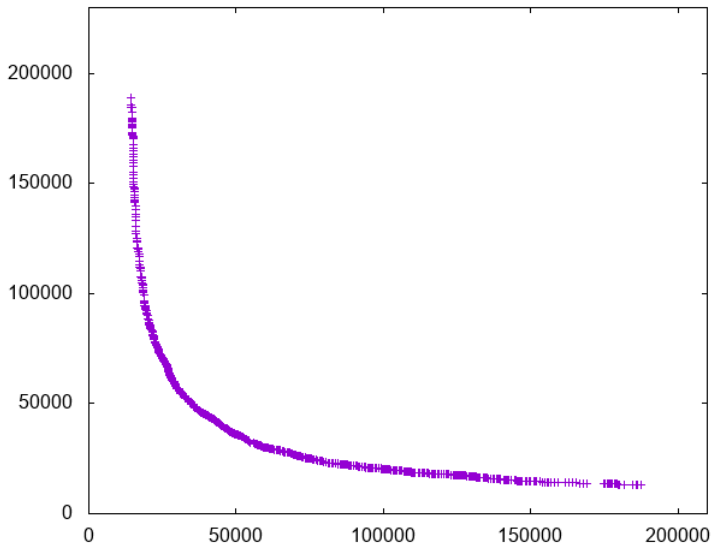
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



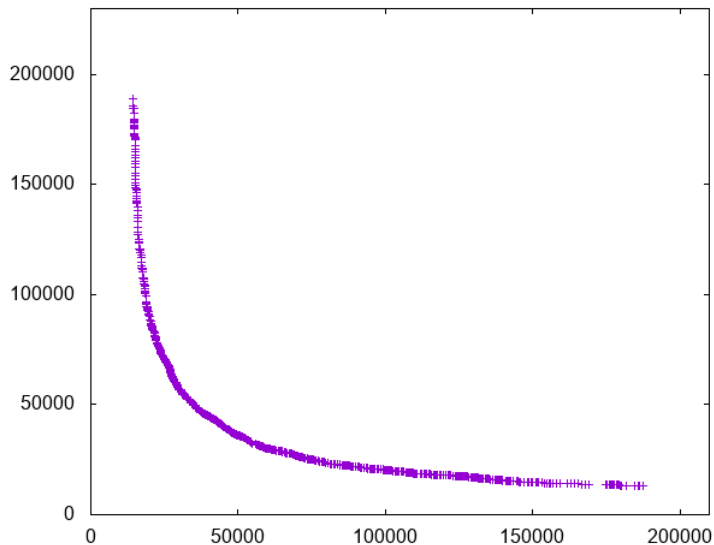
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



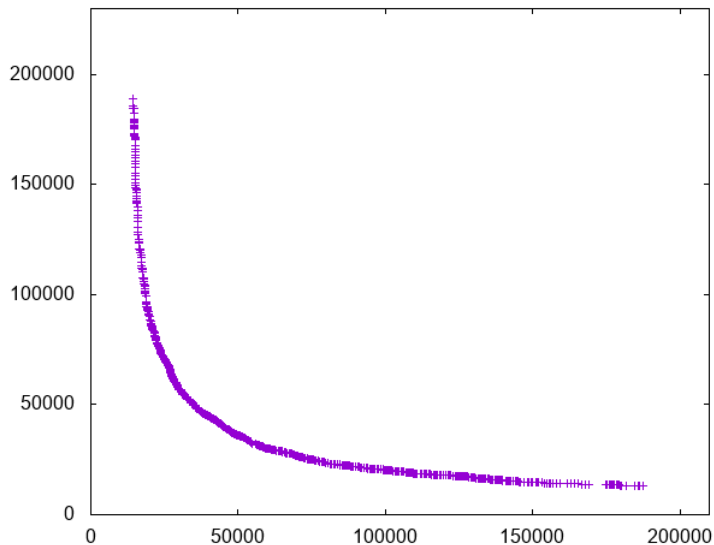
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



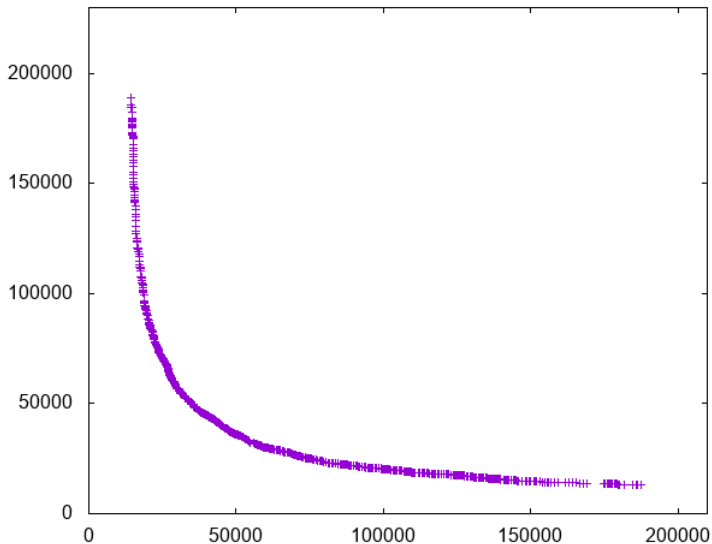
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



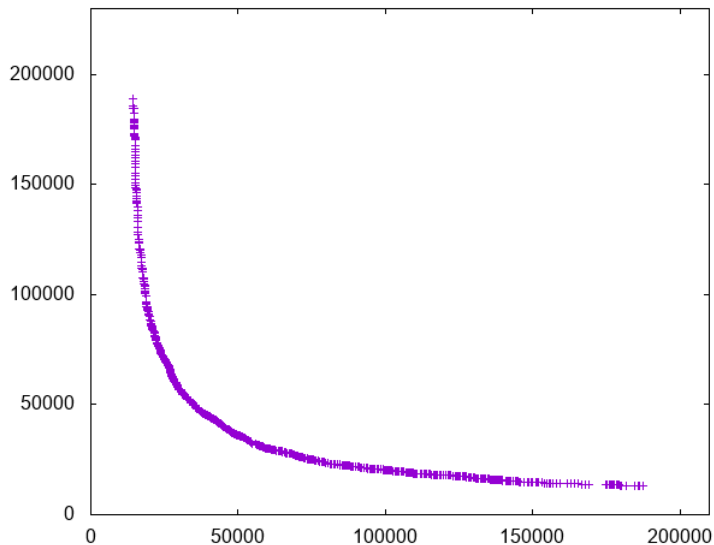
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



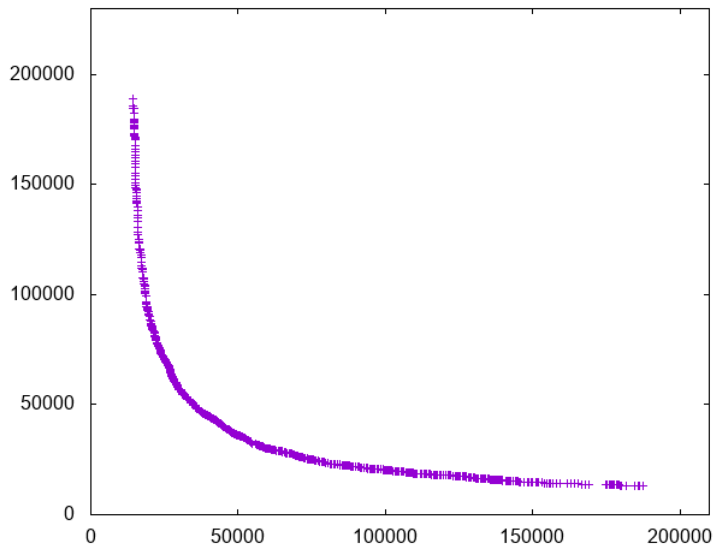
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



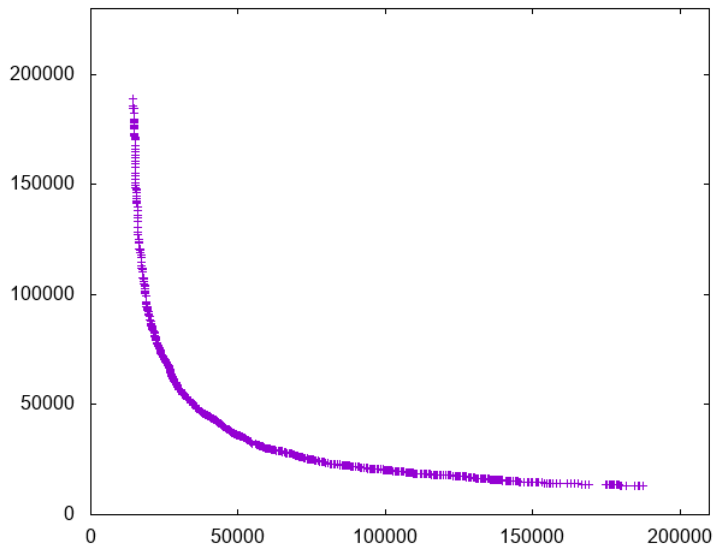
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



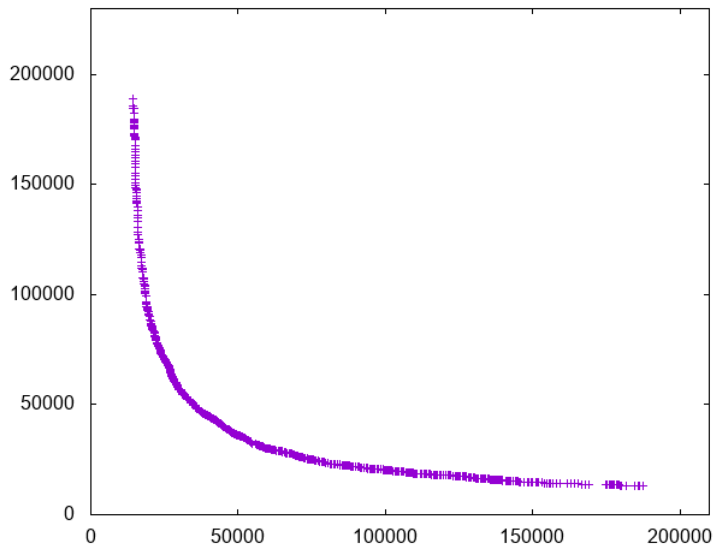
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



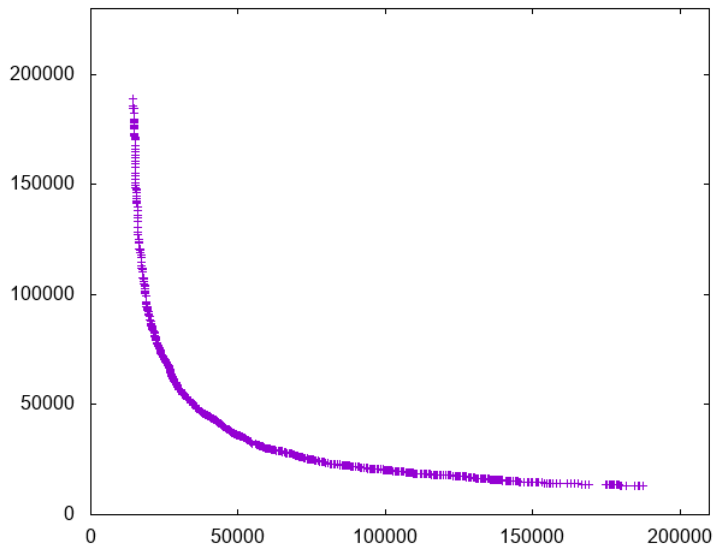
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



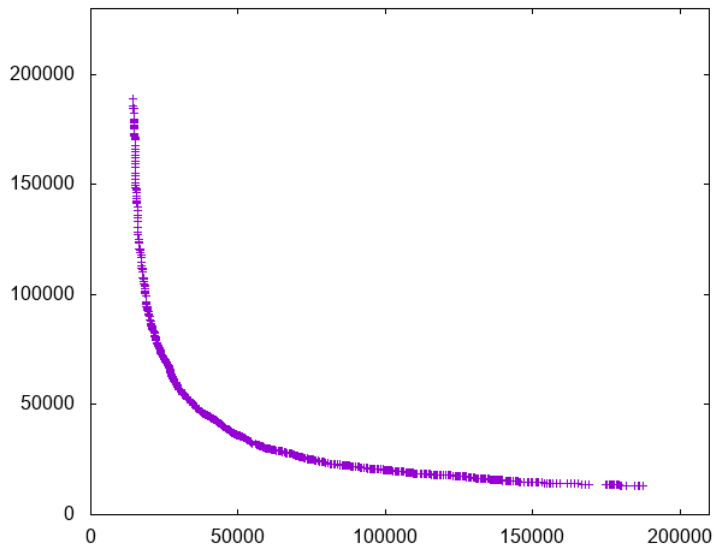
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



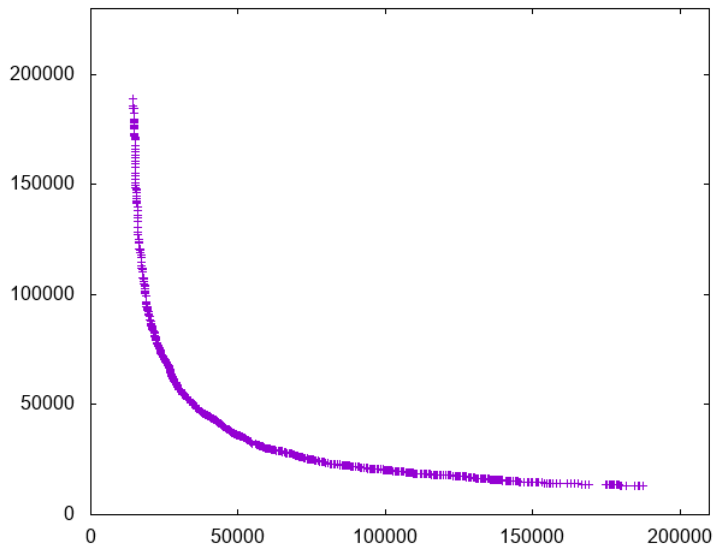
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



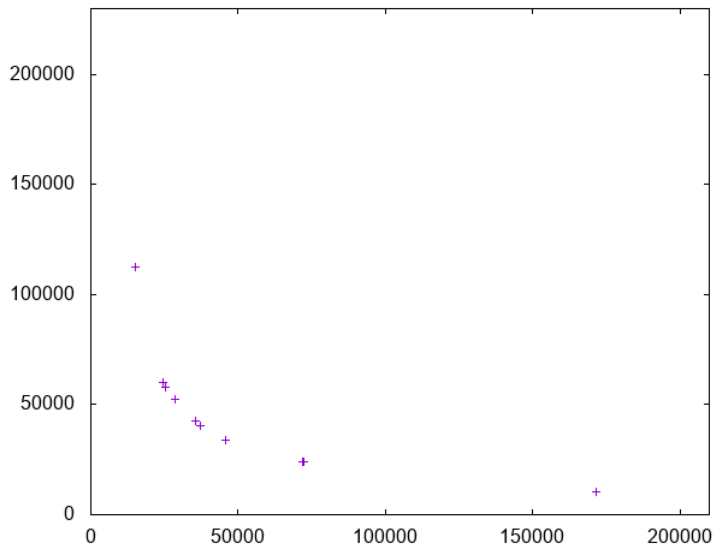
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



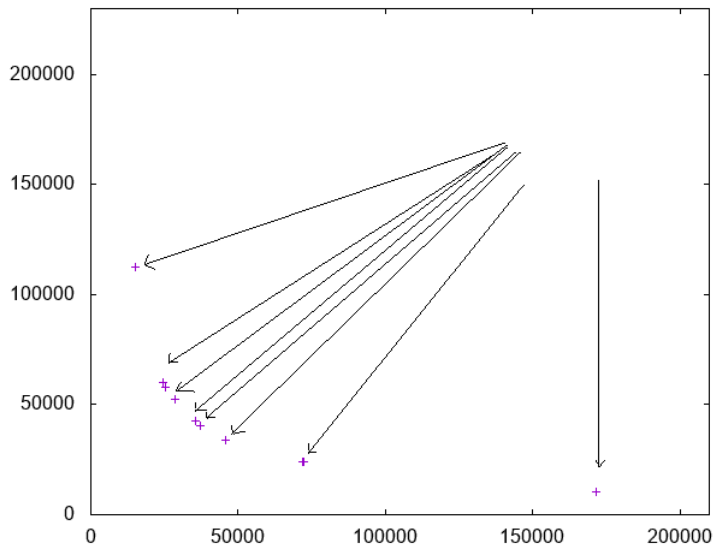
Pareto Local Search (TSP, 2 Obj, 100 cities, 1 Rd)



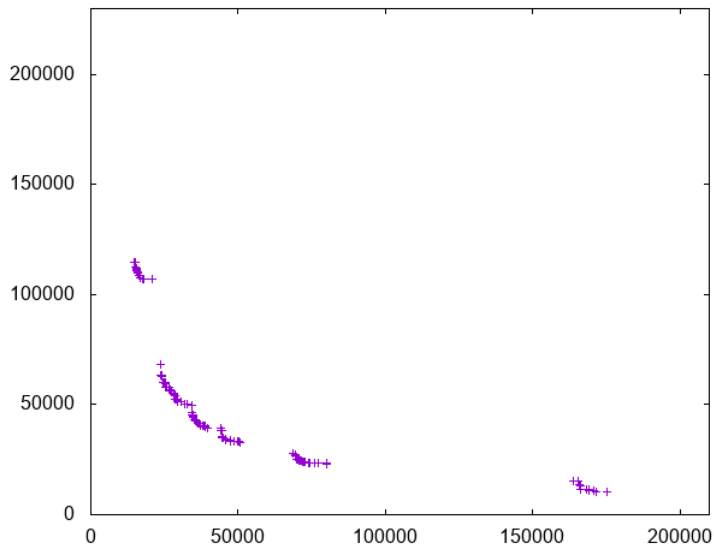
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



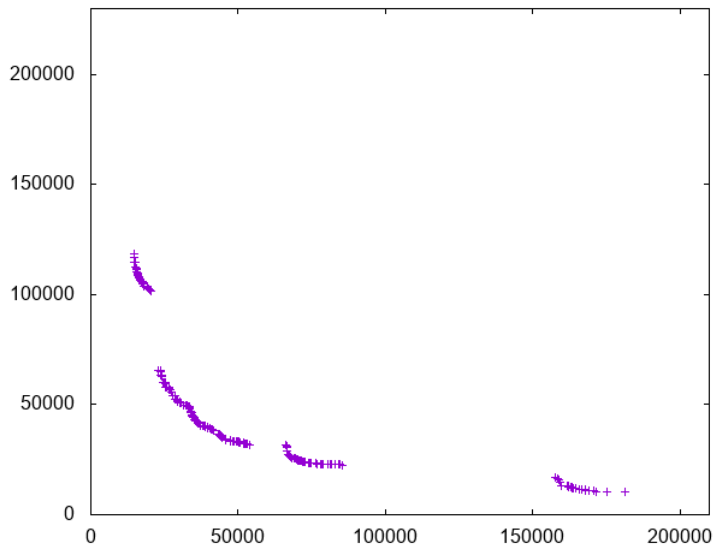
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



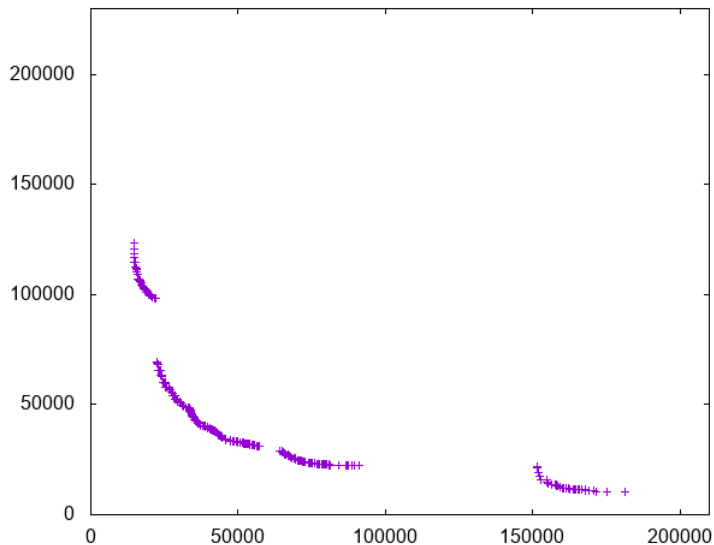
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



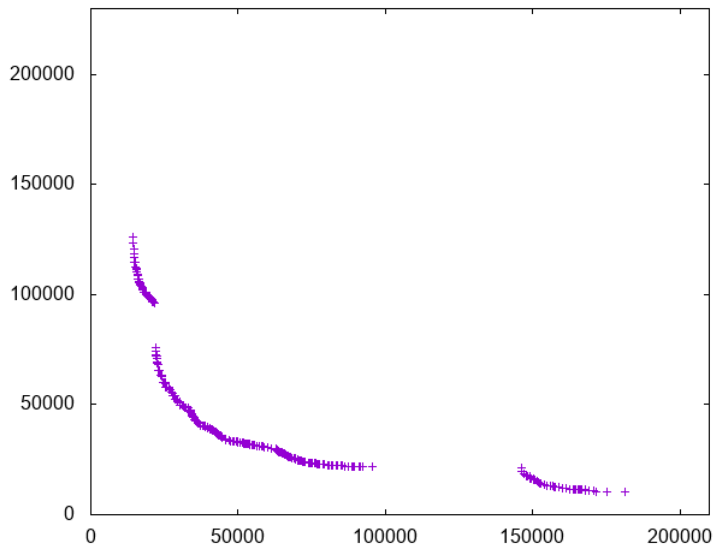
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



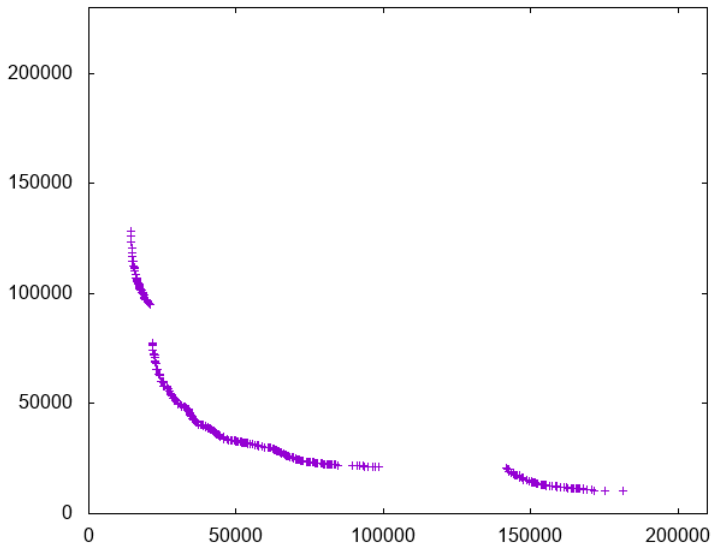
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



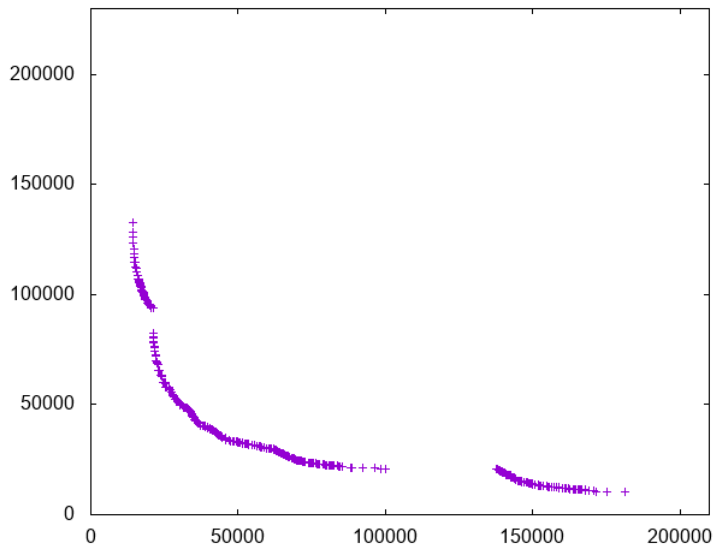
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



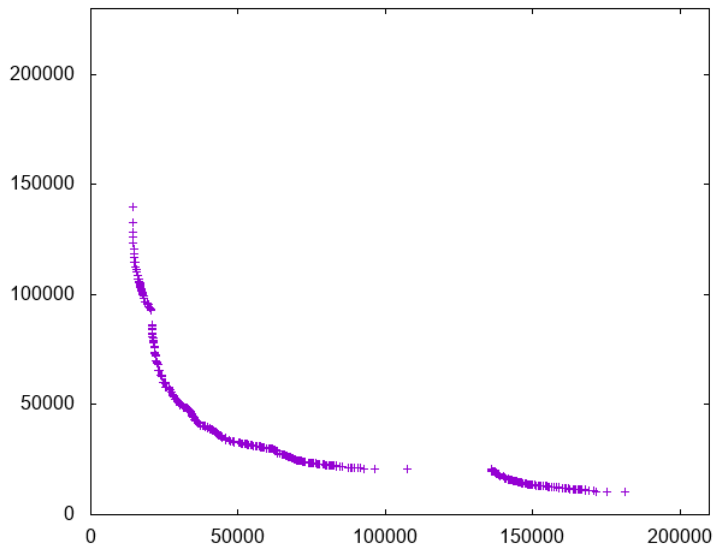
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



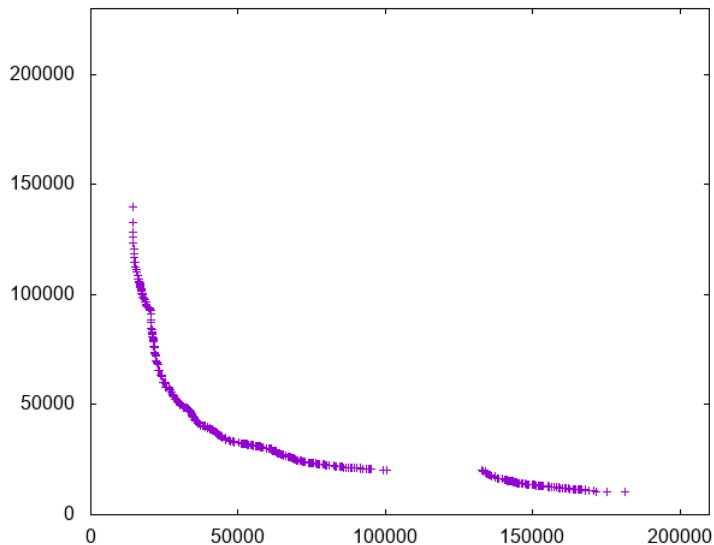
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



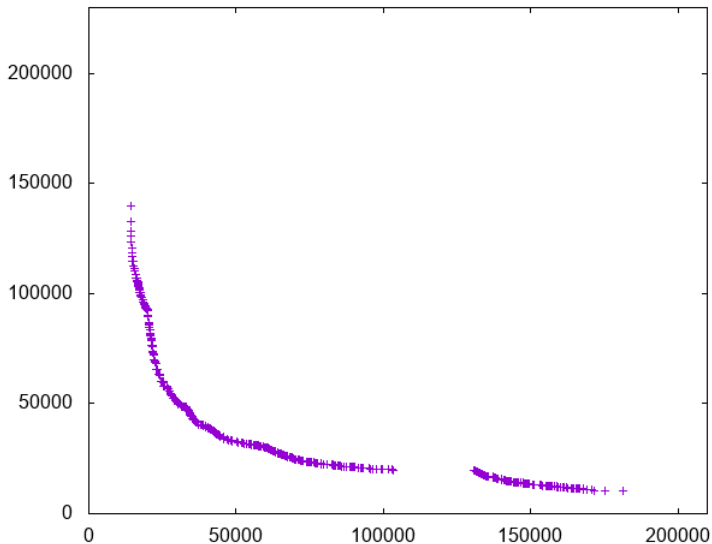
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



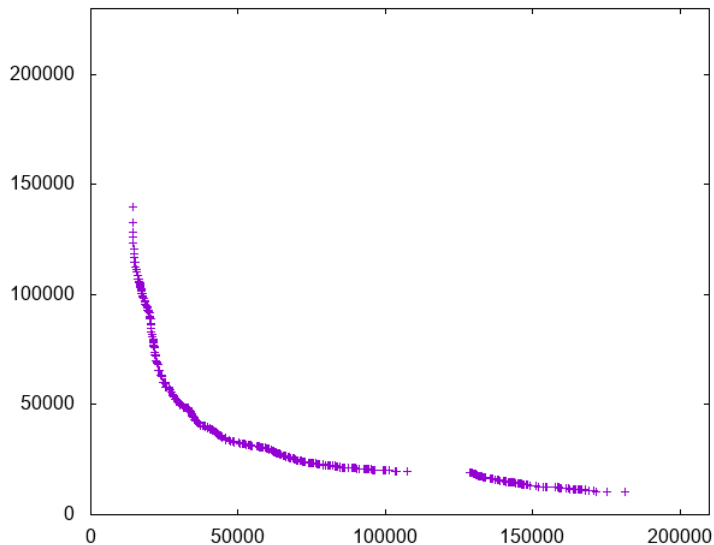
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



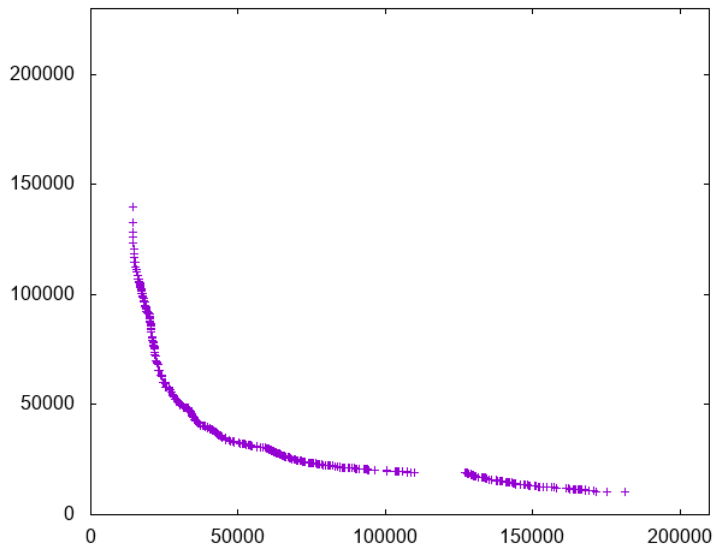
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



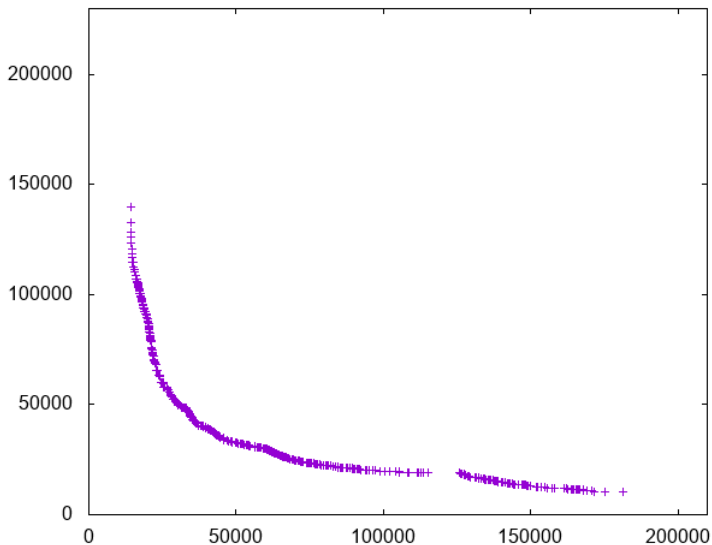
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



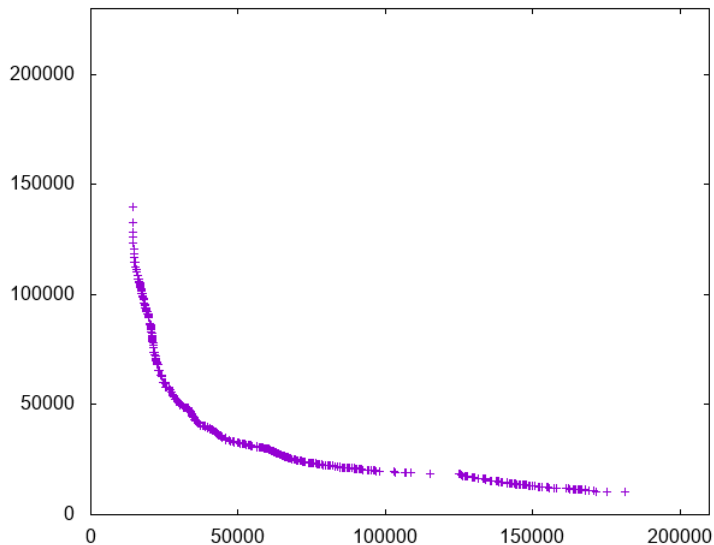
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



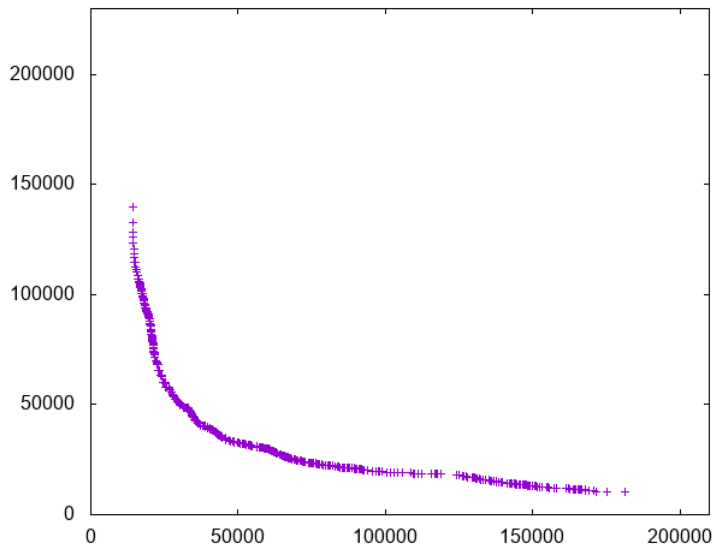
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



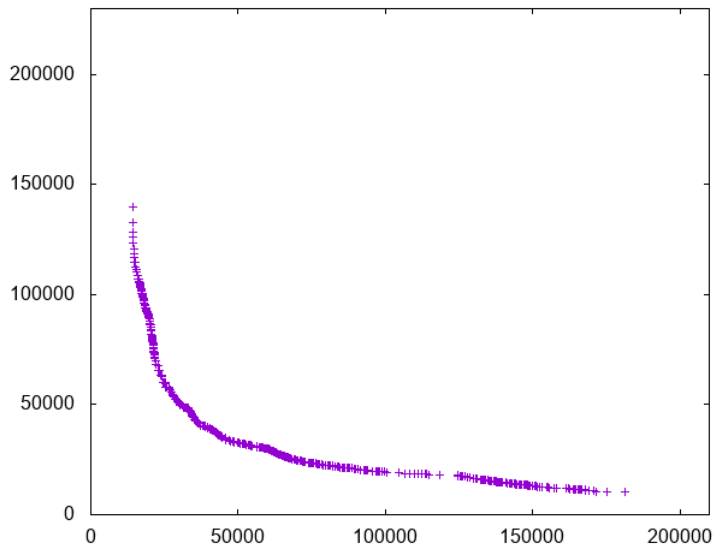
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



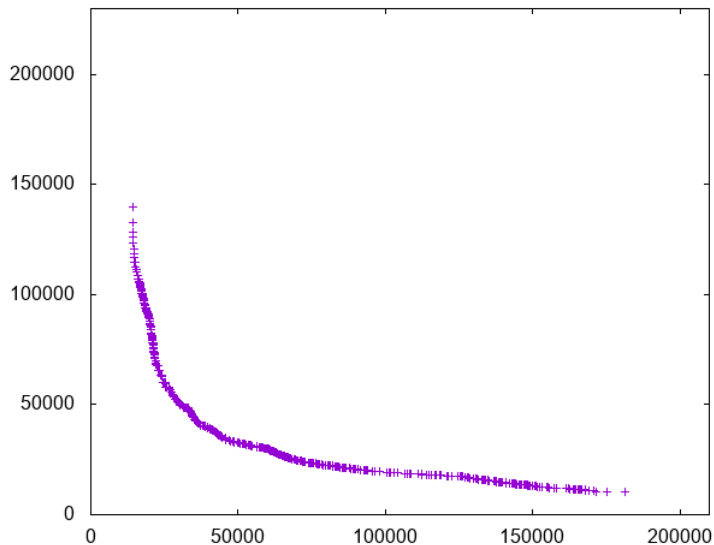
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



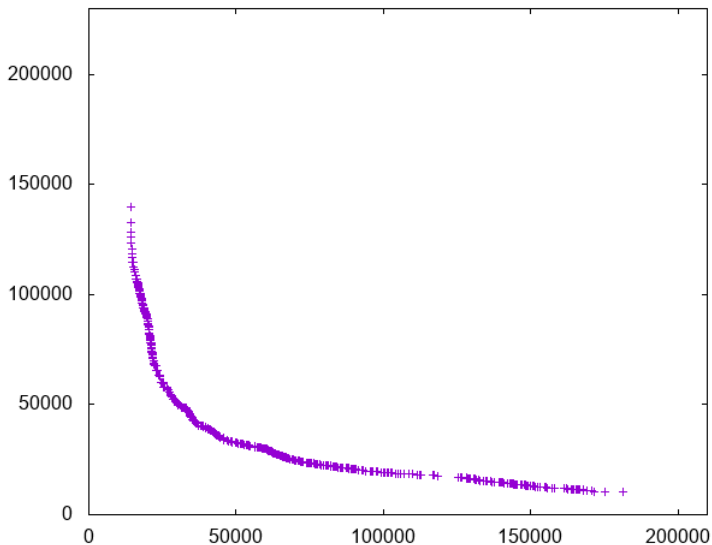
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



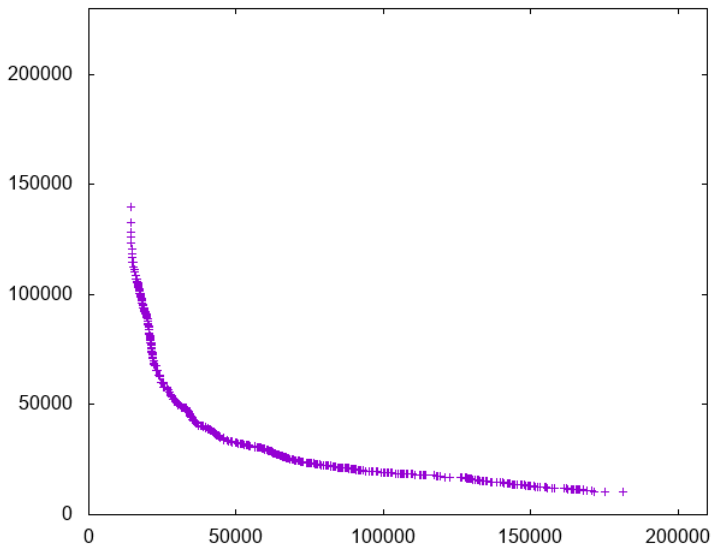
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



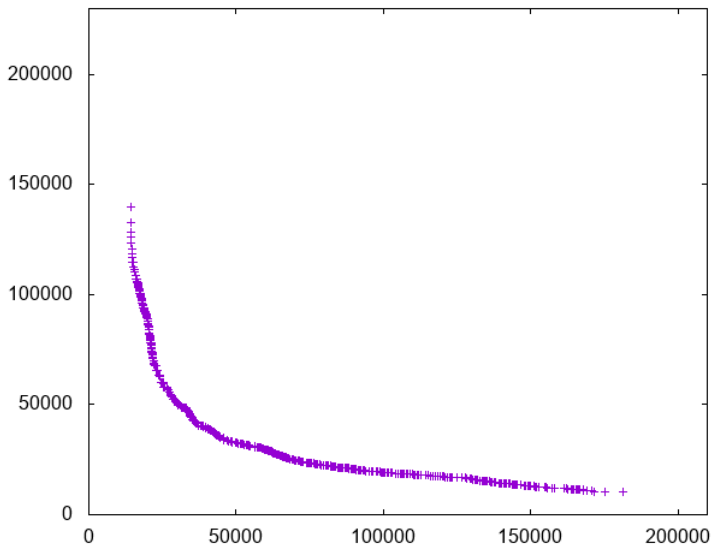
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



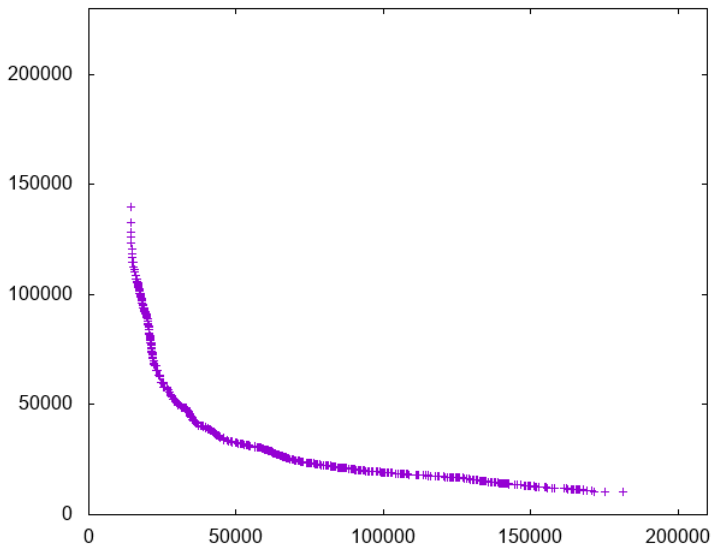
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



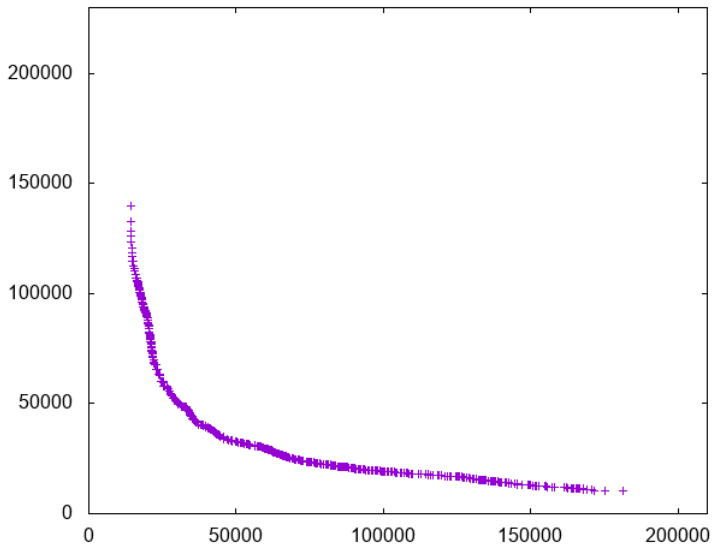
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



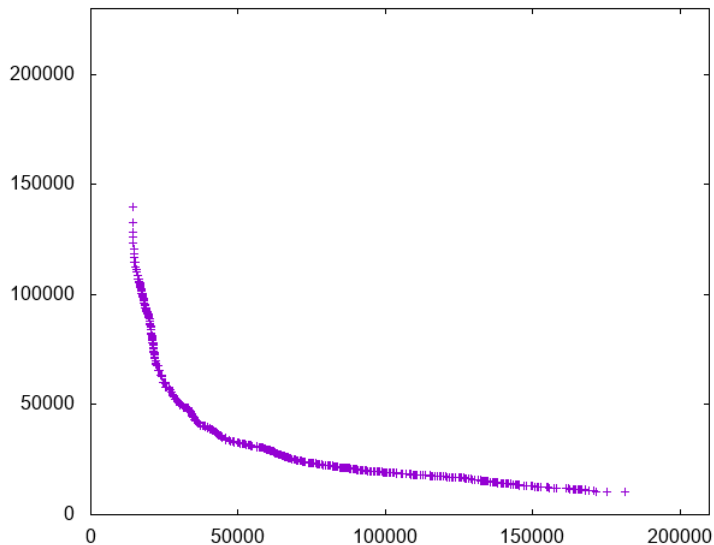
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



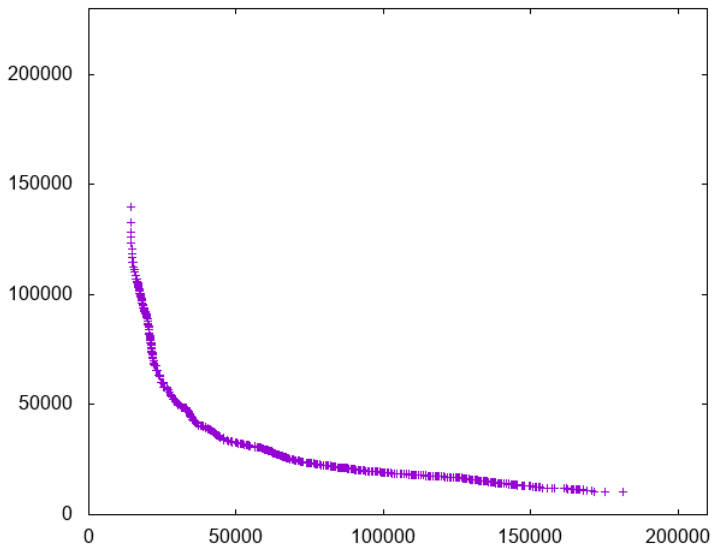
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



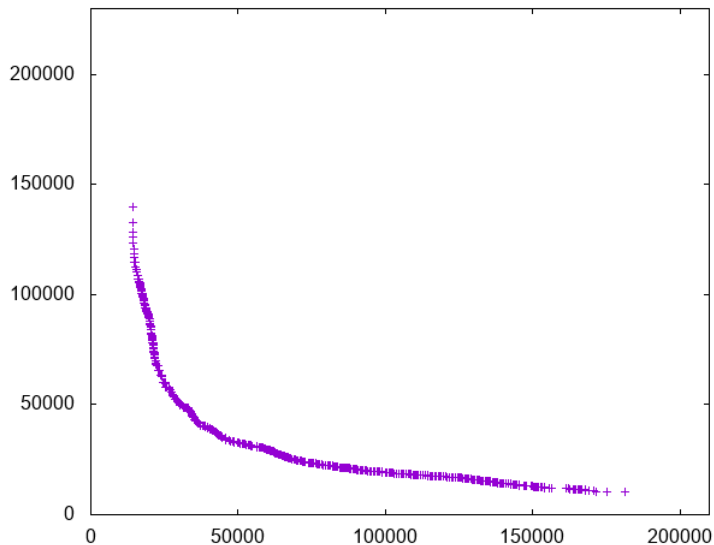
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



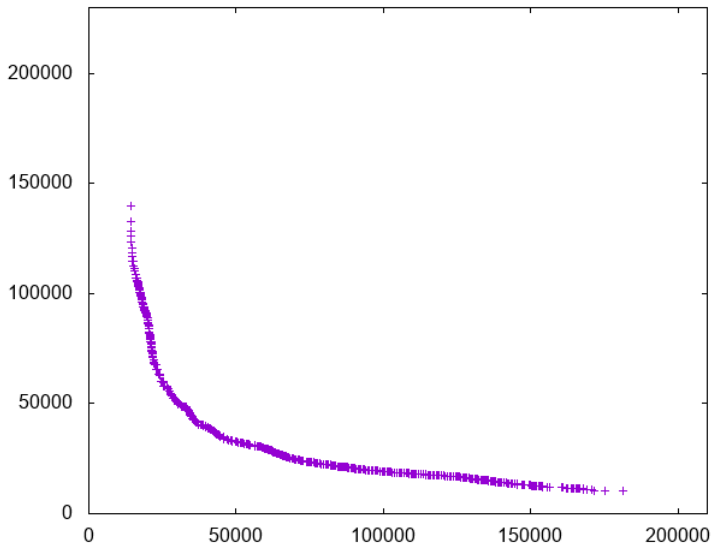
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



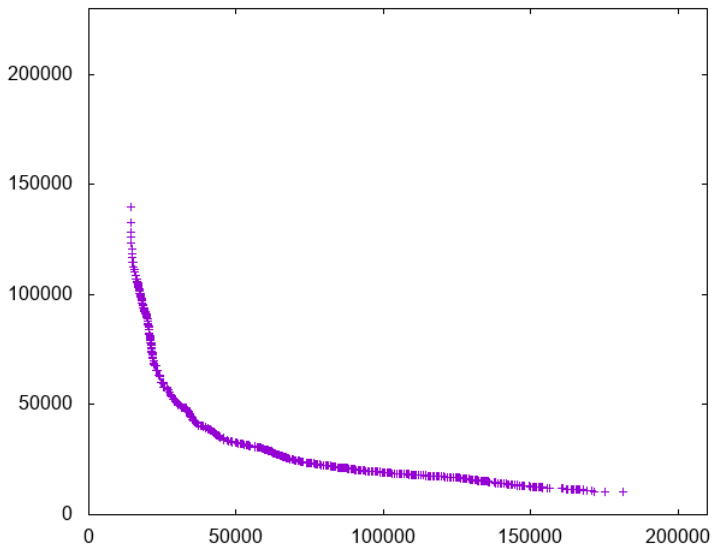
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



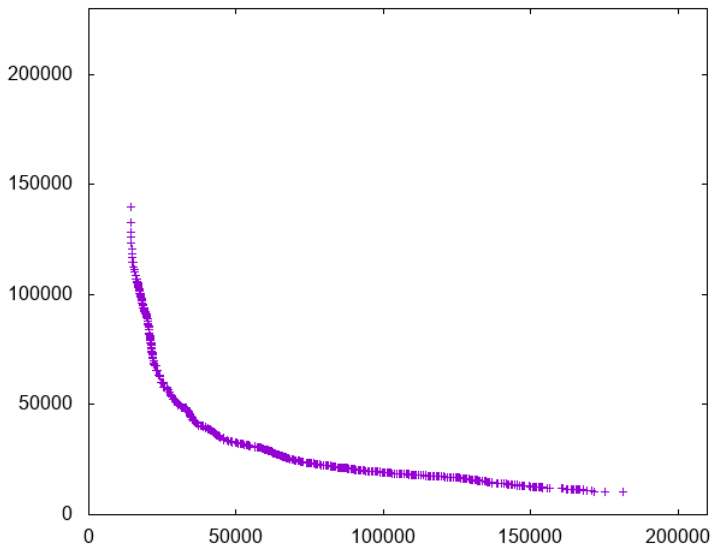
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



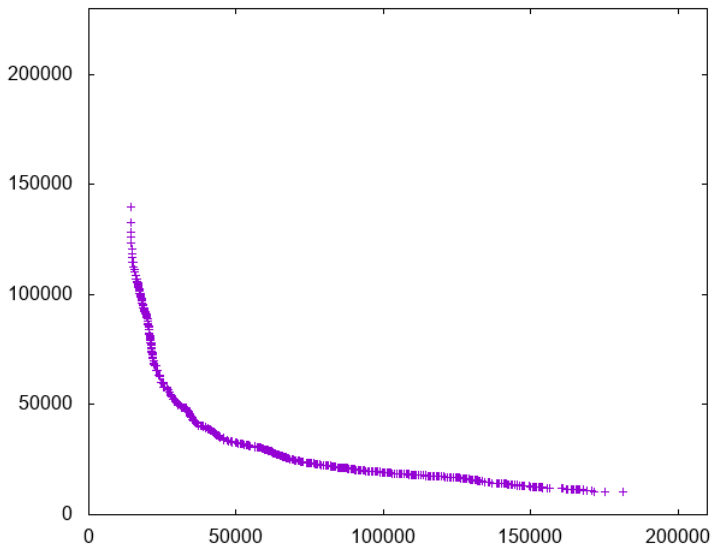
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



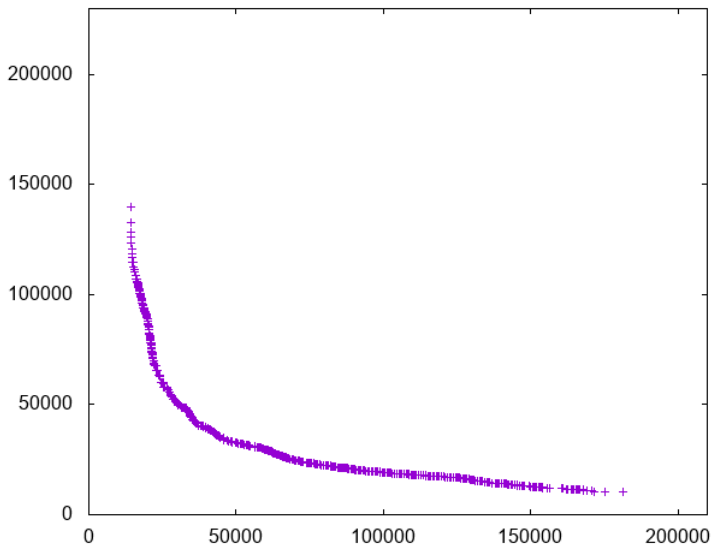
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



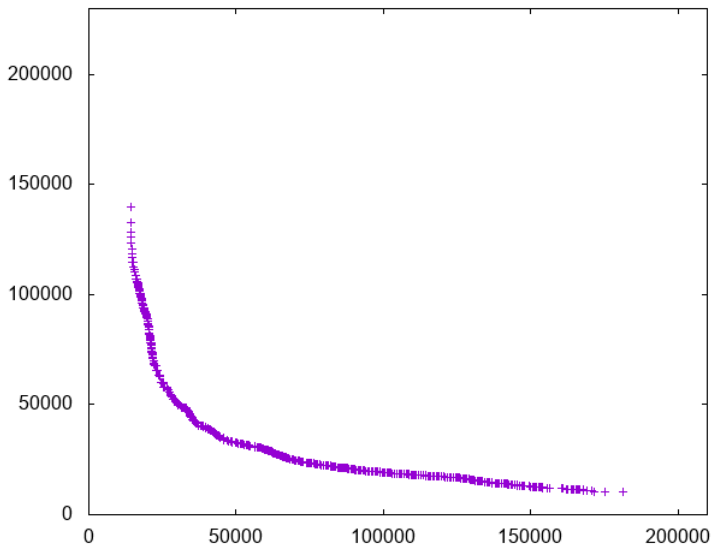
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



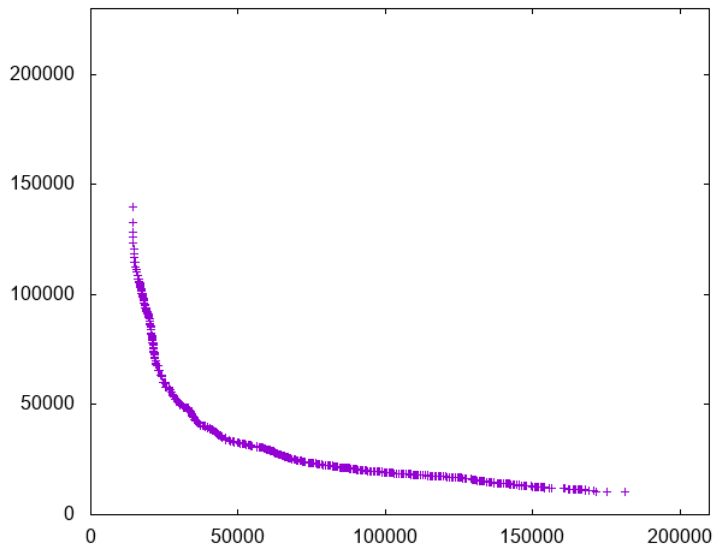
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



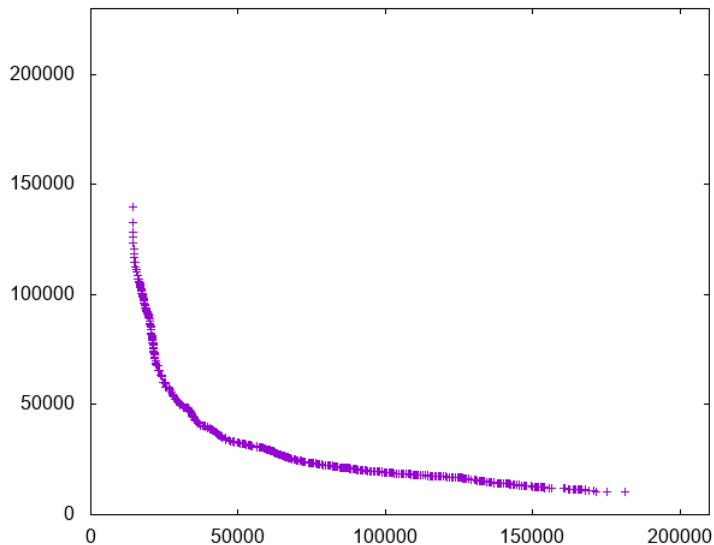
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



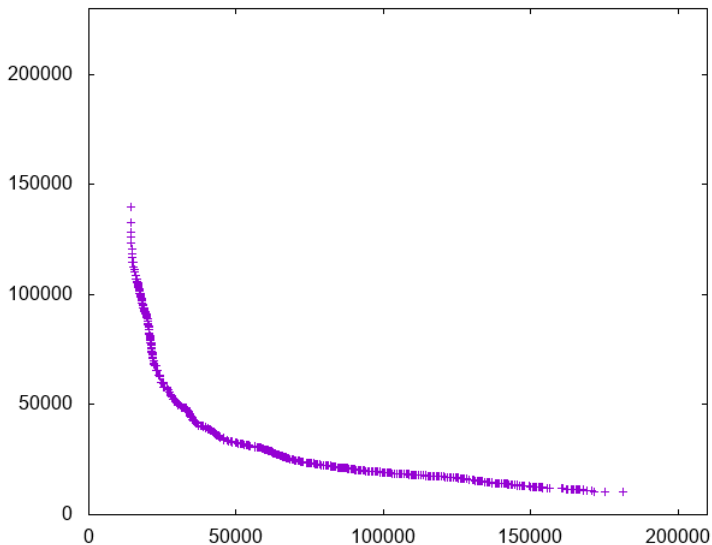
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



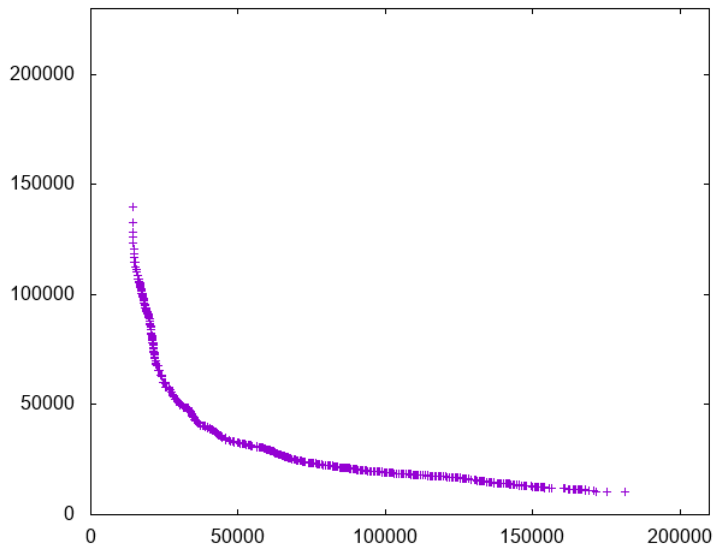
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



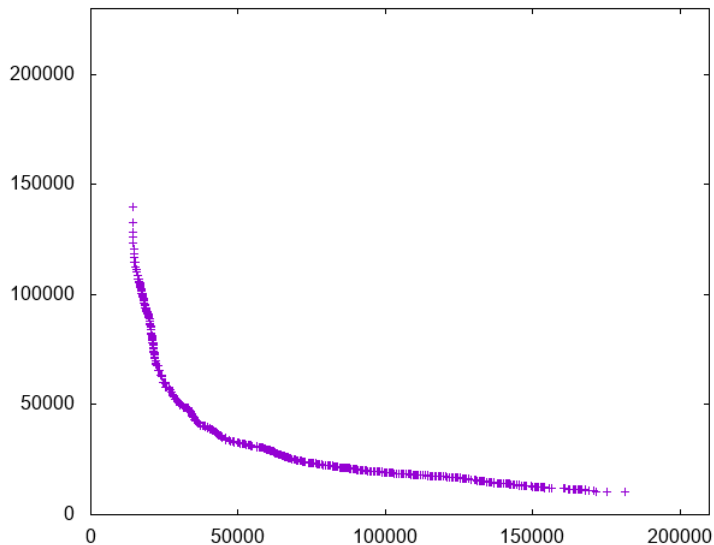
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



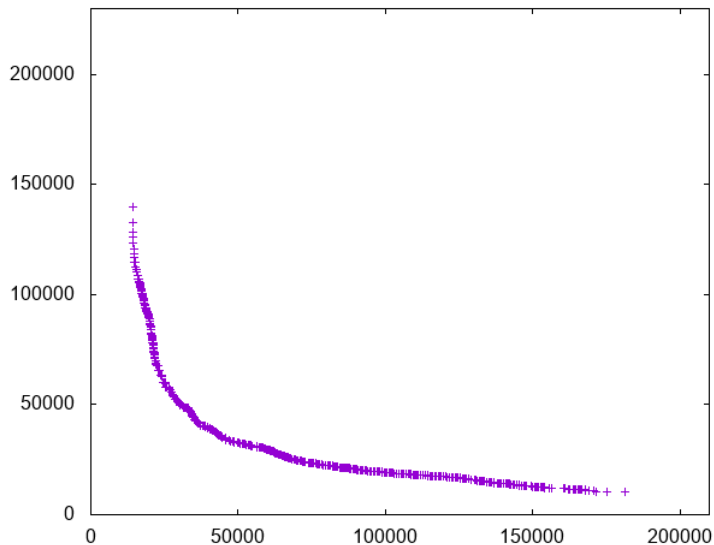
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



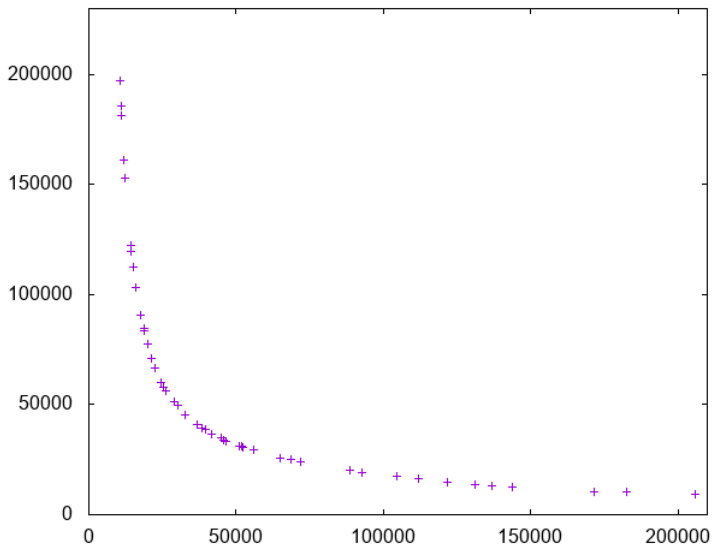
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



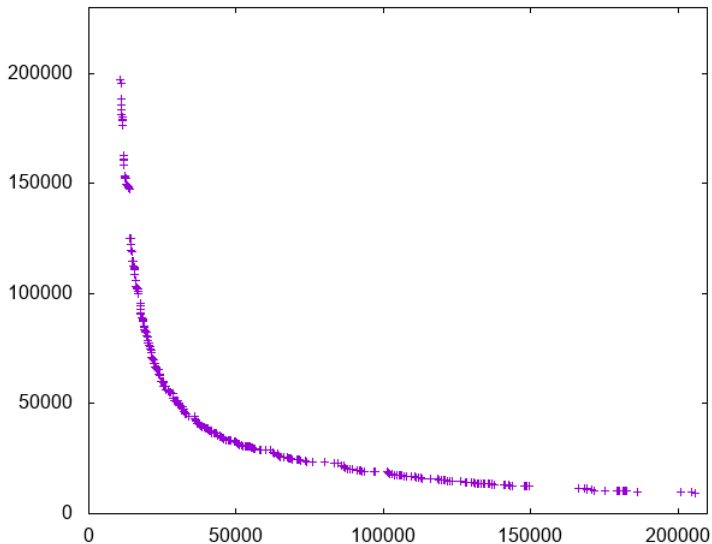
Pareto Local Search (TSP, 2 Obj, 100 cities, 10 WS)



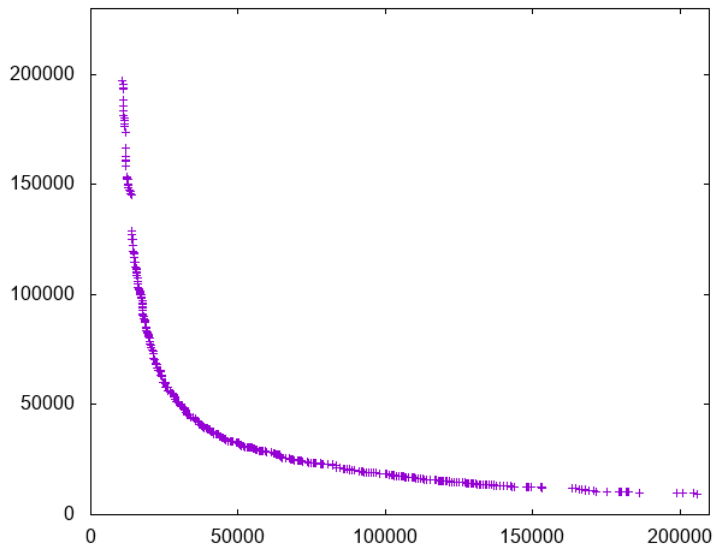
Pareto Local Search (TSP, 2 Obj, 100 cities, 100 WS)



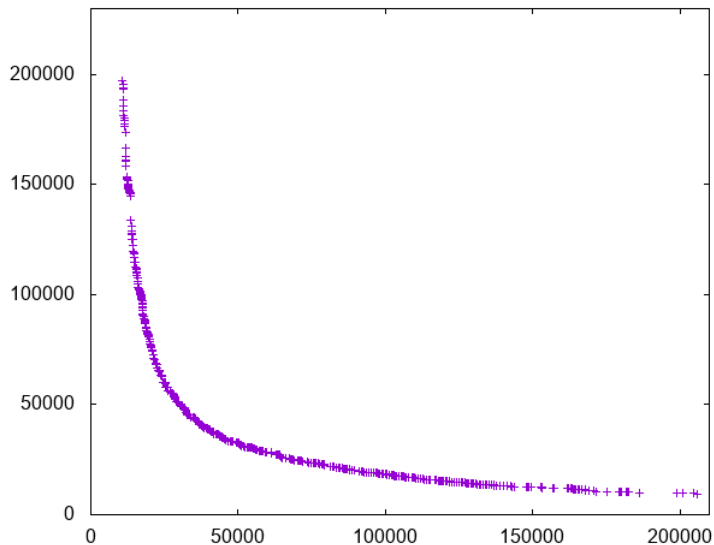
Pareto Local Search (TSP, 2 Obj, 100 cities, 100 WS)



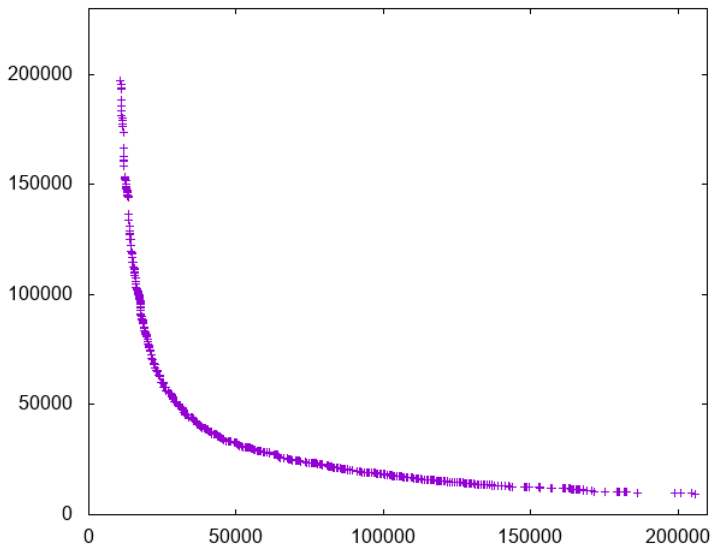
Pareto Local Search (TSP, 2 Obj, 100 cities, 100 WS)



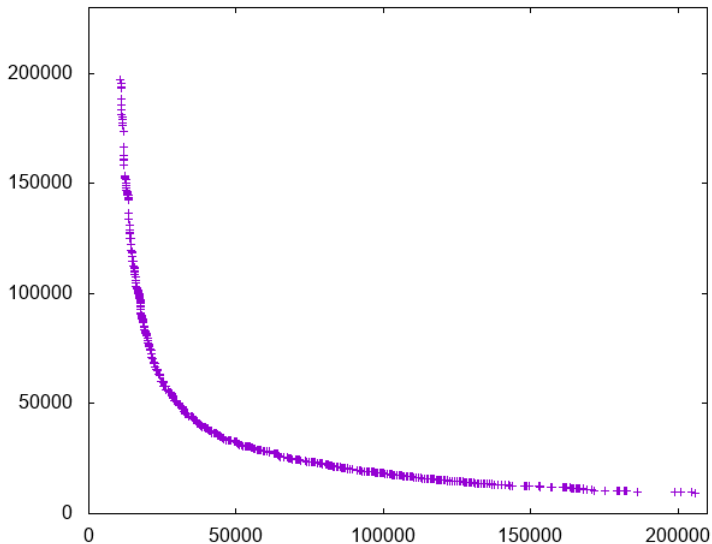
Pareto Local Search (TSP, 2 Obj, 100 cities, 100 WS)



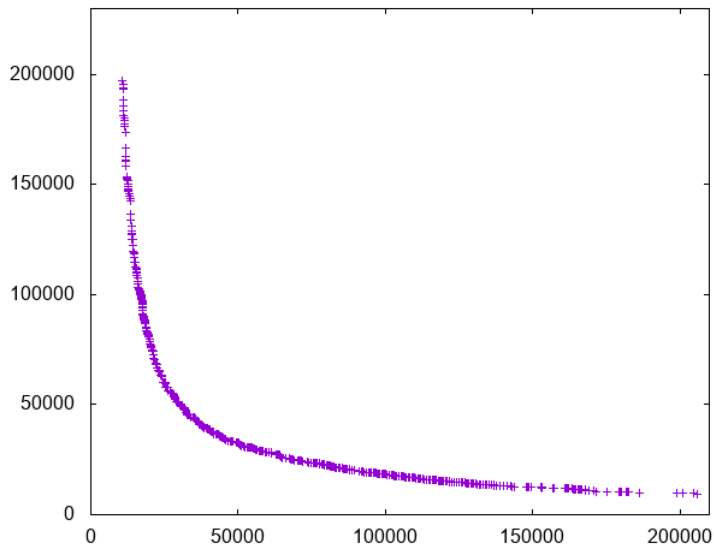
Pareto Local Search (TSP, 2 Obj, 100 cities, 100 WS)



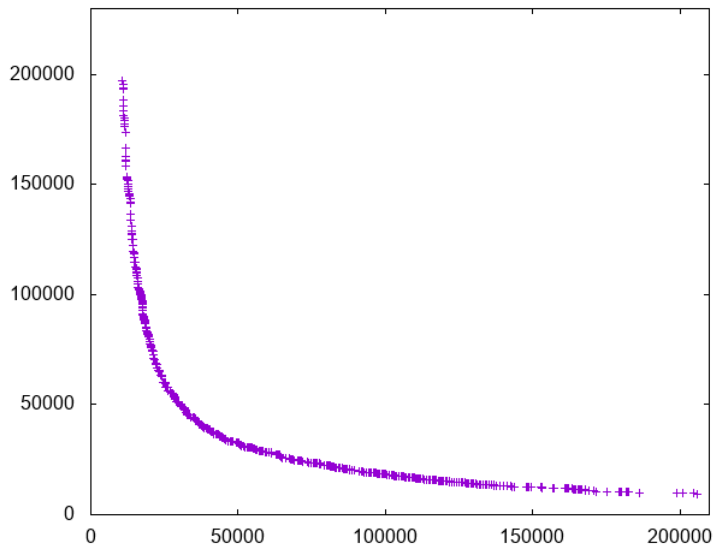
Pareto Local Search (TSP, 2 Obj, 100 cities, 100 WS)



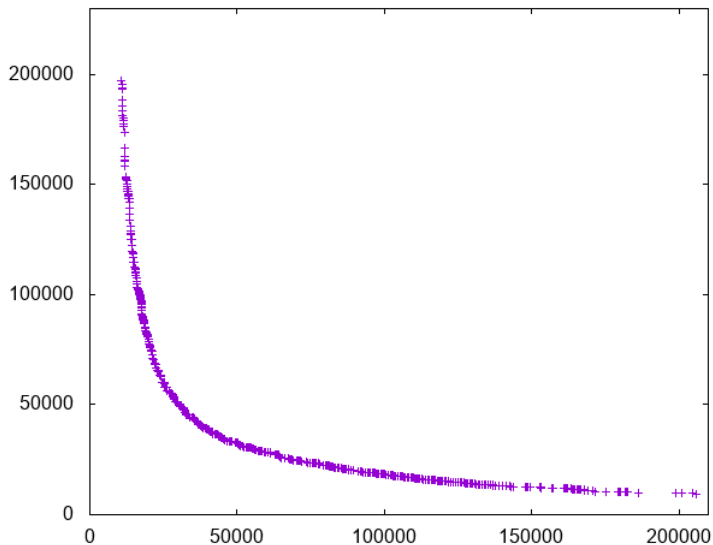
Pareto Local Search (TSP, 2 Obj, 100 cities, 100 WS)



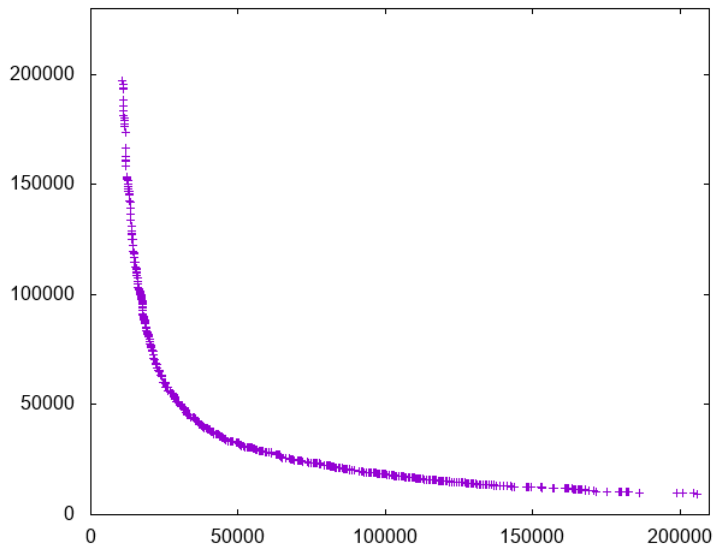
Pareto Local Search (TSP, 2 Obj, 100 cities, 100 WS)



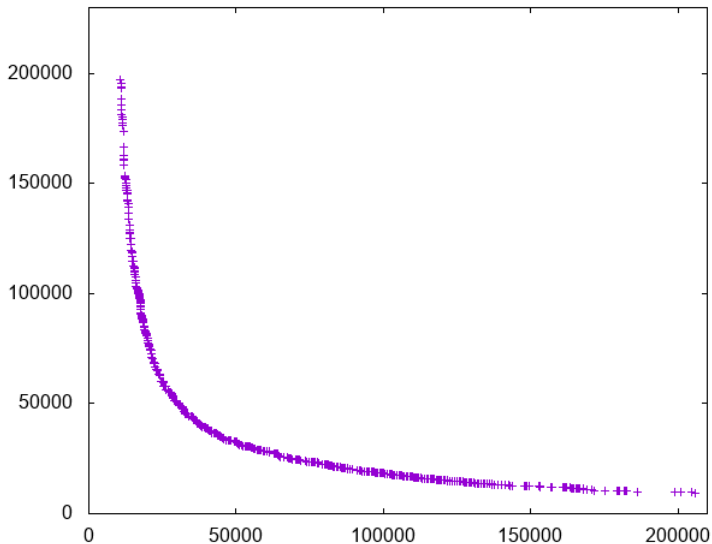
Pareto Local Search (TSP, 2 Obj, 100 cities, 100 WS)



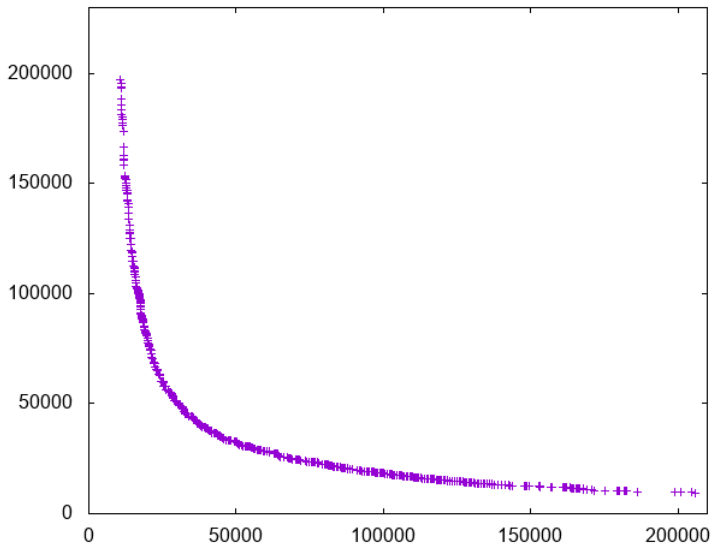
Pareto Local Search (TSP, 2 Obj, 100 cities, 100 WS)



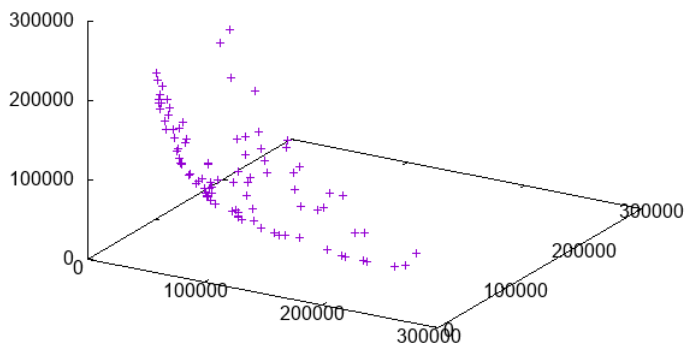
Pareto Local Search (TSP, 2 Obj, 100 cities, 100 WS)



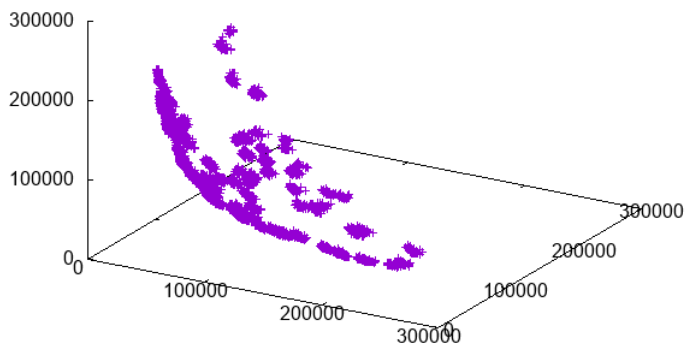
Pareto Local Search (TSP, 2 Obj, 100 cities, 100 WS)



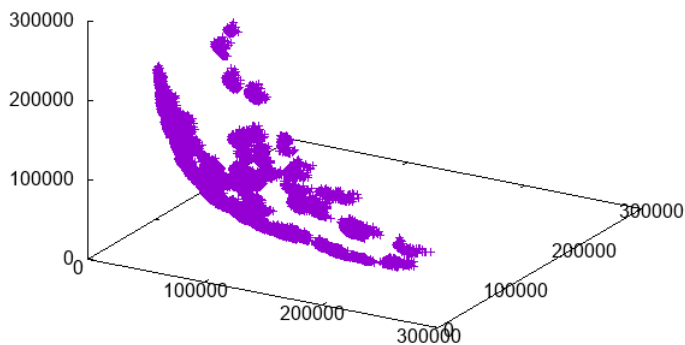
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



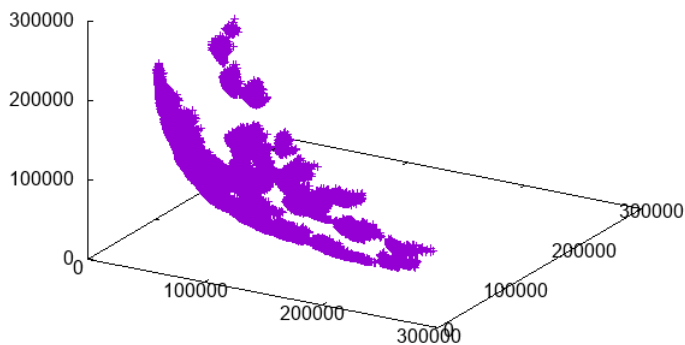
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



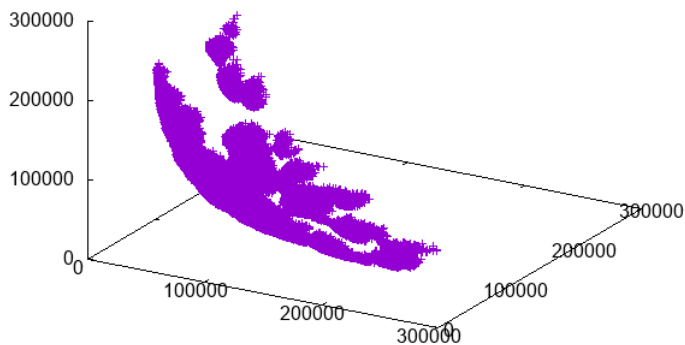
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



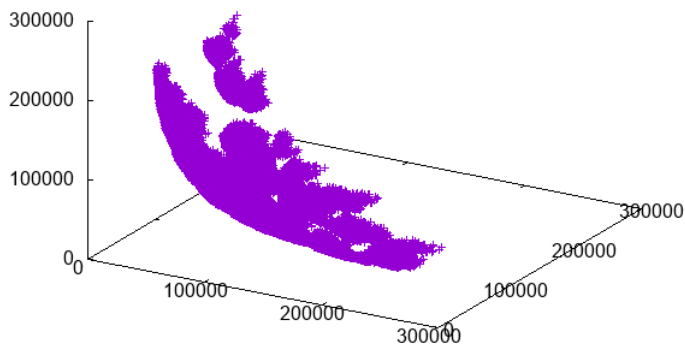
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



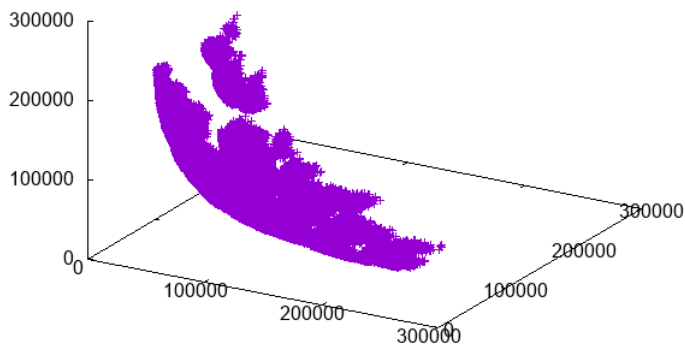
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



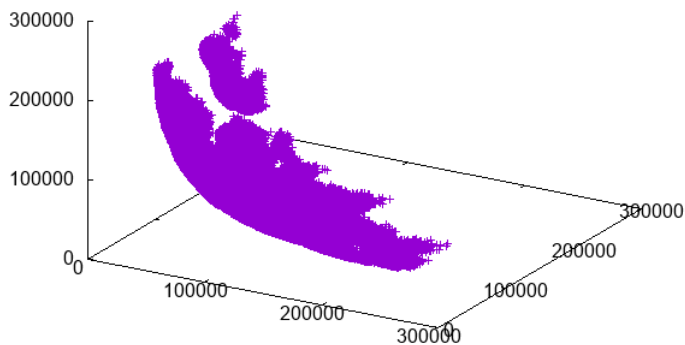
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



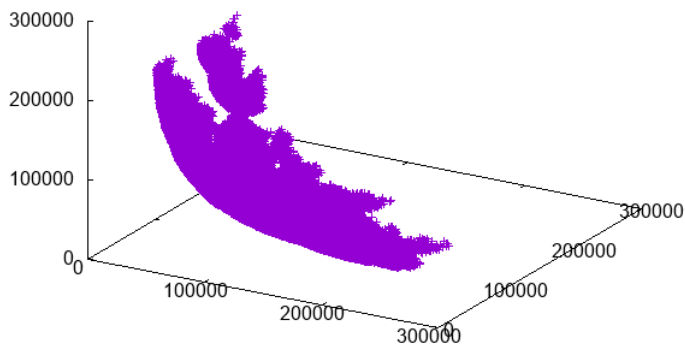
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



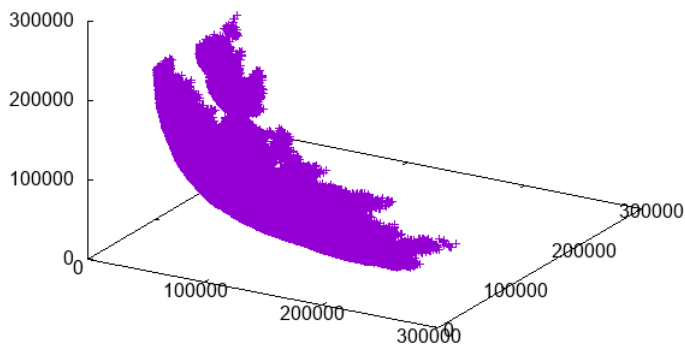
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



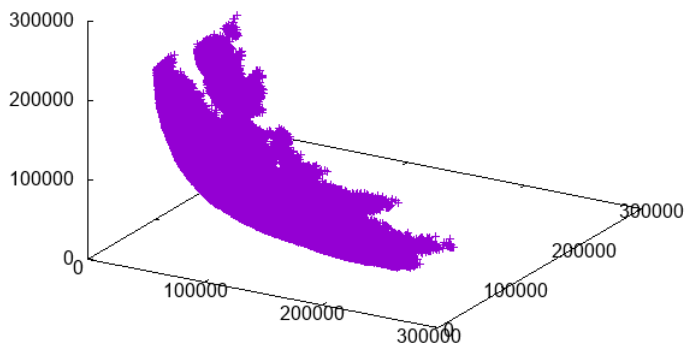
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



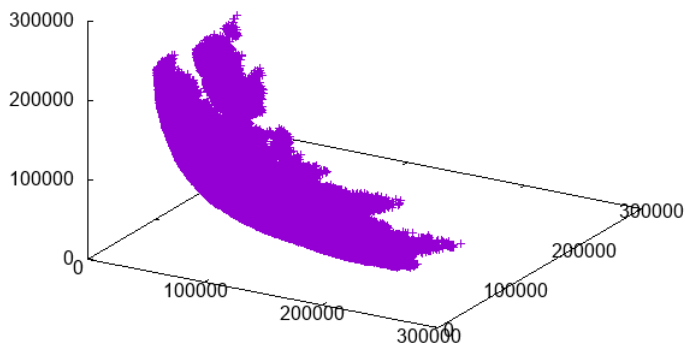
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



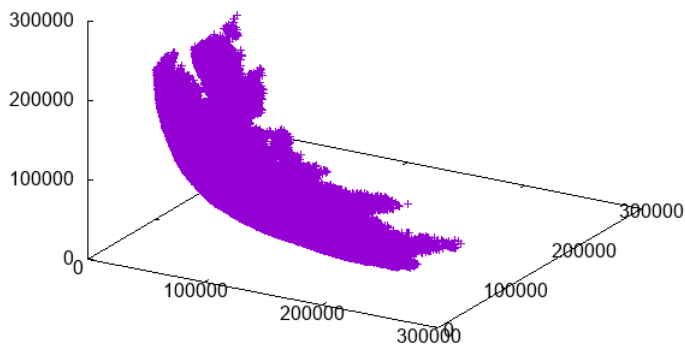
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



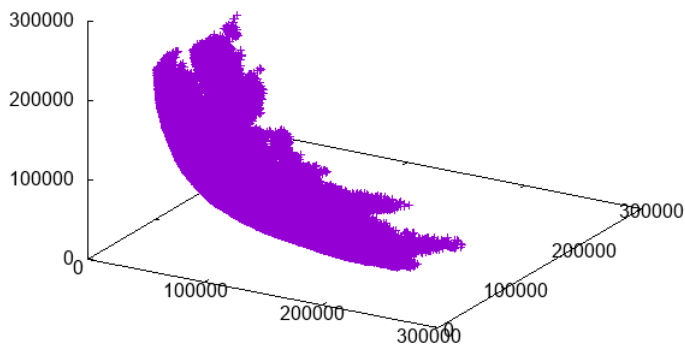
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



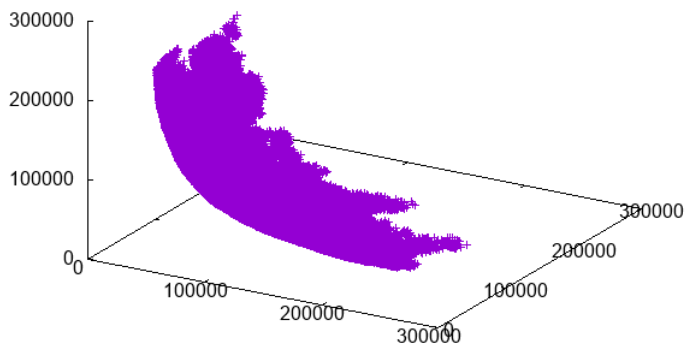
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



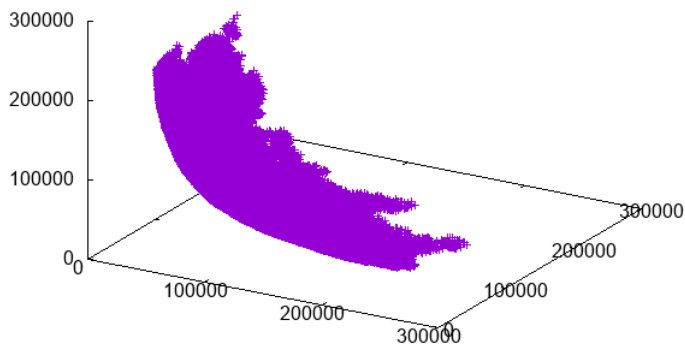
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



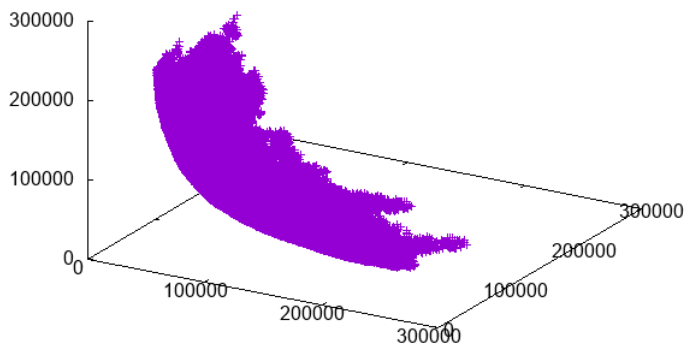
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



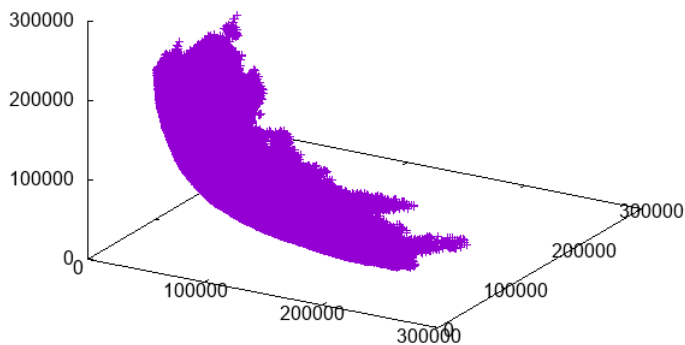
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



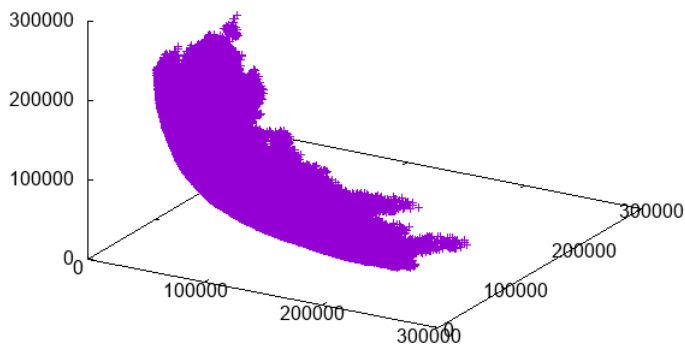
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



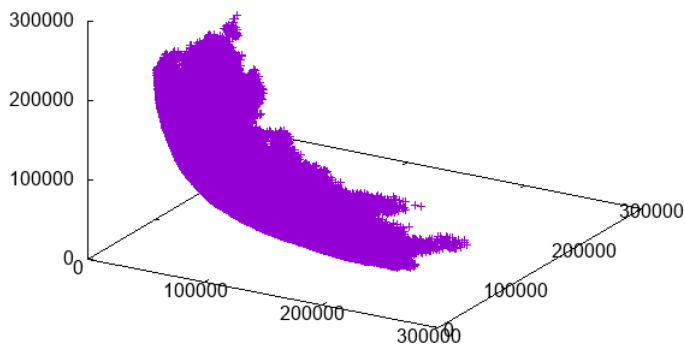
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



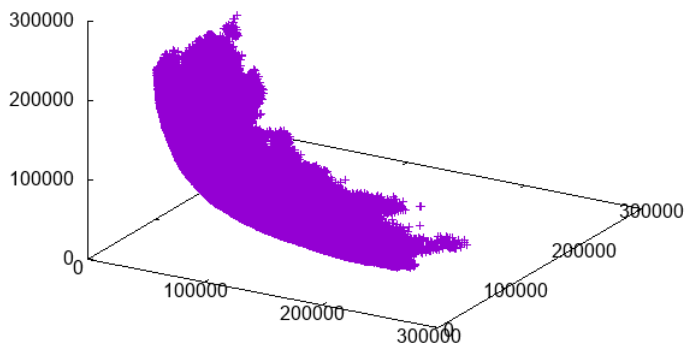
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



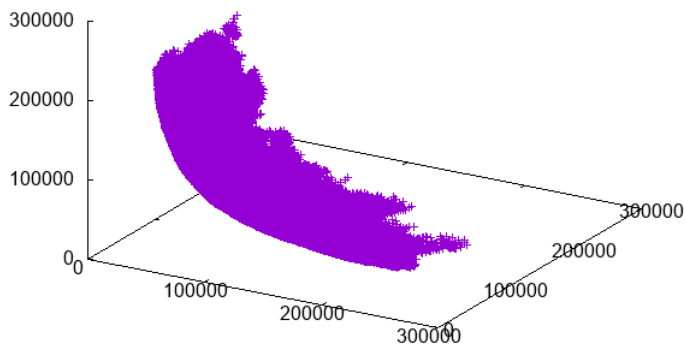
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



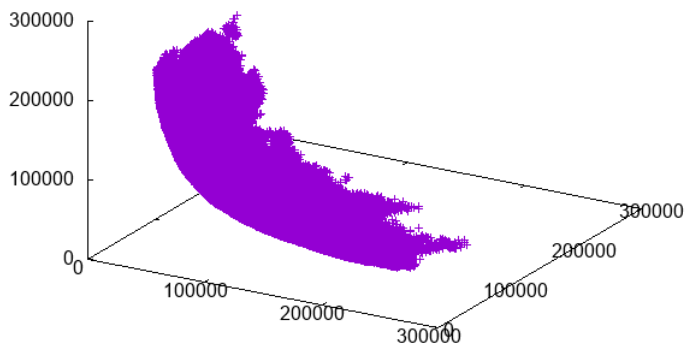
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



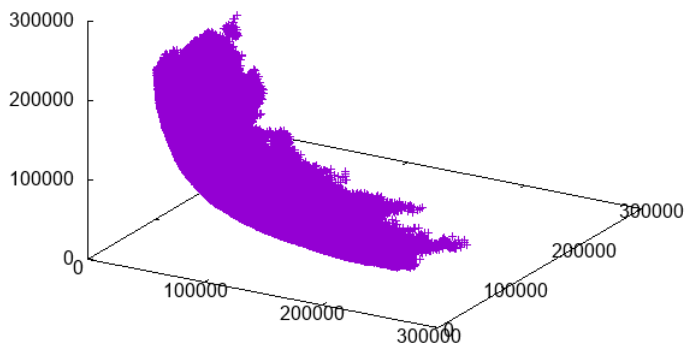
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



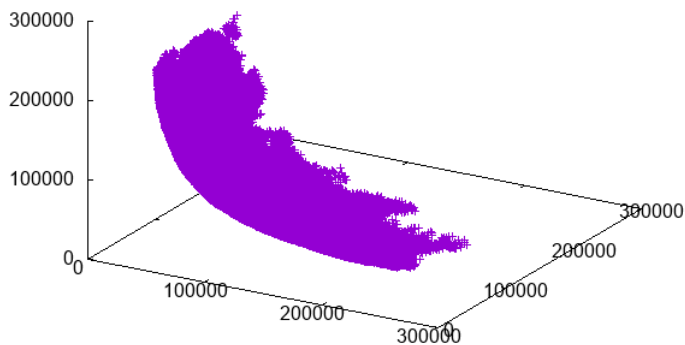
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



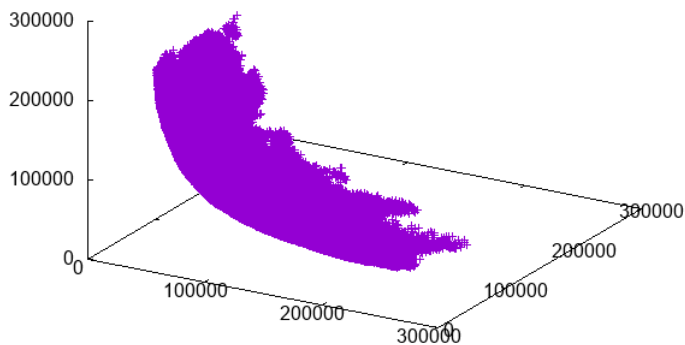
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



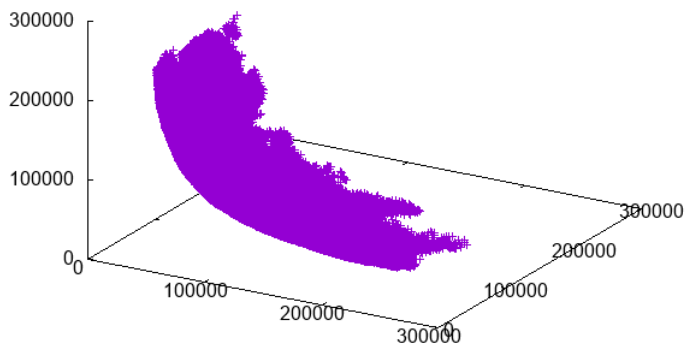
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



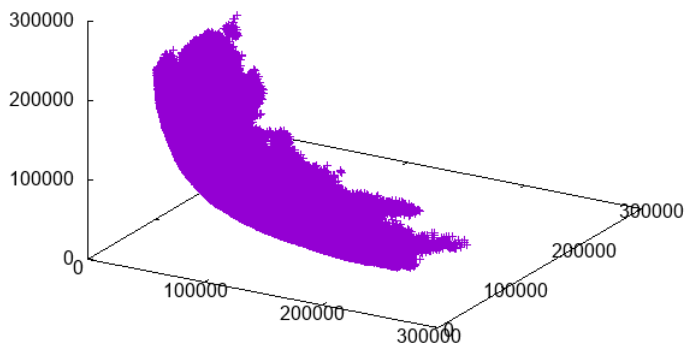
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



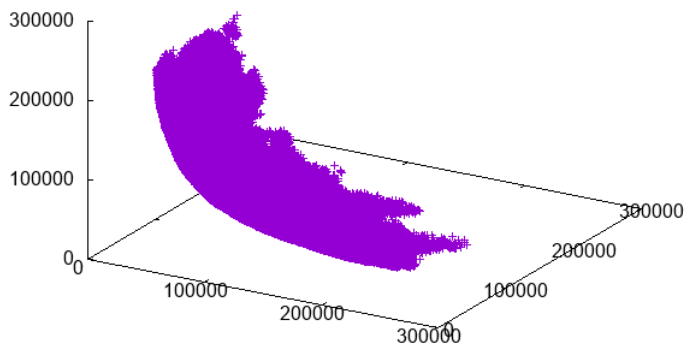
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



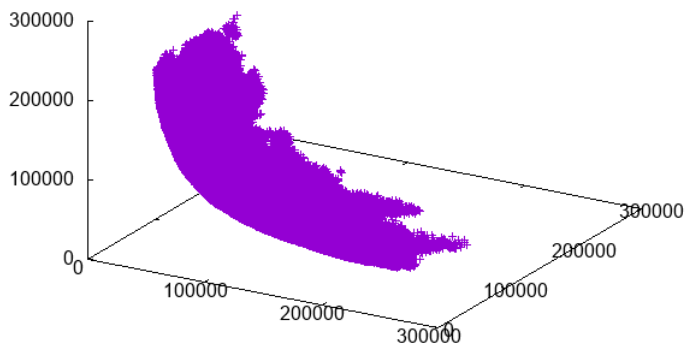
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



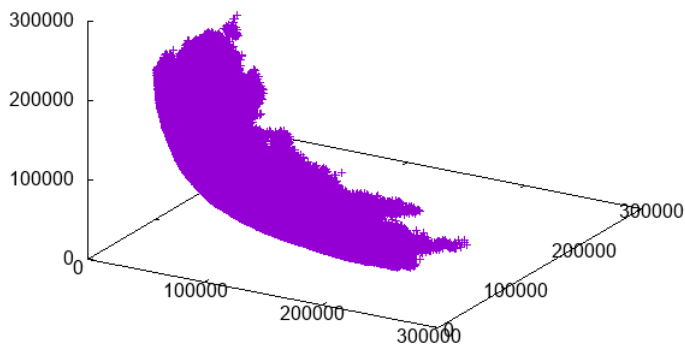
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



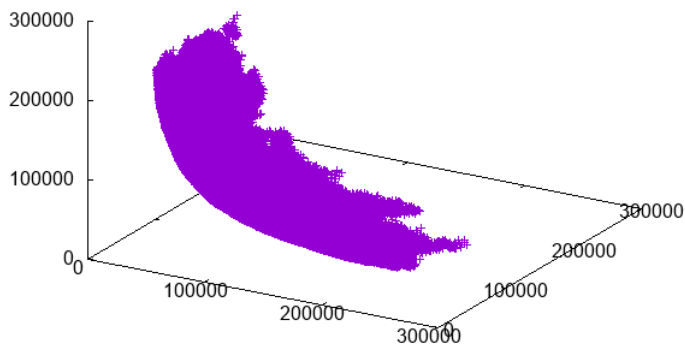
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



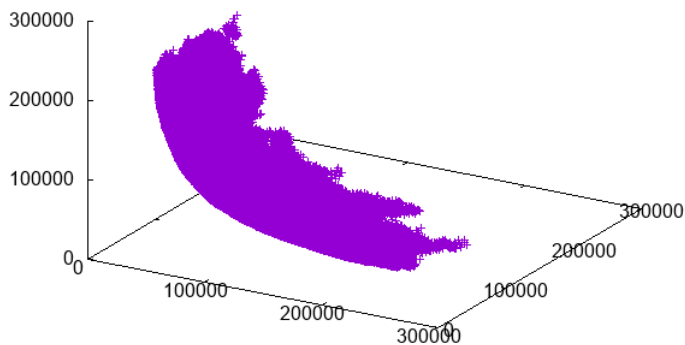
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



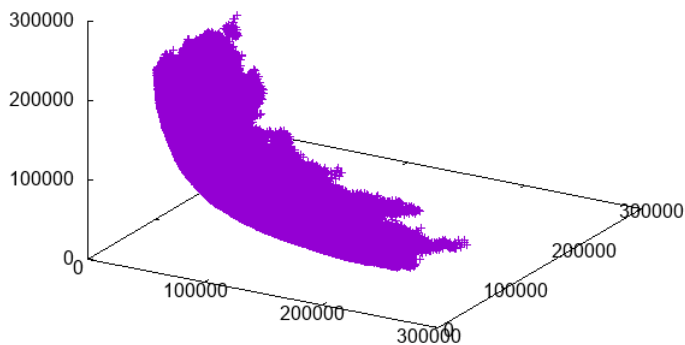
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



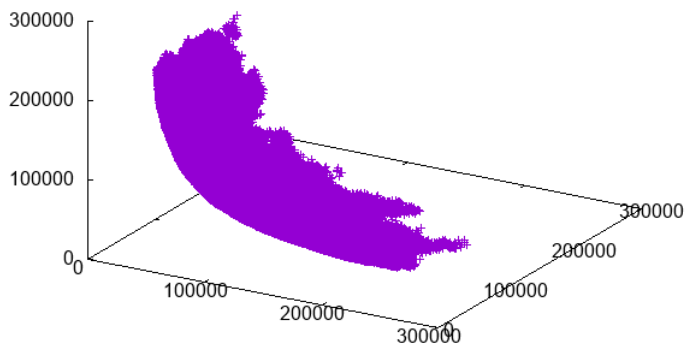
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



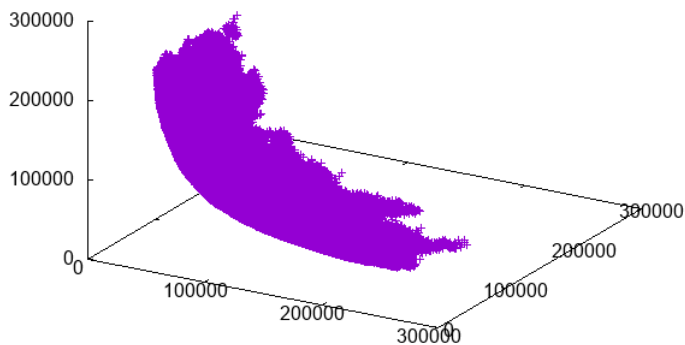
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



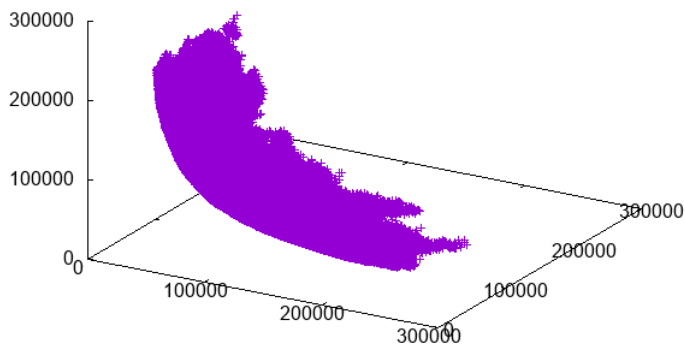
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



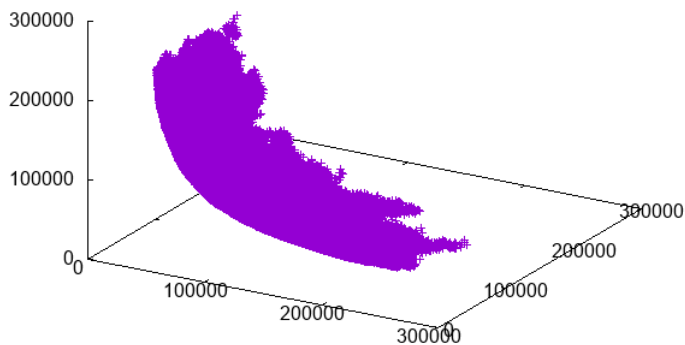
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



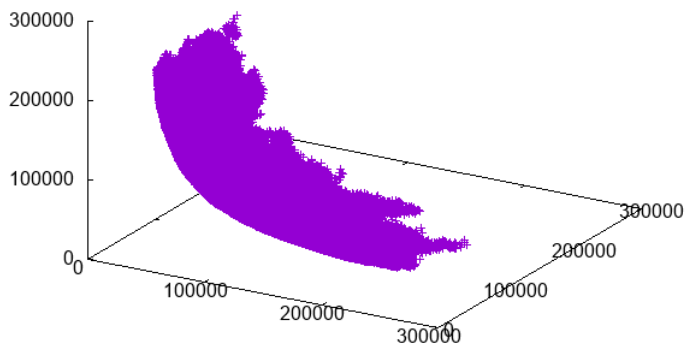
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



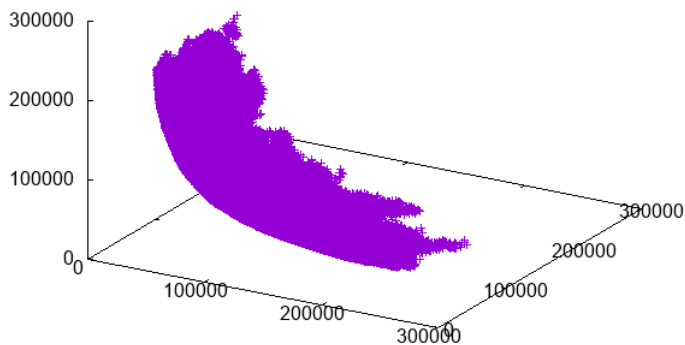
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



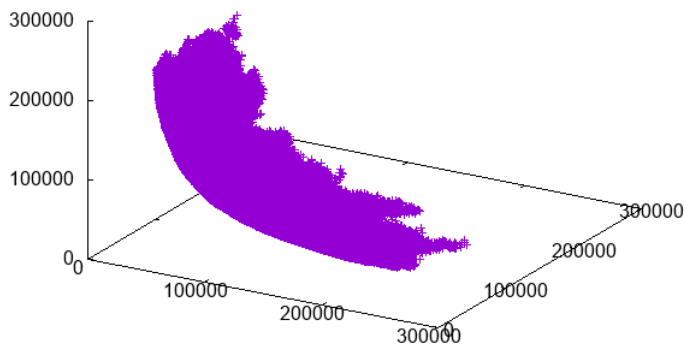
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



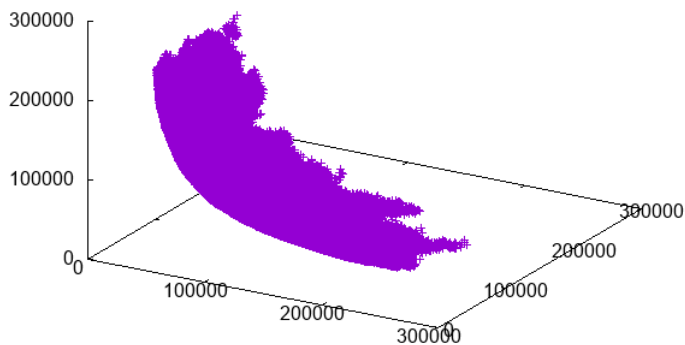
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



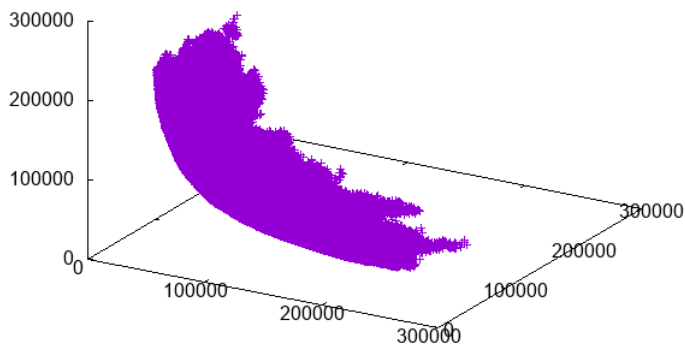
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



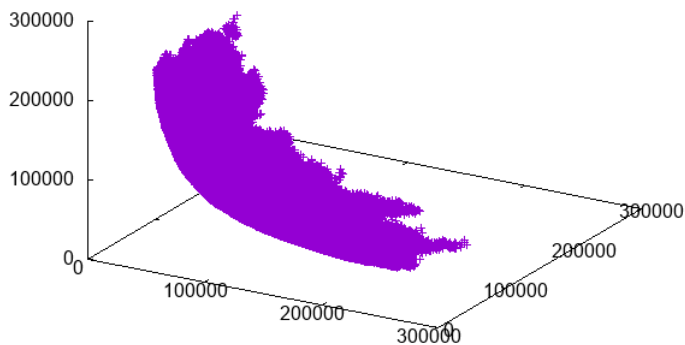
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



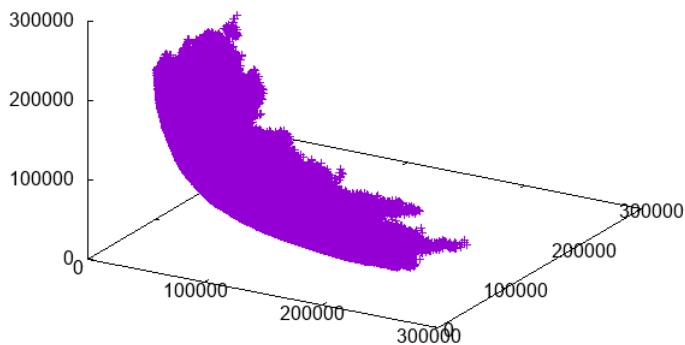
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



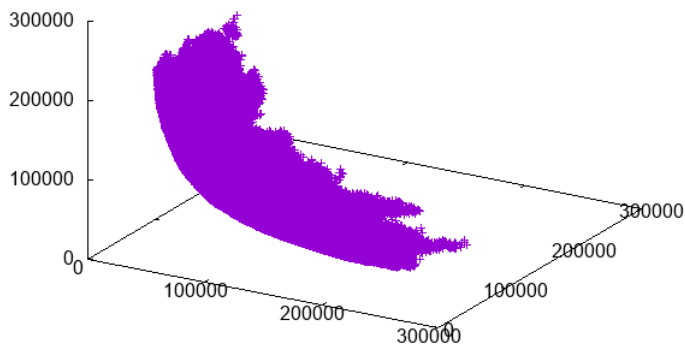
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



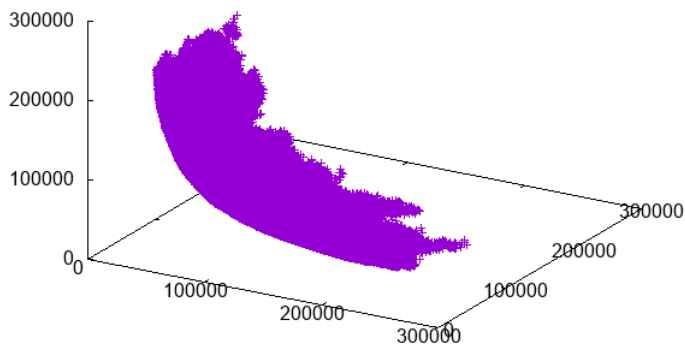
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



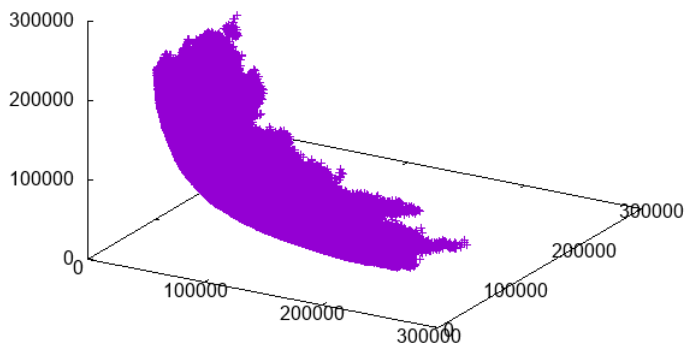
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



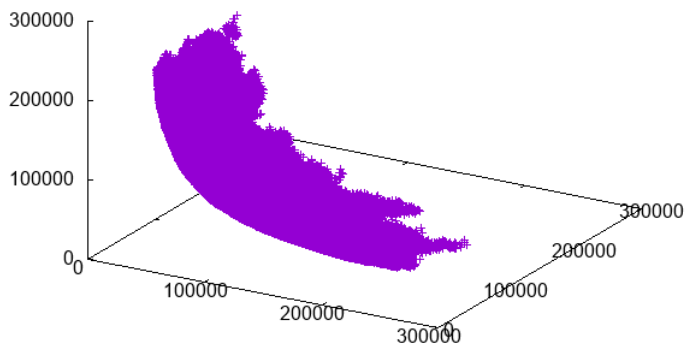
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



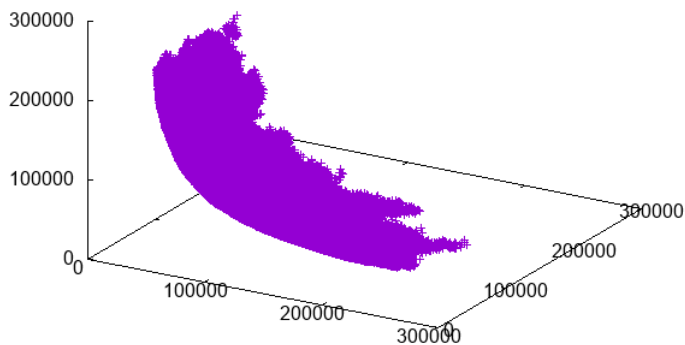
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



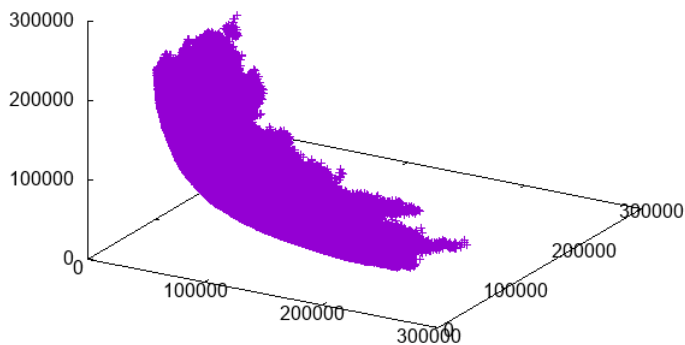
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



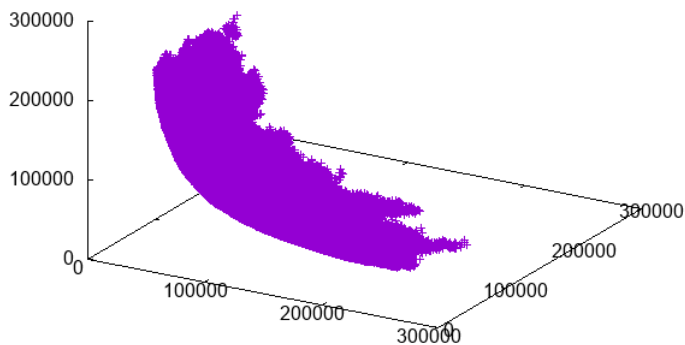
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



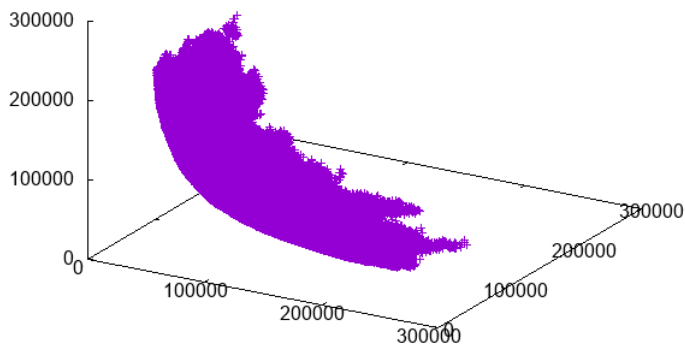
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



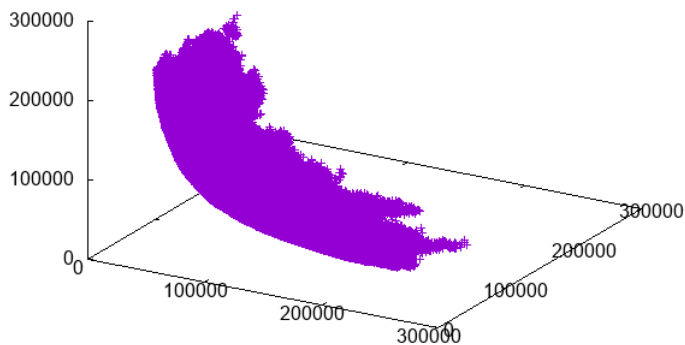
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



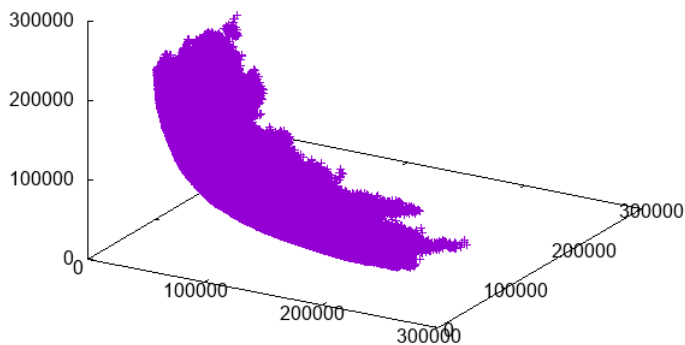
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



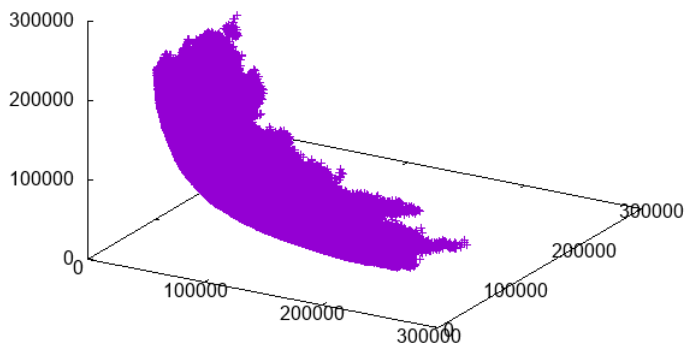
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



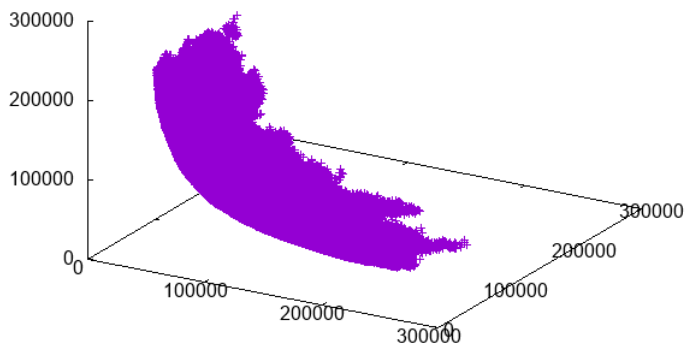
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



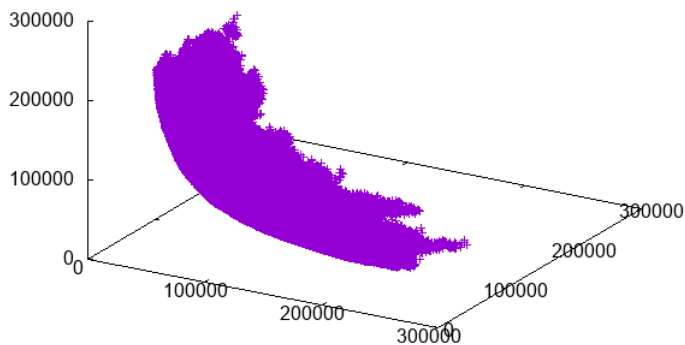
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



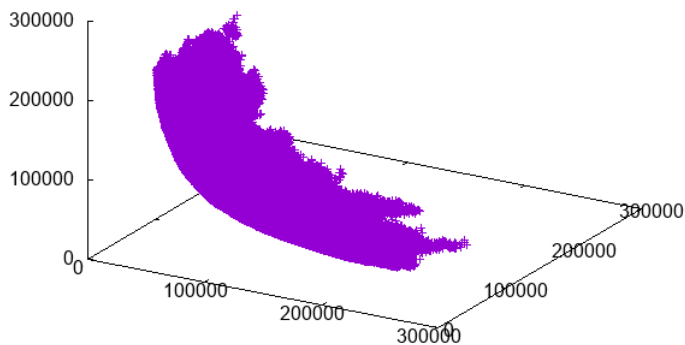
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



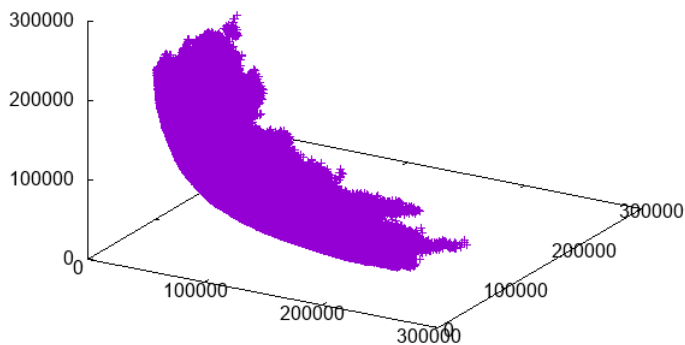
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



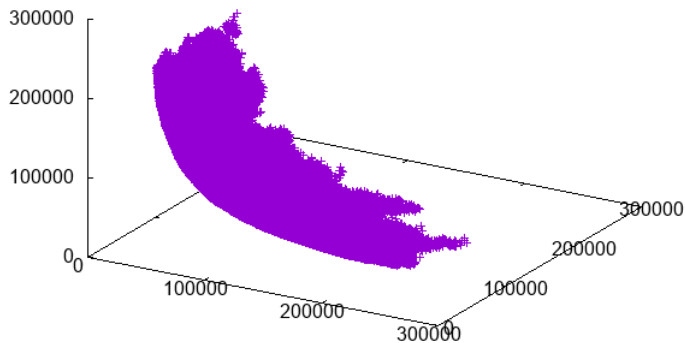
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



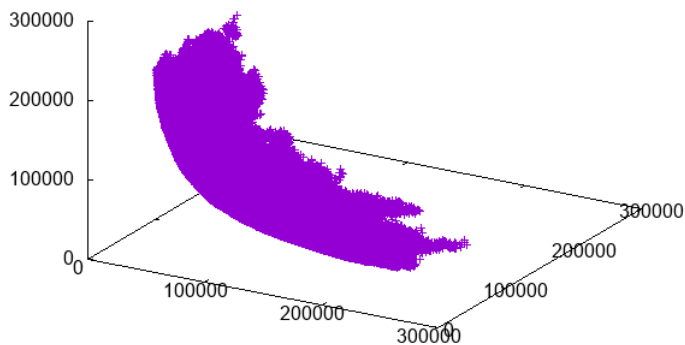
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



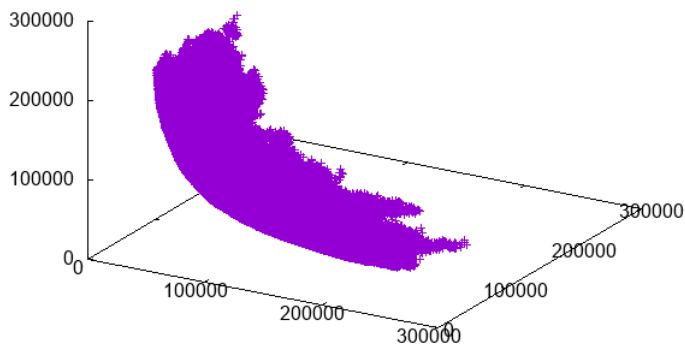
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



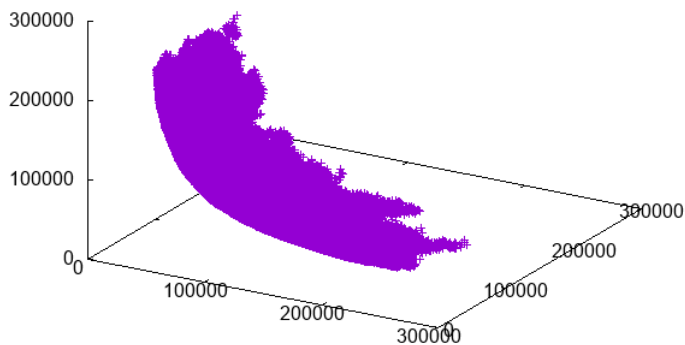
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



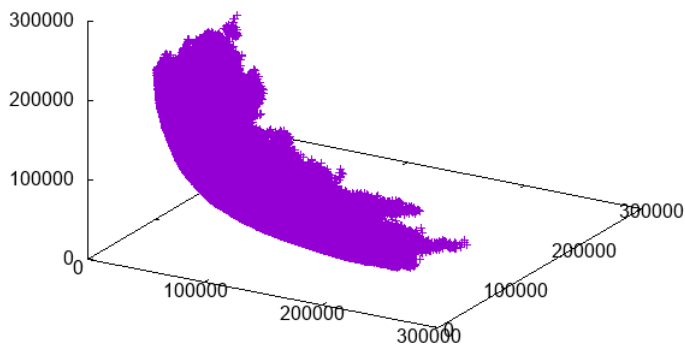
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



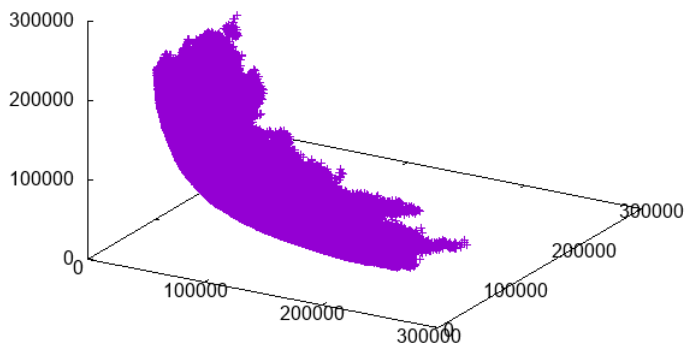
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



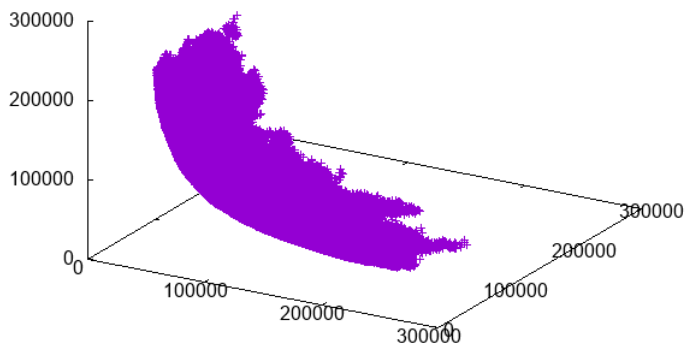
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



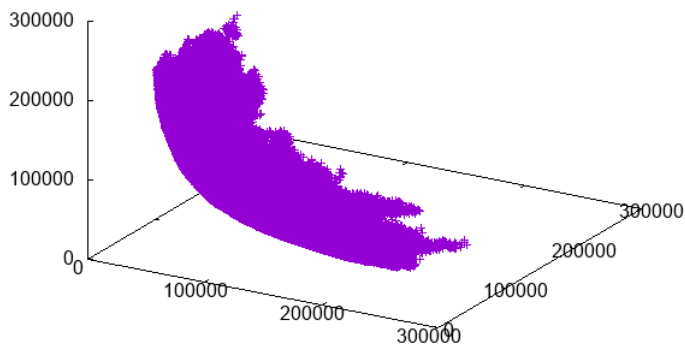
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



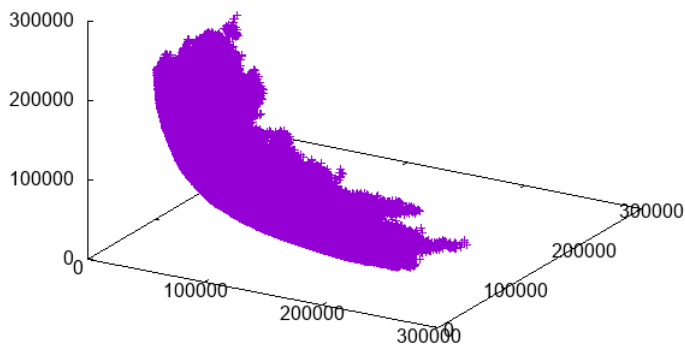
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



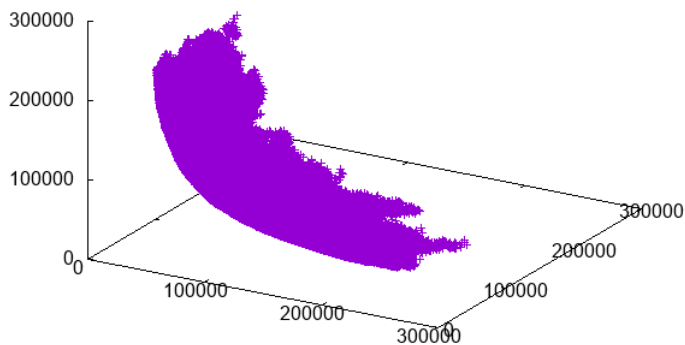
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



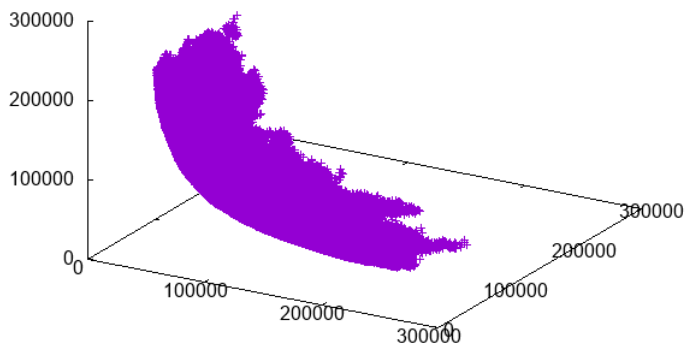
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



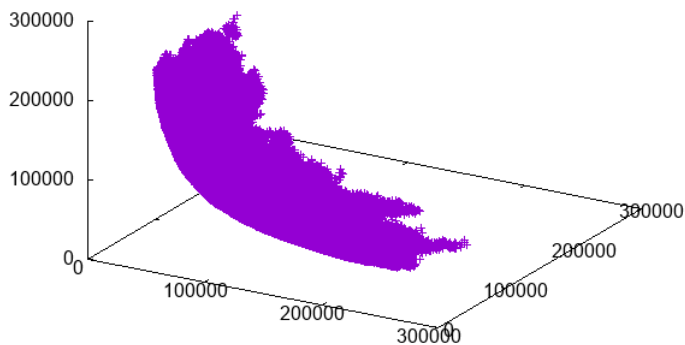
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



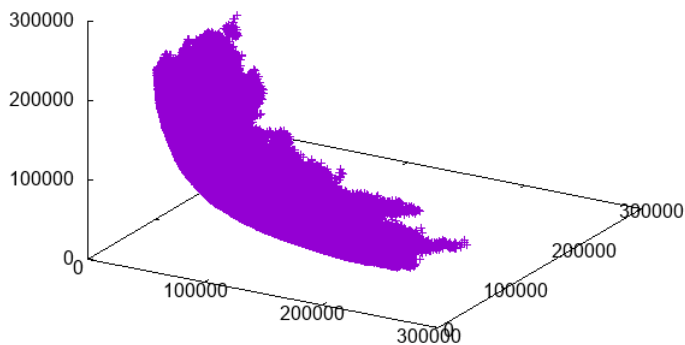
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



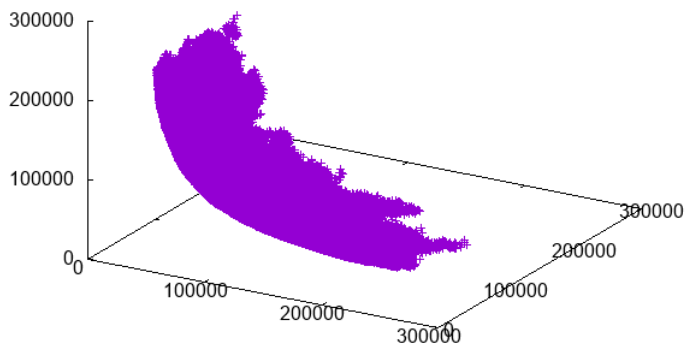
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



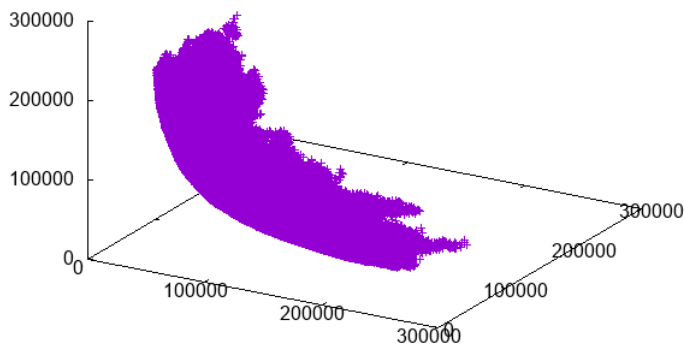
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



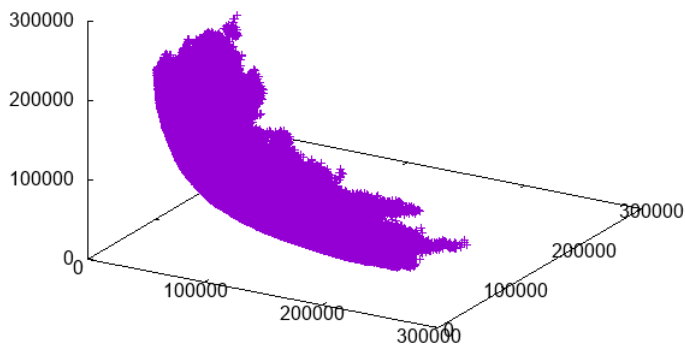
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



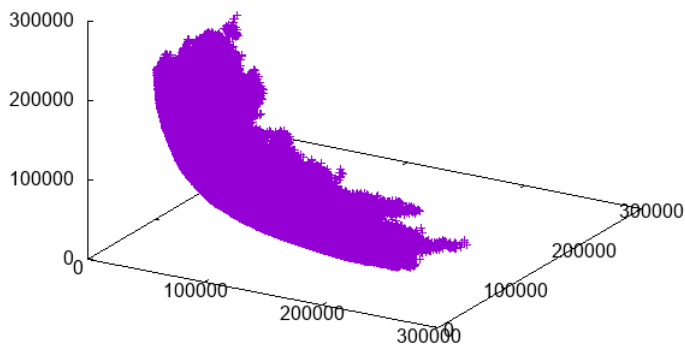
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



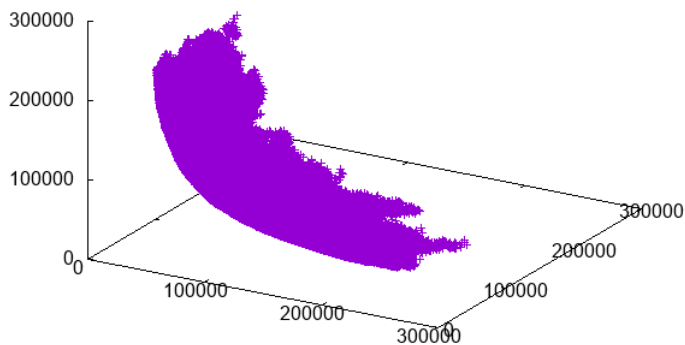
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



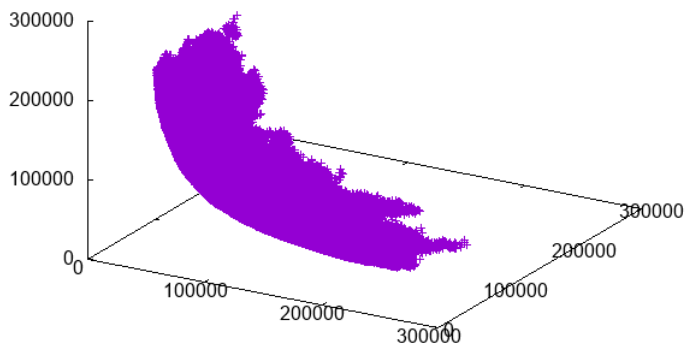
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



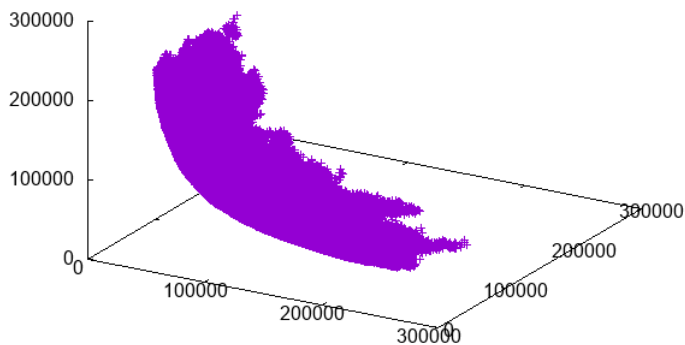
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



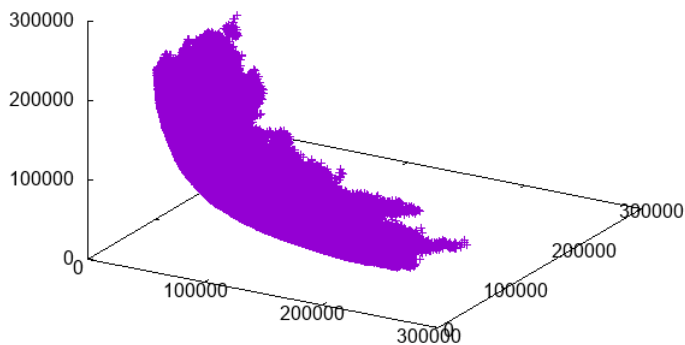
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



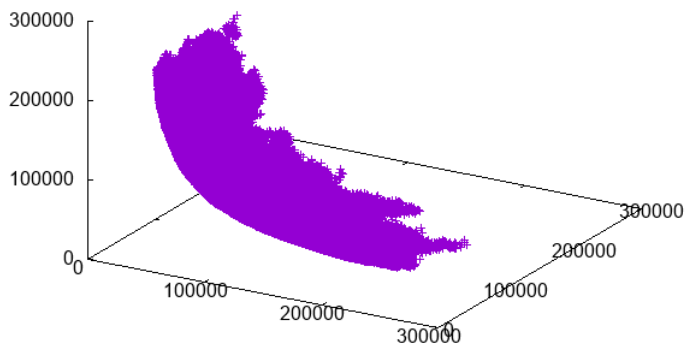
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



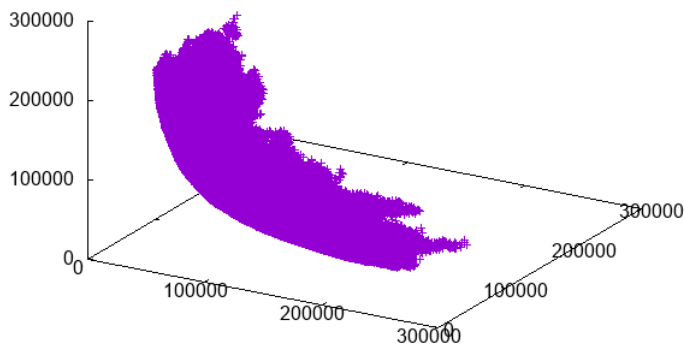
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



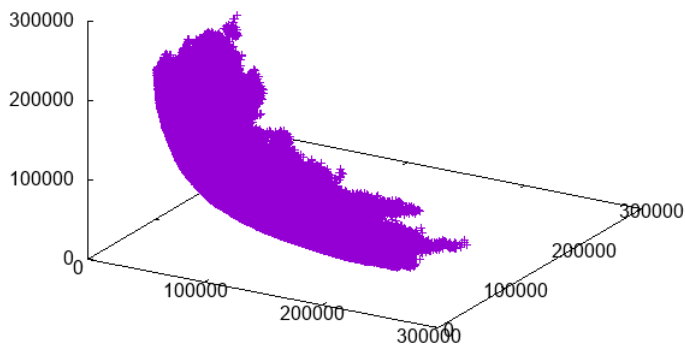
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



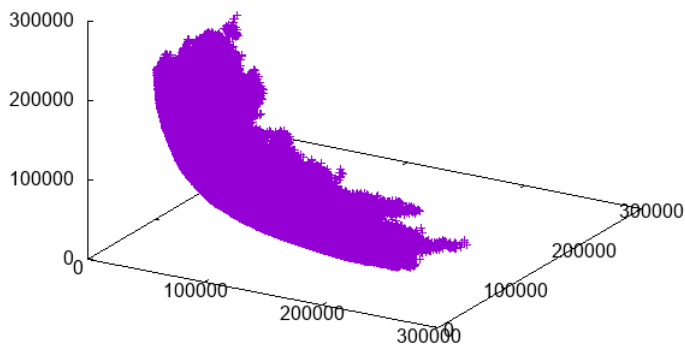
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



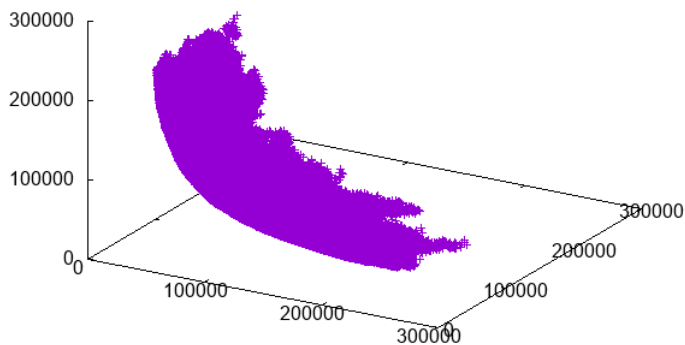
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



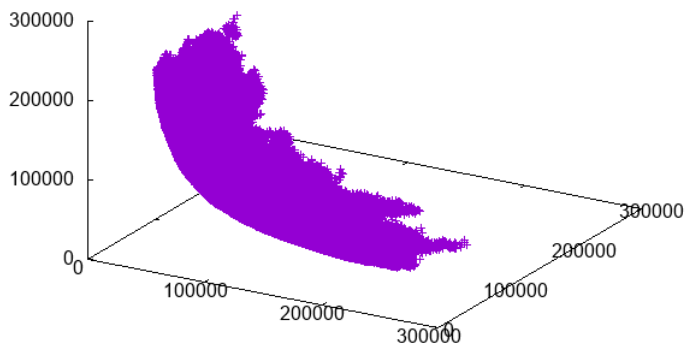
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



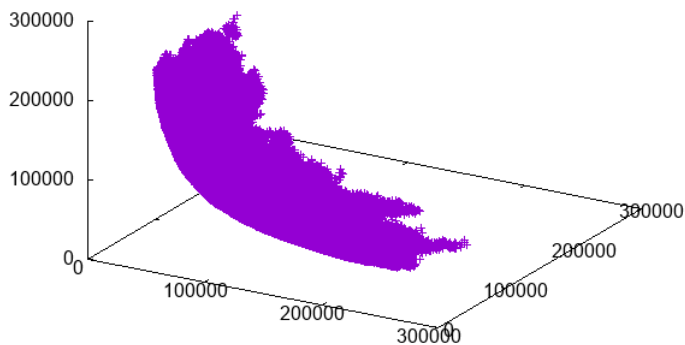
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



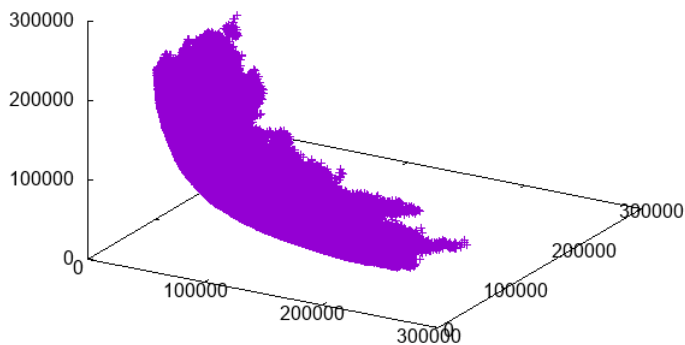
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



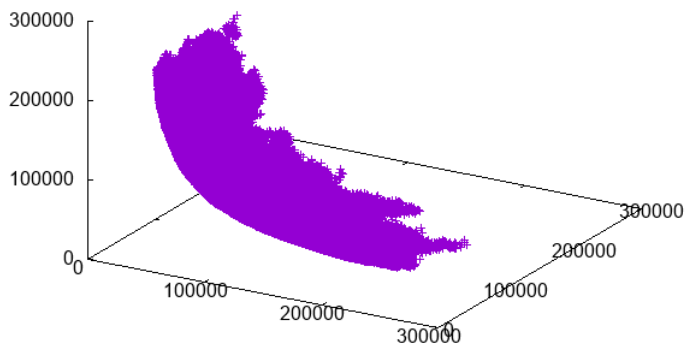
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



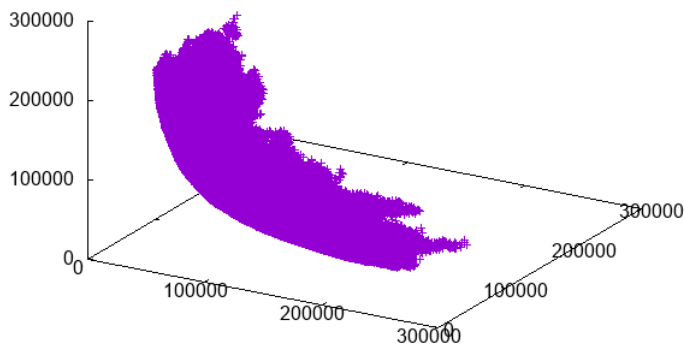
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



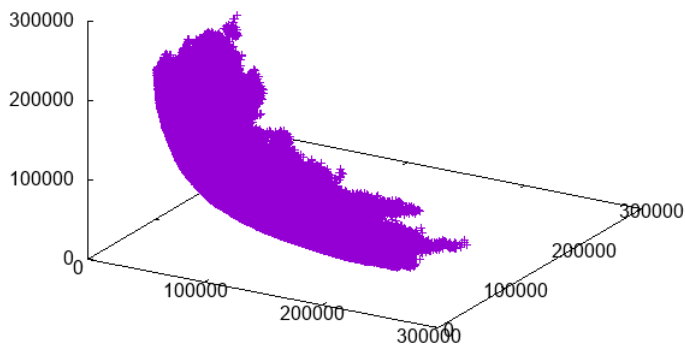
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



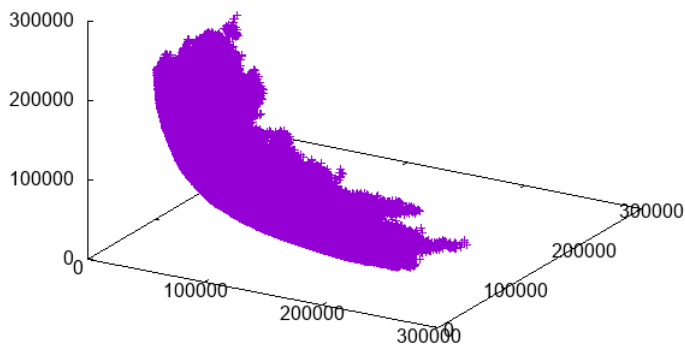
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



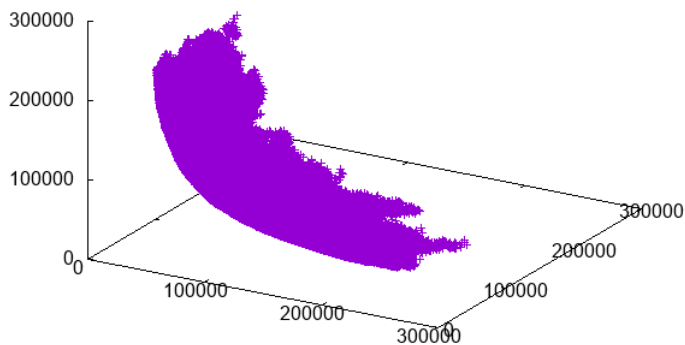
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



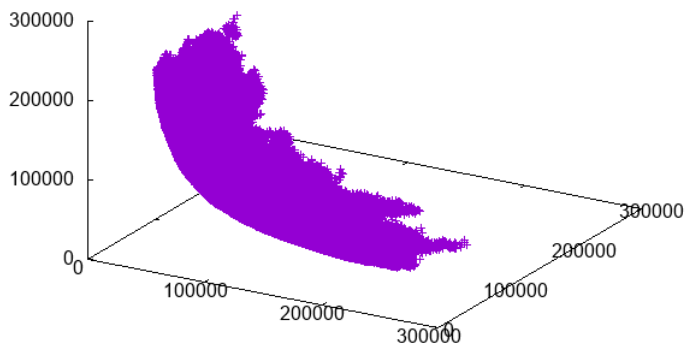
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



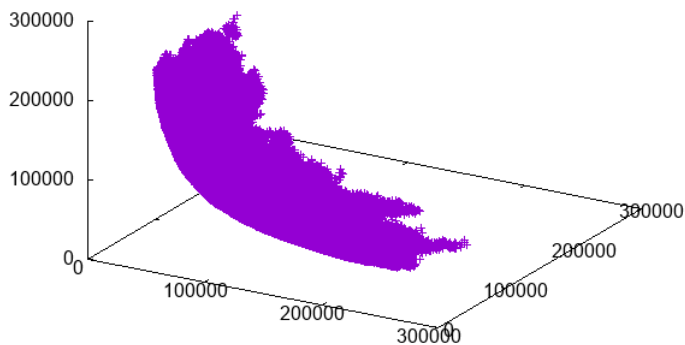
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



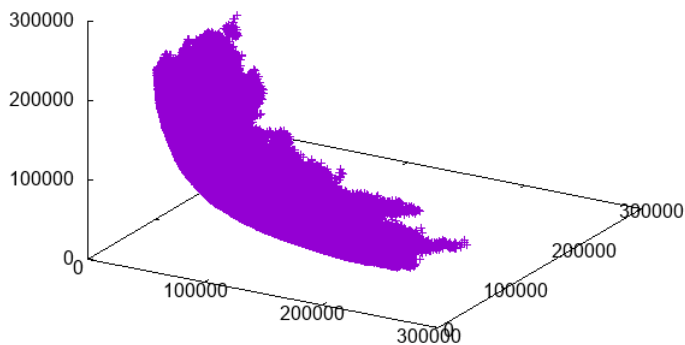
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



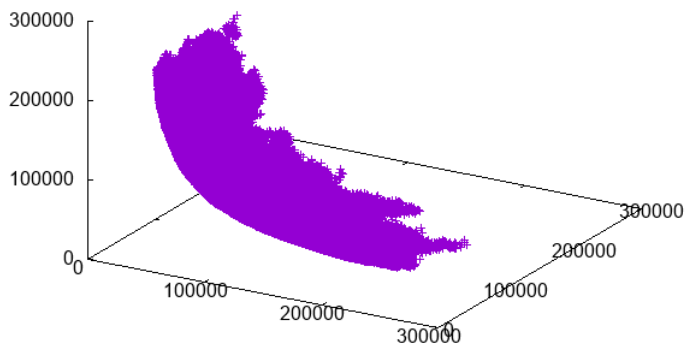
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



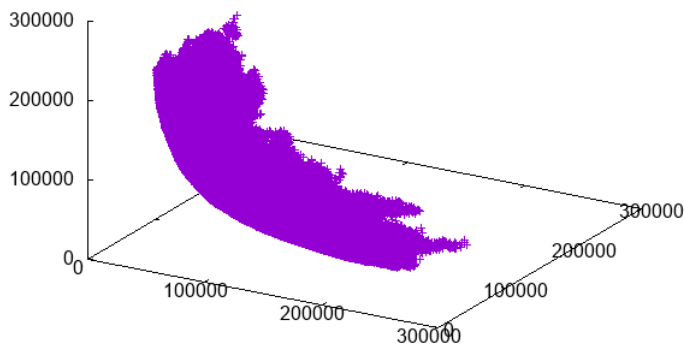
Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



Pareto Local Search (TSP, 3 Obj, 100 cities, 100 WS)



Rappel Métaheuristiques

Solution unique

Population de solutions

Rappel optimisation multi-objectifs

Métaheuristiques pour l'optimisation multi-objectifs

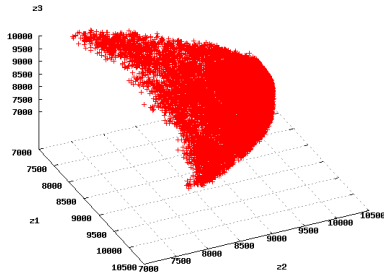
Mise à jour d'une archive de Pareto

Simple liste

Quad-tree

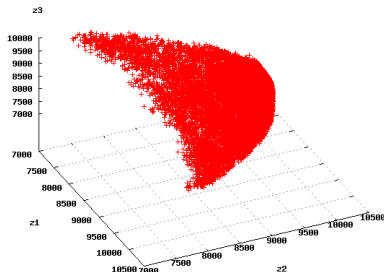
ND-Tree

Motivation



Pareto Archive (PA) : Set of potentially Pareto efficient solutions, i.e set of solutions such that any pair of solutions are mutually non-dominated.

Motivation



Pareto Archive (PA) : Set of potentially Pareto efficient solutions, i.e set of solutions such that any pair of solutions are mutually non-dominated.

Online update of a Pareto archive :

Set L of candidate solutions : $\{x_1, x_2, \dots, x_l\}$

For each solution $x_i \in L$:

- ▶ x_i is added to PA if x_i is non-dominated w.r.t. to any solution of PA.
- ▶ All solutions dominated by x_i are removed from PA.

Pareto local search : Version 1

Parameters $\downarrow$: an initial population P_0 , a neighborhood function $\mathcal{N}(x)$.

Parameters $\uparrow$: an approximation $\tilde{X}_E$ of the efficient set X_E .

--| Initialization of $\tilde{X}_E$ and a population P with the initial population P_0

$\tilde{X}_E \leftarrow P_0$

$P \leftarrow P_0$

--| Initialization of an auxiliary population P_a

$P_a \leftarrow \emptyset$

while $P \neq \emptyset$ **do**

--| Generation of all neighbors p' of each solution $p \in P$

for all $p \in P$ **do**

for all $p' \in \mathcal{N}(p)$ **do**

--| If p' is not dominated by p

if $f(p) \not\leq f(p')$ **then**

if $\text{Update}(\tilde{X}_E \uparrow, p' \downarrow)$ **then**

$\text{Update}(P_a \uparrow, p' \downarrow)$

end if

end if

end for

end for

--| P is composed of the new potentially efficient solutions

$P \leftarrow P_a$

--| Reinitialization of P_a

$P_a \leftarrow \emptyset$

end while

Pareto local search : Updating the neighbors list

Parameters $\downarrow$: an initial population P_0 , a neighborhood function $\mathcal{N}(x)$.

Parameters $\uparrow$: an approximation $\tilde{X}_E$ of the efficient set X_E .

--| Initialization of $\tilde{X}_E$ and a population P with the initial population P_0

$\tilde{X}_E \leftarrow P_0$

$P \leftarrow P_0$

--| Initialization of an auxiliary population P_a

$P_a \leftarrow \emptyset$

while $P \neq \emptyset$ **do**

--| Generation of all neighbors p' of each solution $p \in P$

for all $p \in P$ **do**

--| Initialization of a list of neighbors

$\mathcal{L}_N \leftarrow \emptyset$

for all $p' \in \mathcal{N}(p)$ **do**

--| If p' is not dominated by p

if $f(p) \not\leq f(p')$ **then**

Update($\mathcal{L}_N \uparrow, p' \downarrow$)

end if

end for

for all $p' \in \mathcal{L}_N$ **do**

if Update($\tilde{X}_E \uparrow, p' \downarrow$) **then**

Update($P_a \uparrow, p' \downarrow$)

end if

end for

end for

--| P is composed of the new potentially efficient solutions

$P \leftarrow P_a$

--| Reinitialization of P_a

$P_a \leftarrow \emptyset$

end while

Simple List

$z(x^1)$	20	100	50
$z(x^2)$	50	60	75
$z(x^3)$	30	80	12
$z(x^4)$	12	125	200
$z(x^5)$	40	75	60
$z(x^6)$	80	40	150
$z(x^7)$	55	110	10
...			

Simple List

$z(x^1)$	20	100	50
$z(x^2)$	50	60	75
$z(x^3)$	30	80	12
$z(x^4)$	12	125	200
$z(x^5)$	40	75	60
$z(x^6)$	80	40	150
$z(x^7)$	55	110	10
...			

A new solution x is compared to each solution of PA until a solution dominating x is found or all solutions do not dominate x .

Simple List

$z(x^1)$	20	100	50
$z(x^2)$	50	60	75
$z(x^3)$	30	80	12
$z(x^4)$	12	125	200
$z(x^5)$	40	75	60
$z(x^6)$	80	40	150
$z(x^7)$	55	110	10
...			

A new solution x is compared to each solution of PA until a solution dominating x is found or all solutions do not dominate x .

- ▶ **Fast** if x is found dominated at the beginning (low-quality solution)
- ▶ **Slow** if x has to be compared to all solutions of PA (high-quality solution)

Special case : 2 objectives $\rightarrow$ Sorted List

Special case : 2 objectives $\rightarrow$ Sorted List

If sorted according to the first objective, sorted according to the second objective too.

$z(x^1)$	3	20
$z(x^2)$	5	17
$z(x^3)$	8	12
$z(x^4)$	12	11
$z(x^5)$	16	4
$z(x^6)$	19	1

Special case : 2 objectives $\rightarrow$ Sorted List

If sorted according to the first objective, sorted according to the second objective too.

$z(x^1)$	3	20
$z(x^2)$	5	17
$z(x^3)$	8	12
$z(x^4)$	12	11
$z(x^5)$	16	4
$z(x^6)$	19	1

We want to update PA with a new solution x^7 such that $z(x^7) = (11, 10)$ (minimization)

1. Find the position of x^7 in PA (binary search, $\mathcal{O}(\log n)$) : the solutions preceding x^7 can potentially dominate x^7 and the solutions following x^7 can potentially be dominated by x^7 .

Special case : 2 objectives $\rightarrow$ Sorted List

If sorted according to the first objective, sorted according to the second objective too.

$z(x^1)$	3	20
$z(x^2)$	5	17
$z(x^3)$	8	12
$z(x^4)$	12	11
$z(x^5)$	16	4
$z(x^6)$	19	1

We want to update PA with a new solution x^7 such that $z(x^7) = (11, 10)$ (minimization)

1. Find the position of x^7 in PA (binary search, $\mathcal{O}(\log n)$) : the solutions preceding x^7 can potentially dominate x^7 and the solutions following x^7 can potentially be dominated by x^7 .
2. The first preceding solution x^3 does not dominate x^7 : x^7 is not dominated and could be added.

Special case : 2 objectives $\rightarrow$ Sorted List

If sorted according to the first objective, sorted according to the second objective too.

$z(x^1)$	3	20
$z(x^2)$	5	17
$z(x^3)$	8	12
$z(x^4)$	12	11
$z(x^5)$	16	4
$z(x^6)$	19	1

We want to update PA with a new solution x^7 such that $z(x^7) = (11, 10)$ (minimization)

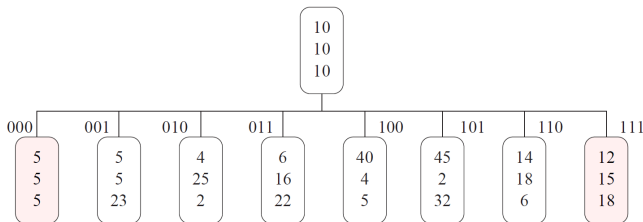
1. Find the position of x^7 in PA (binary search, $\mathcal{O}(\log n)$) : the solutions preceding x^7 can potentially dominate x^7 and the solutions following x^7 can potentially be dominated by x^7 .
2. The first preceding solution x^3 does not dominate x^7 : x^7 is not dominated and could be added.
3. The solution following x^3 is dominated, x^4 should be removed.
4. x^5 is not dominated by x^4 : we can stop the search.

Quad-tree (Mostaghim and Teich, 2005)

- ▶ Tree based data structure for storing objective vectors
- ▶ Each node is a vector with p elements and can have at most 2^p sons
- ▶ The successorship is expressed by a binary string :

$$k_i = \begin{cases} 1 & \text{if } x_i \geq y_i \\ 0 & \text{if } x_i < y_i \end{cases} \quad (1)$$

- ▶ Domination-free data structure : no branches with successorship equal to 0000... or 1111....



Updating quad-tree (minimization)

Suppose $p = 3$, a new solution x , root of quad-tree = y , x is a successor of y and k is the successorship of x

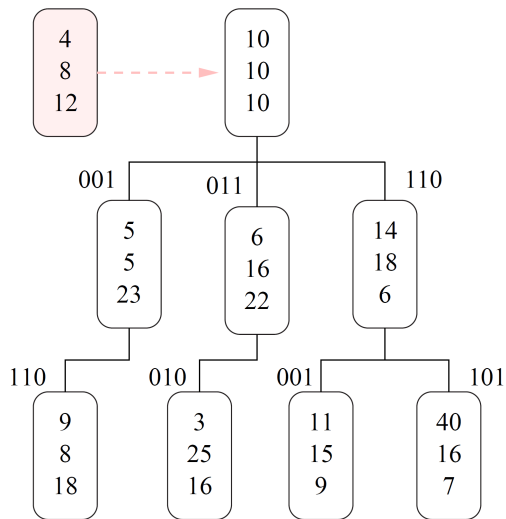
- ▶ **Vectors that dominate x** : Only places in quad-tree where there may be vectors that dominate x are in those subtrees whose roots are sons of y that have **zeros in at least all locations that k does**. For example in the case that $k = 110$, only the 010- and 100-successors of y must be checked whether they dominate x or not.
- ▶ **Vectors that are dominated by x** : Only places in quad-tree where there may be vectors that are dominated by x are in those subtrees whose roots are sons of y that have **ones in at least all location that k does**. For example, in the case that $k = 100$, only the 101- and 110-successors of y must be checked whether they are dominated by x or not.

Main principle of the algorithm

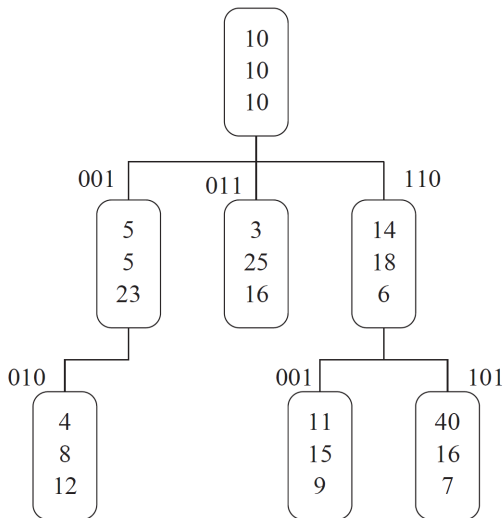
New solution x , k is the successorship of x :

1. Look for nodes that could dominate x . If a solution dominates x , discard x .
2. Look for nodes that could be dominated by x . For each node n that is dominated, delete it and reinsert all the nodes of the subtree of n .
3. Insert x . If a solution y has the same successorship than x : check dominance between x and y . If x is not dominated, consider y as the new local root and apply again the algorithm from this root.

Example : insertion of (4,8,12)



Example : quad-tree obtained after inserting (4,8,12)



Advantages and Disadvantages of Quad-tree

Advantages :

- ▶ Many comparisons are saved
- ▶ Works for any number of objectives
- ▶ Works pretty well with high number of objectives

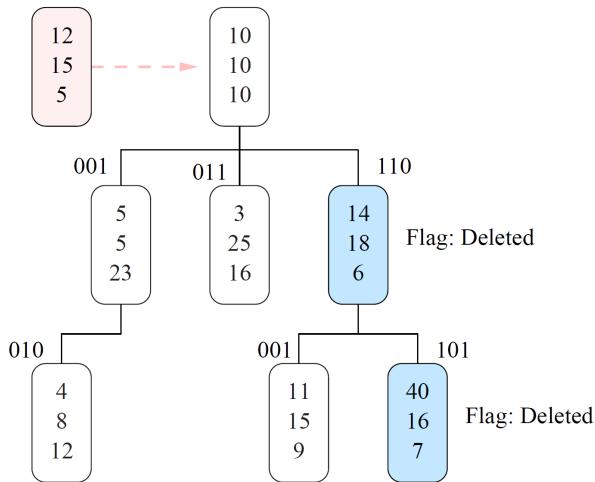
Disadvantages :

- ▶ Form and depth of the quad-tree is dependent on the order of insertion of a given set of vectors
- ▶ Deleting a node destroys the structure of Quad-tree and requires to reinsert all of the nodes of subtrees of a deleted node

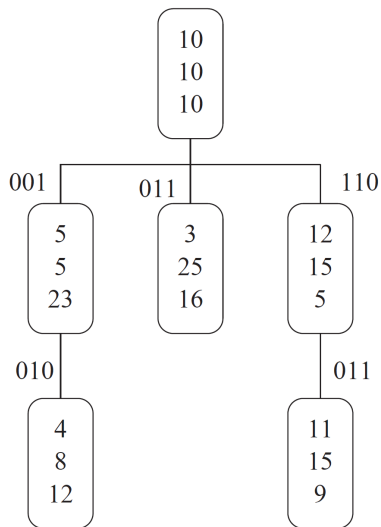
Quad-tree2

- ▶ Quad-tree2 : a discovered dominated node is not deleted immediately, but marked as to be deleted by a flag. Its subtrees are traversed, again setting flags for all encountered dominated nodes instead of resinserting all subtrees immediately. After finishing this recursive descent, we insert the new node and reinsert only those nodes in subtrees that are not marked deleted.

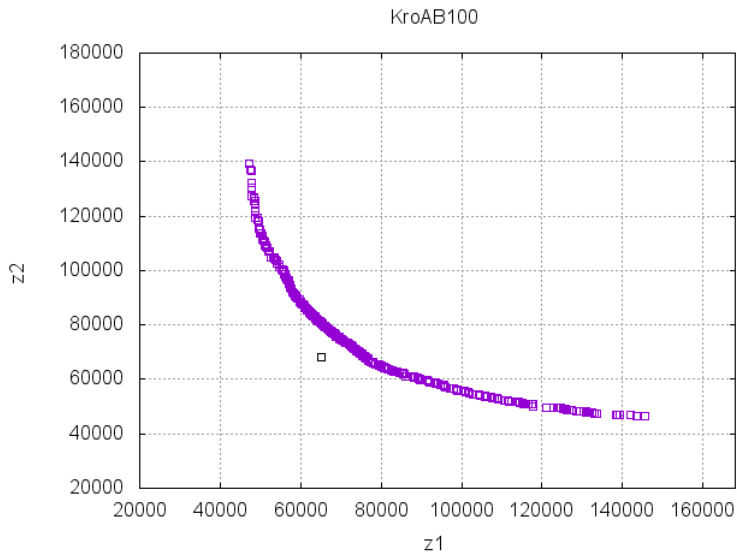
Quad-tree2 : insertion of (12,15,5)



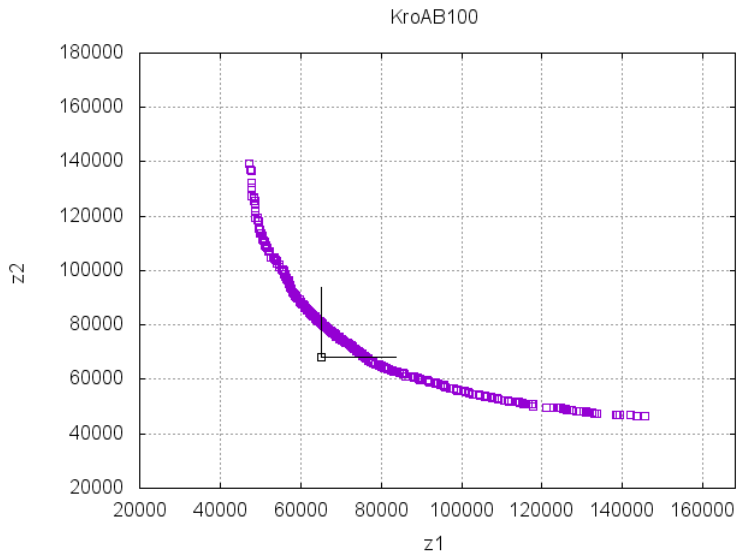
Quad-tree2 : quad-tree obtained after inserting (12,15,5)



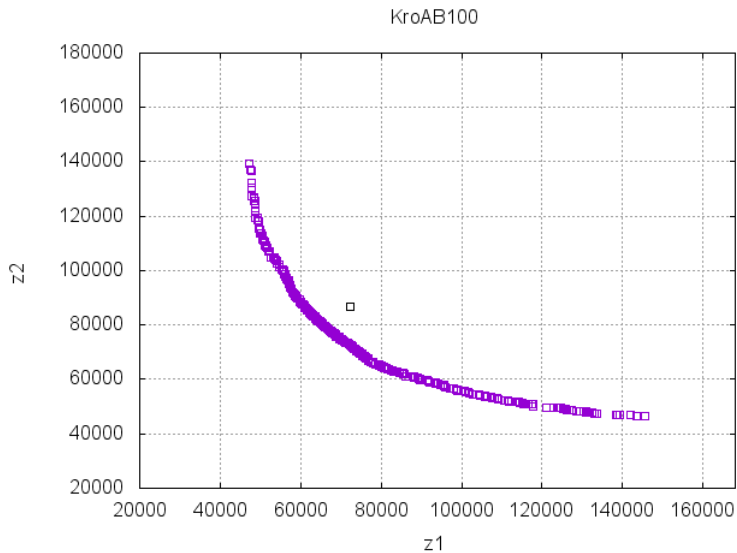
ND-Tree (Jaszekiewicz and Lust, 2018)



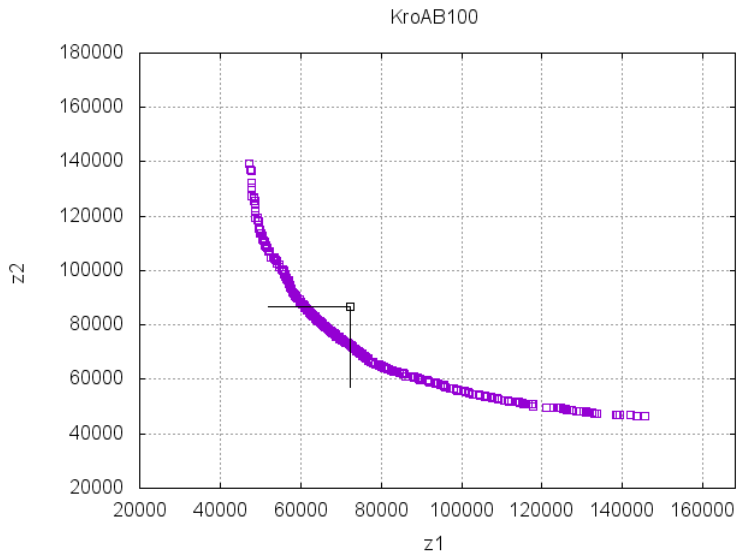
ND-Tree (Jaszekiewicz and Lust, 2018)



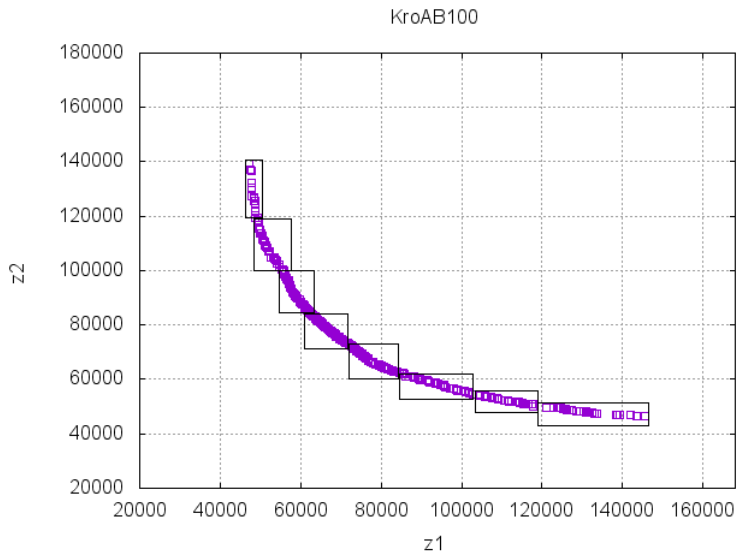
ND-Tree (Jaszekiewicz and Lust, 2018)



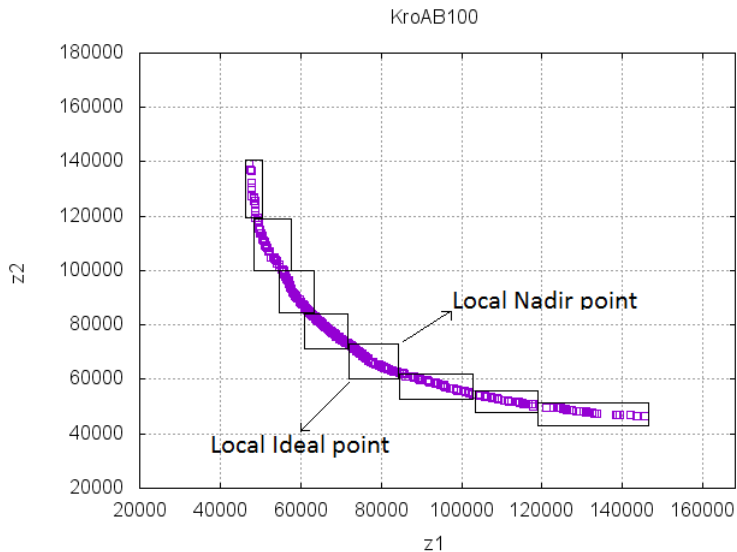
ND-Tree (Jaszekiewicz and Lust, 2018)



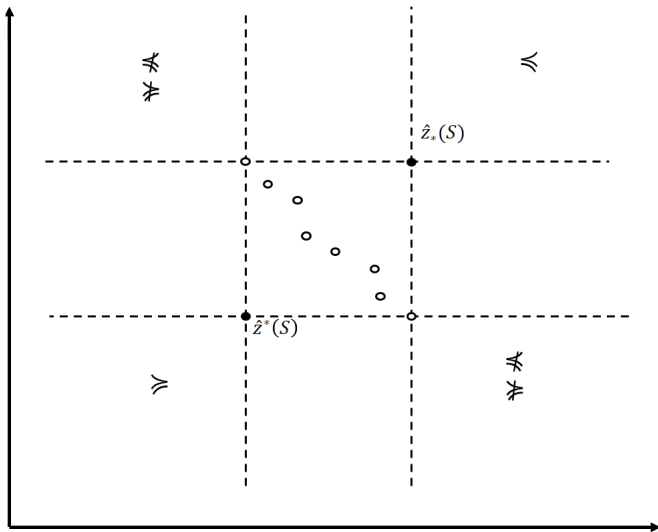
ND-Tree (Jaszekiewicz and Lust, 2018)



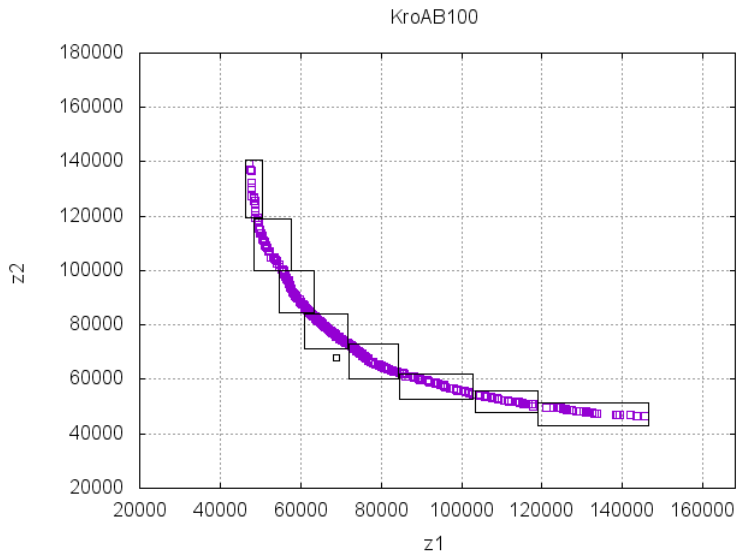
ND-Tree (Jaskiewicz and Lust, 2018)



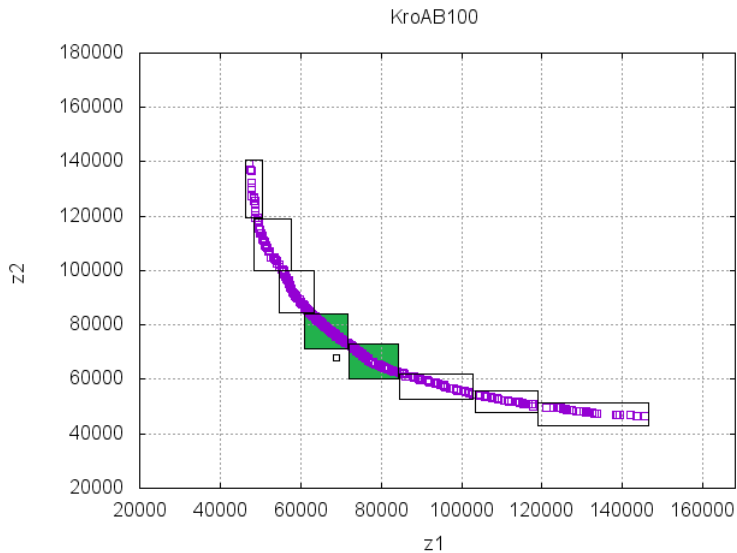
Local Nadir and Ideal points



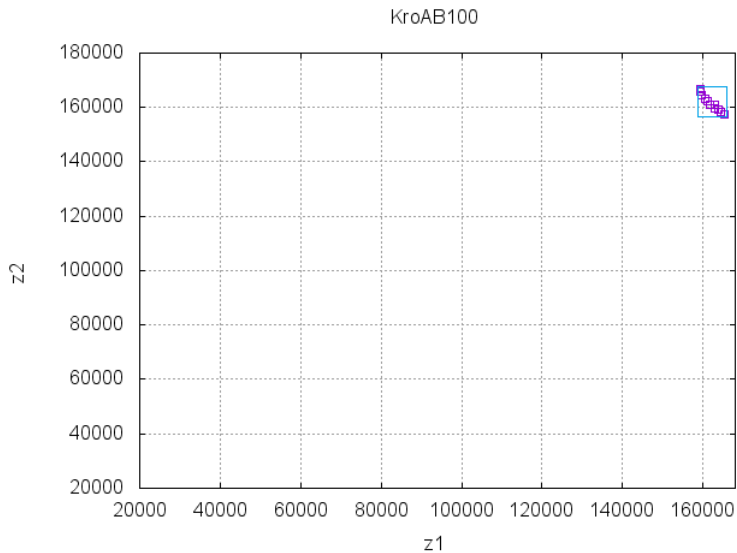
Local Nadir and Ideal points



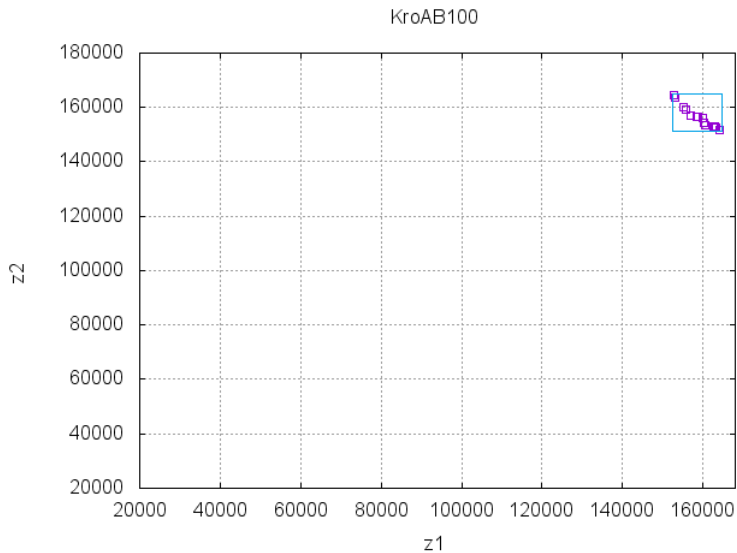
Local Nadir and Ideal points



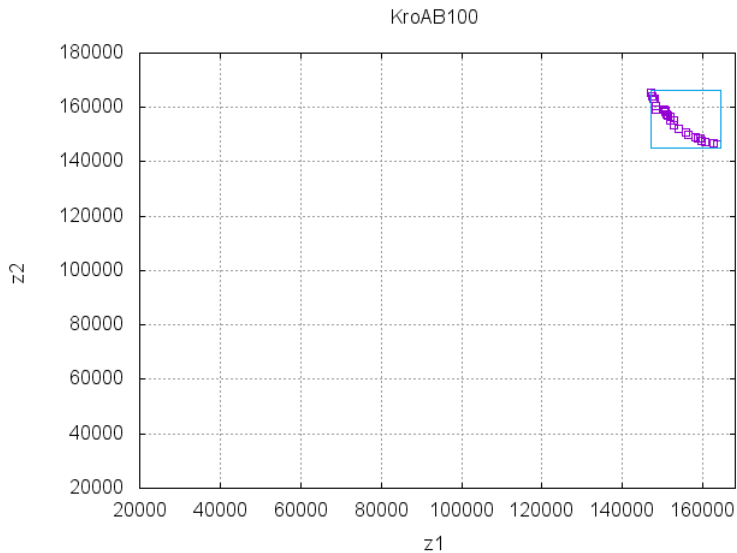
ND-Tree



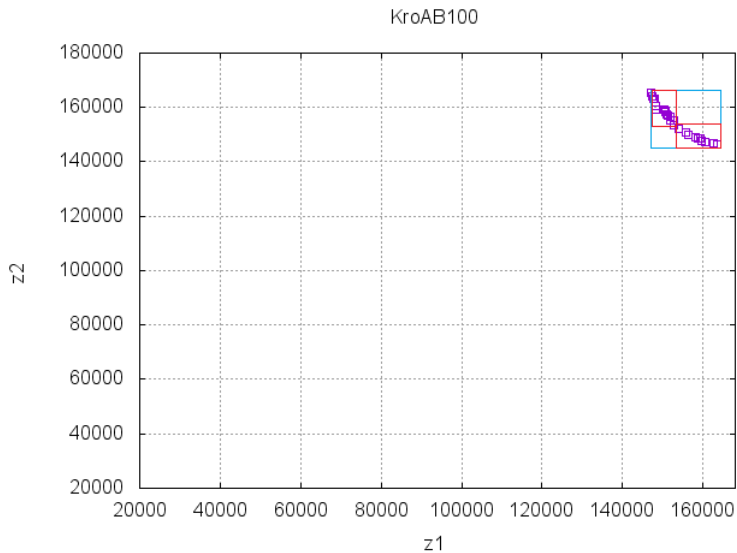
ND-Tree



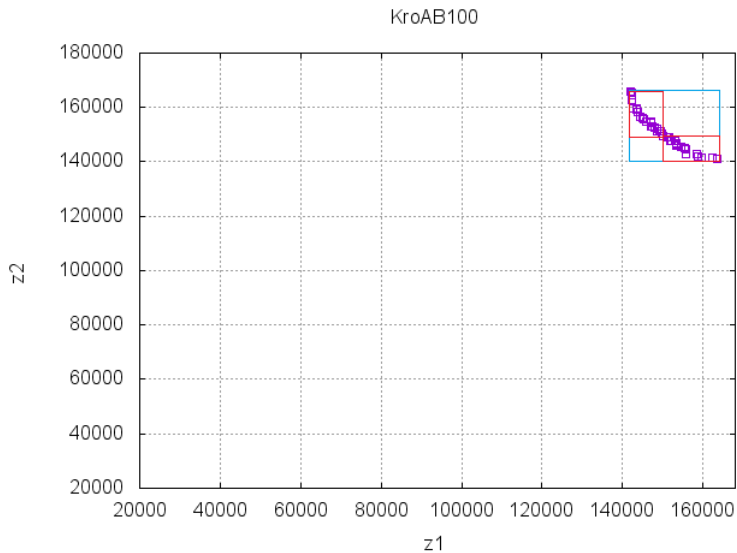
ND-Tree



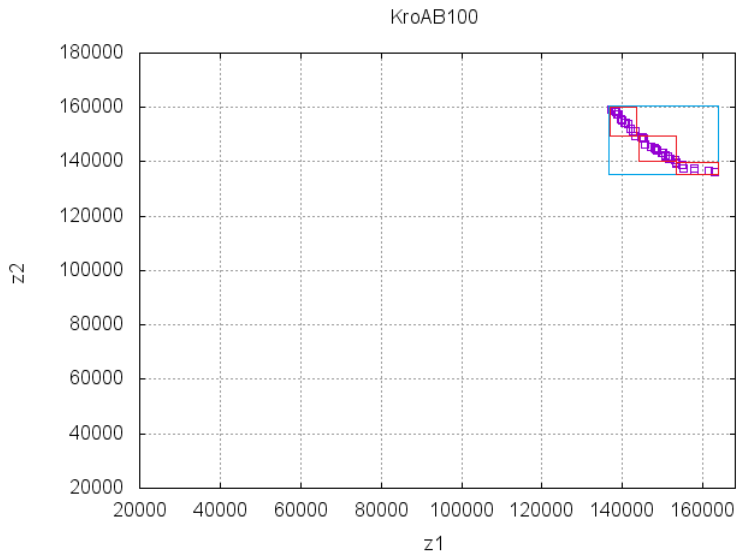
ND-Tree



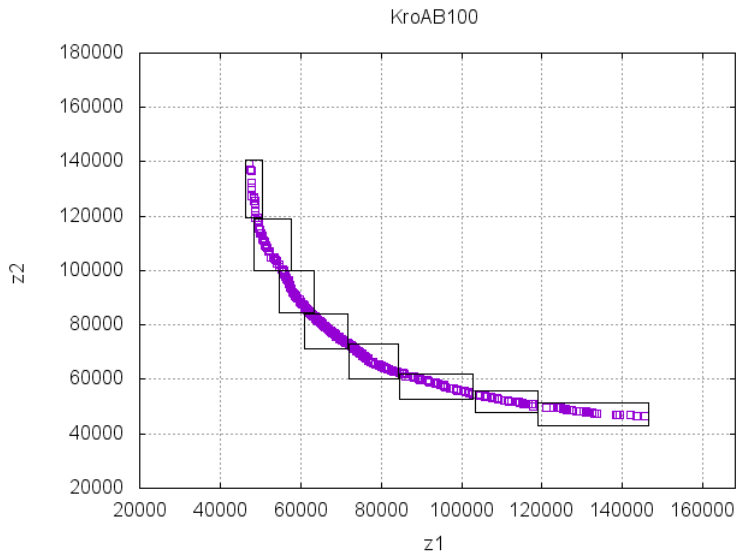
ND-Tree



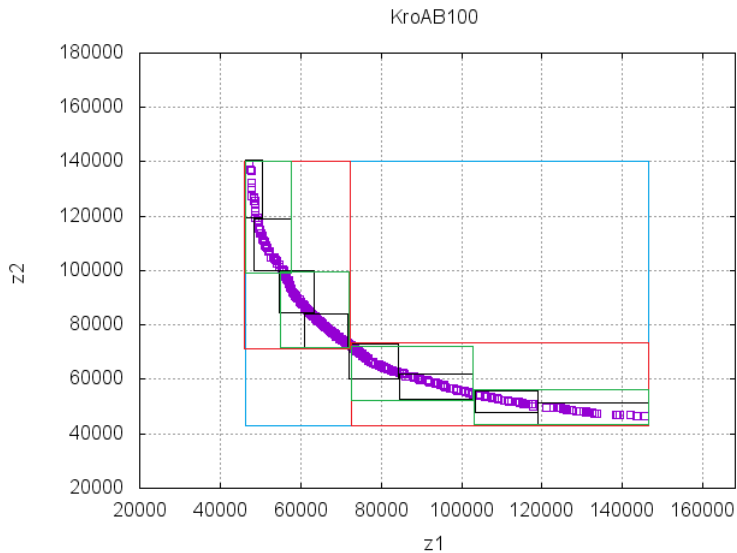
ND-Tree



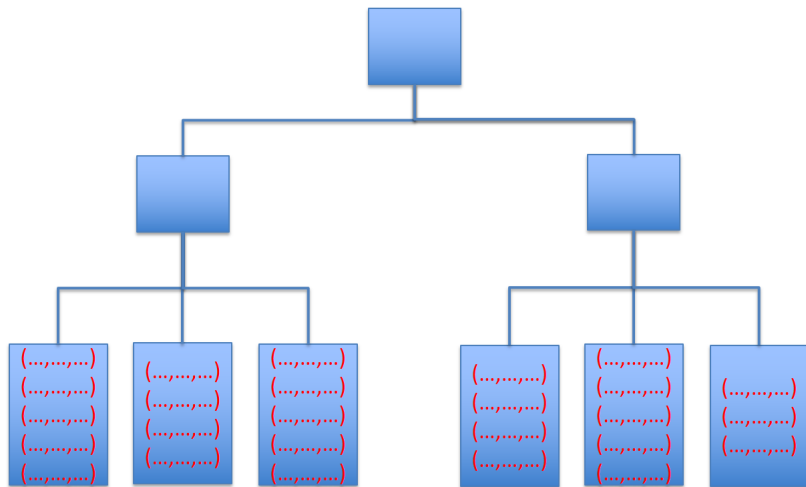
ND-Tree



ND-Tree



ND-Tree data structure



Integration of ND-Tree into PLS to solve the MOTSP

- ▶ Solving multi-objective instances of the TSP with up to 6 objectives
- ▶ ND-Tree is used to update the Pareto archive

Instance			CPU(s)		
p	Size	$ NDP $	List	ND-Tree	Speed-up
2	500	32842	24.26*	24.82	0.98
3	50	94343	3517.52	33.26	106
4	25	84013	1299.30	18.81	69
5	20	359581	20130.35	282.14	71
6	15	49341	257.93	12.09	21